
前言

你知道什么事很轻松吗？当你敬仰的人请你为了一本你如饥似渴读过的、且主题与你本职工作相关的书写前言的时候。

当迈克尔（Michael）和马特（Matt）告诉我们他们正在撰写一本关于PyCharm的书时，坦率地说，我们PyCharm团队非常兴奋。“畅聊Python”（Talk Python）是全球Python领域首屈一指的信息与培训资源之一，它有一种独特的天赋，能够以恰当的语气、适量地传达正确的信息。PyCharm是一款功能强大的大型工具，可能会让人望而生畏。虽然迈克尔的“精通PyCharm”课程有所帮助，但说实话，人们还是喜欢书籍。

所以我们知道PyCharm会得到专业的对待，而这就是：《高效使用PyCharm》。16章内容涵盖了PyCharm的方方面面。

在回顾这些内容时，我得尴尬地承认，我经常发现自己……不知道某些东西，或者在自己的编码中没有充分使用某些东西。例如，《重构》一章中的重复代码。我其实很乐意看到这种情况发生。PyCharm是一款面向专业人士的集成开发环境（IDE），其使用率正疯狂增长，而“IDE”和“专业人士”相结合，就形成了一个功能广泛的工具，需要不断学习。这也使得这本书非常有用且及时。这本书还有一个很好的“代码坏味道”：他们在第4章提醒你，本地历史记录可以帮你摆脱困境。

《精通PyCharm》结构合理，包含代码示例、截图以及一个会更新资料的代码库。这是此类作品的一个重要方面。就像软件一样，你希望知道它会由愿意为自己工作负责的人进行完善和改进。我也很欣赏他们既解释“是什么”又解释“为什么”的做法，这在学习过程中至关重要。

感谢您购买并研读本书。这不仅是因为PyCharm团队感激您使用我们的产品。还因为，感谢您支持那些用心且努力独立出版Python相关内容的优秀人士。我曾在“Python 1994”大会上做过关于Python社区最初几年的演讲。我们当时绝对想不到，到2019年Python会拥有一个能够支持像迈克尔（Michael）和马特（Matt）这样的人的生态系统。当然，正是因为有这样的人，这一切才得以实现。

再次感谢您，希望您能像我一样喜欢这本书。

保罗·埃弗里特（Paul Everitt）——JetBrains公司PyCharm产品的开发者倡导者

第1章

为什么是PyCharm和集成开发环境（IDEs）呢？

1.1 欢迎来到高效使用 PyCharm

大家好，欢迎来到《高效使用PyCharm》。在本书中，我们将探讨用于交互和创建Python代码的最佳环境之一——PyCharm的所有不同功能。PyCharm是一款IDE（集成开发环境），本书将教您如何充分利用这个超级强大的编辑器。让我们直接进入要涵盖的内容。

我们首先要讨论的是，为什么我们一开始就想使用集成开发环境（IDE）呢？像PyCharm这样相对重量级的应用程序能带来什么价值，我们又为什么要用它呢？有很多特性让PyCharm颇具价值。不过，我们先从讨论我们可以使用的各种类型的编辑器以及它们的权衡取舍开始说起。

在使用PyCharm时，我们关注的是*projects*。项目是一组Python文件以及与之相关的其他静态文件和数据文件。它们协同工作，而PyCharm具备将它们整合在一起的功能，这就是IDE中“*I*”（集成）的含义。

我们将首先专注于创建新项目并处理其中的所有文件。你会发现，我们可以设置一些配置开关，以提高效率。然后，我们将直接进入我认为是重头戏的部分——编辑器。

如果你在编写代码，就需要一个编辑器。你会编写大量代码。这包括输入新文本和处理现有文本。编辑器必须非常出色，能在这些任务中为你提供帮助。我们将重点介绍PyCharm编辑器提供的所有酷炫功能。

我们会发现，尤其是版本控制方面，Git和Subversion与PyCharm深度集成。除了git，我们还能做各种强大的事情，包括与GitHub的实际集成。我们将专注于版本控制以及IDE内部的功能。

PyCharm在*refactoring*方面表现出色。重构代码是指以不同的方式对代码进行重组，使用略有不同的算法，同时实际上不改变代码的行为。PyCharm中有许多强大的技术可用于此目的。由于它能一次性理解所有文件，因此可以安全地进行重构。它甚至会重构文档字符串以及其他若不深入理解代码结构就可能被忽略的内容。

PyCharm 中有强大的数据库工具。你可以与大多数数据库进行交互，包括 SQLite、MySQL 和 PostgreSQL。你可以编辑数据、编辑模式、运行查询等等。由于 PyCharm 对代码有深入的理解，因此数据库模式与 Python 文本编辑器之间甚至可以集成。请注意，PyCharm 有免费版和专业版。数据库功能仅在专业版中可用。

1. Why PyCharm and IDEs?

PyCharm 在使用诸如Django、Pyramid或Flask等库构建Web应用程序方面表现出色。它还拥有完整的JavaScript编辑器和环境，因此你可以使用TypeScript或CoffeeScript。我们将研究服务器端和客户端的功能。

PyCharm 有出色的可视化调试器，我们将研究它的所有不同功能。你可以使用它来调试和理解你的应用程序。它具备强大的断点操作和数据可视化功能，而这些通常是其他编辑器所没有的。

在Python中，我们使用打包的概念来捆绑、共享和重用我们的库。PyCharm有许多功能来创建和理解这些包。请记住，PyCharm了解包的文件结构。它可以进行自动导入，甚至自动生成一个`setup.py`文件，这样你就可以与他人共享你的项目，并且他们可以轻松安装它。

如果你想了解代码的运行情况，性能分析是一项常见任务。如果你的应用程序运行缓慢，而你希望它运行得更快，你不应该猜测它哪里运行缓慢。PyCharm 让查看代码并确定哪些部分运行得快、哪些部分运行得慢变得很容易，而不是依赖可能有缺陷的直觉。PyCharm 有一些非常强大的内置可视化工具，帮助我们从根本上了解应用程序的性能。

PyCharm 内置了适用于 `pytest`、`unittest` 以及许多 Python 测试框架的测试运行器。如果你正在进行任何单元测试或集成测试，PyCharm 会助你一臂之力。例如，你可以启用的一项功能是自动测试执行。如果你正在更改代码的某些部分，PyCharm 将自动重新运行测试。

PyCharm 集成了 Jupyter 笔记本。它有一个特殊的数据科学视图，包含文档和工具，可帮助您探索数据。我们将探讨 PyCharm 中的数据科学工具以及它们的工作原理。

还有一些额外的工具，它们并不真正属于上述任何类别。在结尾处有一章介绍这些额外的工具。

最后但同样重要的是，你可以为PyCharm获取数百个插件，使其能够执行其他任务。例如，你是否希望它具有类似Vim的键绑定——有这样的插件。插件有很多，我们将探索其中一些。

1.2 编辑器的范围

编辑器种类繁多。一方面，像Emacs或Vim这类编辑器启动速度极快，由于它们在终端或命令提示符中运行，所以可以在任何地方使用，即便通过ssh登录到服务器也能使用。这些编辑器功能强大，但它们主要专注于编辑单个文件，通常缺少集成开发环境（IDE）的许多“集成”功能。虽然它们功能强大，而且人们可以让这些工具发挥出强大作用，但它们不如PyCharm强大。它们既没有将所有工具整合在一处，也无法理解整个项目以及所有不同文件和组件是如何组合在一起的。

例如，如果我正在处理一个在Web框架中渲染模型数据的Jinja 2或Chameleon HTML模板，那么Emacs不太可能理解静态文件结构、CSS文件、JavaScript文件以及从Web服务器传递过来的模型数据。

Emacs和Vim并不追求拥有完整的集成视图，而是专注于编辑单个文件。这可能会在不经意间导致某些编程风格，即一个组织将大量代码放入单个Python文件中。

在集成开发环境（IDE）和文本编辑器之间的某个范围内，有诸如Visual Studio Code、Atom和Sublime Text之类的工具。这些是轻量级编辑器，但它们是图形用户界面（GUI）。

1. Why PyCharm and IDEs?

另一个例子是“你试图在一个不存在的模块上调用这个函数。你希望我为你编写这个函数，然后你来补充细节吗？”

另外，“我发现你正在使用一个外部库，然而，它并未在你的（此处原文不完整）” requirements.txt “，我要加上它吗？”

即使你清楚自己在做什么，所有这些简洁的“代码意图”都非常棒。你无需花时间去思考这一点，意味着你可以保持思路流畅，继续编写代码。

它具有出色的源代码集成功能。Git、GitHub 以及其他各种源代码控制系统都已集成。如果你不想仅仅为了进行源代码控制就离开当前环境，这一点非常棒。或者，如果你更喜欢可视化环境，PyCharm 在编辑器中就提供了源代码控制功能。

PyCharm中的数据库工具非常棒。理解各种关系型数据库很简单。即使你使用像SQLAlchemy这样的工具，一旦创建了模式，也有很棒的工具来帮助你理解、探索和更改数据库。

PyCharm 支持包管理。这意味着你可以创建新的包，并搭建安装基础架构或框架。然后，你可以在 PyPI 上安装并注册你的包。

最后，单元测试功能非常出色。如果你正在进行单元测试甚至集成测试，可以编写pytest或常规的unittest。PyCharm可以运行这些测试，并为你提供详细的代码覆盖率报告。

除了上述内容之外，还有很多内容，我们将在本书中涵盖其中的大部分。

1.4 集成开发环境（IDE）速度极快

在深入探讨PyCharm的细节之前，让我们先破除一个我认为的误区——即集成开发环境（IDE）运行缓慢。没错，Emacs的启动速度肯定比PyCharm快。然而，我们优化的目标是什么呢？编写软件时最重要的是什么？是程序启动有多快，还是完成工作有多快？正如你将看到的，我认为使用IDE实际上能让你工作得快得多。

虽然启动可能会多花一秒钟，但PyCharm能让你一整天的工作效率大幅提升。反正你启动PyCharm后会让它运行一整天，所以一旦启动并运行起来，它不会变慢。在现代机器上，PyCharm可能需要5秒钟来启动。

另一个问题是功耗。由于PyCharm要进行大量分析，这可能会限制你在飞机上或者当电池电量只剩下10%时创造下一个网络爆款的能力。PyCharm对此有一个解决方案，叫做“性能模式”。

此电源模式允许您关闭部分实时分析和代码补全之类的操作，使PyCharm能耗更低。

1.5 PyCharm不仅仅适用于Python

有一点并非一目了然（实际上，我刚开始接触时就误解了这一点），那就是PyCharm是如何将各个组件组合在一起的。PyCharm显然是一款Python编辑器。然而，正如我所暗示的，它还具备其他功能，比如数据库工具或编辑其他语言。PyCharm是由JetBrains的一系列其他工具组合而成的。

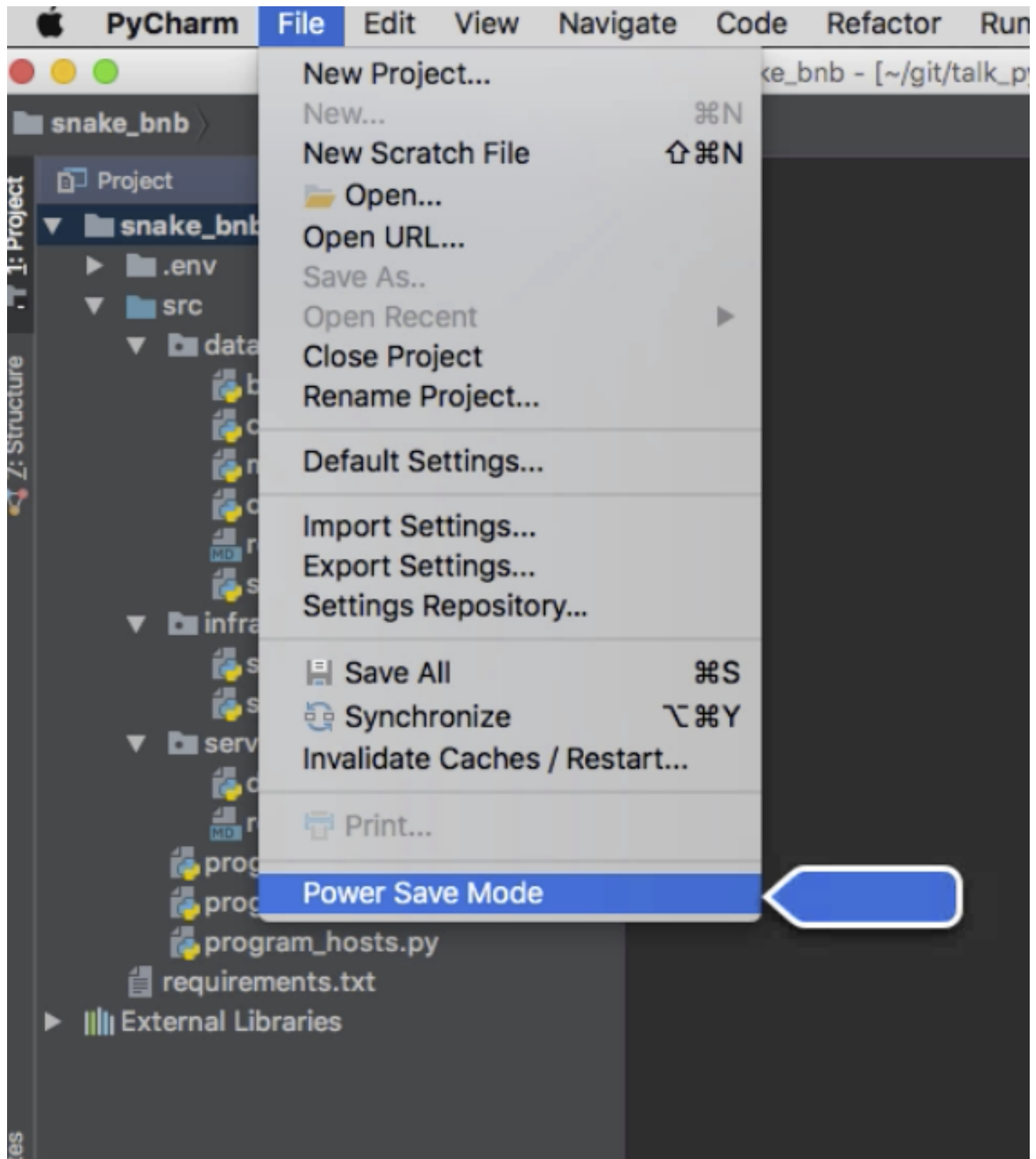


图11: 长途飞行时, 或者一天结束时电量耗尽的情况下, 你可以开启 *Power Save Mode*。

1. Why PyCharm and IDEs?

JetBrains 开发了许多不同的集成开发环境（IDE）：PyCharm、WebStorm（用于 JavaScript 和 HTML）、AppCode（用于构建 iOS 应用程序）、DataGrip（用于编辑数据库）等等。然而，所有这些 IDE 都构建在一个名为 IntelliJ 的通用平台之上。

IntelliJ 最初只是一个 Java 集成开发环境（IDE），但它具有很强的可扩展性和可配置性。它被拆分后，可作为所有这些不同工具的基础。IntelliJ 能做的大多数事情，PyCharm 或其他这些编辑器也能做到。

PyCharm 专业版是付费版本，如果您需要更广泛的工具支持，应该考虑使用它。如果您从事 Web 应用程序、JavaScript 或数据库相关工作，专业版能够与所有这些进行交互。

WebStorm 是一款很棒的 JavaScript 和 TypeScript 编辑器。如果你在做 AngularJS、Ionic、Cordova 之类的项目，它是一个很好的工作平台。它对 JavaScript、CSS、HTML 的理解非常出色。事实上，PyCharm 的专业版包含了 WebStorm 的所有功能。

PyCharm 还具备一款名为 DataGrip 的工具的功能。DataGrip 是 JetBrains 开发的一款数据库集成开发环境（IDE），PyCharm 内的所有数据库功能均源自于此。

当你想到 PyCharm 时，你应该想到作为 Python 编辑器的 PyCharm，再加上 WebStorm 和 DataGrip。这不是一个单一的工具，而是一个全面的全栈工具集，可用于与 Web 开发、数据库工作甚至数据科学相关的任何事务。

1.6 跨平台

PyCharm 的一个出色特性是，你可以在任何操作系统上使用它。无论你使用的是 Windows、Mac 还是某种形式的 Linux，都能使用 PyCharm，而且其表现大致相同。（按键绑定可能略有不同——Mac 上使用 command，而 Windows 或 Linux 上使用 control。）

如果你在团队中工作，团队成员使用不同的平台，或者你想在不同的机器上使用它，都不用担心。PyCharm 很好地遵循了 Python 随处运行的理念。

1.7 版本：专业版和社区版

如果我们访问 PyCharm 官网并点击下载，就会面临一个选择。你是想要需要付费的专业版，还是 100% 免费且开源的社区版呢？

很难说。社区版是免费的，但专业版可能有额外福利。
w 我们可能想要的。我们怎么知道呢？

本书中我们介绍的一些内容仅在专业版中存在。例如，WebStorm 和 DataGrip 属于专业版。JetBrains 有一个页面描述了这些差异。¹

如果你只使用 Python，免费版本的功能相当齐全。其中包括前面提到的代码意图、重构、代码检查、源代码控制以及调试器。

如果你正在进行网页开发，并且需要 WebStorm 的功能，或者需要对各种 Python 框架（Flask、Pyramid 或 Django）的支持，那么你应该考虑完整版。你可以编辑模板，还能使用远程调试功能和 DataGrip 等工具。

如果某些功能仅在专业版中可用，本书将予以指出。

¹https://jetbrains.com/pycharm/features/editions_comparison_matrix.html

1.8 PyCharm是开源的

PyCharm社区版是100%开源的。GitHub上的JetBrains组织²有社区版。如果出于某种原因，你真的在意PyCharm开源这件事，你可以去查看并构建它。

我从未尝试过，也对构建PyCharm不感兴趣。也许我会开发一个插件，但不会从头开始构建整个项目。尽管如此，知道代码就在那里，如果你想查看或修改它，这还是很不错的。

1.9 Python运行时

如果你是Python新手，可能没有意识到Python有许多不同的版本和变体。当你只说Python时，你可能指的是一种叫做“CPython”的东西（如果你访问python.org并为你的操作系统下载Python，得到的就是它）。然而，Python有一系列不同的运行时，有时也称为解释器。（我不喜欢“解释器”这个术语，因为并非所有这些都是解释器，其中一些是即时编译器。）

其中之一是PyPy，它是Python运行时的即时编译器。在某些情况下，PyPy可能比CPython快得多，但它对您可能使用的所有库的覆盖范围没有那么广。许多基于C的扩展在PyPy中不受支持。

Cython是另一种选择。Cython是一种工具，可将类似Python的代码编译为机器指令。这甚至不是即时编译，而是像C语言一样编译。它使用一种类似Python的语言，可以在其中提供类型以提高速度。PyCharm的代码补全引擎支持Cython。

这样我们就有了两种基于插件的Python版本：Jython和IronPython。它们分别与Java和.NET对接。

无论你想如何运行Python代码，PyCharm都支持所有这些Python变体。

1.10 如何了解集成开发环境（IDE）的所有功能？

如果你是PyCharm新手，集成开发环境（IDE）可能会让你应接不暇。从图中可以看到，有许多窗口，而且这些窗口中会发生很多事情。在中间位置，隐藏着编辑器、项目信息、数据库工具和配置。还有许多包含大量选项的菜单，而且你几乎可以右键单击任何内容以调出更多选项。

要熟练使用PyCharm和其他集成开发环境（IDE），需要学习的东西实在太多了。这些工具功能繁多，要记住它们所有的特性和操作真的很难。如果你不能充分利用它们，那么对您来说，它们的价值就和一个简单的编辑器差不多。

你应该致力于学习所有这些小功能（或者至少是你大部分时间使用的80%）。但是你要怎么做呢？你如何逐一学习所有这些东西呢？

我将给你一套我自己或多或少会用到的具体技巧，你也可以使用。首先要发现有哪些功能。我们将在本书中介绍这些功能。在本书之外，你还需要发现更多功能和快捷键。

²<https://blog.jetbrains.com/pycharm/2013/10/pycharm-3-0-community-edition-source-code-now-available/>

1. Why PyCharm and IDEs?

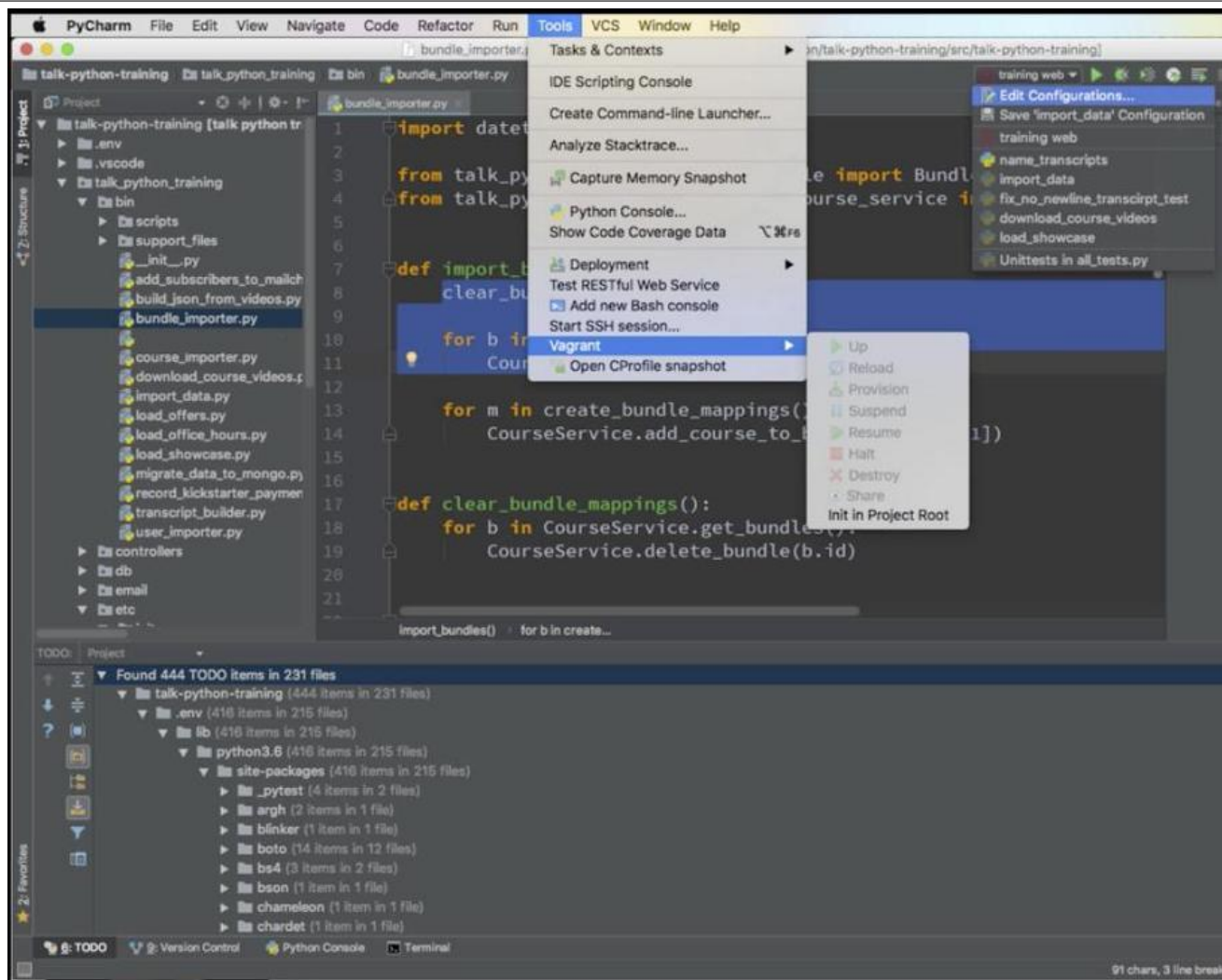


图12：展示PyCharm众多（可能多得让人应接不暇）功能的图

更重要的是，或许也更具挑战性的，你需要切实回忆起这些功能。你希望一直有这样的体验：“哦对，有这个功能，它超棒，而且快捷键在这儿。”要怎么做呢？

我打算提出一些你可以尝试的方法，将它们调整为适合你学习风格的方式。

首先，仅仅阅读有关PyCharm的内容所能带给你的收获是有限的。它会介绍许多功能，但如果你从不尝试使用这些功能，很可能会忘记它们。确保你不只是在阅读这本书，还要在身边准备一台电脑，以便尝试PyCharm的各项功能。

接下来，有一份来自JetBrains的备忘单。³它包含最常见功能的快捷键。把这个打印出来并放在手边备用。

学习快捷键是件苦差事，但它们会带来回报。如果你无需伸手去拿鼠标，你的工作效率会高得多。

最后一个学习技巧是使用便利贴。读书时在身边放一叠便利贴。当提到某个功能时，把该功能以及备忘单上的快捷键记在便利贴上。

³https://blog.jetbrains.com/pycharm/files/2010/07/PyCharm_Reference_Card.pdf

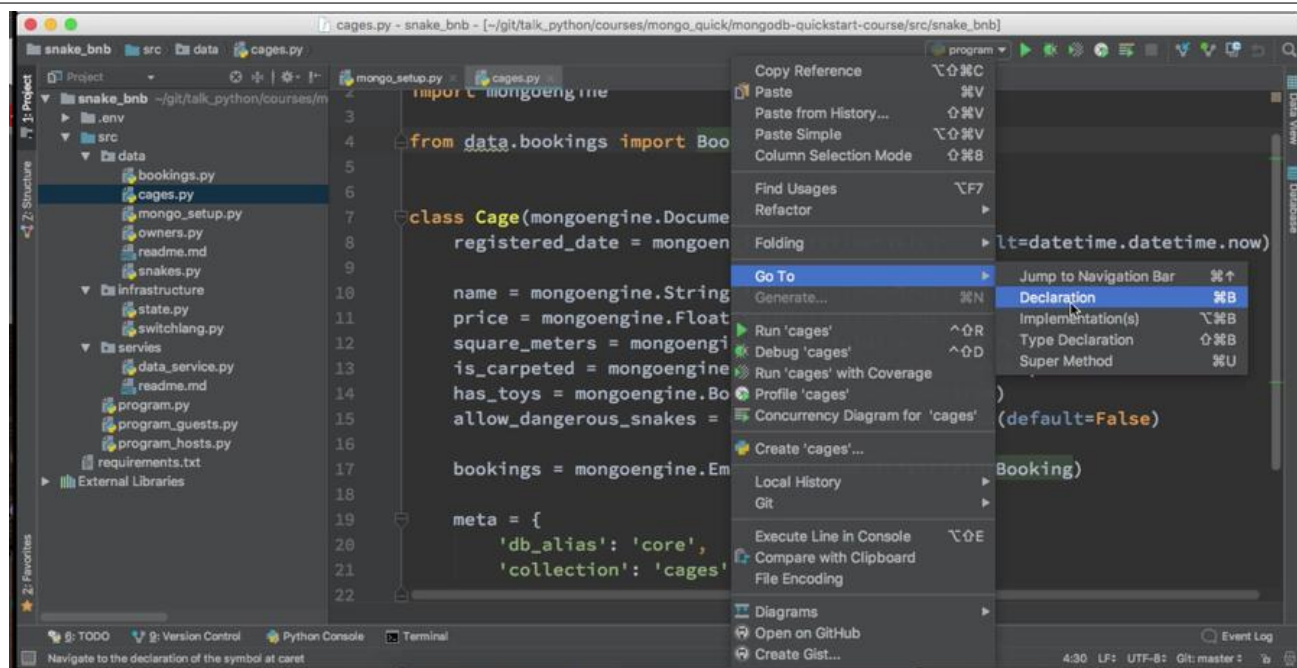


图13: 安装演示助手后, 点击“导航到声明”选项。

你应该准备一叠便利贴。每周，你都应该在显示器的一角贴上一张（或多张）。齐心协力地使用该功能和快捷键，直到它成为你的第二天性。如果没有必要，尽量不要使用鼠标。一旦你熟练掌握了，就把那张便利贴收起来。

[illegible]

最后一个提示。有一个演示助手。如果启用此插件，并且您用鼠标单击某个命令（按钮或菜单项），它会在屏幕底部闪烁显示该命令的热键几秒钟。

要安装演示助手，请转到“偏好设置”页面，输入“插件”。从那里点击“浏览存储库...”。在搜索框中输入“演示助手”，然后点击“安装”按钮。

另一种选择是Key Promoter X⁴插件。当您使用鼠标时，该插件会显示命令的键盘快捷键。搜索“Key Promoter X”来安装该插件。

1.11 在Github上获取源代码

最后一件事——前往GitHub并给本书的资料⁵加星标。

每章都有实践练习，你可以通过完成这些练习来巩固知识。阅读有关PyCharm的内容是一回事。如果你能够应用所学知识，那么你对PyCharm的掌握将会更加深入。

⁴<https://plugins.jetbrains.com/plugin/9792-key-promoter-x>

⁵<https://github.com/talkpython/mastering-pycharm-course>

1. Why PyCharm and IDEs?

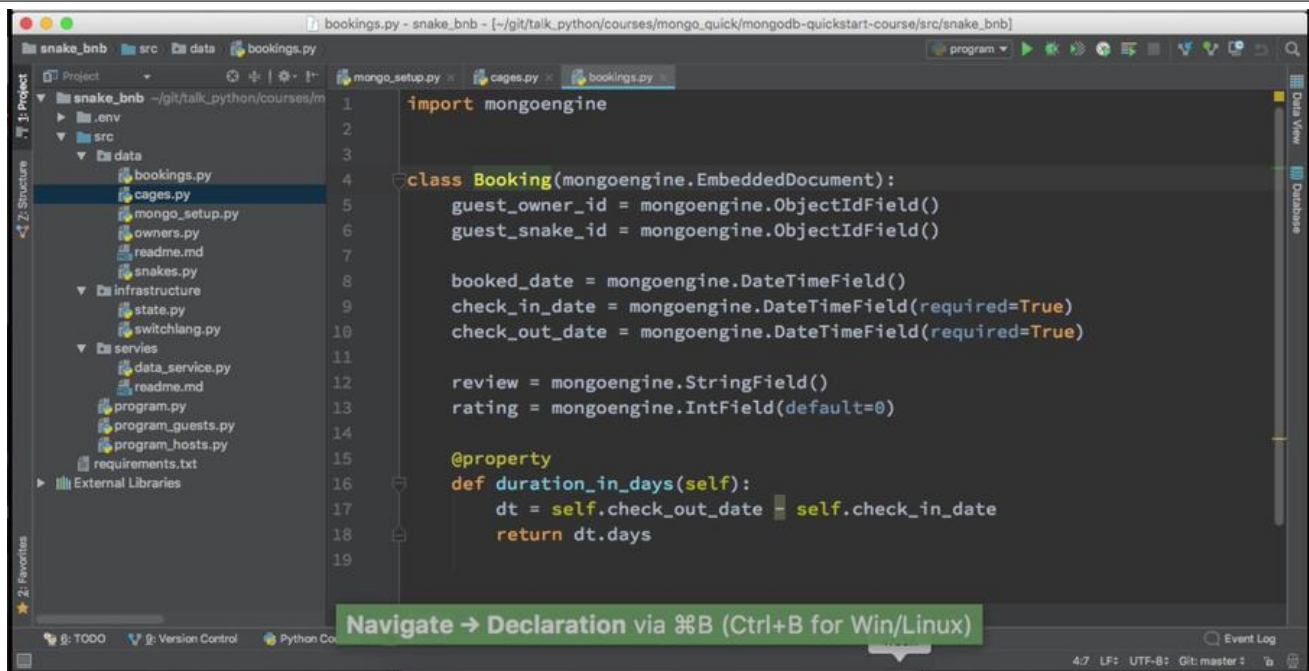


图14：演示助手显示命令的热键。

1.12 总结

PyCharm是功能最强大的Python集成开发环境（IDE）。它可在最常见的操作系统上运行，如Mac、Windows和Linux。开源版本允许您创建和调试Python程序。如果您正在从事Web项目或数据库项目，专业版具有引人注目的功能来协助您。

学习像PyCharm这样功能丰富的工具可能是一项令人生畏的任务。我们建议你打印出速查表，在便利贴上写下快捷键，使用演示助手，并通过练习来实践使用PyCharm。

第2章

机器设置

2.1 PyCharm社区版还是PyCharm专业版？

按照这些说明操作时，你可以选择使用PyCharm专业版（付费）或社区版（免费）插件。你可以选择免费版，但请注意，本书中约30%的主题仅在专业版中可用。好消息是，免费版涵盖了其余70%的内容！如果你想了解哪些功能在哪个版本中可用，请查看它们的对比矩阵⁶。我们建议使用专业版，因为它拥有强大的工具，让我们开发者的工作更加轻松。如果你想试用一下，可以使用免费试用版。

注意

PyCharm专业版对开源开发者和教育用途免费。

2.2 macOS设置

我们来谈谈如何在 Mac 上设置 PyCharm。首先，你要确保自己安装了较新版本的 Python 3。打开终端（按下 CMD+空格键并输入“terminal”），然后输入命令：`python3 --version`。如果已安装 Python，你应该会看到类似以下内容：

```
$ python --versionPython 3.7.1
```

我安装了Python 3.7.1，它能很好地运行本书中的示例。如果你看到的是这样：

```
$ python3 --version -bash: python3: 未找到命令
```

```
PyCharm macOS Python 2 Python 3
Python 3 macOS Python 3
http://python.org Python 3
Homebrew 7 Linux Mac
Homebrew
```

⁶https://www.jetbrains.com/pycharm/features/editions_comparison_matrix.html

2. Machine Setup

如Node、OpenSSL、MongoDB，当然还有Python。

要安装Homebrew，从网页复制命令并在终端中执行：

```
$ /usr/bin/ruby -e "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/master/install)"
```

该脚本会说明它要执行的操作，然后在执行前暂停。还有更多安装选项（OS X 10.8 Mountain Lion及更低版本需要）。你需要安装XCode的命令行工具（详见⁸）。

安装完成后，你应该能够使用`brew`命令。如果你之前已安装过 Homebrew，请务必在继续操作前运行`brew update`。现在你可以运行：

```
$ brew install python
```

它会为你安装最新版本的Python 3。我更喜欢使用Homebrew安装的版本，因为如果我需要编译一个包装C或Fortran库的Python库，Homebrew通常“开箱即用”。

接下来，我们要安装PyCharm。前往下载页面⁹，下载你喜欢的版本。

安装PyCharm后，你可以通过双击“应用程序”文件夹中的图标，或按下“command + 空格”组合键，然后输入“PyCharm”来启动它。我们建议将其固定在程序坞中，以便于访问。

2.3 Windows设置

如果你使用的是Windows系统，首先需要确保已安装Python。Windows系统本身不自带任何版本的Python，你需要自行安装。要检查是否已安装Python，打开命令提示符。你可以通过点击屏幕左下角的搜索框并输入cmd来实现。命令提示符打开后，输入：

```
C:\> python --version
```

此命令将报告您所拥有的 Python 版本，或者提示 python is not recognized。在这种情况下，请访问 <http://python.org> 并下载适用于 Python 3 的最新安装包。确保在安装向导的第一页上勾选“更新 PATH”复选框，以便您可以从命令提示符访问 python。

□ □ □ □ □ □ □ □ □ PyCharm □ □ □ □ □ □ □ □ □ □ 9 □ □ □ □ □ □ □ □ □ □ □ □ □ □

现在你应该可以从开始菜单运行PyCharm了。我们建议将其固定到任务栏以便快速访问。

⁷<https://brew.sh>

⁸<https://docs.brew.sh/Installation>

⁹<https://www.jetbrains.com/pycharm/download/>

¹⁰<https://www.jetbrains.com/pycharm/download/>

2.4 Linux设置

大多数Linux机器都预装了Python。通过打开终端并输入以下内容，检查是否安装了Python 3：

```
$ python3 --version  
Python 3.6.6
```

如果有最新版本，那就没问题。否则，你可以使用包管理器安装更新的版本。

接下来，我们要安装PyCharm。前往下载页面¹¹，下载你喜欢的版本。

然后你可以将其固定到收藏夹，这样在翻阅书籍时就很容易找到。

¹¹<https://www.jetbrains.com/pycharm/download/>

第3章

PyCharm项目

3.1 项目介绍

我们已经讨论了PyCharm、它的一些特性、为什么需要它以及如何安装它。现在是时候开始使用PyCharm来做一些很酷的事情了。我们将从关注项目开始。PyCharm是以项目为核心组织的——一组文件及其相互关系。这些项目实现的一些特性包括通过文件关系进行导航、跨多个模块跳转到定义处以及重构方法。如果你正在处理一个Web应用程序，你可以编辑静态文件并获得自动补全功能。

我们将重点关注项目的关键部分，并展示如何将一堆文件组合成一个单一的应用程序，在这些文件间导航，以及运行它们。

3.2 创建项目

让我们从创建一个新项目开始。我们将使用一个现有项目。输入：

```
$ git clone https://github.com/talkpython/mastering-pycharm-course.git
```

我们可以用PyCharm克隆这个，但你会发现应用程序中可能会遗漏一个部分。

启动PyCharm并点击“创建新项目”。我们将创建一个“纯Python”项目。在位置区域使用类似first_project的路径。它还会要求您选择一个Python解释器。PyCharm应该能找到我们安装的最新版本的Python，如果找不到，请将“解释器”选择指向该版本。

如果您在已经由版本控制系统（如git）管理的文件夹中创建了一个项目，PyCharm会弹出一个窗口，询问您是否要将该项目添加到版本控制中。大多数情况下，选择注册版本控制系统根目录是个好主意。它会激活PyCharm中围绕文件和源代码控制的大量功能。

在PyCharm的左侧是项目区域。您可以看到您的文件夹、创建项目时选择的Python环境以及外部库。

如果您右键单击项目，可以通过单击“新建>”“Python文件”来添加新文件。

PyCharm会要求您输入文件名。将该文件命名为hello。同样，如果您位于一个受版本控制的目录中，它会询问您是否要添加该文件。在该文件中输入以下代码：`print("hello world")`

创建另一个名为otherfile.py的文件。在该文件中添加：

3. PyCharm Projects

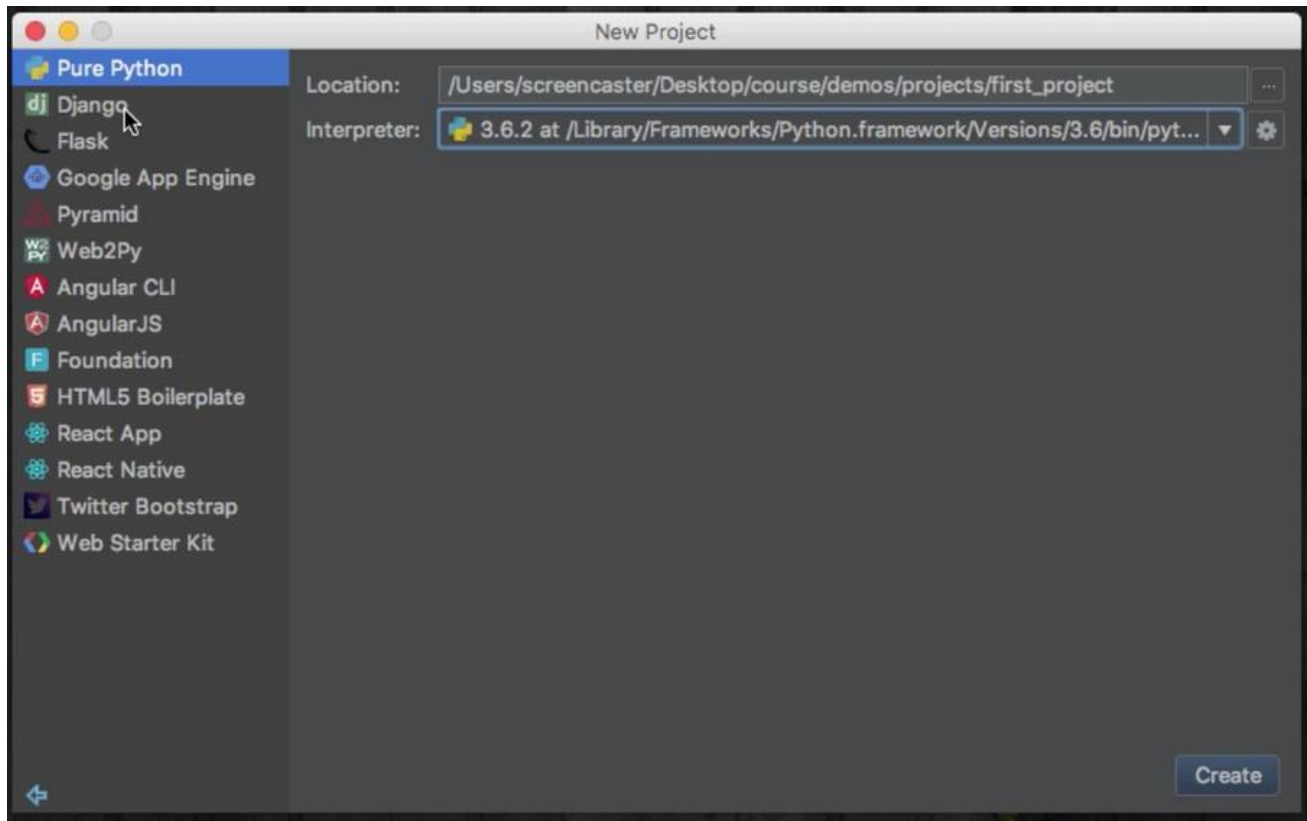


图31：说明第一个项目的位置和解释器的图。

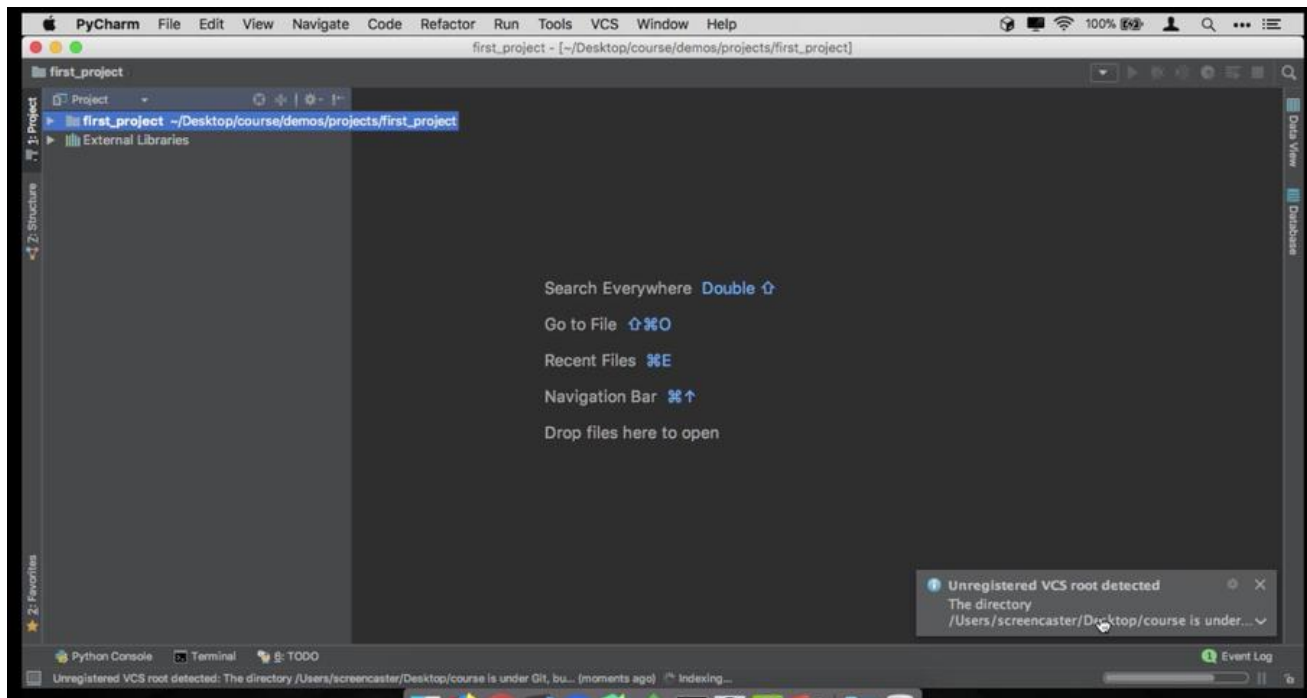


图32：展示一个新的空项目的图。注意右下角的版本控制系统（VCS）弹出窗口

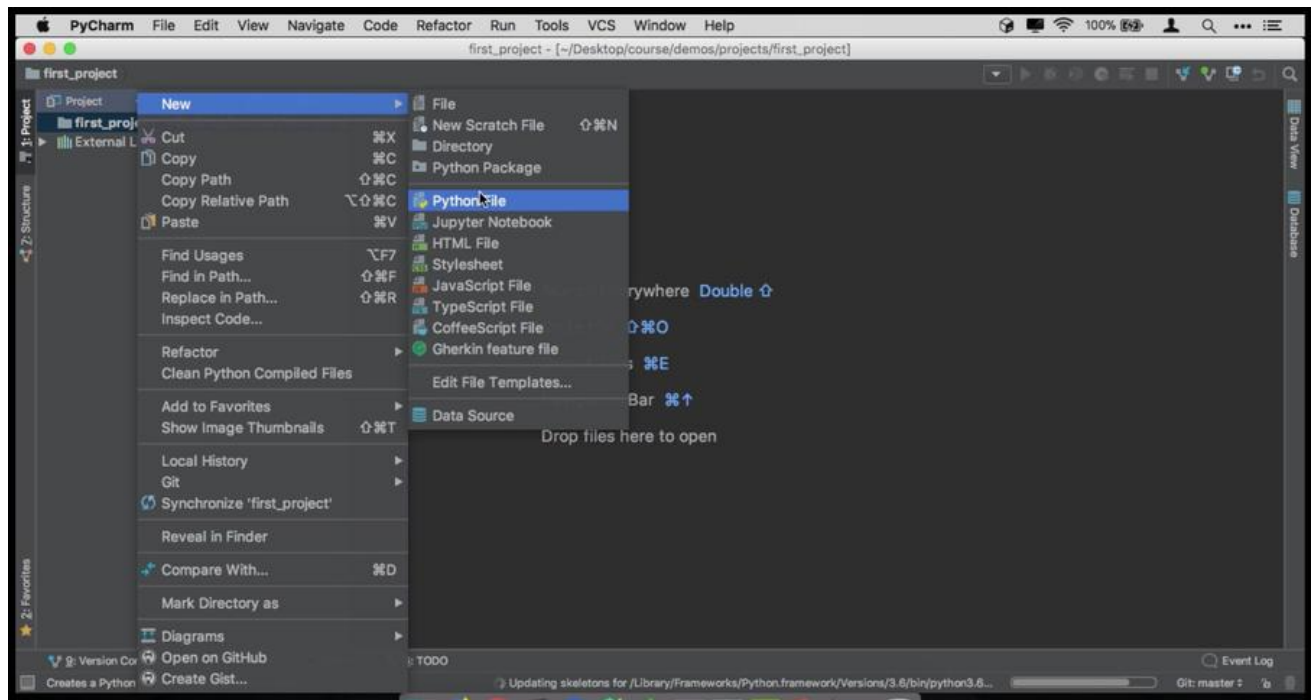


图33：说明创建新Python文件的图

```
def other_method():
    print("This is the other method")
```

要运行一个文件，你需要在项目窗口中右键单击hello.py文件，然后点击“运行’hello’”。

以这种方式运行一个文件将生成一个 *run configuration*。如果你查看PyCharm的右上角，你会看到一个指向“hello”配置的下拉菜单。有用于运行、调试、报告代码覆盖率以及分析该配置的按钮。

这些是从头开始创建项目的基础知识。你在PyCharm中指定一个文件夹，选择一个Python解释器，并为你添加的顶级文件设置运行配置。

3.3 使用现有项目

大多数时候，你会打开现有的项目。毕竟，我们会在项目（包、应用程序和网站）上花费数年时间，很少创建新项目。让我们找一个现有项目，并在PyCharm中打开它。

这次，让我们克隆《Talk Python 快速入门》课程的代码仓库：

```
$ git clone https://github.com/mikeckennedy/python-jumpstart-course-demos
```

在这个仓库中有一个apps文件夹，里面包含各种不同的应用程序或项目。你可以在其中创建一个包含子项目的大型项目，或者根据这里的不同目录创建一堆较小的项目。我们将选择“大型项目”。不过，在开始之前，让我们先为该项目创建一个虚拟环境。虚拟环境是一个独立的Python安装环境，你可以在其中安装你的软件包和第三方依赖项。

创建虚拟环境主要有两种方式。一种是通过终端。这种方式很好，因为它在任何地方都能使用，并且不依赖于任何工具。下面将对此进行描述。另一种选择是使用PyCharm来创建环境。

3. PyCharm Projects

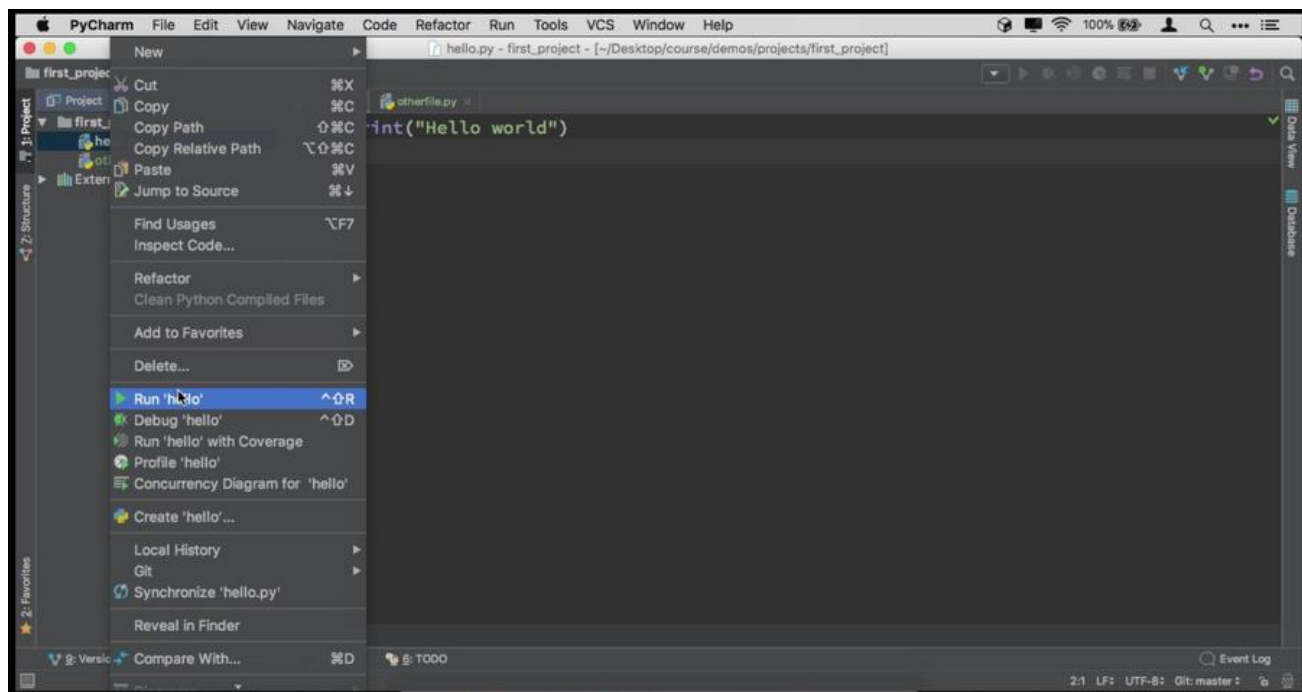


图34：说明配置文件以运行的示意图

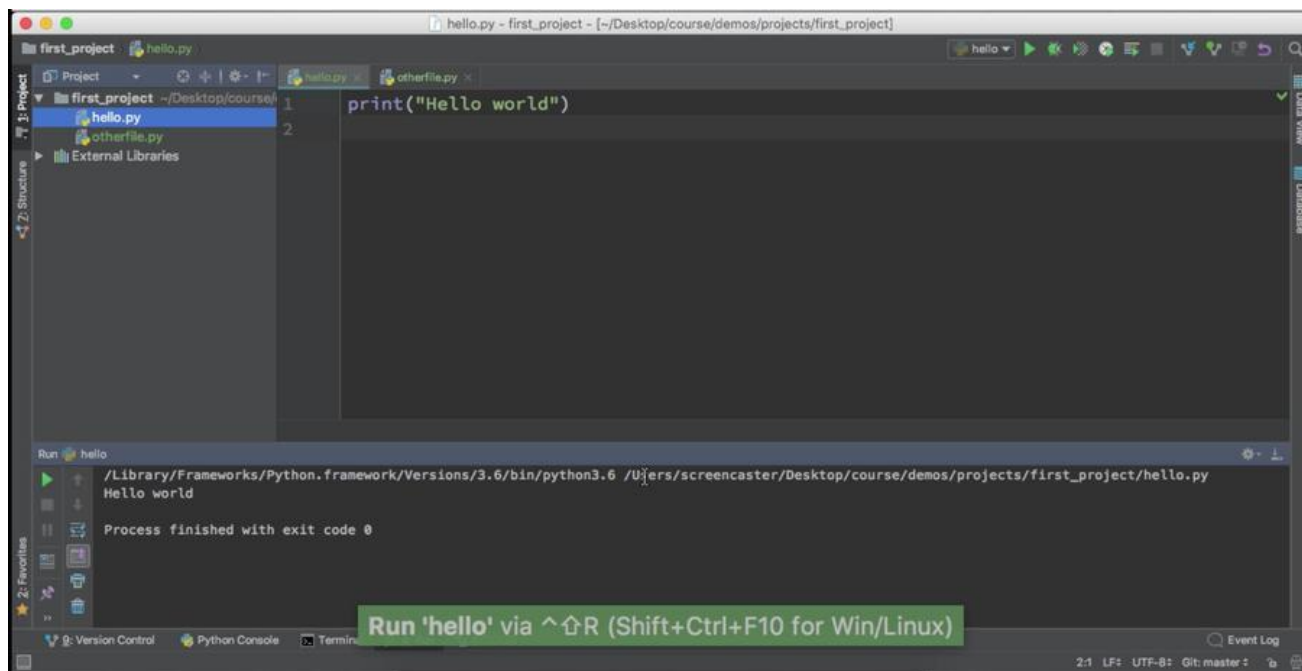


图35：说明运行文件的图。输出显示在下方窗格中，“演示助手”会显示快捷键（Mac 上为 shift command R）。

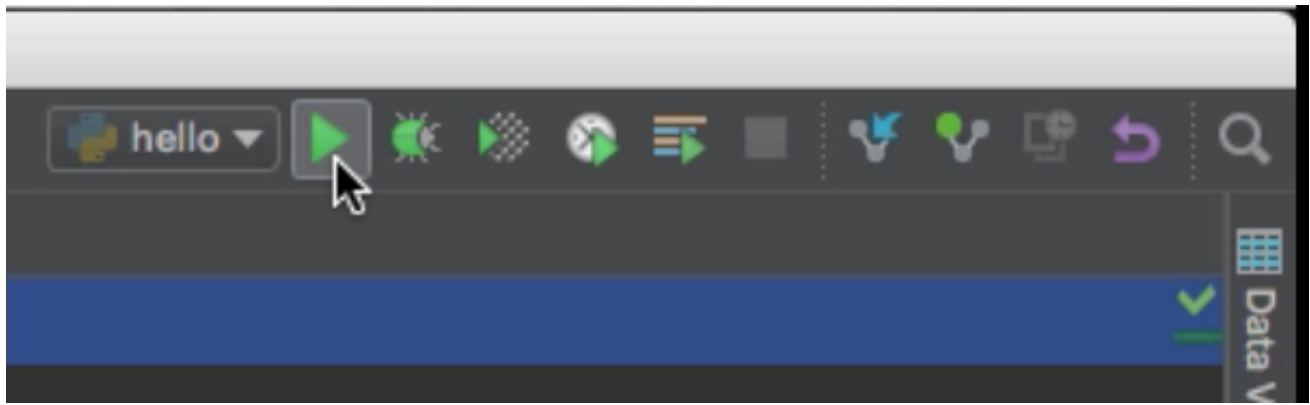


图36: 说明运行现有配置的按钮的图。有用于运行、调试、报告代码覆盖率以及分析配置的按钮。

要从终端创建环境，请执行以下操作。在终端/命令提示符中，切换到刚刚检出的文件夹（即 `python-jumpstart-course-demos`）。我们将使用以下命令创建一个虚拟环境：

```
python3 -m venv .env
```

注意

对于在所有平台（Mac、Windows 和 Linux）上都能使用的命令，我们不会在其前面加上适用于 Mac 或 Linux 的 `$` (Unix 提示符)，也不会加上 `C:\>` (Windows 提示符)。当我们针对这个跨平台命令调用 Python 时，我们会指定 `python3`，尽管 Windows 上的命令会是 `python`。在 Windows 上，由于 Python 不支持 `python3` (vs `python`) 命令，你需要确保 Python 3 的 `python.exe` 在你的路径中排在首位。测试这一点的一个简单方法是在命令提示符中输入 `where python`。

```
$ python3 -m venv .env
```

而Windows版本则为：

```
C:\> python -m venv .env
```

要使用PyCharm创建虚拟环境，打开 *Preferences* 窗口并搜索“项目解释器”。如果单击默认解释器旁边的齿轮，您可以单击“添加...”。在“添加Python解释器”窗口中，您可以创建“新环境”或使用“现有环境”。如果您的计算机上安装了多个Python版本，请确保选择了合适的“基础解释器”。请注意，PyCharm会将环境放在您主文件夹中的 `.virtualenvs` 目录中。

PyCharm 允许你打开 `python-jumpstart-course-demos` 文件夹来创建这个大型项目。或者，在 Mac 系统上，如果你在程序坞中有 PyCharm，你可以将该文件夹拖到应用程序图标上。由于我们有一个包含虚拟环境的 `.env` 文件夹，PyCharm 可以识别它，并将其用作默认运行配置。PyCharm 识别为虚拟环境的名称包括 `venv` 和 `.env`。

我们来谈谈PyCharm中一个名为 *Sources Root* 的概念。由于这个项目包含多个子项目，PyCharm 会假定项目的Python包位于根目录中。如果我们导航到 `apps/07_wizard_battle/final` 文件夹，会看到有两个文件：`actors.py` 和 `program.py`。如果你打开 `program.py`，你会发现它引用了以下代码

3. PyCharm Projects

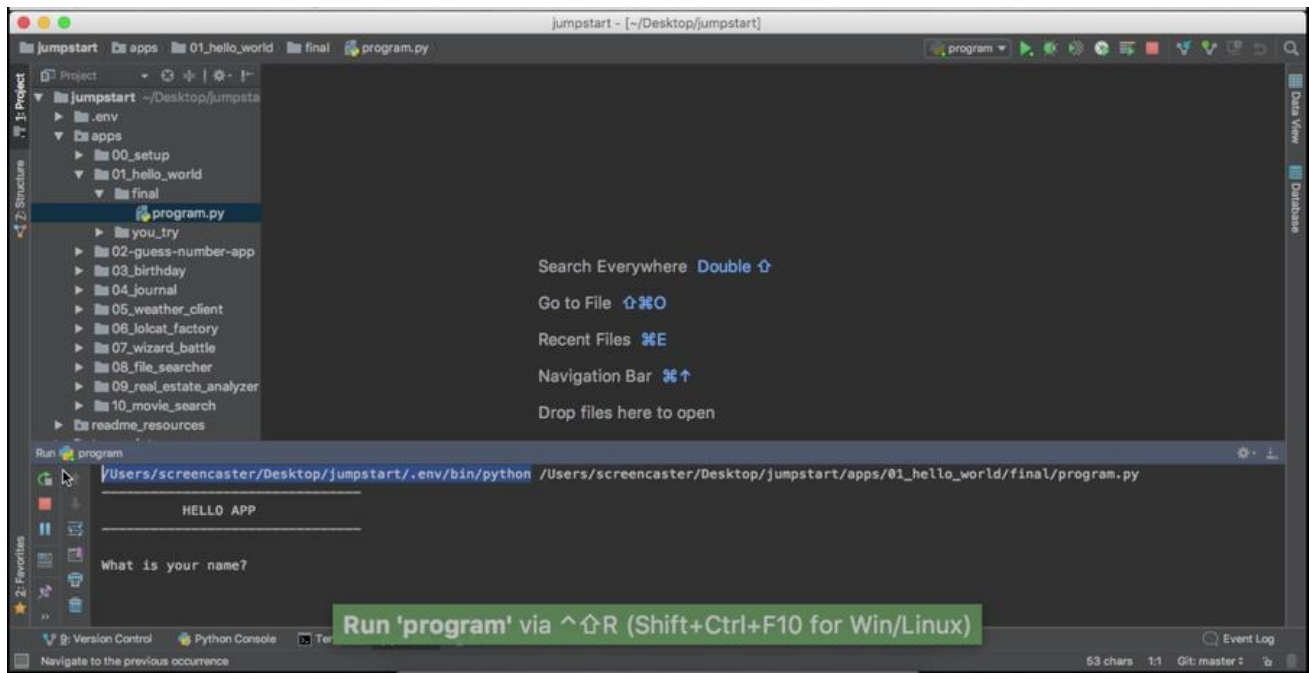


图37: 说明在包含现有虚拟环境（.env文件夹）的文件夹中运行文件的示意图

actors.py 并且在这些引用下面会出现红色波浪线。这是因为PyCharm会在顶级目录中查找 actors.py (和虚拟环境)，但却找不到。然而，如果我们在PyCharm中运行program.py，它就能顺利运行。当程序执行时，Python会将包含program.py的目录添加到其用于查找库的路径列表中。

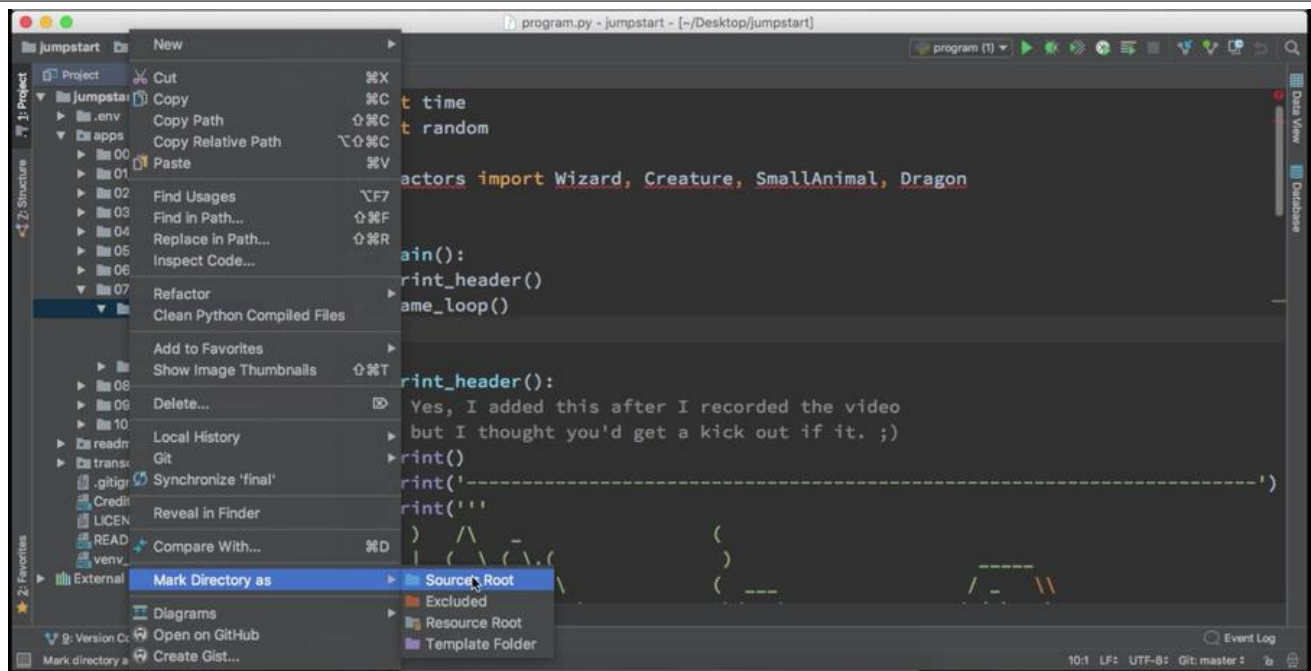
要让PyCharm将final文件夹识别为一个独立于项目其余部分的包含源代码的文件夹，我们可以右键单击该文件夹，选择“Mark Directory as”（标记目录为），然后选择“Sources Root”（源根目录）。这样PyCharm就能发挥其智能功能，并启用自动补全等特性。

同样，如果你正在开发一个Web应用程序，并且有类似apps/static/site.css的结构，PyCharm不会将/static/site.css与该文件夹中的URL关联起来。但你也可以解决这个问题。右键单击并选择“将目录标记为”，然后单击“资源根目录”（就像网站资源那样）。PyCharm还允许你创建模板文件夹（用于Django、Pyramid和Flask）。你甚至可以告诉PyCharm忽略某些文件夹，不对其进行分析或索引。这既可以提高索引速度，又可以在自动补全中省略该内容。

总之，在PyCharm中使用现有项目需要打开文件夹。如果该文件夹中有虚拟环境，PyCharm将自动检测到它，并使用该环境来执行文件（仅在首次创建项目时）。我们可能需要将文件夹标记为源文件夹、资源文件夹或模板文件夹，以便告知PyCharm如何对其进行索引和集成。要运行一个文件，可以右键单击该文件并选择“运行”，或者显式创建一个运行配置。

3.4 Python环境

PyCharm 可以识别现有项目中的虚拟环境。或者，你可以让 PyCharm 使用系统 Python（Mac 或 Linux 上的 Python 2），或系统范围内安装的 Python 3。你甚至可以让 PyCharm 使用 PyPy、IronPython 或 Jython。在本书中，我们使用来自 python.org 的 Python 3 版本（也称为 CPython，因为它用 C 语言实现的）。



38 PyCharm

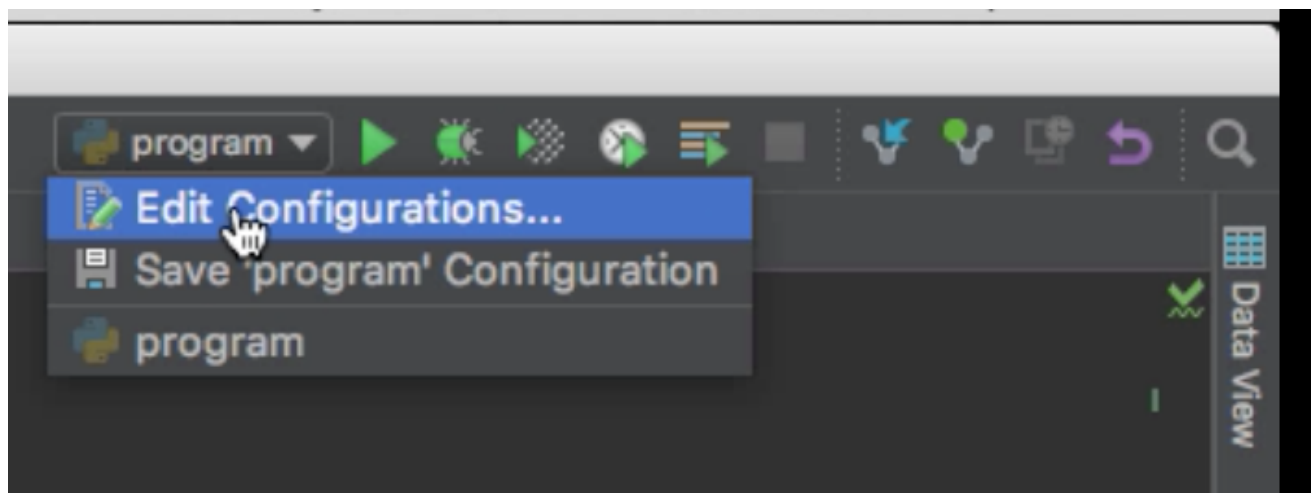


图39: 说明启动配置编辑器的图。

如果您想更改运行配置的环境，可以点击编辑器顶部的运行配置按钮。按钮上要么显示“添加配置”，要么显示最近执行的文件的名称，并带有一个下拉选项。点击下拉选项，然后点击“编辑配置...”。请注意，这只会更改运行配置的解释器，而不会更改项目的解释器。（如果您想更改项目的解释器，则需要在“偏好设置”->“项目解释器”页面中进行操作。）

打开“运行/调试配置”窗口后，您可以更改“Python 解释器”。对于基本脚本，使用系统 Python 可能就足够了。对于任何大型项目，我们强烈建议您创建一个虚拟环境并使用它来管理依赖项。

3. PyCharm项目

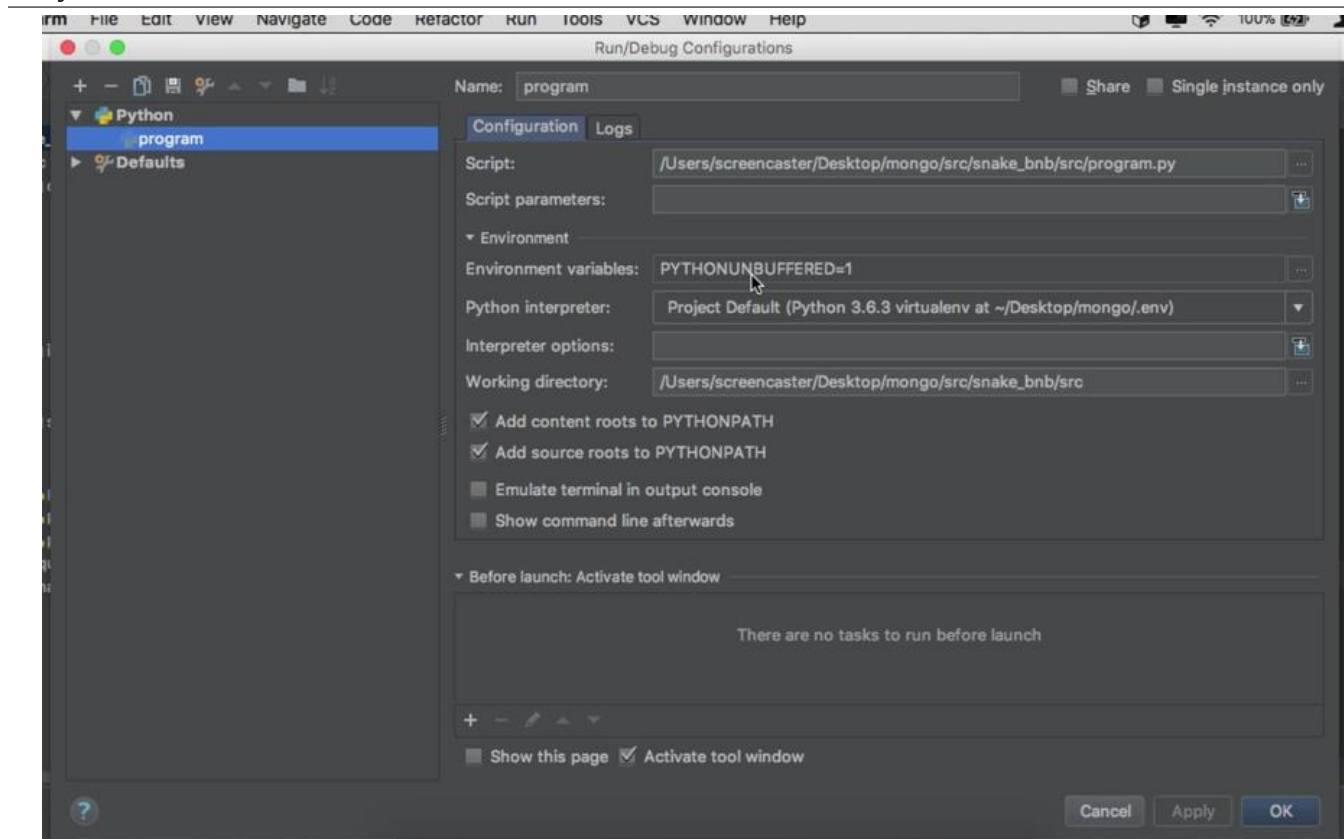


图310：启动配置编辑器的示意图。从这里我们可以选择用于配置的Python解释器。请注意，每个文件都可以有自己的配置。

如果在将检出的存储库作为项目添加到PyCharm中之前，该存储库内已有一个名为`.env`的虚拟环境，PyCharm会识别到它。如果之后需要添加一个环境，则需要创建该环境并将其作为“项目解释器”添加。然后，您可以从运行配置窗口使用该环境。

你可以像我们一直以来那样使用命令行创建环境。但是，你也可以在PyCharm中创建这些虚拟环境。要做到这一点，通过点击扳手图标打开“Settings”（Windows/Linux系统下按`Ctrl+Alt+S`），或“Preferences”（Mac系统下的屏幕（`command+”,”`））。在搜索区域中输入“Interpreter”。

通过此窗口，您可以查看项目解释器中的所有软件包。您还可以分别点击加号、减号、向上箭头或眼睛图标来添加、移除、更新或刷新软件包。这些图标位于窗口底部。如果您点击右侧的齿轮图标，会有一个“添加...”新环境的选项。从那里，您可以使用Python的`venv`命令、`pipenv`、`conda`（如果已安装Anaconda）等创建一个虚拟环境。

如果您熟悉`pip`并在命令行下工作，在主编辑器窗口底部有一个标有“Terminal”（`alt - F12`）的按钮，可启动一个终端。

在此窗口中，你可以运行所有标准的`pip`命令（例如，如果你创建了需求文件，可运行`pip install -r requirements.txt`）。无论你将哪个解释器/环境设置为项目默认值，都会在此窗口中自动激活。



3. PyCharm项目

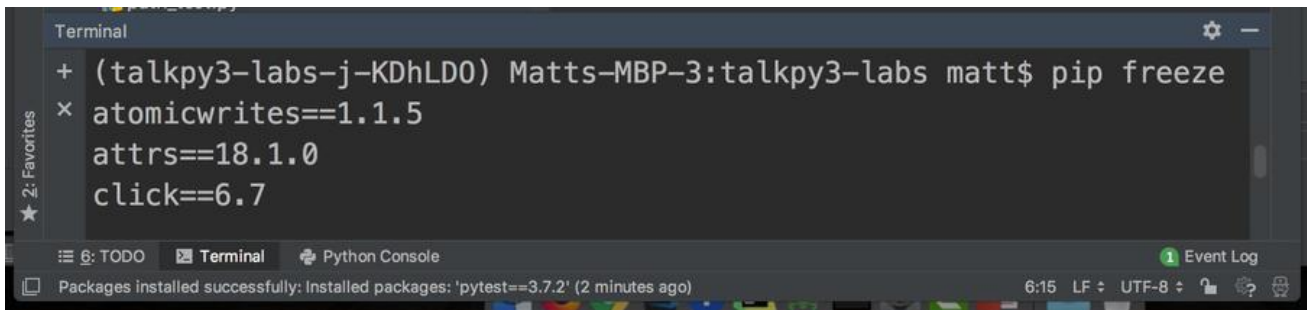


图313：展示使用终端访问pip和虚拟环境的示意图。注意窗口底部的“终端”按钮。

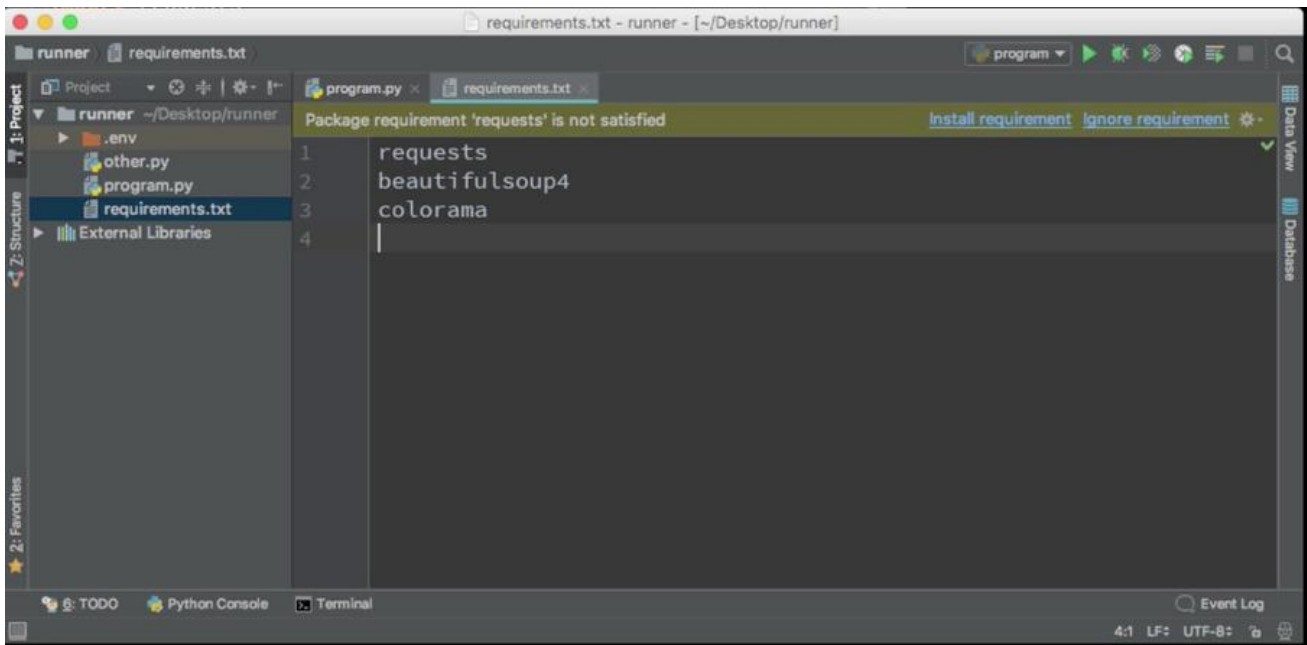


图314：说明打开requirements.txt文件且PyCharm提示安装缺失依赖项的示意图

3.5 需求管理

我们看到，PyCharm 可以管理为特定解释器安装的现有软件包。PyCharm 还将帮助我们安装在 requirements.txt 文件中找到的依赖项。这些 requirements.txt 文件是在 Python 应用程序中声明依赖项的常用方法。需求文件可以只包含软件包名称，也可以指定版本条件。一个示例的内容可能如下：

```
pyOpenSSL==0.13.1
pyparsing==2.0.1
python-dateutil==2.4.2
```

设想将requirements.txt文件的每一行依次输入到pip install VALUE中。

如果你在PyCharm中打开此文件，它将识别所有依赖项是否都已满足。它会提示你安装缺失的软件包。

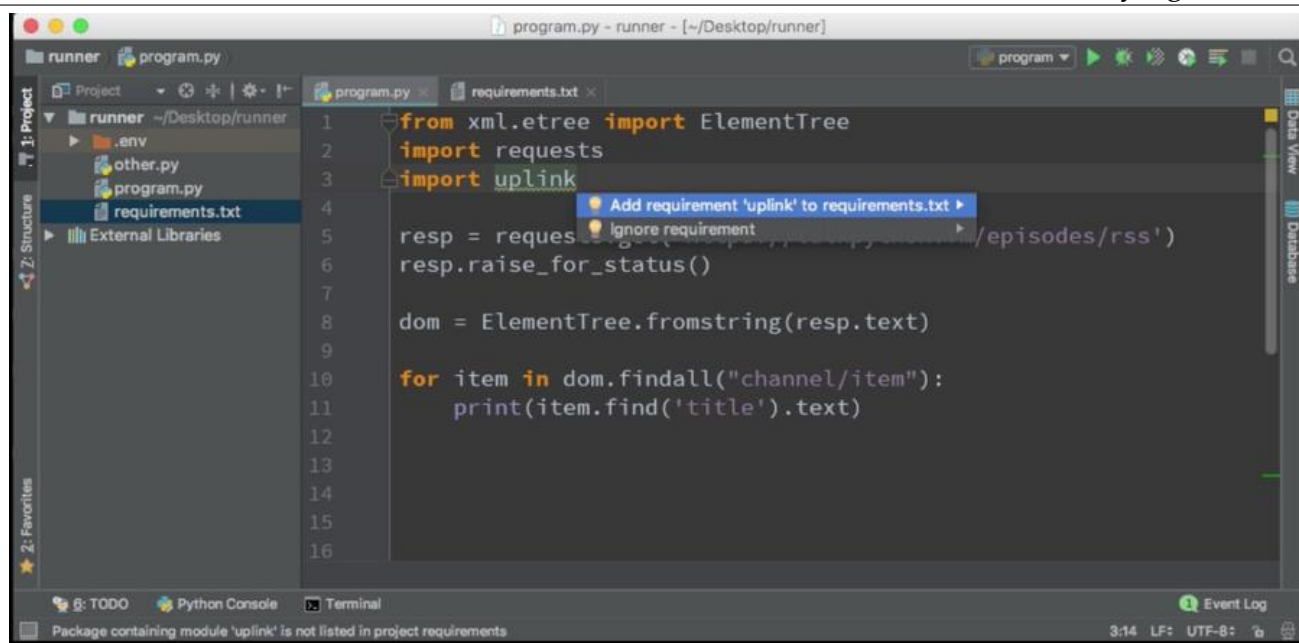


图315: 说明在向Python源文件添加新包的导入后更新requirements.txt文件的示意图

相反，如果您安装并导入了一个新包，而该包未安装在您的活动环境中，PyCharm 会提示您是否要更新requirements.txt，将新包声明为依赖项。

3.6 对目录进行分类

在大型应用程序或大型目录结构中，标记目录的概念确实很有帮助。这在开发Web应用程序时尤其有用。如果右键单击该目录，会有四个标记选项。

第一个是*sources root*。PyCharm 使用源根目录作为解析导入的起点。在之前的示例中，我们看到program.py引用了actors.py中的代码。PyCharm 声称 actors 模块不存在。通过将final目录标记为“源根”，PyCharm 将对在那里找到的代码建立索引。

我们还有*resource roots*。这些目录用于存放样式表、图像、JavaScript等静态内容。这对Web开发很有帮助，因为当你创建Pyramid Web应用程序时，有一整套可定制的文件夹结构。并非每个项目都有完全相同的结构。然而，你所要做的就是右键单击并将其标记为资源根目录，其余的事情PyCharm会处理。

Excluded roots 用于PyCharm忽略的文件夹。这些文件夹不会被索引，因此使用它们可以让PyCharm消耗更少的内存并提高运行速度。如果项目层级结构中有与代码无关的文档文件夹、虚拟环境文件夹或资源，可以使用此选项。包含频繁变化的构建输出的文件夹（如webpack）也会从中受益。

最后，还有*templates roots*。这里存放着像Jinja2这样的网页模板或Chameleon文件。由于PyCharm能识别这些文件，因此您可以在其中使用自动补全功能。

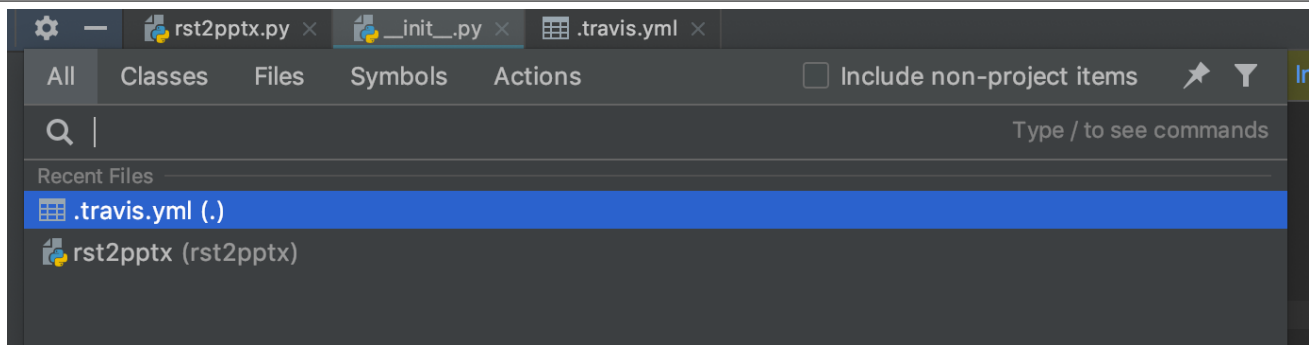


图316：显示未输入搜索词的“全局搜索”对话框的图。在这种情况下，它会显示最近的文件。

3.7 全局搜索

在加载项目、设置解释器、安装依赖项并对目录进行分类后，您就可以开始编码了。这可能看起来很繁琐，但通常只需设置一次项目。设置好项目后，您将充分享受PyCharm的强大功能带来的好处。在讨论在PyCharm中输入和编辑代码的机制之前，让我们先探索一个用于理解现有代码的功能：代码导航。

这里有个快捷方式（碰巧不在PyCharm快捷键速查表中），即连续按两次Shift键（或者你也可以点击右上角的搜索图标），这会调出“*Search Everywhere*” dialog。这个名称不太准确，因为它的功能不止搜索。当你调出“随处搜索”时，首先会看到“全部”“类”“文件”“符号”和“操作”这些标签。默认标签是“全部”，如果你在搜索框中输入内容，PyCharm会显示所有搜索结果。你可以通过点击某个标签来筛选结果。除了记住最近打开的文件，它还会记住光标在该文件中的最后位置，并且通过点击文件名或使用箭头键选中文件名（然后按回车键）就可以导航到该位置。此外，如果你在搜索框中什么都不输入

如果您在搜索框中输入文本，窗口将显示类、文件、符号和操作。您可以通过单击结果导航到代码的相应区域，或执行这些操作。

此外，“*Search Everywhere*”工具并不局限于仅处理Python代码。它可以在JavaScript文件、CSV文件或任何其他由PyCharm建立索引的文件中进行搜索。记住这个命令的快捷键（shift-shift），因为你会经常用到它。还有一个在“非项目项”中搜索的选项，如果勾选该选项，你还可以在系统库和第三方库中进行搜索。当你想要阅读正在使用的某个包的代码时，这会非常有用（我们鼓励这么做，因为Python注重可读性，通过阅读他人的代码你会学到很多）。

提示

“随处搜索”命令是你首先应该记住的命令之一。如果你忘记了其他键盘快捷键或操作，可以使用“随处搜索”来查找。

3.8 在项目内导航

除了全局搜索之外，PyCharm还有另一个很酷的功能。如果你按住command键（在Windows和Linux系统上是control键），你的源代码就会像网页一样可以导航。如果你点击某个内容，比如函数调用，编辑器就会“*Navigate*

3. PyCharm Projects

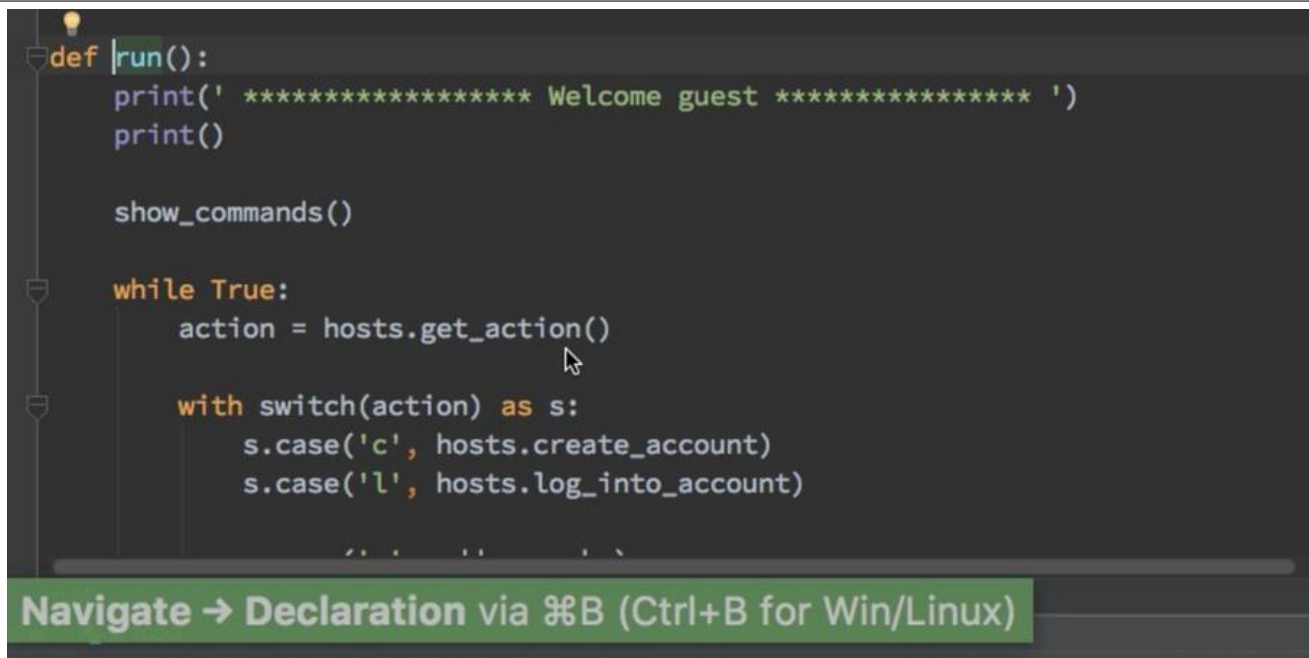


图319：说明导航到run声明结果的图。

3.9 总结

为了充分利用PyCharm，你需要为新项目配置它。这将启用自动补全和代码导航等强大工具。我们了解到，如果项目的顶级文件夹中存在现有的虚拟环境，PyCharm会识别它们。或者，你可以从终端或在PyCharm内部创建虚拟环境。如果你不想在命令行中使用pip，PyCharm还将管理第三方依赖项的安装。

在某些情况下，我们需要向PyCharm提供一些关于如何处理目录的提示。配置好项目后，我们就可以利用强大的工具了。在本章中，我们还了解了一些搜索和导航功能，但PyCharm还有更多功能有待探索。我们将在后续章节中看到这些内容。

3.10 命令

- "Run File" - (按Ctrl+Shift+F10)
- "Search Everywhere" - (双移)
- "Navigation to declaration" - (按住 Command 键或 Control 键并点击)
- "Find Usages" - (Command+F7 或 Ctrl+F7)
- "Mark Directory as" - (右键点击文件夹)
- "Settings/Preferences" - (命令键 - 或 Ctrl + Alt + S)
- "Terminal" - (按 Alt+F12)

3.11 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/1-projects` 中关于新项目的说明创建一个新项目，运行它，并验证其中的环境。
2. 为上一个练习创建一个新的虚拟环境，并设置一个运行配置来使用它。
3. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/1-projects` 中复杂项目的说明操作。你将导入现有项目、标记目录、验证PyCharm是否正在对代码建立索引，然后运行代码。
4. 使用上一练习中的项目，试用“导航到声明”和“查找所有引用”命令。

第4章

编辑器

4.1 PyCharm编辑器

在本章中，我们将更深入地了解编辑器。它是创建Python代码的强大工具。此外，它还可以处理其他文件类型，如HTML、JavaScript和CSS。在本书中，我们将主要关注Python，并介绍一些出色的编辑器功能，包括一些可能并非一目了然的功能。

4.2 简单项目

我们将通过编写一个简单的项目来检索有关Talk Python播客的信息。我们将从创建一个名为podcast的项目开始，该项目包含一个虚拟环境，并添加两个文件。program.py将包含我们的主要逻辑，而service.py将包含一些辅助功能。

program.py文件应包含：

```
def main():  
    print("Welcome to the talk python info downloader")  
    print()
```

service.py文件应该有一个简单的存根。我们可以稍后再补充细节，但关键是这个逻辑将检索有关播客的数据。将以下内容放入其中：

```
def download_info():  
    pass  
  
def get_episode(show_id):  
    pass
```

现在让我们看看PyCharm是如何帮你解决问题的。我们将尝试从program.py内部调用download_info方法。如果你只是将download_info()放入main函数的主体中，PyCharm会给这个调用加上下划线。这条红色下划线表明PyCharm发现了你的代码存在问题，并且可能会给出一些建议。

如果将鼠标悬停在带下划线的代码上，左侧会出现一个灯泡图标。这表明PyCharm有一个Code Intention (自动修复)功能可供您激活。红色灯泡图标表示它可以修复错误。如果灯泡图标为黄色或橙色，则表示对已能正常运行的代码提出的建议或改进。

当你执行“Show Intention Actions”操作（按Alt+Enter或点击灯泡图标）时，会出现一个下拉选项列表。其中包含诸如“导入”之类的建议。

4. 编辑器

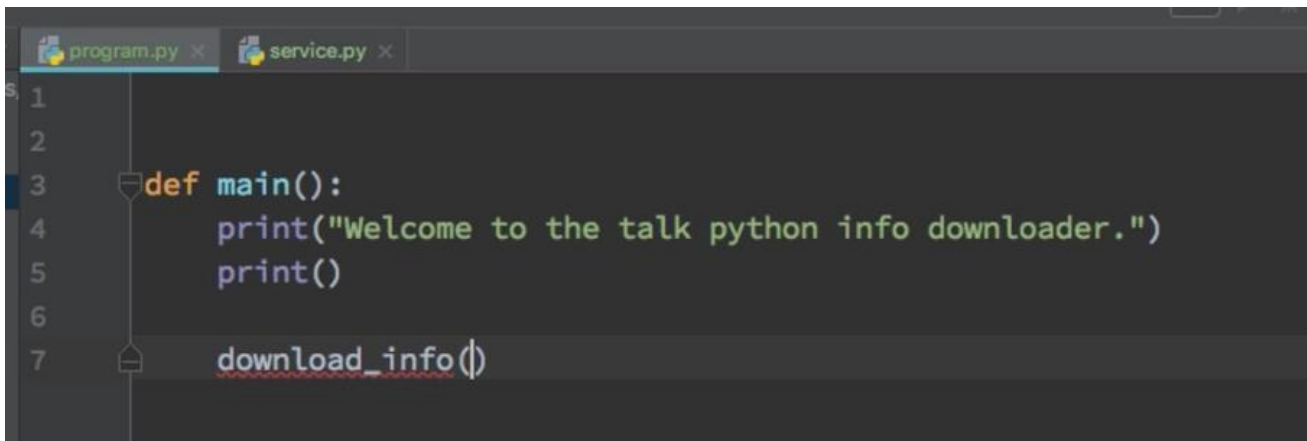


图41：该图说明，当我们插入模块命名空间中不存在的代码时，它会给代码加上下划线，表明存在问题及可能的解决方案。

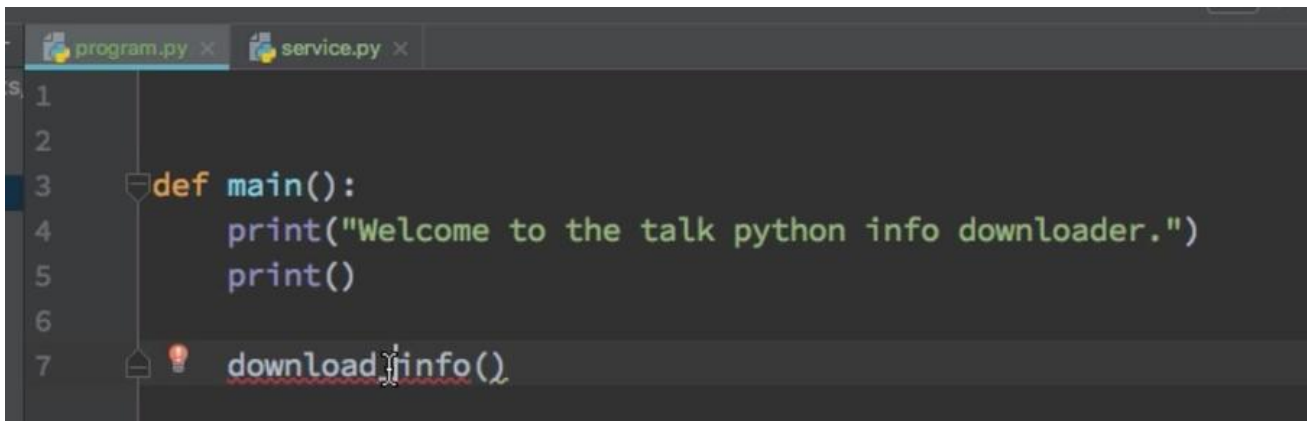


图42：该图展示了将光标移至带下划线的代码附近时出现的灯泡。

`service.download_info`创建函数`download_info`、重命名引用，或者完全忽略此类错误（此处需谨慎，这是编辑器中的全局更改）。

第一个选项，导入`service.download_info`，正是我们想要的。如果你选择该选项，它将在代码顶部插入合适的导入语句：

```
from service import download_info
```

记住，我们现在可以使用“导航到声明”在文件之间导航，或者按两次 Shift 键跳转到上一个文件。你会看到 PyCharm 有语法高亮显示。关键字、内置函数、字符串和用户定义的代码都有各自的颜色。此外，如果你导航回到 `service.py`，你会看到 `get_episode` 函数中的参数 `show_id` 被加了下划线，因为它未被使用。

未使用的代码是一种代码坏味道，可能表明你的逻辑存在错误。解决这个错误是 PyCharm 免费提供给你的一项功能。其他一些编辑器也有此功能，但更常见的情况是，Python 程序员会使用像 `pyflakes` 这样的工具来检查这类问题。在编辑器中直接拥有此功能，你可以节省时间，无需担心因离开编辑器并切换到终端或其他工具而打断工作流程。

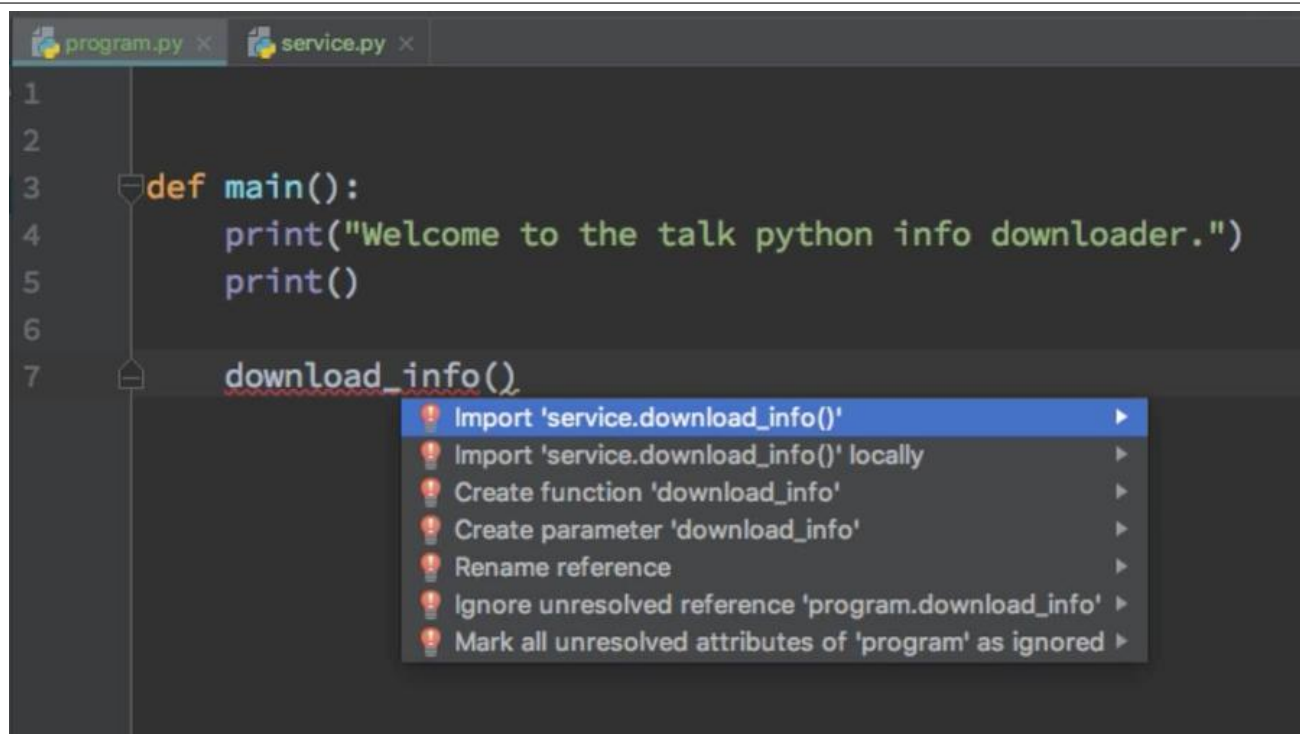


图43：说明当光标位于带下划线的代码上方时的意图操作（按alt-enter）的图

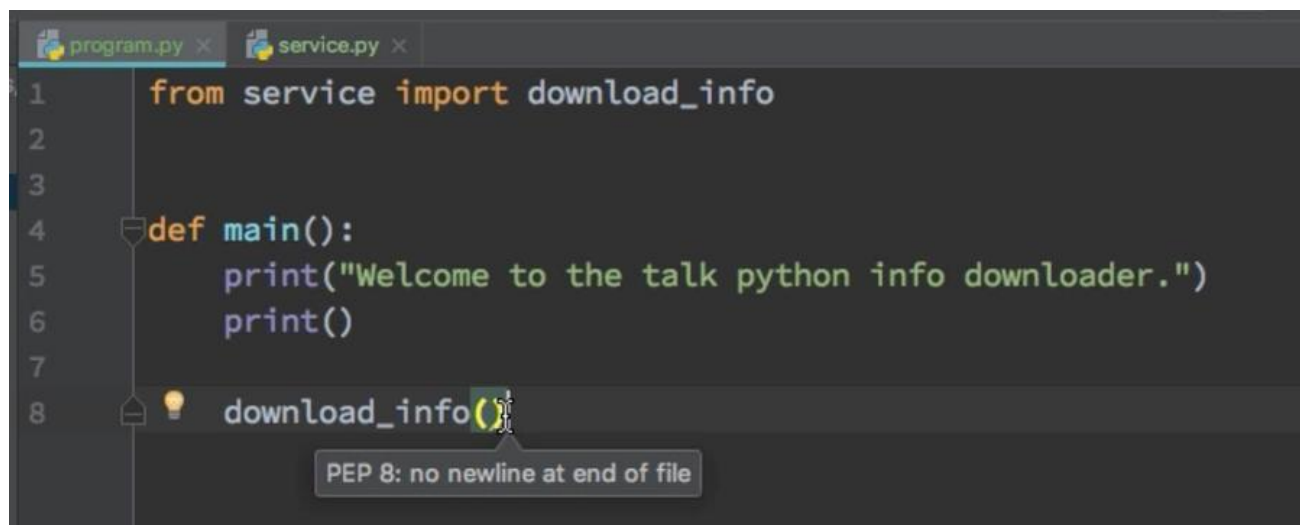


图44：说明违反PEP 8的图。使用重新格式化代码来清理这些问题。

PyCharm 也会对违反 PEP 8 的情况发出警告。PyCharm 的优点在于，它不仅会指出问题，还会为你提供问题的解决方案（或多种解决方案）。通过运行 “Reformat Code” 命令（alt-command-L 或 ctrl-alt-L），PyCharm 会帮你修正这些问题。

另一种选择是将导入语句从：

```
from service import download_info
```

至：

4. The Editor

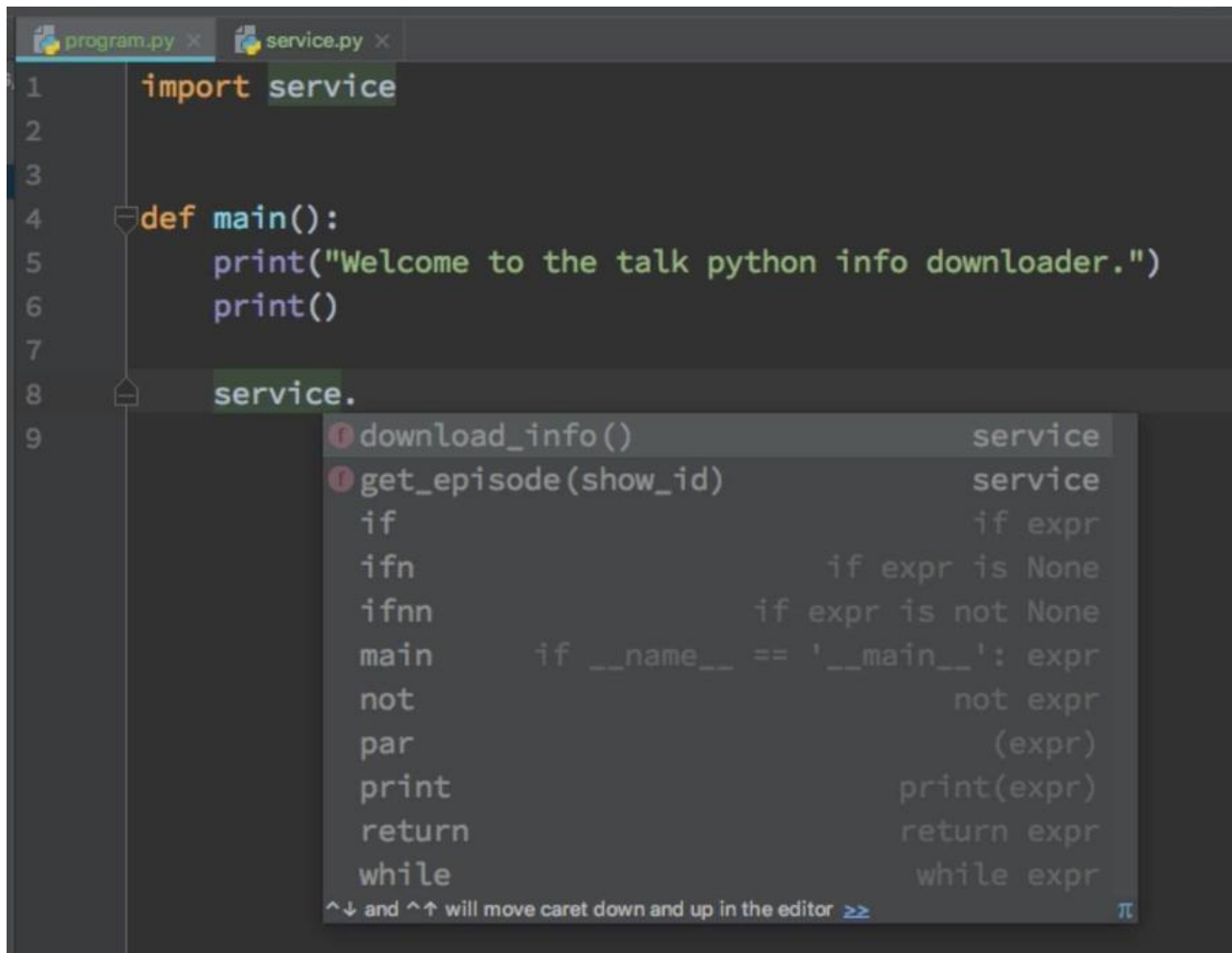


图45：说明模块上智能感知完成情况的图。

```
import service
```

现在，如果你进入main函数并输入service.（注意末尾的句号），PyCharm将显示来自service命名空间的选项。在这些选项中有download_info。

在main函数内部，我们添加一个循环，以获取100到129之间的节目ID的信息。
U将main函数更新为以下代码：

```
def main()
    print("Welcome to the talk python info downloader")
    print()

    service.download_info()

    for show_id in range(100, 130):
        info = service.get_episode(show_id)
```

现在，我们将填写download_info的实现。你可以通过按住Command或Control键并点击download_info导航到声明处。这将打开service.py。我们希望此函数使用requests¹²库从RSS源获取信息。将download_info的实现更改为：

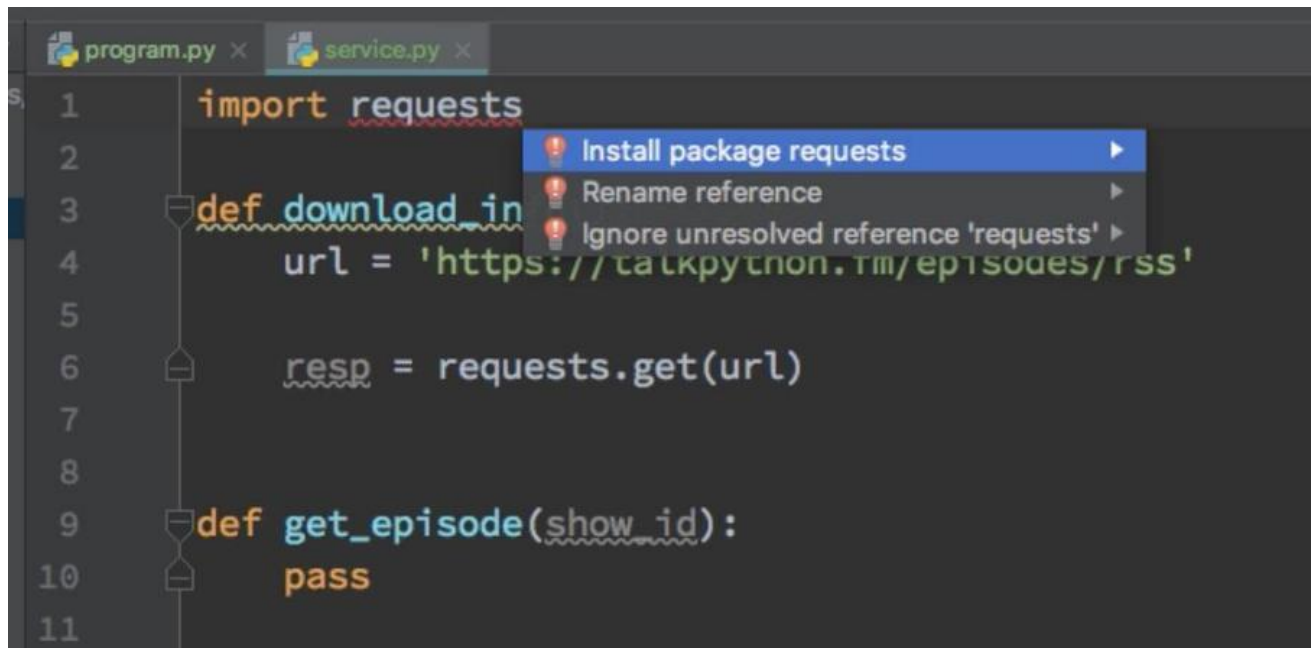


图46：说明从意图操作安装requests库的示意图

```
def download_info():
    url = 'https://talkpython.fm/episodes/rss'

    resp = requests.get(url)
```

PyCharm 会给 `requests` 加上下划线，因为它不在该文件的命名空间中。在这种情况下，意图操作选项不会执行正确的操作（即导入 `requests` 库）。相反，应跳到文件顶部并添加以下行：

```
import requests
```

在这一行的意图操作中有一个“安装requests包”的选项。选择该选项将为我们安装依赖项。如果有一个`requirements.txt`文件，我们也可以选择更新它。

这有多酷？我们从尝试使用一个未安装、未导入的外部包，到将其安装、导入并（有可能）添加到我们的需求中。而这一切，只需给PyCharm一点小小的提示。

现在我们将遍历RSS源的结果，并使用元素树库提取项目。像这样更新`download_info`函数以打印出项目的长度：

```
def download_info():
    url = 'https://talkpython.fm/episodes/rss'

    resp = requests.get(url)
    resp.raise_for_status()

    dom = ElementTree.fromstring(resp.text)

    items = dom.findall('channel/item')
    print(len(items))
```

¹²<https://python-requests.org/>

4. The Editor

你应该能够使用意图操作来为 `ElementTree` 创建正确的导入，因为它在标准库中。如果你跳回到 `program.py` 文件并尝试运行它，不会有任何反应，因为我们没有调用主函数。我们可以输入下面这种常见的代码模式，它能让一个 Python 模块既可以作为可导入模块，又可以作为应用程序运行：

```
if __name__ == '__main__':
    main()
```

但是，让我们让 PyCharm 帮我们完成这件事。如果你使用 Python 有一段时间了，这段代码通常用于表明我们正在执行一个文件（而不是将其作为库导入使用）。PyCharm 有一个名为“实时模板”的功能。如果你输入 `main`，PyCharm 会为你提供一个实时模板，其中将该条件作为首要意向操作。在选中该模板时按下 TAB 键，PyCharm 就会为你插入那段代码。

提示

当你创建一个主函数时，PyCharm 会在其旁边添加一个绿色的 `gutter` 图标。这使得运行你的代码变得非常容易。

实时模板对于节省时间、避免错误，甚至有助于预防腕管综合征非常有用。你可以利用内置的实时模板，稍后我们将展示如何创建自己的模板。

如果你现在运行代码，它应该会告诉你有多少集。让我们继续更新代码以完善它，并进一步探索 PyCharm。在 `service.py` 文件中添加对 `namedtuple` 的导入，定义一个 `namedtuple` 来存储剧集信息，并更新 `download_info` 逻辑，代码如下：

```
from xml.etree import ElementTree
from collections import namedtuple
import requests

Episode = namedtuple('Episode', 'title link pubdate show_id')
episode_data = {}

def download_info():
    url = 'https://talkpython.fm/episodes/rss'

    resp = requests.get(url)
    resp.raise_for_status()

    dom = ElementTree.fromstring(resp.text)

    items = dom.findall('channel/item')
    episode_count = len(items)

    for item in items:
        episode = Episode()

def get_episode(show_id):
    pass
```

将光标置于 `for` 循环中对 `Episode` 的调用内部。如果按下 *Parameter Info* (`command-p` 或 `ctrl-p`)，PyCharm 会识别具名元组，并显示调用它所需的参数。

在调用类初始化器、方法或函数时，拥有参数信息非常有用。

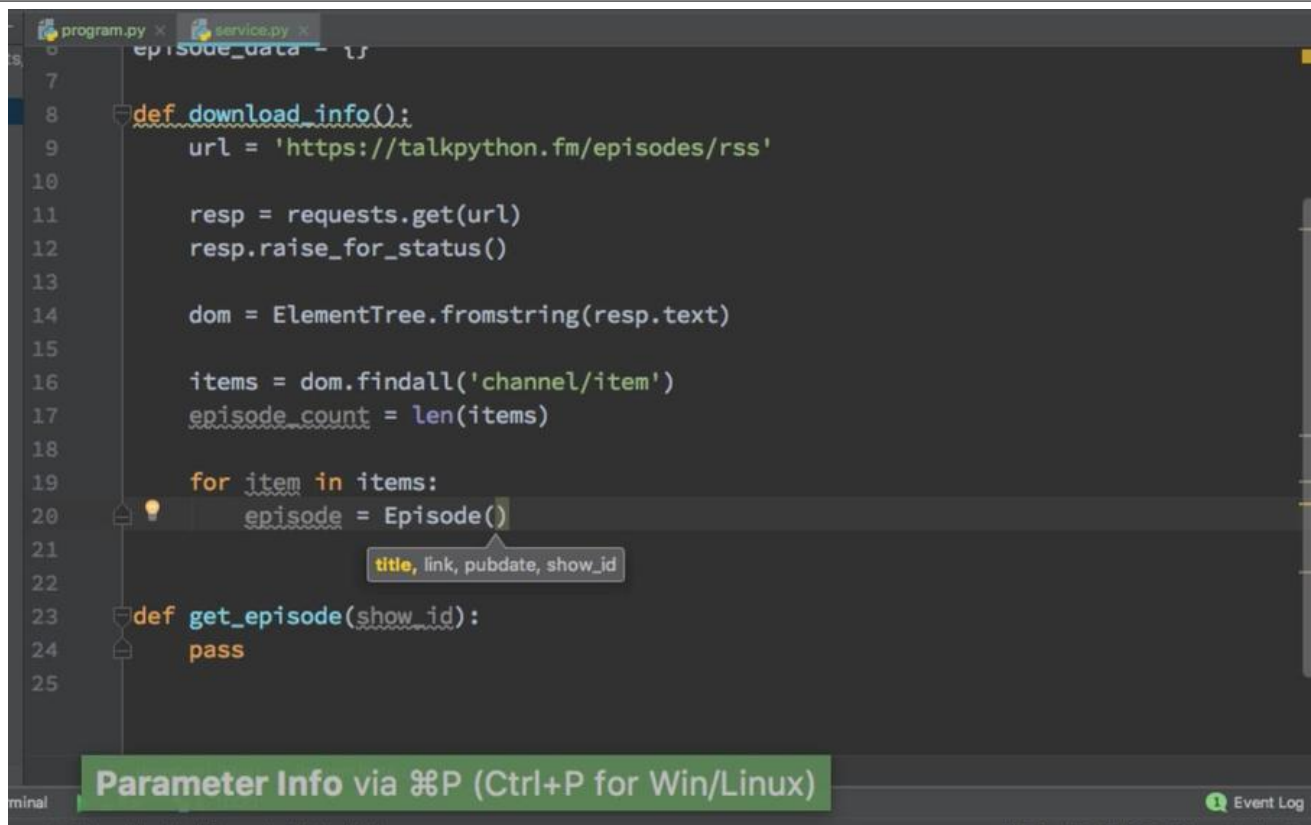


图47: 说明光标位于具名元组内部时的参数信息的图。

在我们这个小型下载器应用程序中，下一步是根据……填充这些Episode具名元组。RSS源。我们需要将for循环改成类似这样的形式：

e

```

for idx, item in enumerate(items):
    episode = Episode(
        item.find('title').text,
        item.find('link').text,
        item.find('pubDate').text,
        episode_count - idx - 1
    )
    episode_data[episode.show_id] = episode

```

在get_episode函数中添加以下实现：

```

def get_episode(show_id):
    return episode_data.get(show_id)
def main():
    print("Welcome to the talk python info downloader")
    print(f"{info.show_id}, {info.title}")
    service.download_info()

for show_id in range(100, 130):
    info = service.get_episode(show_id)

```

将program.py main函数更新为：

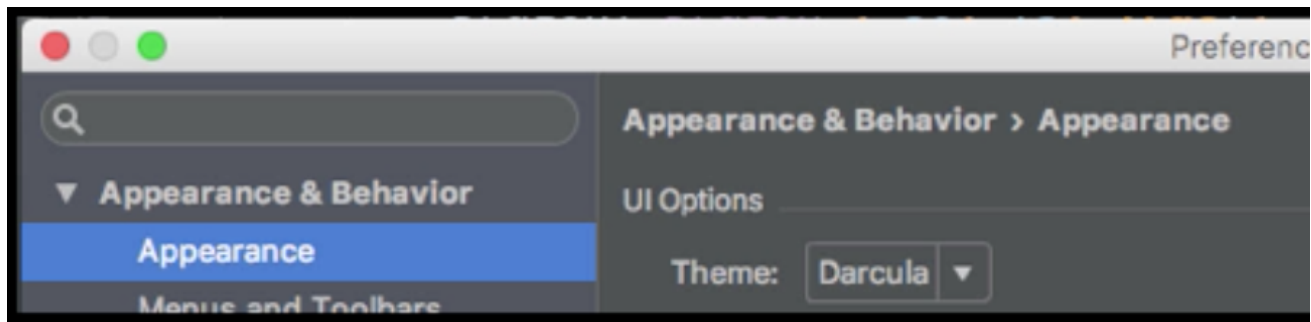


图48：说明主题选择的图。

请注意，PyCharm 支持 Python 3.6 及更高版本中的 f 字符串。它还支持在 f 字符串内进行代码补全。不过，有时 PyCharm 不够智能，无法为我们提供代码补全。在上述代码中，即使我们知道 `info` 对象是一个名为 `Episode` 的具名元组，它也不会为该对象提供代码补全。为了帮助 PyCharm，我们可以利用 Python 3 的另一个特性——类型注解。将 `service.py` `get_episode` 函数更新为：

```
def get_episode(show_id: int) -> Episode:
    return episode_data.get(show_id)
```

有了上述代码，PyCharm 会为我们提供对调用 `get_episode` 的结果的自动补全功能！

注意

由于 `.get` 方法可以返回 `None`，`get_episode` 的返回结果实际上应该是 `typing.Optional[Episode]`。

在进行这个项目时，有几件事需要注意。你实际上不需要保存文件。每当你运行文件时，PyCharm 会为你处理保存事宜。此外，如果运行过程中出现错误，终端窗口中的堆栈跟踪信息会包含超链接，你可以点击这些链接跳转到错误源。

这篇概述应该让你对 PyCharm 的强大功能有了初步了解。虽然这些功能已经很出色，但这仅仅是它所能实现功能的冰山一角。

4.3 语法高亮

如我们所见，编辑器具有语法高亮功能。PyCharm 会以不同颜色显示（并处理）关键字、内置函数、变量、字符串和用户定义的代码。这是可配置的。在本书的图中，截图使用的是“Darcula”主题。如果你在 macOS 上按 *Preferences* (command+,) 或按 *Settings* (command-,)，在 Windows 和 Linux 上按 `ctrl-alt-S`，然后搜索“Appearance”，就可以更改主题。在“UI Options”下有一个“Theme”选择框。

如果你想安装自己的主题，可以前往“配色方案”偏好设置页面。在该页面上，方案名称旁边有一个齿轮图标，你可以通过它导入新主题。“颜色主题¹³”网站有大量主题可供下载。

另一种选择是自定义现有主题。在“配色方案”偏好设置页面中，您可以为许多不同的文件类型自定义字体和颜色。您还可以导出该方案以与他人（或其他计算机）共享。

¹³<http://color-themes.com>

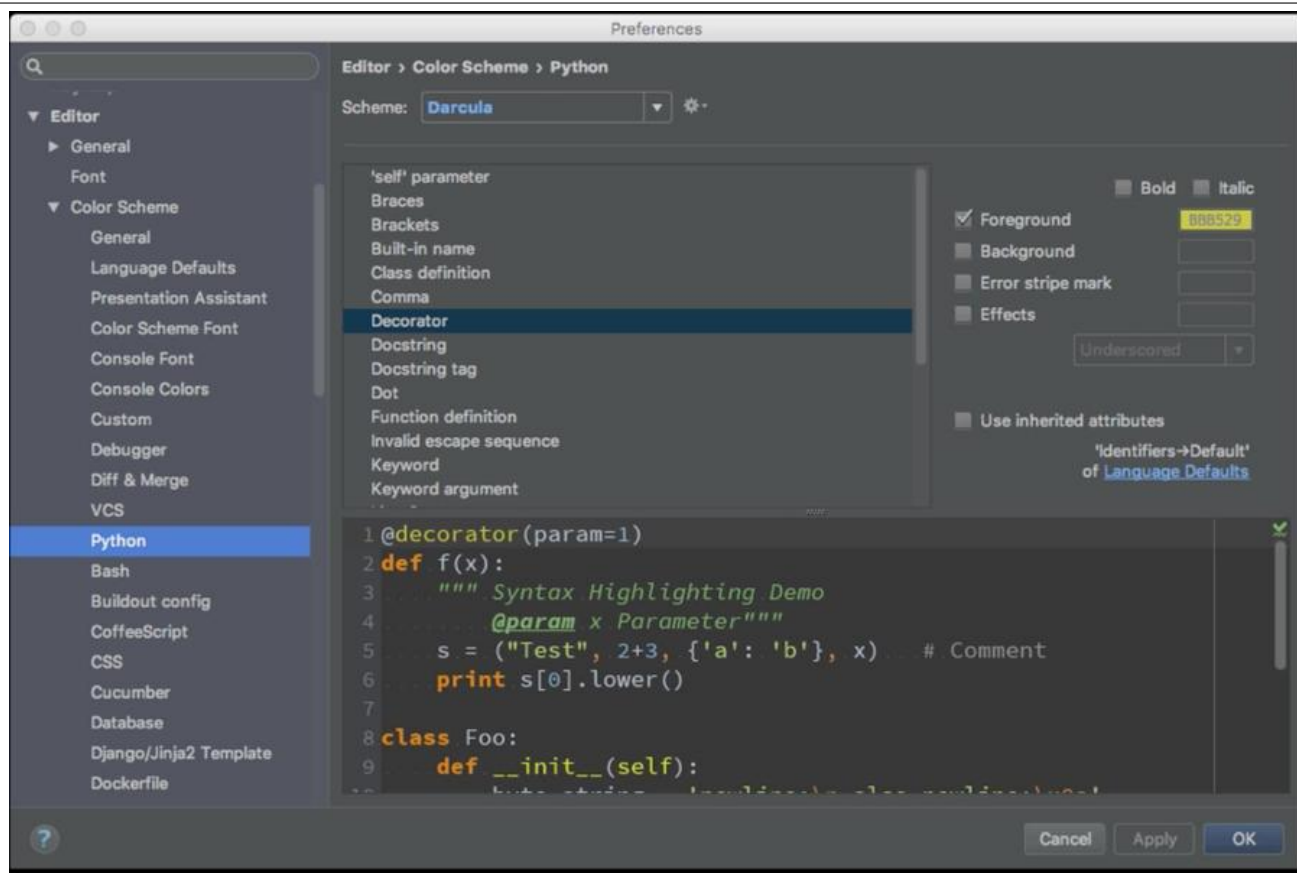


图49: 重新说明如何自定义装饰器选项，用于 Python 代码。

还有一个“外观”偏好设置页面，用于日后调整编辑器。你可以调整辅助功能、用户界面选项、窗口选项等。

4.4 代码补全

PyCharm 对它能理解的代码有 *code completion*。正如我们在示例中看到的，有时我们需要提供注释，以帮助 PyCharm 给出建议。让我们探索代码补全功能

s
名词

如果你有一个名为 `name` 的变量，并且在代码中稍后输入 `na`，PyCharm 会将 `name` 作为 *basic completion* 进行提示。如果你按下 TAB 键，它会为你补全缺失的字符。或者，如果由于某种原因代码补全被关闭或未激活，你可以按 CTRL + 空格键调出代码补全。

如果再次按下 CTRL + 空格键，PyCharm 将为您提供更多选项。它将显示具有该名称的类、函数和模块。

PyCharm 还会针对对象属性（句点后的任何内容）提供补全建议。通常，在您输入 `.` 后，选项会自动弹出，但如果您想返回并执行新的查找，请再次按 CTRL + Space。左侧图标中的字母表示补全的类型：类 (c)、字段 (f)、函数 (F)、方法 (m)、参数 (P)、属性 (p) 和变量 (v)。在对象实例上，补全内容的右侧指示找到该属性的类。

有时 PyCharm 无法提供智能补全。这是由于 Python 的动态特性。如果 PyCharm 中的补全没有意义，很可能是因为 PyCharm



图410：说明变量名基本完成情况的图。

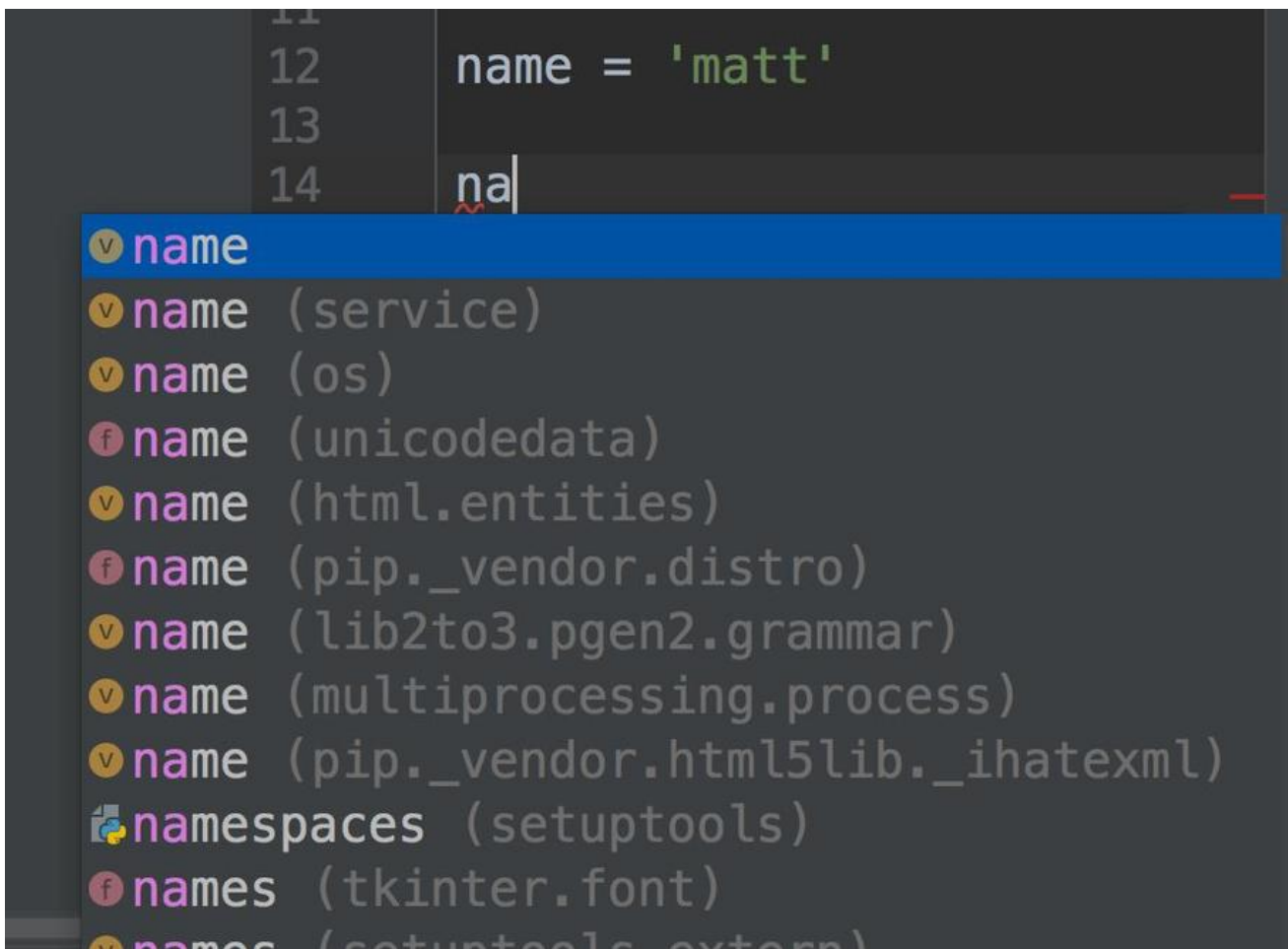


图411：该图展示了再次按下控制空格键以获取更多代码完成选项。

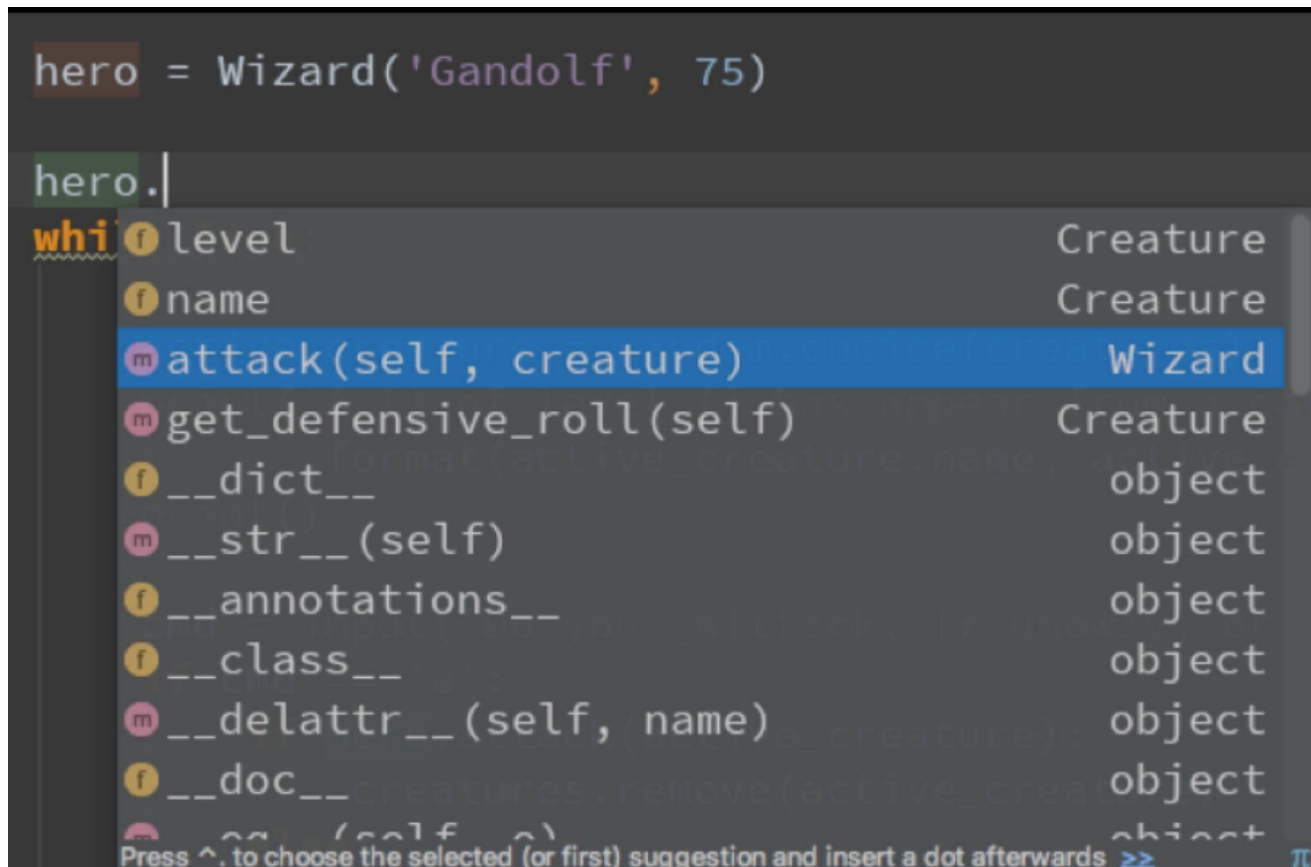


图412：左侧为说明完成图标含义的图。“f”代表字段，“m”代表方法。右侧是定义该属性的类。Wizard是Creature的子类，而Creature是object的子类。

无法推断变量的类型。你有几种选择，可以不使用代码补全功能，给变量添加类型注释，或者给函数的返回类型添加类型注释。“Show Intention”命令（按 alt + 并回车）将包含“添加类型提示”选项。

4.5 意向行动

当PyCharm显示灯泡图标时，表明有办法更改您的代码。如果灯泡为红色且带有感叹号，则表示存在错误，并且PyCharm有相关的修复方法。通过单击灯泡（或按 alt + 回车键），会弹出“Show Intention”下拉菜单。常见意图包括：根据用法创建代码、快速修复（为模块添加缺失的导入）、微重构、删除无效代码、PEP 8清理、解决性能问题、修复Python 2或3不匹配问题、包维护等。

不要忽视这些选项。花些时间阅读它们，甚至可以尝试一下，看看它们有什么作用。你随时可以撤销某个操作。我们将在本书中讨论其中的许多选项。如果你发现某些警告是误报或者对你造成了困扰，你总是可以抑制它们。从意图中点击“设置...”下拉菜单，然后选择抑制选项。

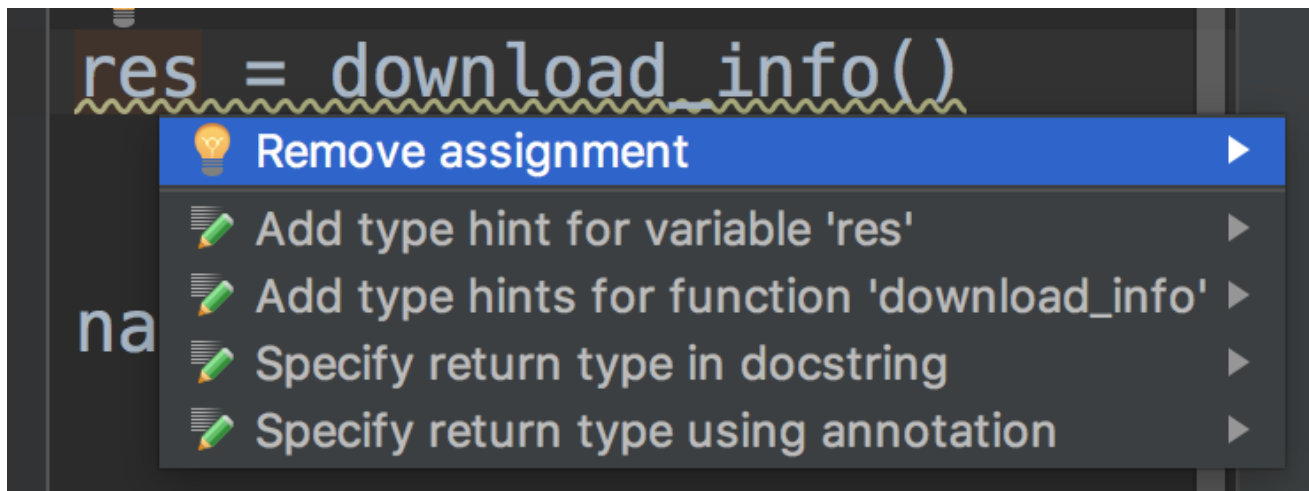


图413：展示“显示意图（备用+返回）”对res变量的结果的示意图。我们可以给变量或download_info函数添加类型提示。这将启用智能代码补全。

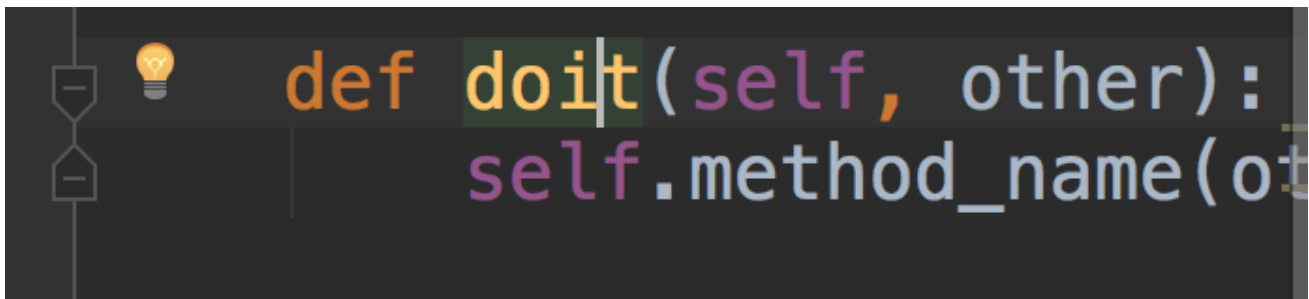


图414：阐释灯泡图标的示意图。您可以点击它，或者输入alt + 回车以调出意图选项。

4.6 发现新特征

PyCharm 包含广泛的文档和帮助主题。如果您忘记如何执行某个命令，可以在帮助菜单中搜索。例如，如果因为不符合 PEP 8 规范而看到大量下划线，并且想要重新格式化代码但忘记了操作方法，可以使用帮助菜单。在其中有一个“Find Action...” (Command-Shift-A (或 Ctrl-Shift-A))。如果在其中输入“reformat”，它将显示一个建议列表。接近顶部的是带有快捷键 (Command-Option-L 或 Ctrl-Alt-L) 的“Reformat Code”命令。

注意

Mac 用户还有另一种选择。“帮助”菜单中有一个搜索框。如果你在其中输入“重新格式化”，它会弹出一个菜单项列表，如果你选择某个菜单项，它会打开菜单并指向该项所在的位置。如果你更喜欢使用鼠标，这种方法还不错，但记住常用命令的快捷键会更好。

“查找操作”是一个强大的命令，也是你首先应该记住的命令之一。例如，如果你想将一些代码重构为一个函数，你可以突出显示那段代码，打开“查找操作”窗口，然后输入“重构”。你会看到你可以打开“Refactor”菜单本身，或者直接调用“Refactor Method” (Command+Option+M (在Mac上) 或 Ctrl+Alt+M) (在Windows/Linux上)。

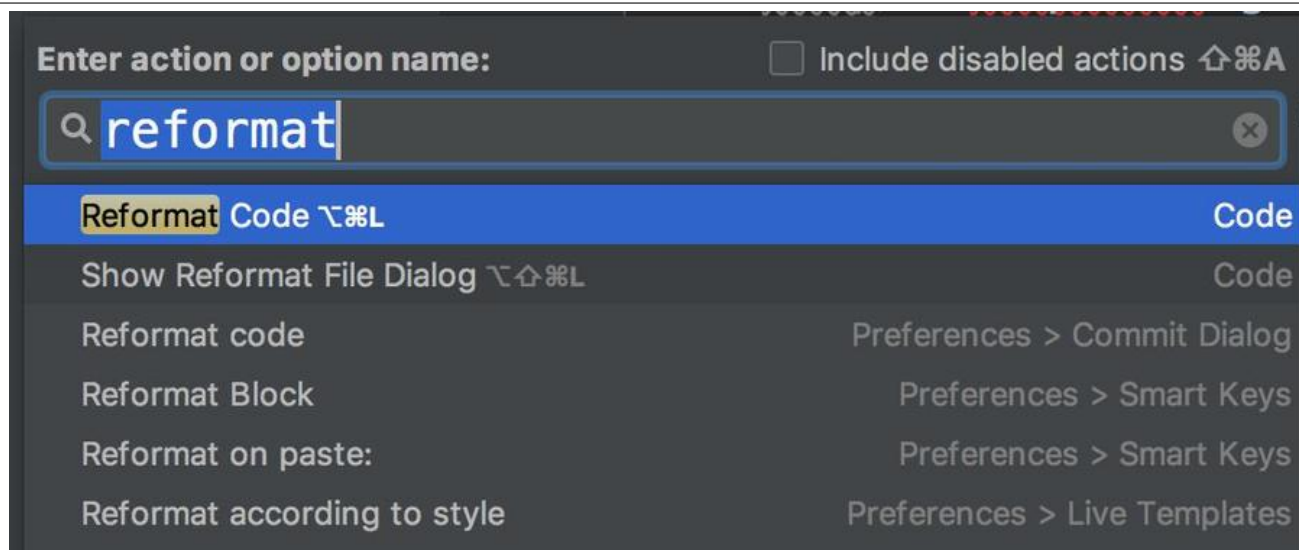


图415: 说明使用“查找操作”弹出窗口搜索“重新格式化代码”命令的示意图。

注意

PyCharm 使用“方法”一词来表示重构函数（全局级别的代码或函数中的代码）或方法（绑定到类中的函数）。

4.7 键映射

PyCharm 中的所有操作都是可配置的。默认情况下，这些操作已为您预先配置，并使用前面引用的备忘单中的映射。如果您想自定义任何键码，可以在“偏好设置”窗口中进行。搜索“键盘映射”以打开用于调整它们的界面。从那里，您可以自定义单个命令，也可以让 PyCharm 使用不同的键盘映射。例如，如果您习惯使用 Emacs，“键盘映射”下拉菜单中有一个 Emacs 选项，它对 Emacs 的模拟效果相当不错。

另外，如果你是 Vim 用户，有一个名为 IdeaVim 的插件，它支持许多 Vim 特性。你需要进入偏好设置页面的“插件”界面，搜索“vim”。在那里你可以点击“安装”按钮。

注意

如果你更改了键映射，此命令中的许多书籍将有不同的命令。例如，Emacs 使用 Ctrl + 空格来标记选区，因此它不适用于代码补全。

4.8 格式化与代码清理

你可能有存在各种 PEP 8 问题的代码。其中一些问题很容易修复。而另一些问题，比如使用 *camel-case* 命名约定（单词间用大写字母分隔）而不是 *snake-case*（单词间用下划线分隔），并不总是容易更改。如果你正在使用第三方库（甚至是标准库中的某个模块，比如 `unittest`），你不能随意重命名违规的函数或方法。

考虑以下代码：

4. The Editor

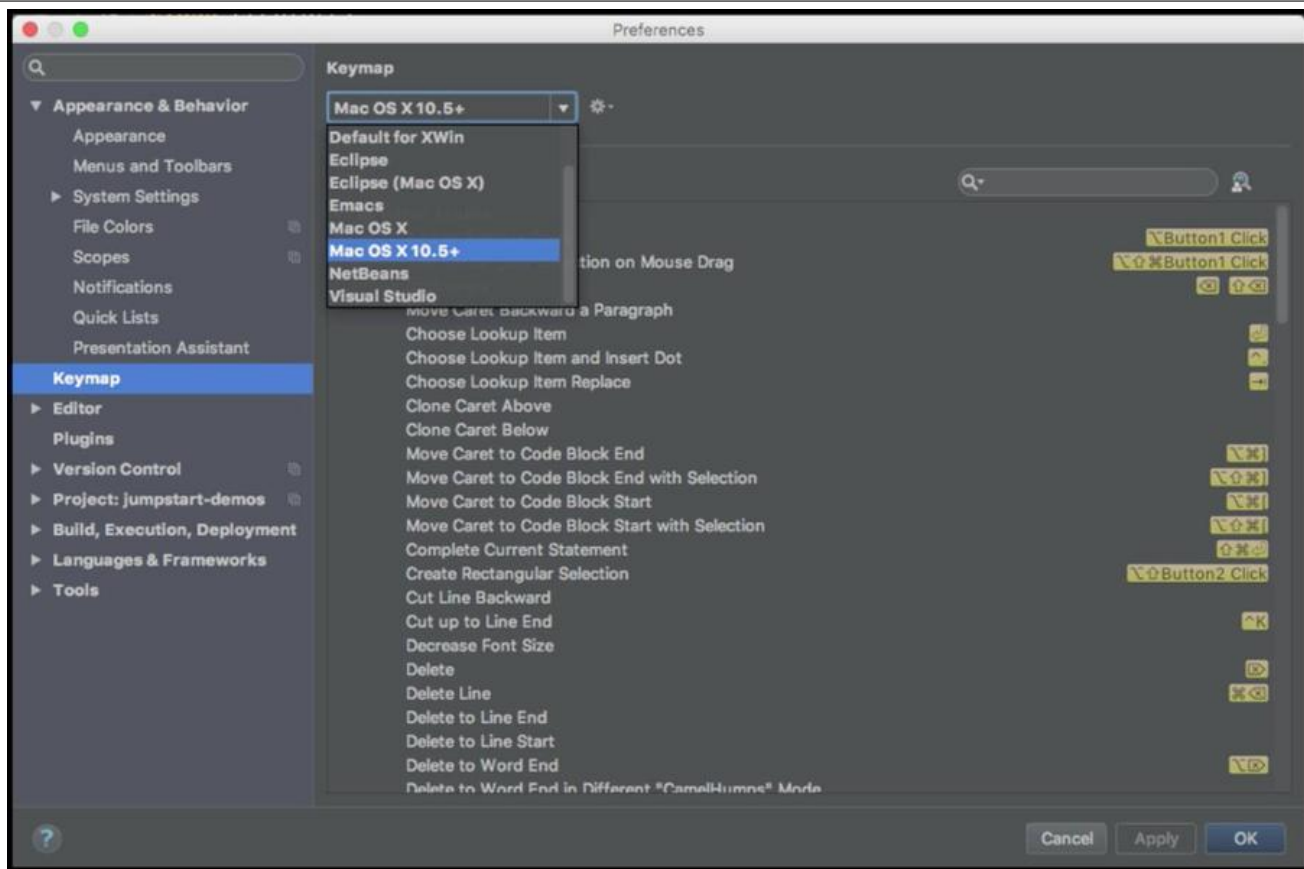


图416：说明更改热键或全局键盘映射的图。

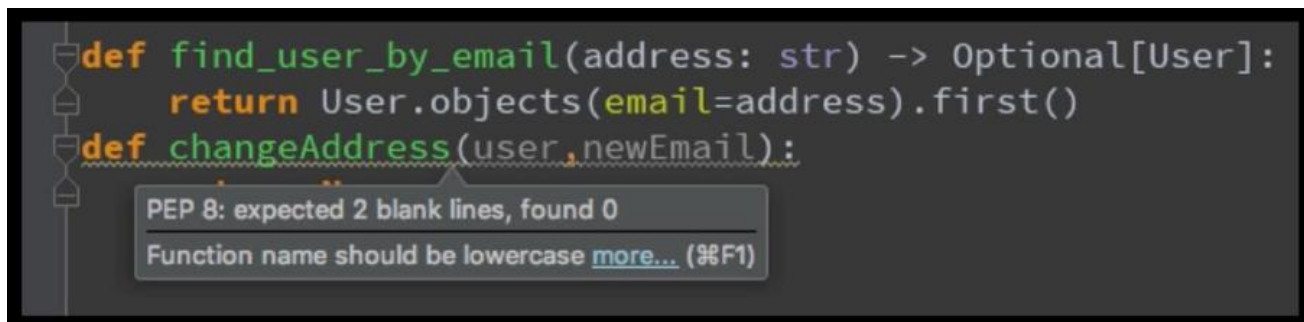


图417：说明代码中发现问题的图。没有空行，且函数名不是小写。

```
def find_user(username):
    return User.objects(name=username).first()
def changeAddress(user, newEmail):
    return None
```

它违反了PEP 8规范，因为函数之间没有空格。此外，changeAddress和newEmail采用了驼峰命名法，并且newEmail之前没有空格。

有一个命令“Reformat Code” (cmd + L，或者按ctrl + L)，这将解决其中一些问题。执行该命令后，代码将如下所示：

```
def find_user(username):  
    return User.objects(name=username).first()  
  
def changeAddress(user, newEmail):  
    return None
```

注意

如果你想格式化整个项目，可以运行“运行->”*Code Cleanup...*”操作。（此命令没有快捷键，但按下shift + shift并输入命令的首字母将调出该操作。如果你更喜欢打字而不是使用鼠标，那么无需伸手去拿鼠标就可以运行此操作。）

请注意，changeAddress 的名称并未转换为符合PEP 8规范的名称。在这种情况下，我们可以在函数声明上使用“*Rename*”命令（shift - F6）。请注意，不能在该函数的调用上进行此重构，必须在函数本身进行。如果你遇到不符合PEP 8规范命名的库，除非你是第三方库的作者，否则你只能接受现状。

4.9 团队代码格式化

如果你在一个团队中工作，会有多个开发人员将代码检入到源代码控制系统。如果你的团队没有采用风格指南，管理这些变更可能会令人头疼。一个不错的目标是消除由缩进偏好、换行符、逗号位置和命名引起的麻烦。通过采用风格指南，可以消除许多此类问题。

市面上有很多风格指南。PEP 8是一种风格指南，但它相当灵活。我们建议采用更严格一些的风格指南。谷歌有针对Python代码的风格指南¹⁴，OpenStack有另一种¹⁵。还有第三方命令行工具，如yapf¹⁶和black¹⁷，你可以从命令行运行它们。这两种工具都有插件¹⁸，因此可以从PyCharm或命令行运行（如果你有同事更喜欢其他编辑器的话）。

或者，如果您有特定需求，可以自定义 PyCharm 的“*Code Style*”选项。然后，您可以导出这些选项，并与团队成员共享。

如果你有多个风格各异的团队，或者还参与开源项目或其他标准不同的项目，PyCharm 也能应对自如。在 Python 的代码样式偏好设置中，使用“方案”下拉菜单并选择“项目”。这样你就可以按项目指定样式。

¹⁴<https://github.com/google/styleguide/blob/gh-pages/pyguide.md>

¹⁵<https://docs.openstack.org/hacking/latest/user/hacking.html>

¹⁶<https://github.com/google/yapf>

¹⁷<https://github.com/ambv/black>

¹⁸<https://plugins.jetbrains.com/plugin/10563-black-pycharm>

4. The Editor

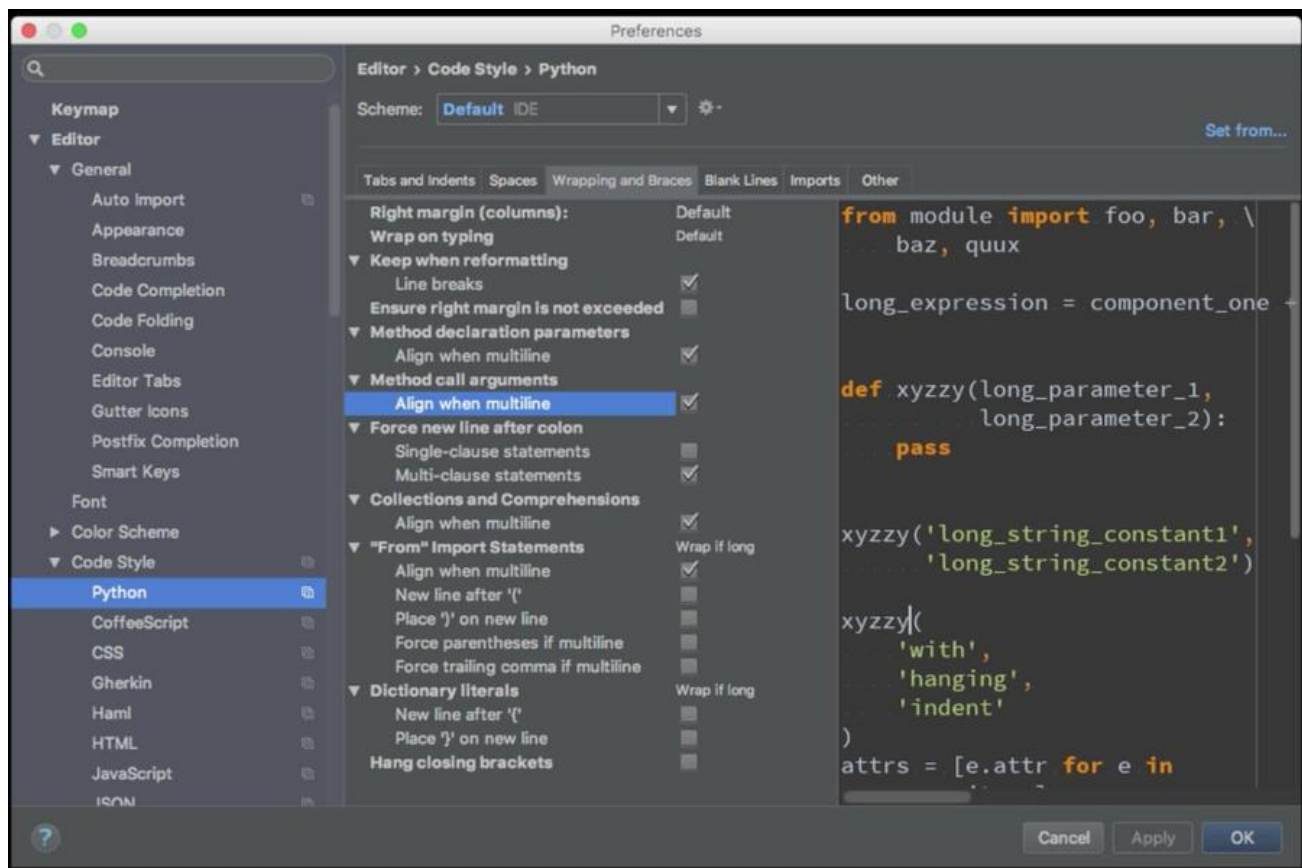
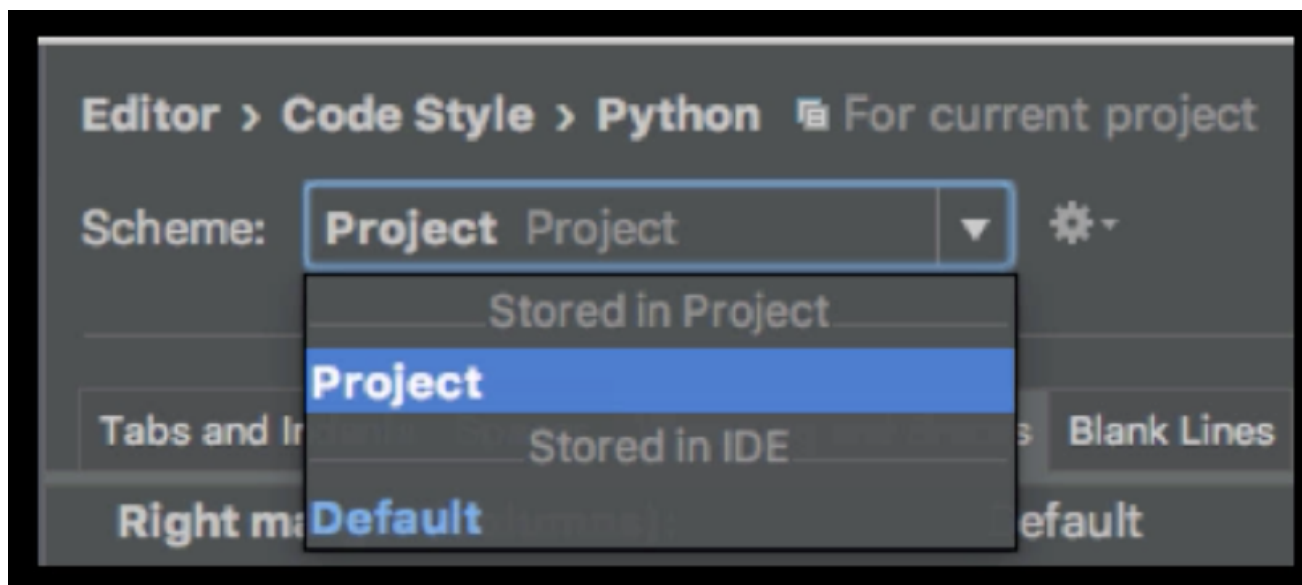


图418：说明代码风格偏好变化的图。您还可以从该页面导入和导出这些设置。



F图419：该图展示了将方案设置为“项目”，以便项目格式设置不是全局的。

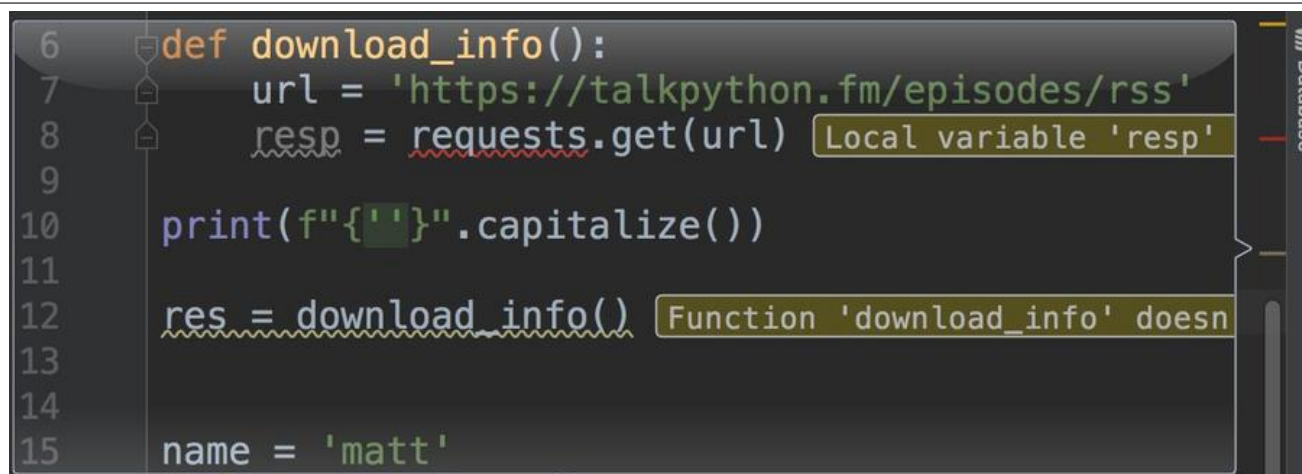


图420：说明镜头模式的图。要激活镜头模式，请将鼠标悬停在滚动条中的条纹上。如果它们在滚动框之外，镜头模式弹出窗口将出现。

配置格式很重要，因为它使与他人的交互变得容易得多。代码审查者在审查代码时最不想看到的是一堆空白调整，而对代码的唯一有意义的更改却淹没在这些干扰中。

4.10 镜头模式

在PyCharm评估您的代码时，您会注意到滚动条中嵌入了条纹。这些条纹用于突出显示错误或警告。如果您点击它们，PyCharm将导航到文件中的相应位置并弹出灯泡图标，以便您可以修复问题。您还可以将鼠标悬停在条纹上。如果将鼠标悬停在当前屏幕上代码对应的条纹上，PyCharm会显示一个包含错误或警告的工具提示。如果将鼠标悬停在窗口视图上方或下方的条纹上，PyCharm将弹出*Lens Mode*。镜头模式工具提示将显示问题以及周围的一些代码，以便提供相关上下文。

4.11 面向对象和基于类的特性

Python是一种多范式语言，支持多种编码风格。你可以采用命令式风格（如C或Basic）、函数式风格（如Lisp或Scheme）以及面向对象风格（如C++或Java）进行编码。如果你正在定义类并嵌套类层次结构，PyCharm具备一些功能可帮助你更好地理解代码。PyCharm拥有*gutter icons*来帮助你浏览和理解代码。

假设我们正在开发一款游戏（有没有人喜欢老式的多用户地下城游戏？）。我们可能已经创建了一个Creature基类，Wizard和Dragon是它的子类。如果你查看Creature的源代码，左侧边栏会有边栏图标。边栏图标旁边还有箭头。向上的箭头表示存在父类方法。向下的箭头告诉你哪些子类重写了该类或方法。你可以将鼠标悬停在边栏图标上，查看引用了哪些类。或者你可以点击边栏图标导航到实现处。

gutter 图标对于在面向对象程序中浏览类型层次结构非常有用。

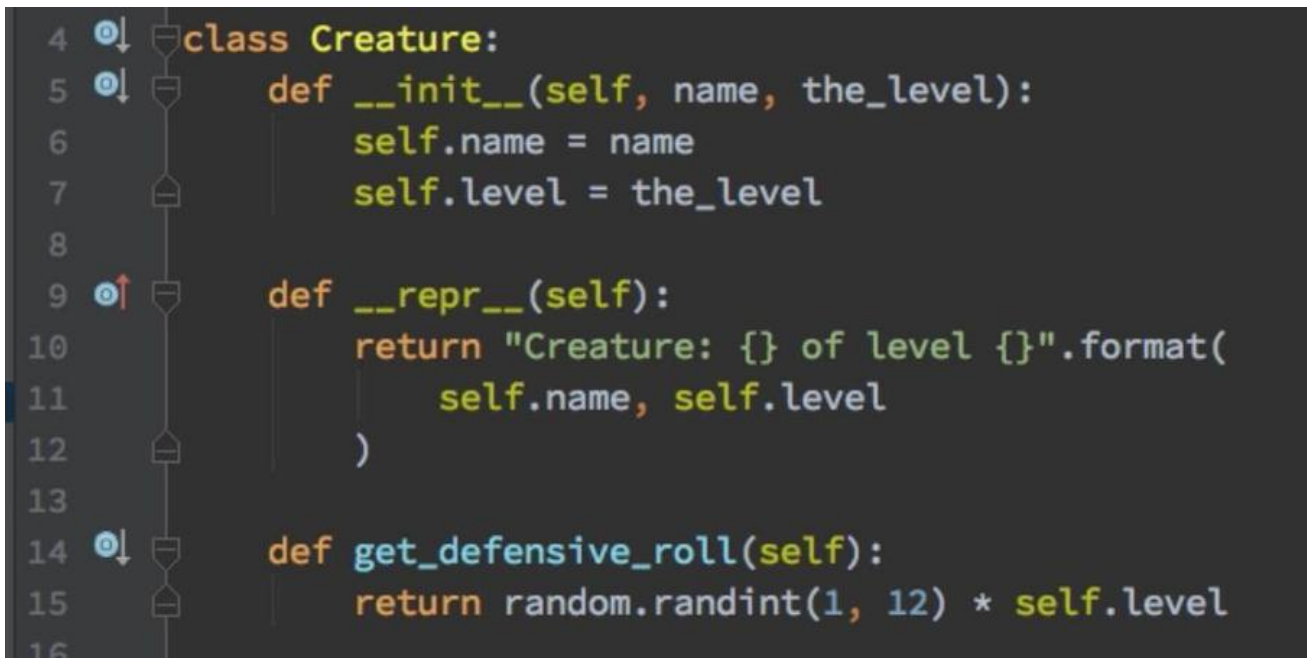


图421: 说明边栏图标示意图。在`class`、`__init__`、`__repr__`和`get_defensive_roll`的左侧是边栏图标。向下箭头表示存在子类（在类上），或者子类重写了方法。向上箭头表示该方法正在重写基类（object是Python 3中隐含的基类）

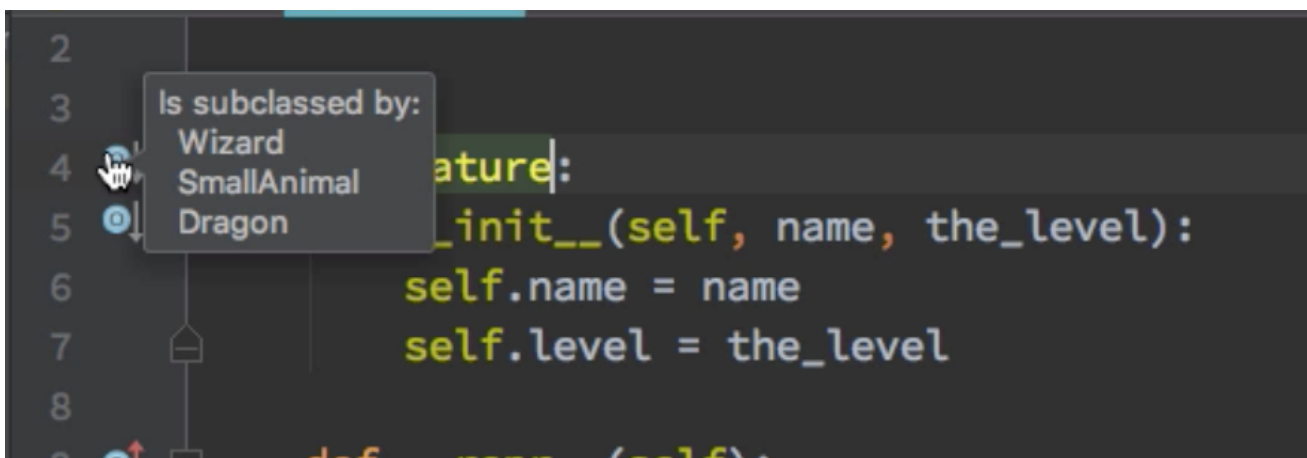


图422: 说明悬停在装订线图标上的图示。在这种情况下，它指示哪些类是Creature类的子类。我们可以点击该图标导航到这些类。

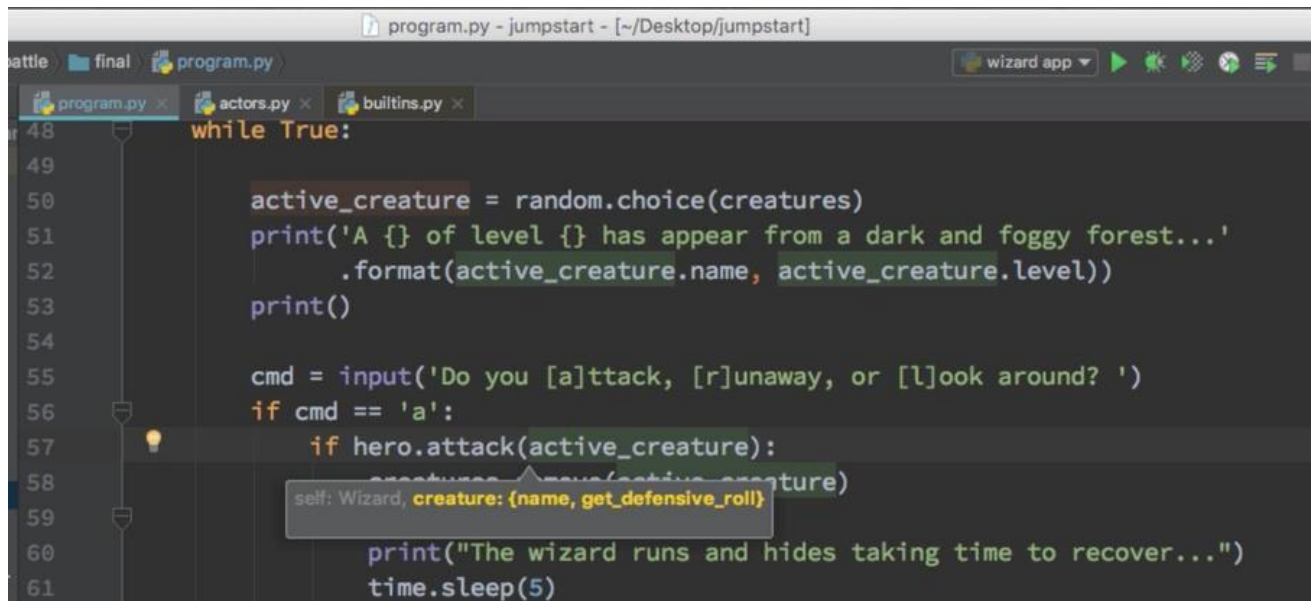


图423：说明`.attack`方法参数的图。请注意，它表明一个生物应该具有`name`和`get_defensive_roll`属性。

4.12 查看文档

让我们探索一下PyCharm提供的其他一些帮助。当你调用函数或方法时，如果你刚刚输入了函数名，PyCharm会调出“*Parameter Info*” (command - p (在Mac系统上)，或者)ctrl - p (在Windows和Linux系统上)。这个提示对于了解要提供哪些参数很有用。如果之后你的光标不在函数调用内部，这个工具提示不会弹出，你需要显式调用它。

如果您想查看该方法的文档，请使用“*Quick Documentation*”命令 (Mac 上为 ctrl-j，Windows/Linux 上为 ctrl-q)。这将弹出一个包含文档字符串以及任何类型信息或推断类型的窗口。

4.13 创建文档

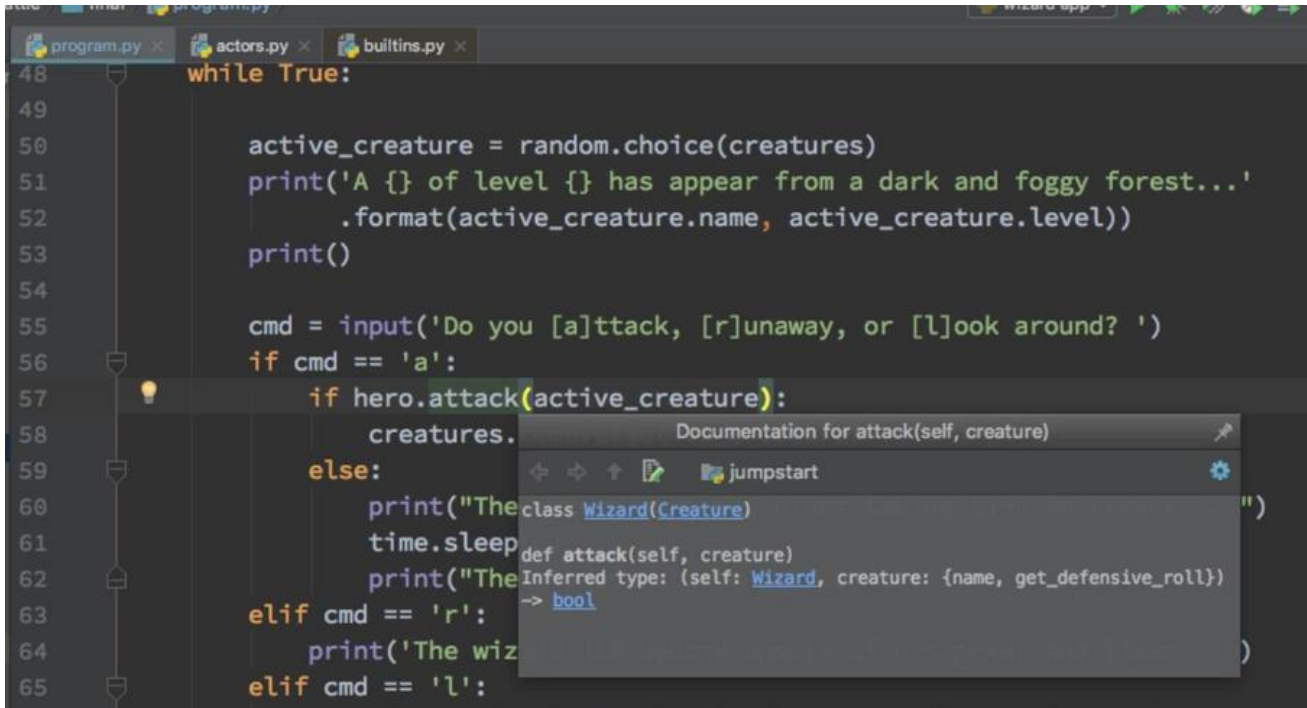
让我们看看如何创建文档。如果你在某个函数或方法上查看快速文档时发现它为空，你可以自己添加文档。从该函数点击“*导航到声明*”来定位它。如果你输入一个三引号字符串（连续三个”字符）并回车，PyCharm 会生成一些文档框架。它会填入参数名称和返回值。这没有快捷键，是自动完成的，但如果你想更改文档字符串的格式，可以在偏好设置的“Python 集成工具”部分进行编辑。

PyCharm 还会跟踪函数中的参数名称。如果您重命名这些参数，PyCharm 也会重命名文档。此外，快速文档中的任何类型都是超链接，因此您也可以快速导航到相关文档。

4.14 查找用法

如果你正在考虑重构，你可能会问自己，这个函数还会被调用吗？PyCharm 有一个“*Find Usages*”命令 (Alt+F7)，它会列举出调用该函数的位置。

4. The Editor



424 .attack

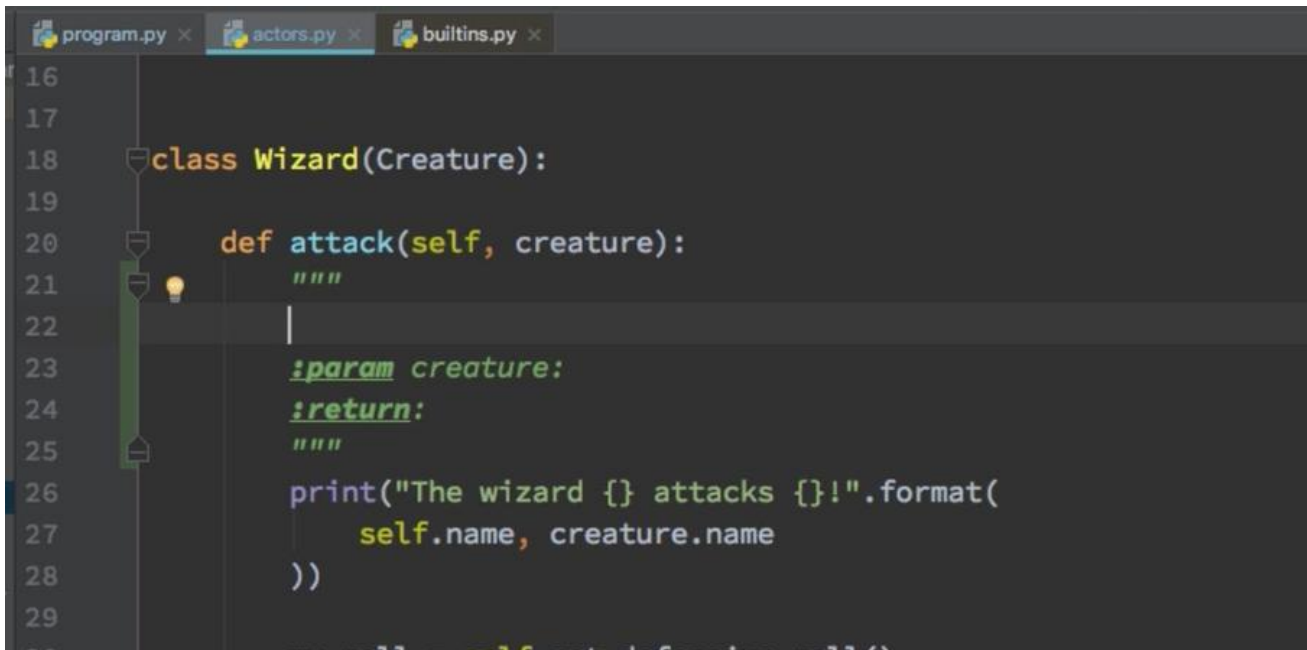


图425：说明插入三引号并回车后的结果的图。PyCharm自动生成了参数和返回值。

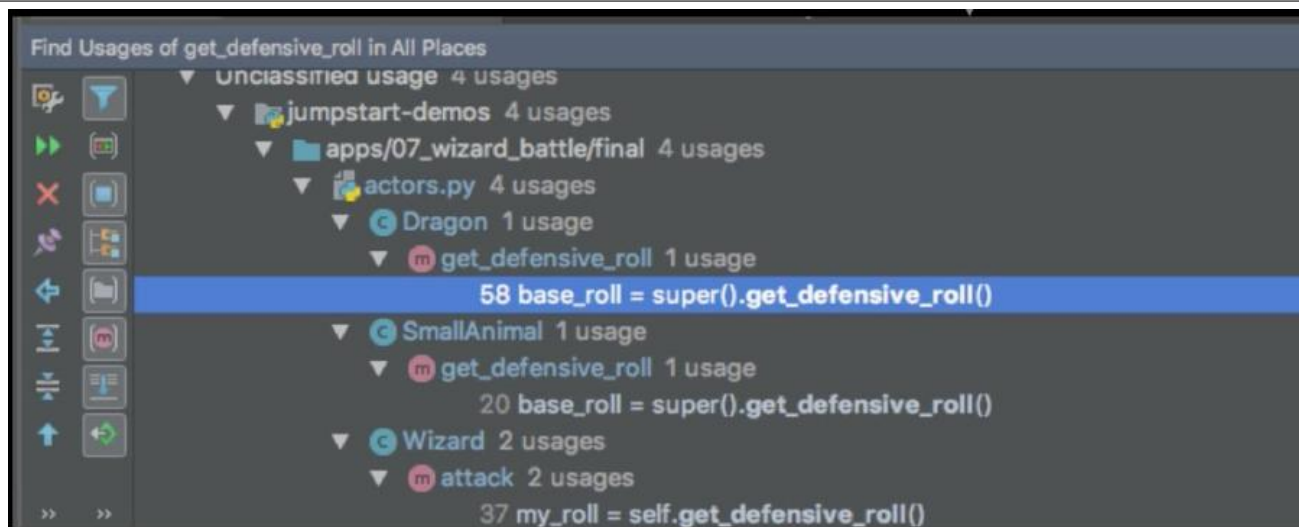


图426: 说明对`.get_defensive_roll`方法调用“查找用法”的结果的图。

同样，这不仅仅是对文件进行简单的grep搜索。PyCharm理解代码的引用方式，并且还会显示模板结果。如果有多个类碰巧使用相同的方法名，PyCharm应该能够找到正确的方法，而不会显示其他方法。PyCharm还能够在整个项目进行智能重命名。

4.15 实时模板

之前我们看到，如果输入`main`并按下TAB键（或回车键），PyCharm会使用*Live Template*为以下内容创建框架代码：

```
if __name__ == '__main__':
    #
```

它还会将光标定位到上述代码中注释所在的位置。这是一个简单的模板，还有其他更复杂的模板，比如输入`iter`。如果你输入`iter`并按下制表键，你会看到类似这样的代码，其中CURSOR代表实际的光标，而不是这个词：

```
for in CURSOR:
```

你可以输入想要枚举的内容，比如`range(10)`，然后再次按下TAB键。你会看到光标移动到`for`关键字之后：

```
for CURSOR in range(10):
```

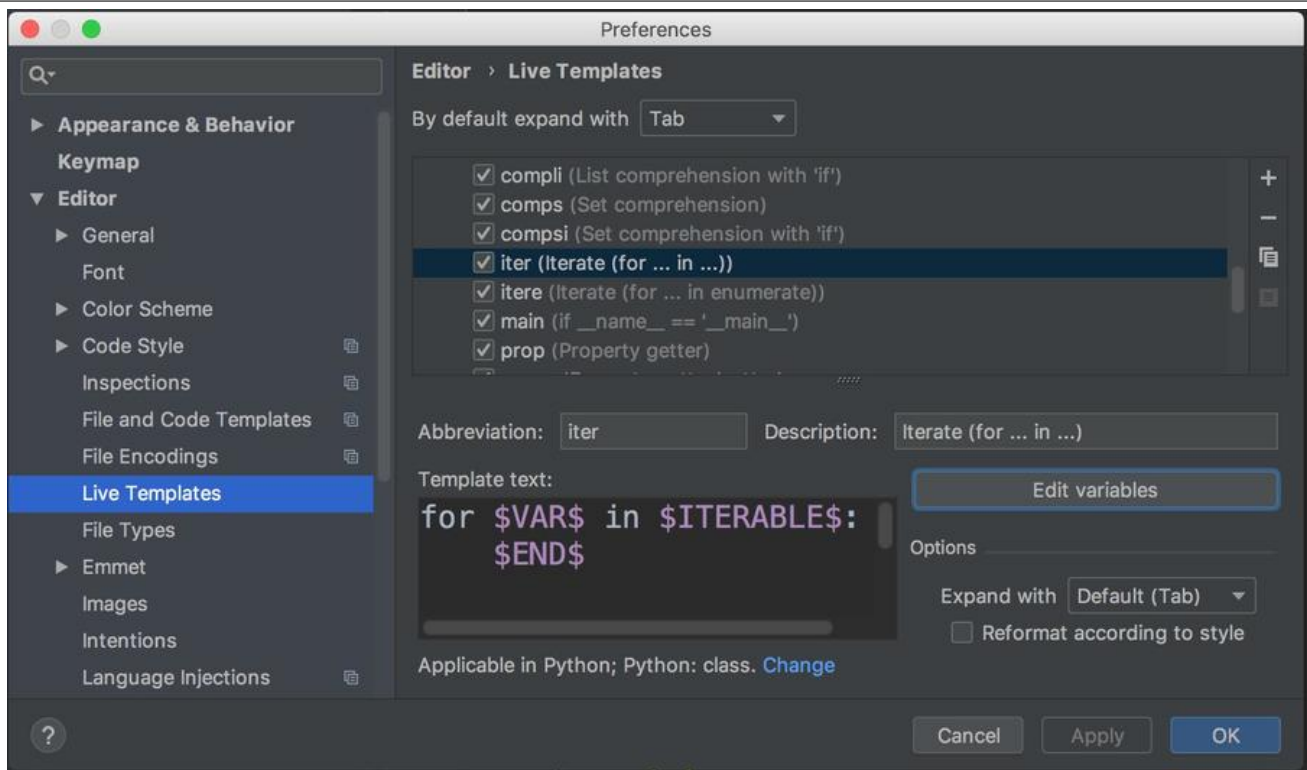
你可以输入一个用于迭代的变量名，比如`i`，然后按TAB键。你会看到以下内容：

```
for i in range(10):
    CURSOR
```

现在你可以填写`for`循环的主体部分。

有许多实时模板。你会想要探索可用的模板。打开偏好设置窗口并搜索“实时模板”。如果你展开Python部分，你会看到一个模板列表。还有针对其他文件类型的模板。如果你愿意，你可以创建更多。如果你点击`iter`模板，你会看到它的“模板文本”内容为：

```
for $VAR$ in $ITERABLE$:
    $END$
```

[illegible]

你可能想知道，根据这个模板，为什么光标首先移到了`$ITERABLE%`而不是`$VAR$`。右侧有一个名为“编辑变量”的按钮。如果你点击它，会显示输入变量的顺序。你可以根据需要重新排列它们。

如果你发现自己反复输入相同的代码，可以为其创建一个模板。点击加号按钮创建新模板，为其设置缩写和描述，并指定它在何种上下文环境中运行（如Python、XML等）。这是提高工作效率的简便方法，尤其是当你可以与其他团队成员共享这些模板时。

4.16 总结

本章介绍了许多能让你在PyCharm中提高工作效率的命令。如果你开始使用这些命令以及上一章中的命令，你会发现自己充分利用了PyCharm的诸多强大功能。我们探讨了如何使用PyCharm清理代码、提出建议、自动完成以及浏览文档。PyCharm拥有一个功能非常强大且可配置的编辑器。由于你会经常使用它，花些时间学习其功能是很有好处的。

4.17 命令

- *"Show Intention Actions"* - (按 Alt + Enter 或点击灯泡)
- *"Reformat Code"* - (Option + Command + L 或 Ctrl + Alt + L)

- "Code Cleanup..."
- "Live Template" - (输入模板开头并按制表符)
- "Parameter Info" - (command - p 或 ctrl - p)
- "Basic Completion" - (输入变量起始部分并按制表符)
- "Code Completion" - (按Ctrl + 空格)
- "Find Action..." - (Command + Shift + A 或 Ctrl + Shift + A)
- "Refactor Method" (Command + Option + M 或 Ctrl + Alt + M)
- "Rename" - (按shift + f6组合键)
- "Lens Mode" - (悬停在滚动条上)
- "Quick Documentation" - (Mac 上按 ctrl-j, Windows/Linux 上按 ctrl-q)
- "Find Usages" - (按 F7 键或 Alt+F7 组合键)

4.18 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/2-the-editor` 中关于新项目的说明, 为一个类添加一项功能。
2. 访问<https://github.com/talkpython/mastering-pycharm-course> , 按照 `tree/master/your-turn/2-the-editor` 中新项目的说明清理代码, 使其符合PEP8规范。
3. 访问<https://github.com/talkpython/mastering-pycharm-course> , 按照 `tree/master/your-turn/2-the-editor` 中关于新项目的说明删除未使用的导入。
4. 访问<https://github.com/talkpython/mastering-pycharm-course> , 按照 `tree/master/your-turn/2-the-editor` 中关于新项目的说明, 为方法添加文档。

第5章

源代码控制

一旦开始与团队合作，就需要某种版本控制系统。这会让团队开发变得容易得多，因为你可以进行分支、合并和提交更改。PyCharm 对许多常见的版本控制系统都有内置支持。如果你一直希望为自己喜欢的版本控制系统配备一个用户界面，那么你很幸运，因为 PyCharm 支持常见的版本控制系统。它支持 Git、Subversion、Mercurial、Perforce 和 CVS。除此之外，你还可以找到适用于 PyCharm 的插件，以添加诸如微软的 TFS 等额外功能。

5.1 访问源代码控制

让我们来看一个使用Git的示例。如果你在上一章从GitHub检出了一个Git项目。如今Git非常常见，所以我们将重点关注它。当你打开一个Git项目时，你会看到有一个“VCS”菜单，其中有许多与版本控制系统集成的命令。我们暂时忽略这个菜单选项，看看PyCharm的右下角。你会看到一个写着“Git: master”的按钮。这个按钮也被称为Git小部件。

如果您点击该按钮，PyCharm 会弹出一个菜单，用于创建新分支或标签，或与其他本地和远程分支进行交互。

在PyCharm顶部还有一组按钮。在这些按钮左侧，你会看到“Git:”。有一个用于“Update Project” (command-t (或ctrl-t))、“Commit” (command-k (或ctrl-k))、“Compare with the Same Repository Version”、“Show History”以及“Revert” (command-alt-z (或ctrl-alt-z))的按钮。在项目文件浏览器中，文件也会根据其状态显示不同颜色。蓝色表示文件已更新，红色表示文件不在版本控制中，绿色表示文件已添加到版本控制但尚未提交。

如果您想查看所做的任何更改，请点击“与同一存储库版本比较”按钮。这将显示一个带有两个代码窗格的屏幕。左侧是存储库中的原始副本，右侧是您的版本。任何差异都将突出显示。

如果你有想要分享的更新，常见的工作流程是首先运行“更新项目”来拉取并自动合并最新更改。然后你可以“提交”你的更改。这将打开一个“提交更改”窗口。你可以选择要包含哪些更改并提供提交消息。还有一些有用的预提交更改，比如重新格式化代码或优化导入。根据你团队的编码标准，这些更改可能会有帮助。这里的“提交”按钮功能丰富，有多个选项。在Git中点击“提交”将更新你的本地仓库。如果你想将更改推送到远程仓库，请点击

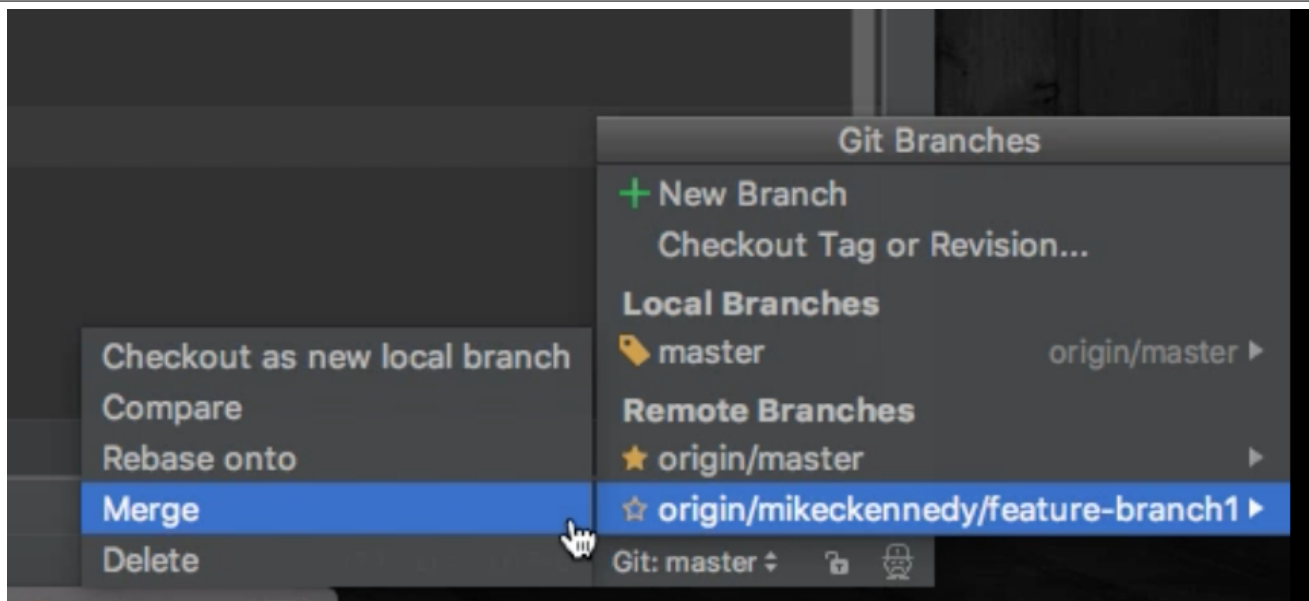


图51：说明点击右下角Git按钮的图示。请注意，按钮本身会显示当前分支。点击该按钮时，会出现创建新分支或标签的菜单选项，以及与其他分支交互的子菜单。



图52：该图展示了用于更新项目、提交更改、与存储库版本进行比较、显示历史记录以及恢复的按钮。这些按钮位于编辑器的右上角。

“提交并推送...”。如果您决定推送，将打开一个“推送提交”窗口，用于添加标签或强制推送。

在这个用户界面中，PyCharm 有一些不错的选项，而使用命令行工具时，这些操作要么很麻烦，要么很恼人。如果你想从提交中移除某项更改，需要取消选中该更改旁边的复选框。这样你就可以选择要提交的更改。

Git集成的另一个很酷的功能是“Annotate”命令（版本控制系统-> Git -> 批注）。这使您可以查看对文件的所有签入。如果运行此命令，左侧会列出进行更改的人员和时间。如果单击某项更改，将出现另一个显示差异的屏幕。

5.2 更多版本控制

让我们深入探讨一下。除了用于查看差异的按钮以及指示文件状态的文件颜色之外，PyCharm还有一些其他功能。在任何已更改文件的空白边距处

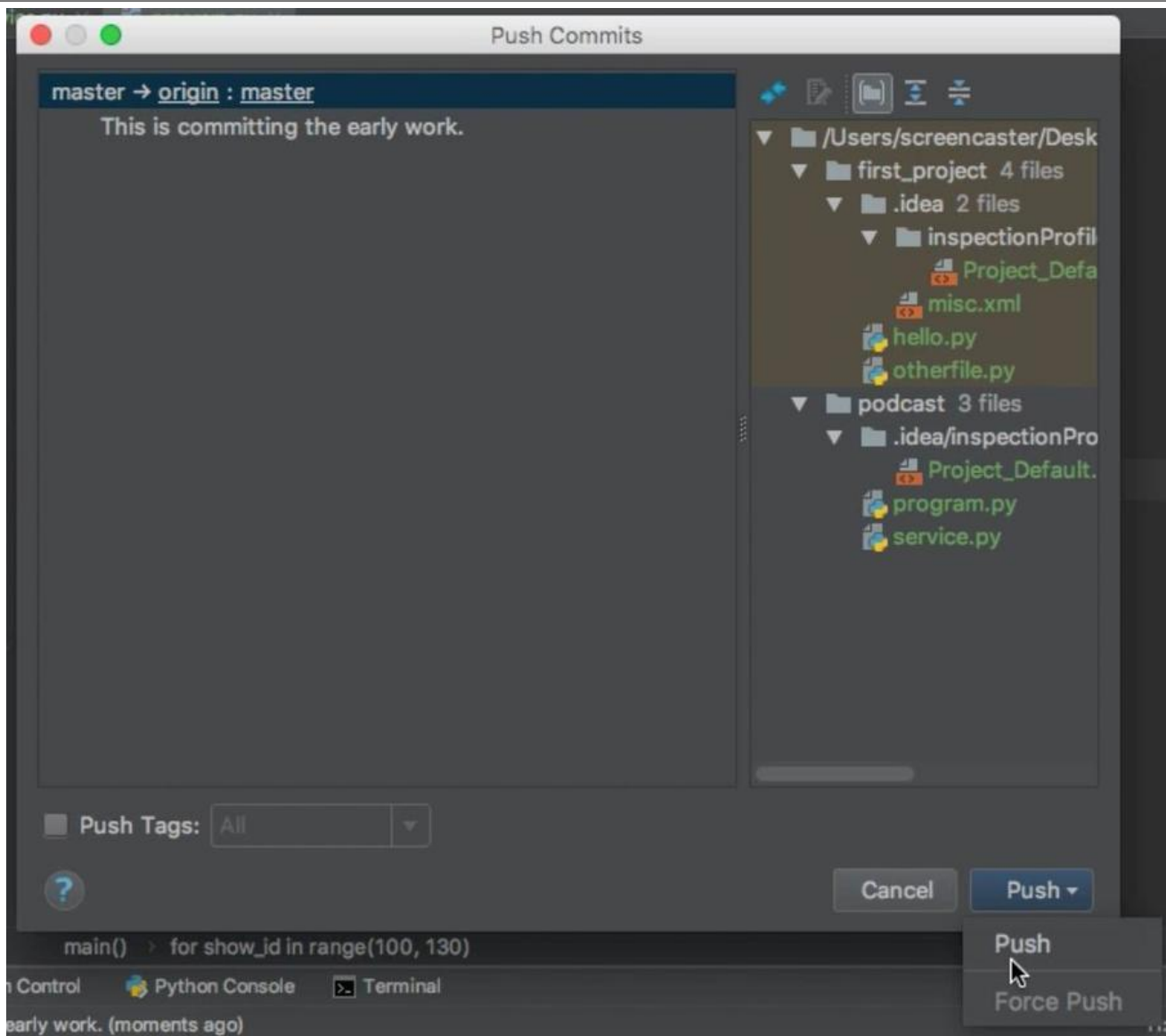


图55：说明“推送提交”窗口的图。在此处，您可以添加标签，或强制推送。

代码是突出显示部分。如果行号栏颜色为绿色，则表示这是新行，在前一次提交中不存在。如果颜色为蓝色，则表明该行在前一次提交中存在，但后来已更新。

如果您点击代码行号栏中的高亮部分，会出现一些按钮。您可以跳转到“*Previous Change*” (Ctrl+Shift+Alt+向上箭头)、“*Next Change*” (Ctrl+Shift+Alt+向下箭头)、“*Rollback Lines*”(Command+Alt+Z 或 Ctrl+Alt+Z)、“*Show Diff*” (Command+D 或 Ctrl+D)、“*Copy*” (Command+C 或 Ctrl+C)，以及“*Highlight Words*”。点击蓝色的代码行号栏也会显示内联差异。请注意，复制命令将复制原始版本，而非当前版本。

PyCharm 还会插入一个向右的箭头。这表明从存储库版本中删除了一个部分。这些提示很友好且不突兀，能让您了解正在编辑的文件中发生了哪些更改。

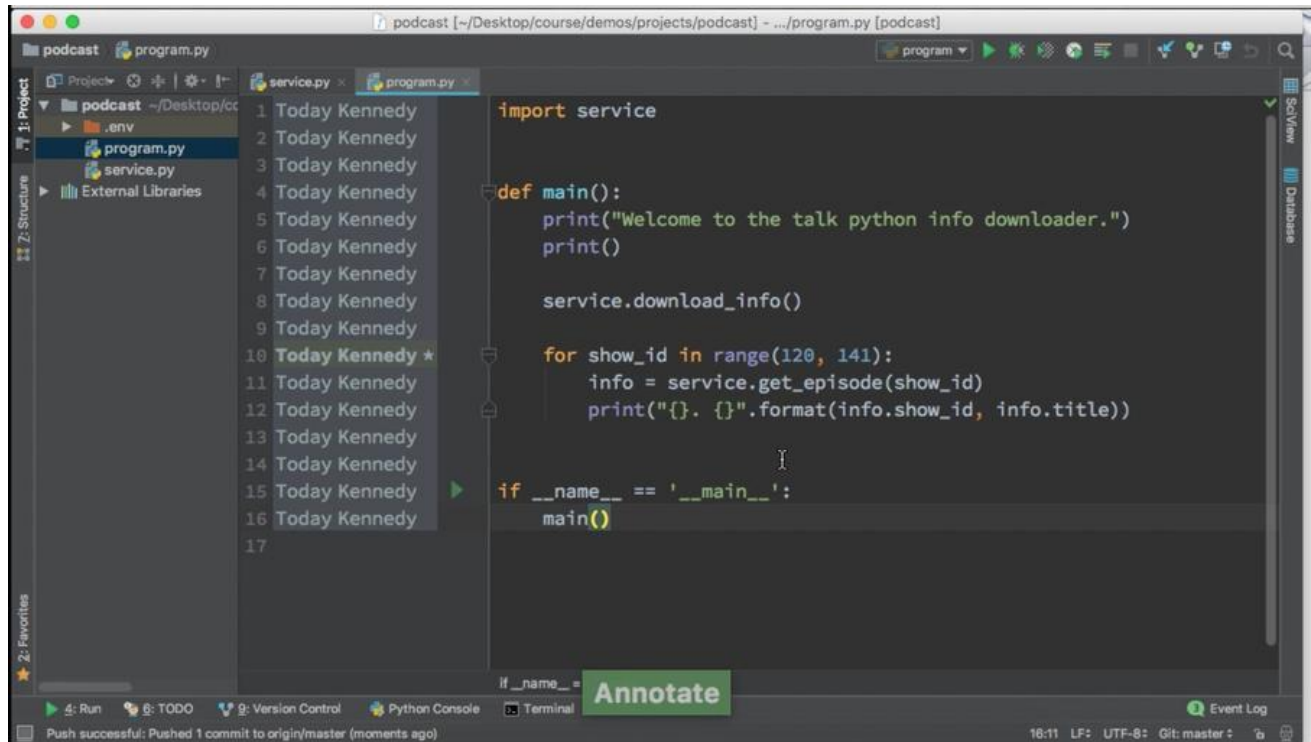


图56：阐释注释视图的图。左侧是对文件所做更改的列表。

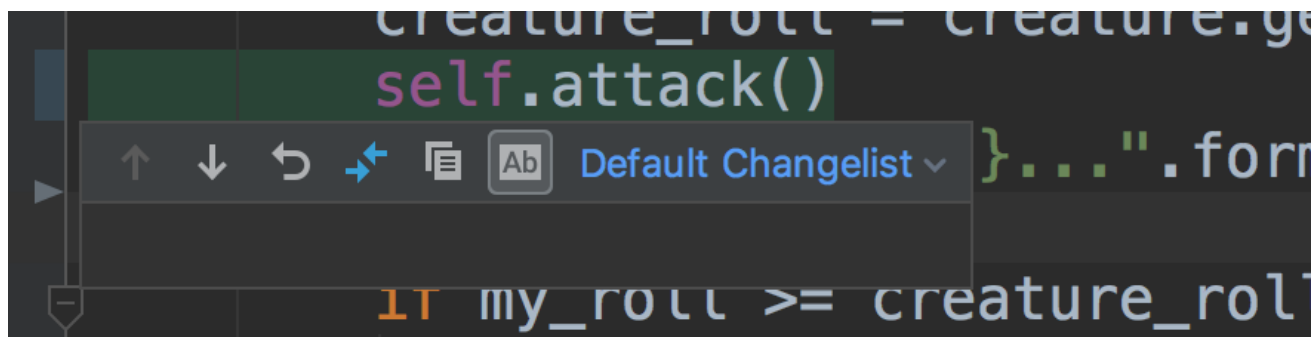


图57：该图展示了装订线中的突出显示部分以及点击它时出现的上下文菜单。还展示了代码被删除处的箭头。

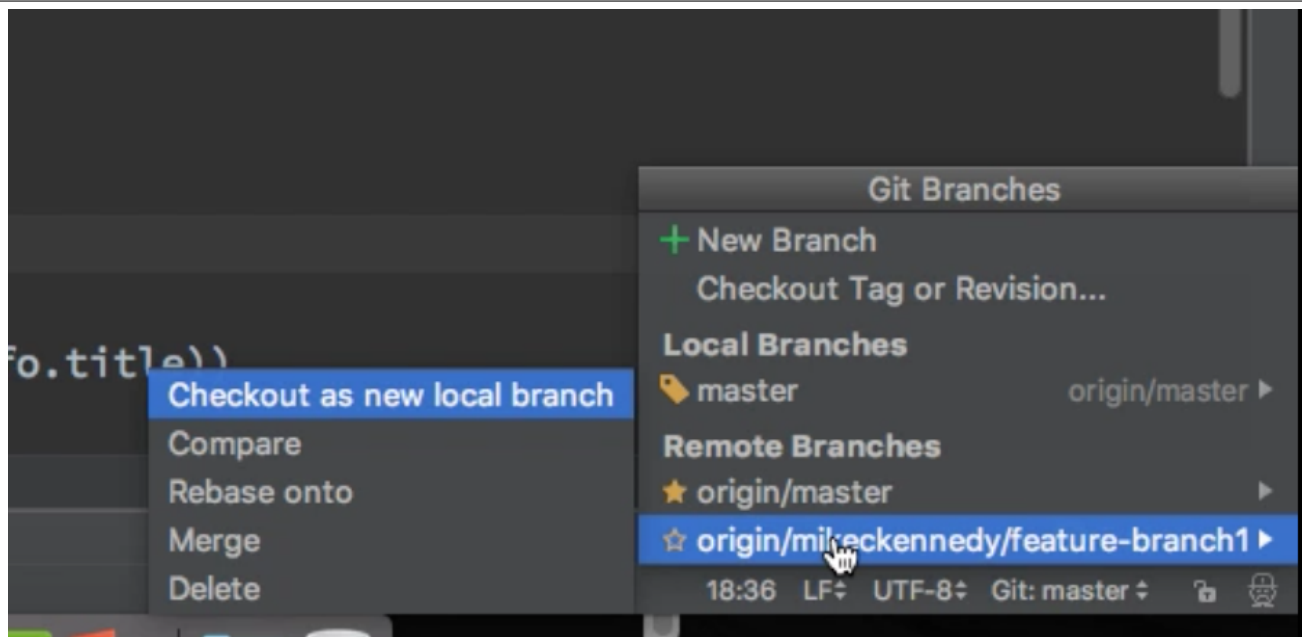


图58：说明检出远程分支的示意图。

5.3 分支

在本节中，我们将了解如何创建一个新分支并使用它。回想一下，在PyCharm的右下角有一个Git小部件按钮。如果你点击它，就会有处理分支的选项。如果你想在远程功能分支上工作，可以在弹出窗口中点击远程分支名称，然后点击“检出为新的本地分支”。此时，你的项目将使用该分支的代码进行更新。你可以按照前面描述的工作流程将更改提交到远程分支。

创建新的本地分支后，您可以通过点击本地分支名称，然后点击“检出”，在分支间快速切换。如果您有本地更改，则有两个选项。一个是“强制检出”，这将丢弃本地更改。另一个是“智能检出”，这将暂存更改、检出分支，然后取消暂存更改。

PyCharm 还允许您合并分支或将其变基。检出您希望用更改进行更新的分支。点击 Git 小部件，然后选择您希望从中获取更改的分支。您可以选择“合并到当前分支”或“变基并检出”。如果存在冲突，您将有机会合并它们。

5.4 本地历史

PyCharm “ ” Git

PyCharm

“ ” -> “ ”

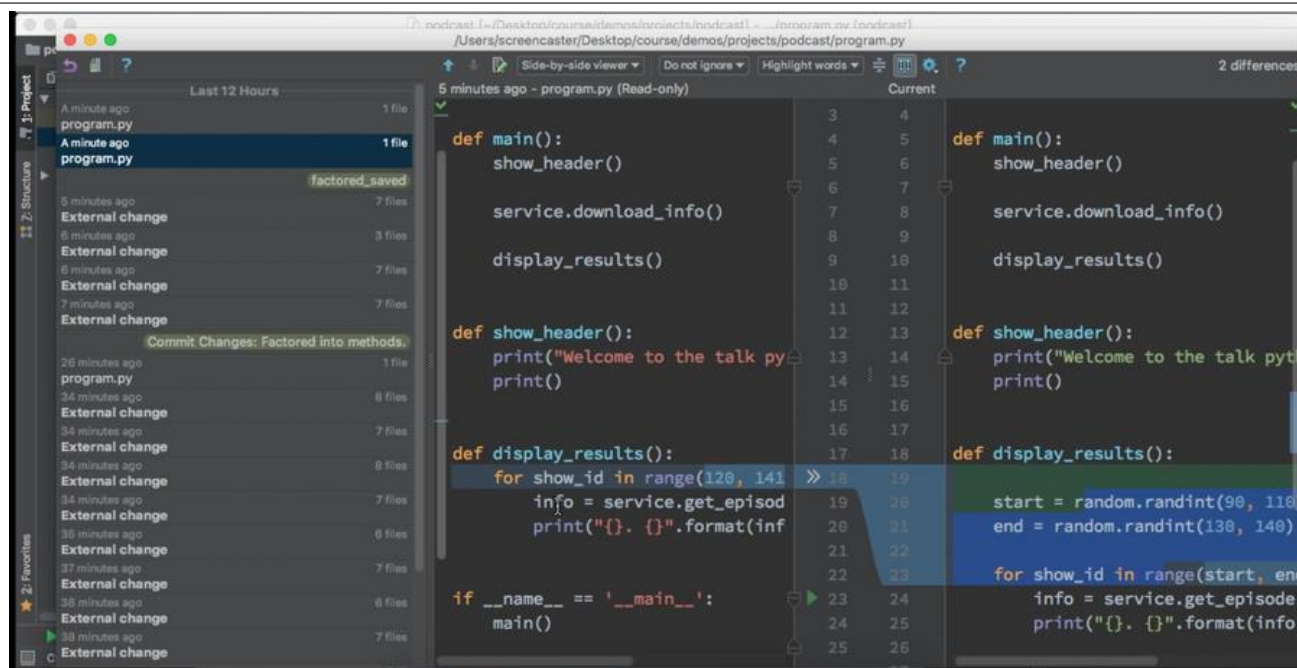


图59: 说明本地历史记录的图。与版本控制无关, PyCharm管理其自身对项目所做更改的版本。

PyCharm
PyCharm
localHistory.daysToKeep PyCharm

5.5 PyCharm与Git工作流程

Git是一个功能强大的工具，有许多选项。一种简单的Git推荐工作流程称为Git Flow¹⁹。这个工作流程的要点是有两个主要分支，`master`和`develop`。只有在我们有确定的发布版本时才会将内容推送到`master`分支。`develop`分支包含下一个发布版本的工作内容。还有其他分支可用于创建固定版本、测试发布版本或进行热修复。然而，这些都是短期存在的分支，最终会合并回`develop`分支（当发布版本确定时，`develop`分支最终会合并到`master`分支）。

让我们逐步添加一个新功能。首先，我们将在GitHub上为该功能创建一个问题。然后使用该功能创建一个新分支。最后，我们会将该功能合并回主分支。假设我们已经在GitHub上提交了一个错误。我们使用PyCharm与GitHub的集成功能，这样就能查看项目的任务。要进行配置，请转到“工具”->“任务与上下文”->“配置服务器...”。

在配置好GitHub后，PyCharm右上角会出现一个“默认任务”按钮。如果点击“打开任务...”，它将显示项目中的问题（将其称为“任务”）。这将打开“打开任务”窗口，并允许您创建一个新分支来解决此问题。点击“确定”后，您会看到PyCharm已为您创建了一个新的本地分支。Git小部件将不再显示“master”，而是显示分支名称。

¹⁹<https://nvie.com/posts/a-successful-git-branching-model/>

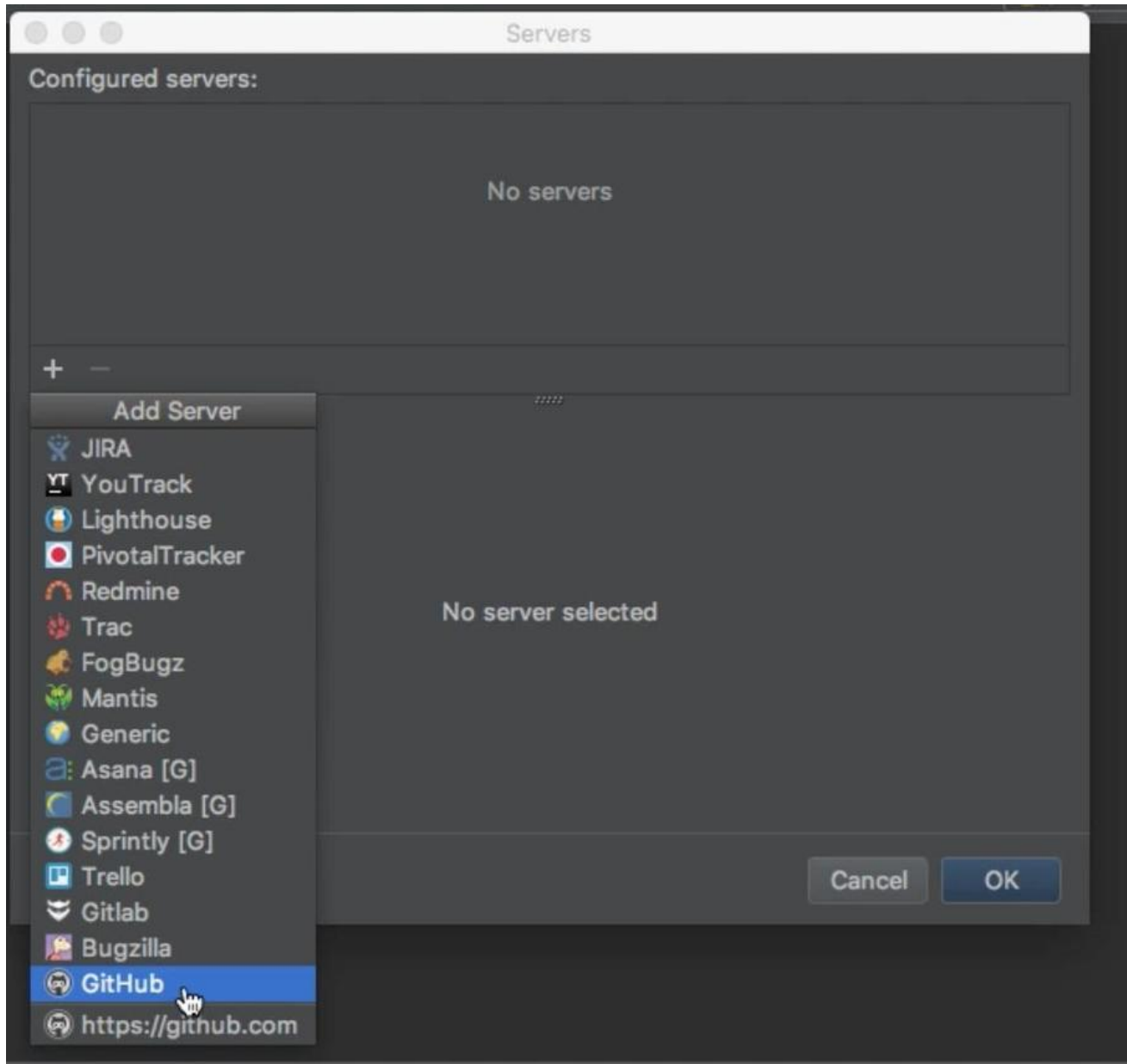


图510：说明为GitHub添加服务器的示意图。

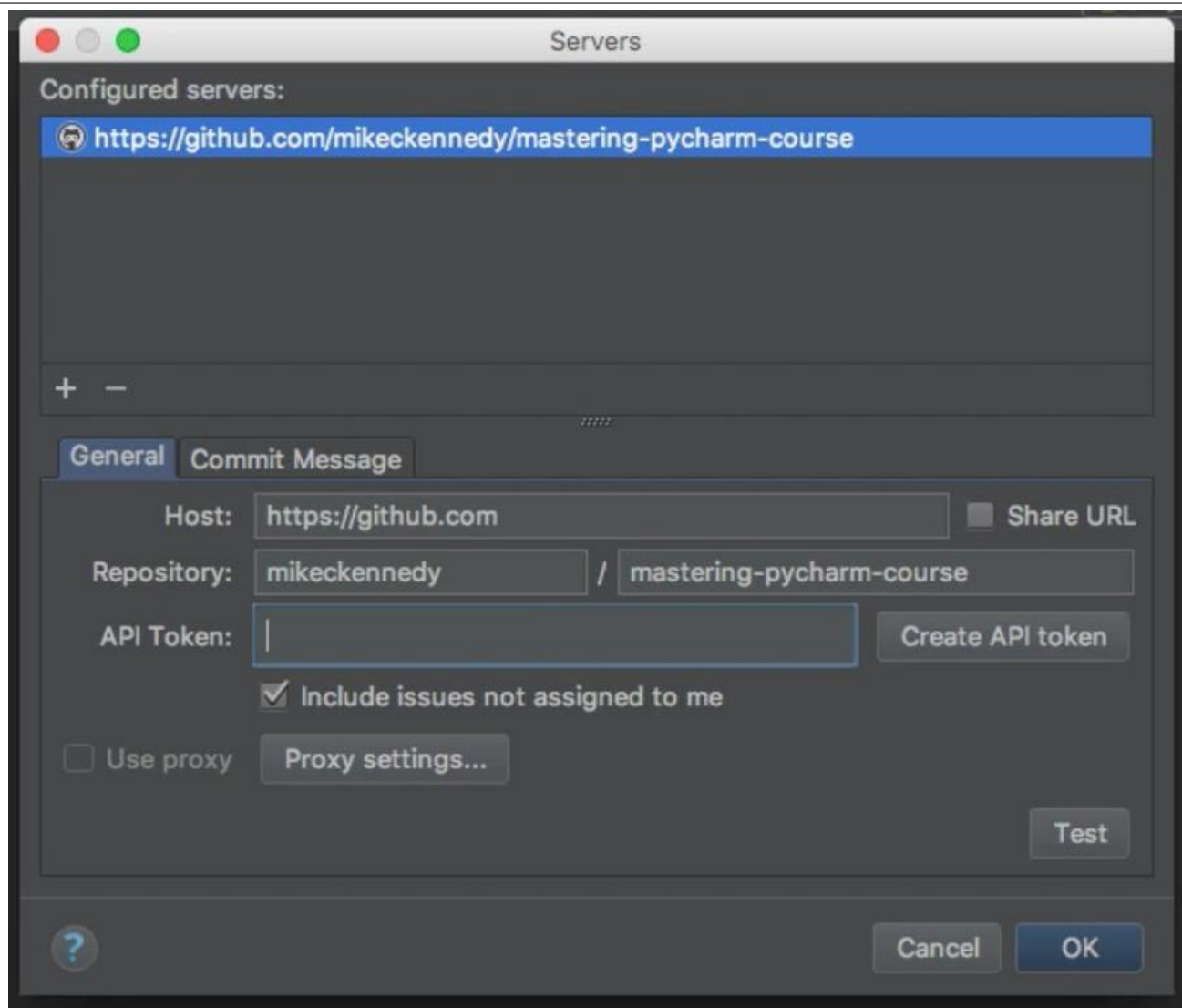


图511: 说明为GitHub配置服务器的图。

此时，你就可以编写代码了。添加了解决GitHub问题所需的代码后，你需要关闭任务。要做到这一点，请运行“Close Active Task” (Alt+Shift+W)。点击“确定”后，你将有机会推送分支。如果你推送分支，PyCharm会将该分支提交到GitHub上，并让你回到主分支的最新版本。你需要回到GitHub创建一个拉取请求，合并该请求并关闭问题。

通过使用PyCharm中的“任务”功能，你将能够访问项目的问题。如果需要，你还可以轻松地基于这些问题创建新分支，并遵循Git Flow类型的工作流程。

5.6 拉取请求

[illegible]

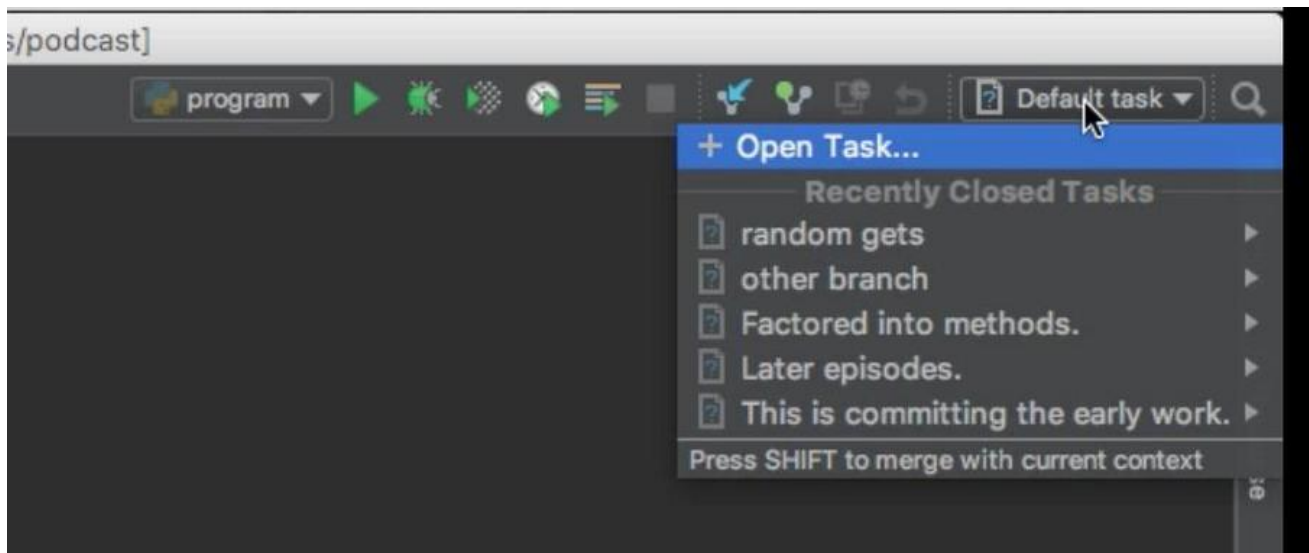


图512：展示成功配置GitHub且右上角出现任务按钮的示意图。

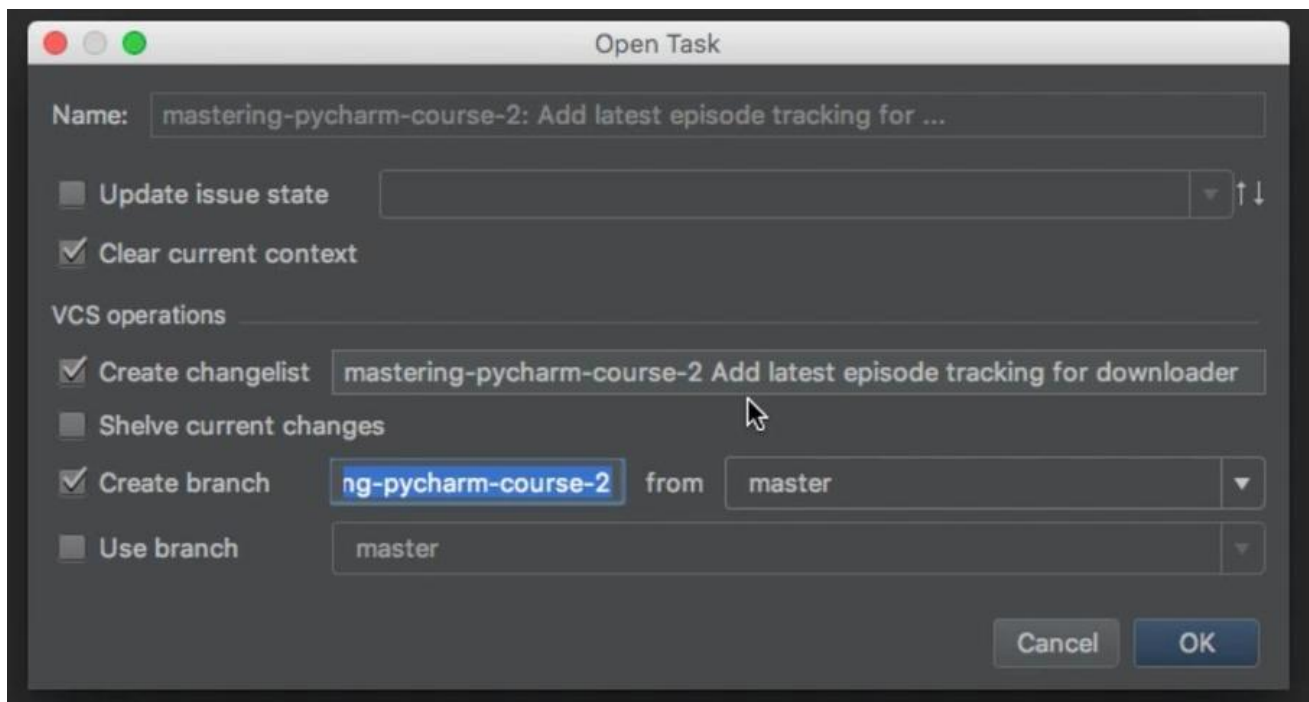


图513：用于说明为GitHub问题创建新分支的“打开任务”窗口的图。

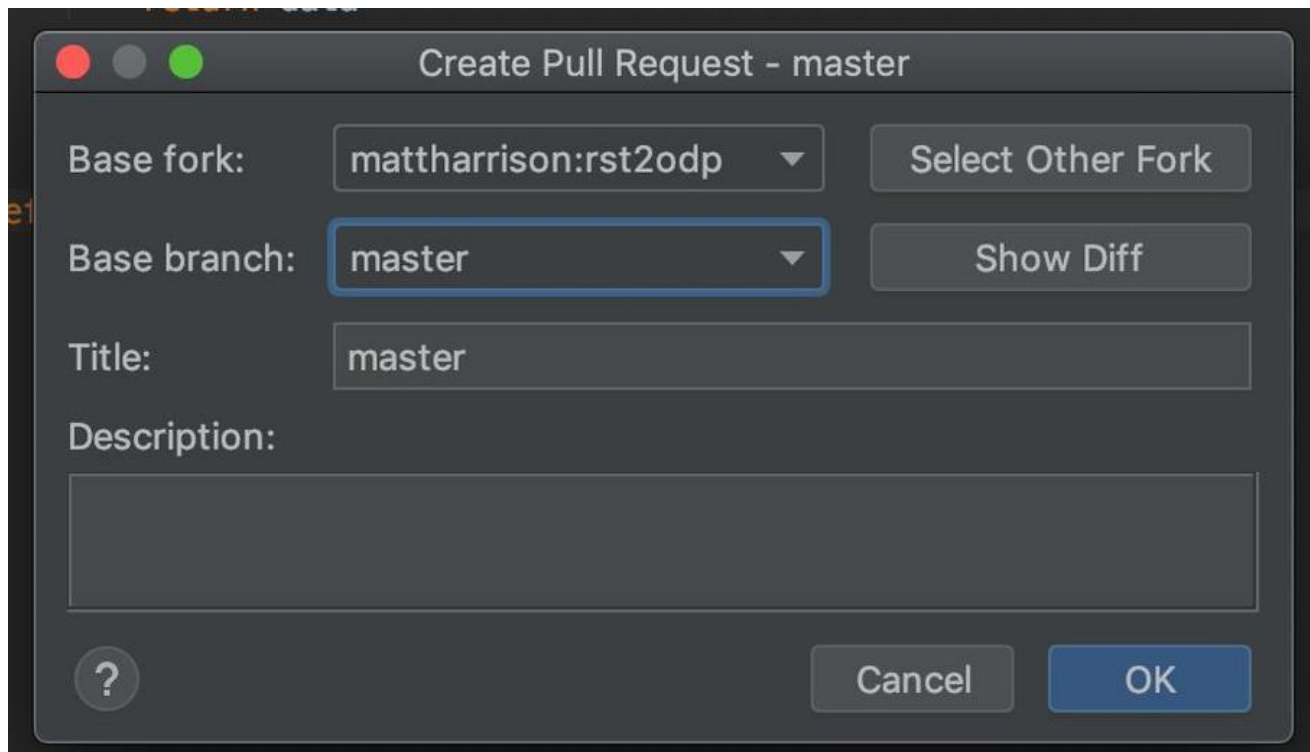


图515：说明创建拉取请求的图。

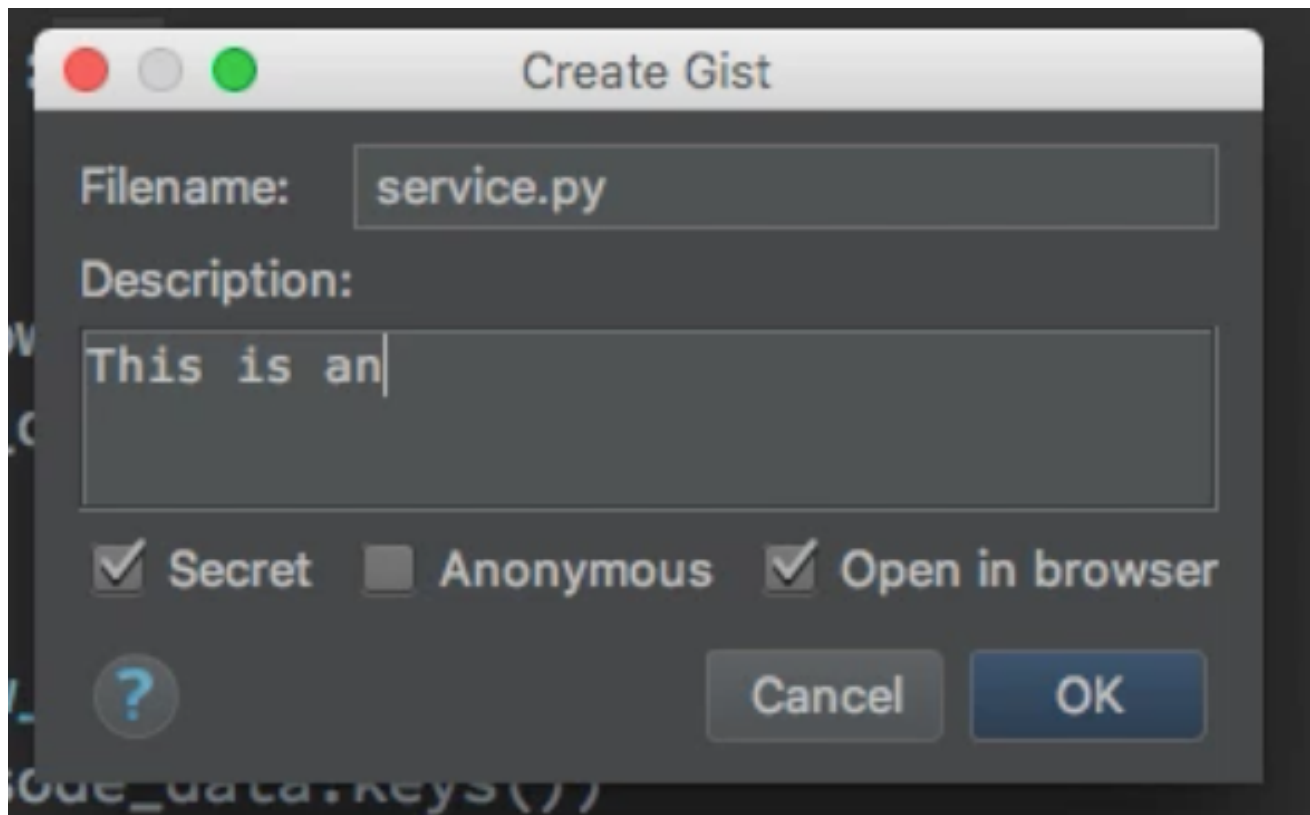


图516：说明创建要点的图。

我们特别研究了Git集成以及GitHub集成。一旦项目与源代码管理关联，右下角就会出现一个Git小部件，右上角则会出现执行常见源代码管理操作的按钮。使用Git小部件，我们可以切换分支并执行其他常见的分支操作。我们可以将任务与GitHub集成，以便根据问题创建新分支并遵循Git Flow流程。

5.9 命令

- *"Update Project"* - (command - t 或 ctrl - t)
- *"Commit"* - (Command-K 或 Ctrl-K)
- *"Compare with the Same Repository Version"*
- *"Show History"*
- *"Revert"* - (Command + Option + Z 或 Ctrl + Alt + Z)
- *"Annotate"* - (版本控制系统 -> Git -> 批注)
- *"Previous Change"* - (同时按下Ctrl + Shift + Alt + 向上箭头)
- *"Next Change"* - (同时按下Ctrl+Shift+Alt+向下箭头)
- *"Rollback Lines"* - (Command + Option + Z 或 Ctrl + Alt + Z)
- *"Show Diff"* - (Command - D 或 Ctrl - D)
- *"Copy"* - (command-c 或 ctrl-c)
- *"Highlight Words"*
- *"Close Active Task"* - (按Alt+Shift+W组合键)
- *"Create Pull Request"*
- *"View Pull Request"*
- *"Create New Local Branch..."*
- *"Create Gist..."*

5.10 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照tree/master/your-turn/3-source-control中新项目的说明，在PyCharm中试用Git。2. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照tree/master/your-turn/3-source-control中新项目的说明，在PyCharm中试用本地历史记录。

重构

在本章中，我们将讨论PyCharm中的重构支持。重构远不止是在一些文件中进行搜索和替换。PyCharm会将你的文件解析为抽象语法树。这使其能够在上下文中完全理解代码，从而可以对你的整个项目进行强大的更改。想要一个例子吗？假设你有两个类，每个类都有一个方法`to_json`。你想重命名其中一个。PyCharm知道这两个符号之间的区别，只会重命名你正在重构的那个。符号不是字符串。

代码重构是在不改变现有计算机代码外部行为的前提下，对其进行结构重组（即改变代码的分解方式）的过程。重构可提升软件的非功能性属性。其优点包括提高代码可读性和降低复杂度；这些优点能够提升源代码的可维护性，并创建一个更具表现力的内部架构或对象模型，以提高可扩展性。

我们如何知道何时进行重构呢？你可能遇到过`code smells`的概念。这些可以成为何时以及如何进行重构的有力指导。代码异味（code smell）是肯特·贝克（Kent Beck）创造的一个术语，用于描述代码中存在某些令人反感（或难闻气味）的地方，这往往表明系统中存在更深层次的问题。

如果你有重复的代码，这可能表明你需要创建一个函数或方法。这个函数可以封装代码，减少重复的错误，或者便于将来更轻松地进行修改。

[illegible]

71

6. Refactoring

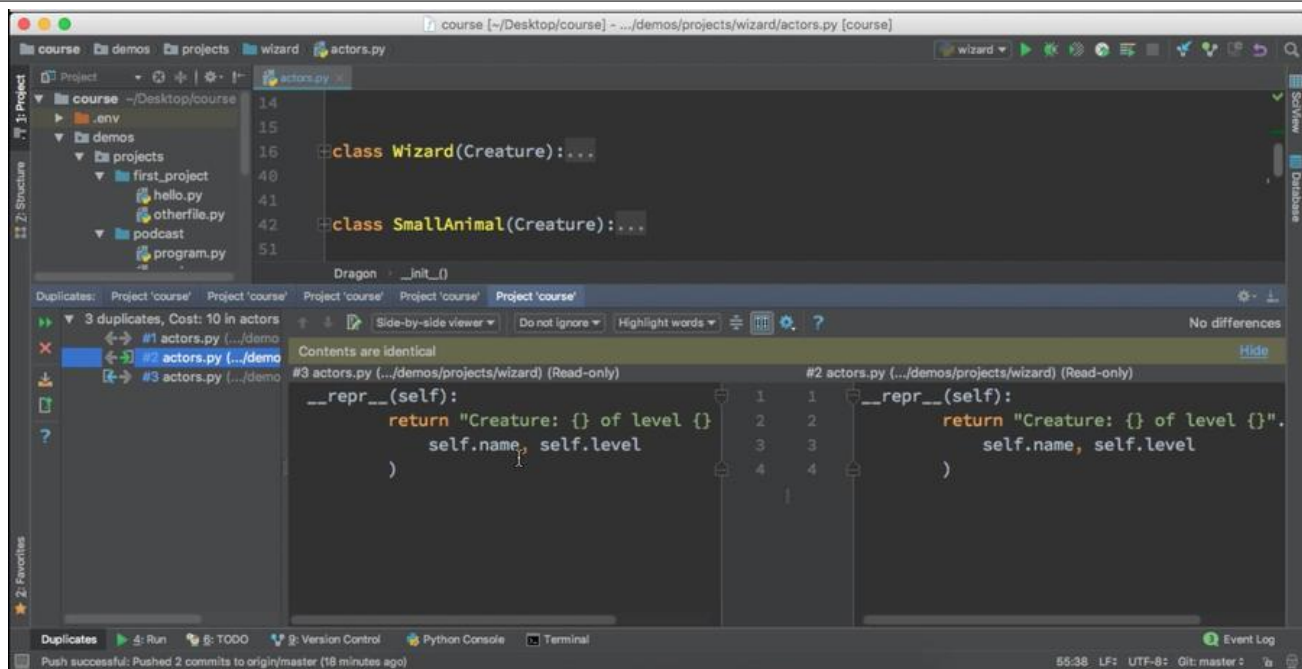


图61：展示PyCharm底部“重复”工具窗口的示意图。

参数过多可能会造成混淆。可能很难记住每个参数的作用。这也可能表明该函数承担的任务过多，或者这些参数应该组合到它们自己的配置类中。在现代Python中，可以使用Python 3.7的数据类功能来实现这一点。对于旧版本，`namedtuples` 是一种替代方案。

当你考虑重构时，要留意这些代码异味。它们能为你何时应该改变代码的非功能方面提供线索。重构是一项投资，未来可能会带来巨大回报。

6.2 查找重复代码

重复代码是指在不同地方重复出现的代码。重构可以解决这个问题。然而，在进行重构之前，你需要先找到重复代码。找到重复代码可能颇具挑战性。有时代码非常相似，但存在细微差异。PyCharm 可以找出这些代码，并让你决定如何处理它们。

PyCharm 有一个“Locate Duplicates”命令（代码->“查找重复项...”），它将查找重复代码。如果运行该命令，会弹出一系列窗口。第一个窗口会询问你是要将其应用于项目、单个文件还是自定义范围。下一个窗口会询问你希望在何种语言中查找重复项。我们将选择 Python，但请记住，PyCharm 也支持其他语言，如果你正在开发 Web 应用程序，这些语言会很有用。我们将选择 Python 作为语言。对于 PyCharm 如何检测重复项还有一些其他选项，但通常你可以使用默认设置。如果你点击“确定”，会看到“重复项工具窗口”出现在 PyCharm 的底部。

在“重复项”工具窗口中，您可以看到PyCharm发现的重复项。左侧是一个包含所有重复项的树状结构。您可以展开这些重复项，并且在以下情况下

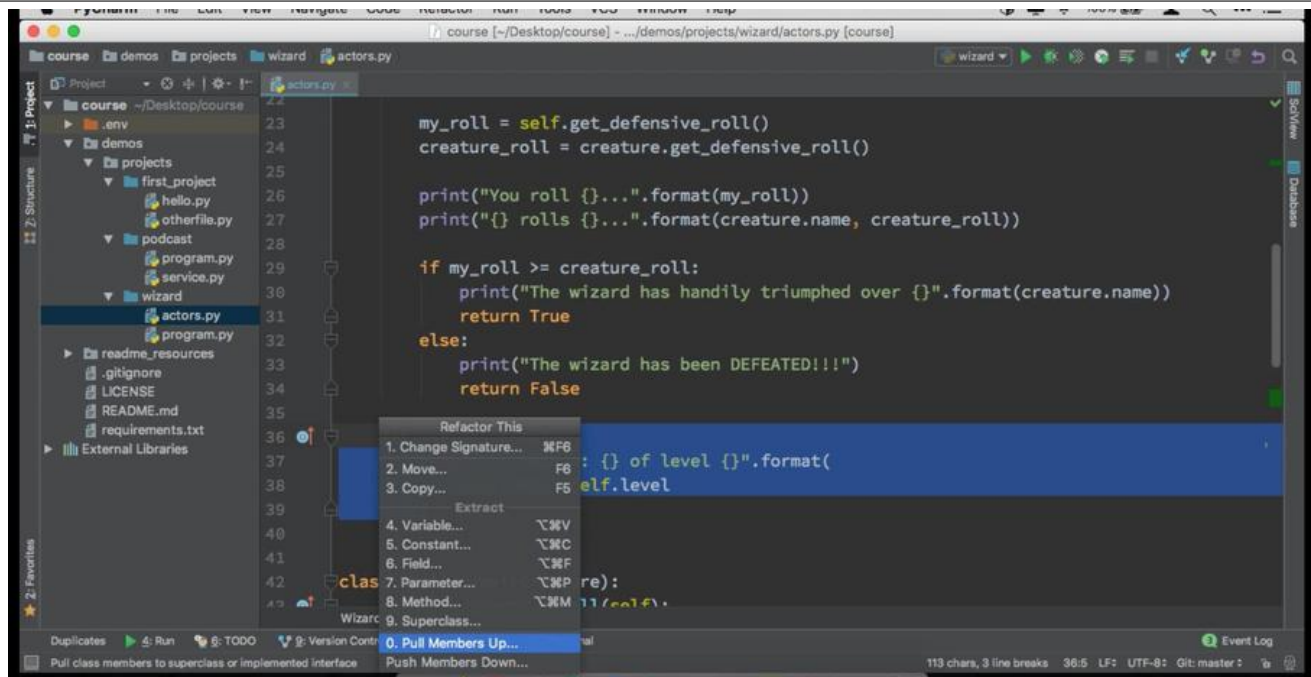


图62：说明运行“重构此项”命令时出现的上下文菜单的图。

如果有两个以上的重复项，您可以点击它们，将其插入到右侧的差异视图中。

以下是一个类层次结构，其中Wizard和Dragon是Creature的子类。这两个子类都有相同的方法。

```
class Creature:
    def __init__(self, name, level):
        self.name = name
        self.level = level

class Wizard(Creature):
    def __repr__(self):
        return f"Creature: {self.name} level: {self.level}"

class Dragon(Creature):
    def __repr__(self):
        return f"Creature: {self.name} level: {self.level}"
```

如果你遇到与以下代码类似的情况，你可能希望将__repr__方法重构到Creature的基类中。要做到这一点，选择你想要重构的方法。这种重构称为“向上提取”，因为我们正在将一个方法从子类移动到父类。我们会在Wizard中选择__repr__方法，然后使用“Refactor This”命令（command-alt-shift-t或shift-ctrl-alt-t）调出重构弹出窗口。选择“Pull Members Up...”以开始重构。

将会出现一个窗口，您可以选择要将该方法移动到的类。在这种情况下，选择Creature。点击“重构”，您的代码应该如下所示：

```
class Creature:
    def __init__(self, name, level):
        self.name = name
        self.level = level
```

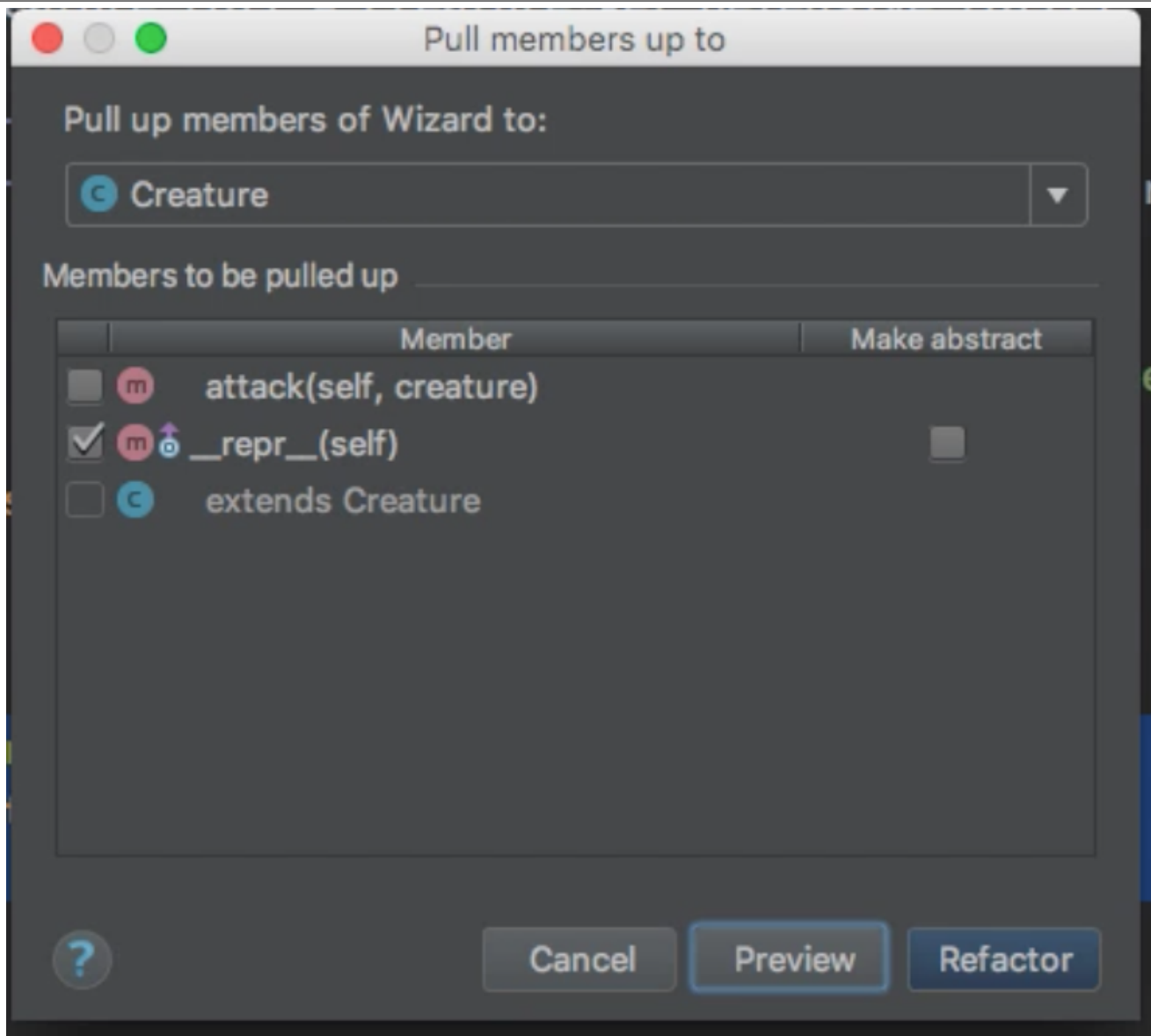


图63：说明“向上拉动成员至”窗口的示意图。您可以选择要上移的方法以及希望它们移至的类。

```
def __repr__(self):
    return f"Creature: {self.name} level: {self.level}"

class Wizard(Creature):
    pass

class Dragon(Creature):
    def __repr__(self):
        return f"Creature: {self.name} level: {self.level}"
```

请注意，Dragon 仍然有 `__repr__` 方法。最简单的做法是删除它。PyCharm 还不够智能，不知道在重构时应该删除它。如果你

在删除Dragon的__repr__方法后重新运行查找重复项命令，你应该会看到这个重复项现已消失。

6.3 重命名项目

另一种常见的重构是重命名符号（变量、方法、类型）。这种重构的一个主要用途是使代码越来越具有自解释性。考虑一个具有3D元素的游戏，该游戏使用边界框算法进行碰撞检测。如果你看到这段代码，在没有注释的情况下，你能确定它的功能吗？

```
# compute the bounding box of this shape
shape.compute()
```

我不太确定。不过，如果我们把compute更明确地重命名为它所做的事情，你马上就会知道，而且可以去掉注释：

```
shape.compute_bounding_box()
```

如果没有找出所有的使用情况，这些重命名操作可能会让人担心。但对于自动化重构来说，这不是问题。

回到我们的龙与地下城游戏：如果在冒险游戏中有一个名为player的变量，而你想将其名称更改为hero，可以通过搜索和替换来实现。这可能行得通，但如果你还有另一个名为player_2的变量或一个名为attack_player的方法，它们可能会被错误地重命名为hero_2和attack_hero。查找和替换也可能会遗漏不同文件中的重命名。即使你精确匹配了“player”，但它可能是一个不同的符号，名称相同但来自不同的模块。

“Rename...”命令（shift - F6）将以安全的方式执行重命名操作。你可以选择想要重命名的变量并运行此命令。如果你不想记住所有重构快捷键，也可以通过“重构此项”命令访问重构命令。

重构变量时，您只需输入新名称，PyCharm 便会将其插入到正确的位置。

如果你想重命名一个方法，比如用fight替换attack方法，突出显示该方法的一个实例（可以是调用或方法本身），然后对其运行“重命名...”命令。你可以告诉PyCharm在注释和字符串中包含重命名。

底部将出现一个“重构预览”窗口。您可以查看找到引用的位置，并指示 PyCharm 执行或取消重构。

你也可以对文件应用“重命名...”。PyCharm 会确保导入重命名文件的任何文件都将得到更新。此外，VCS 集成将让你更轻松地跟踪文件重命名。当然，如果你意识到自己犯了严重错误，始终可以“Undo”（按 Ctrl+Z 或 Command+Z），PyCharm 将以单元形式撤销更改（包括文件名更改）。

下次你想在使用查找和替换之前重命名某个内容时，考虑我使用更安全、更智能的“重命名...”命令。

6.4 引入变量

让我们看看代码中另一个常见问题，*magic numbers*。魔法数字是内联的字面数字，这些数字对代码作者有特定含义，但之后对代码读者（甚至作者本人）来说可能并不明确。下面是一个包含魔法数字的方法示例：

6. Refactoring

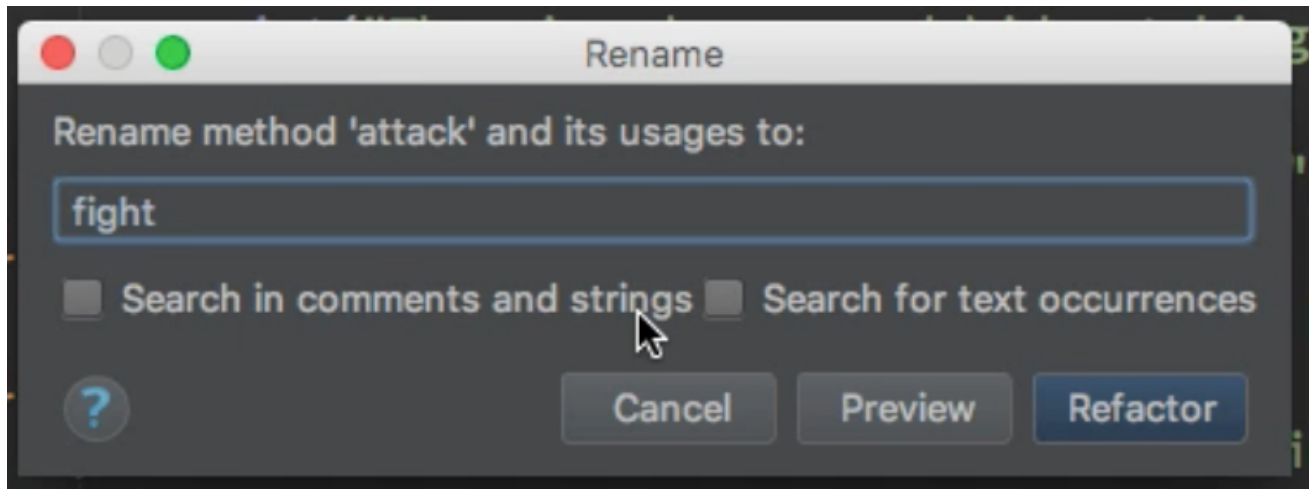


图64：说明重命名方法选项的图。

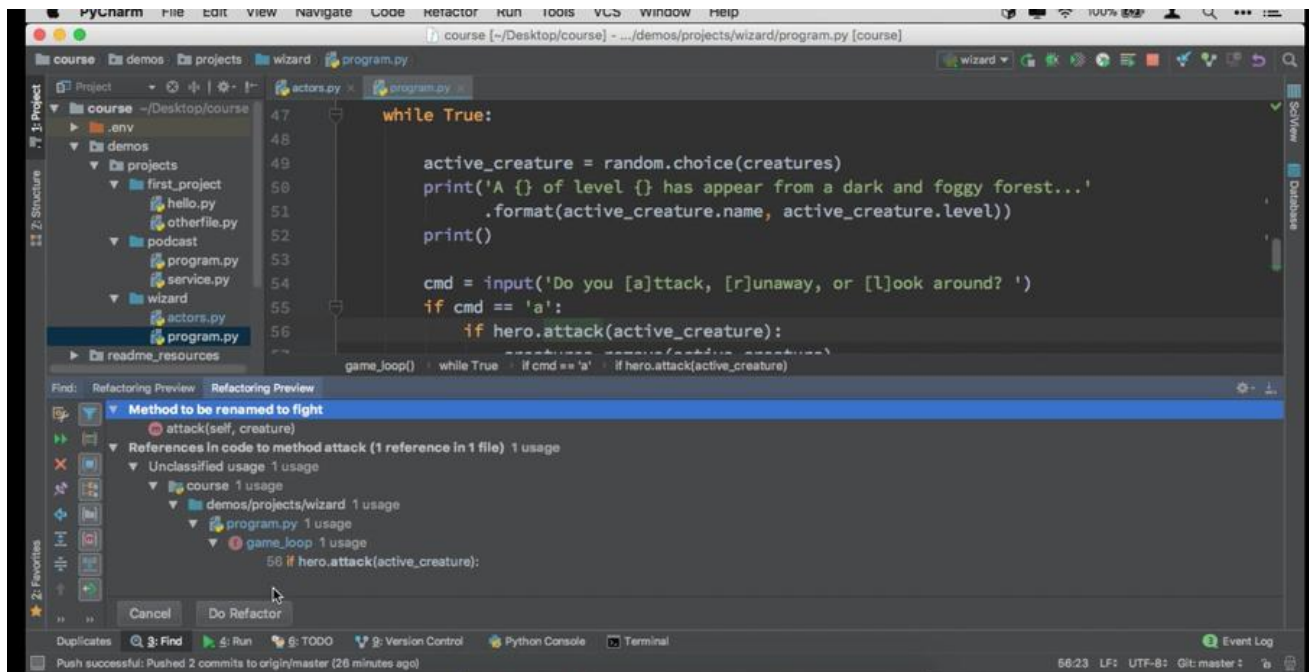


图65：说明重构预览的图。在此处，您可以看到在批准重构之前引用的位置。


```
def get_defensive_roll(self):
    return 3 * random.randint(1, 12) * self.level
```

内联的字面值有3、1和12。3是什么意思。大概来说，其他数字用于在1到12（含）之间随机选择一个数字（注意，`randint`不像`range`那样遵循半开区间，并且将12也作为一个选项）。这可能会让之后查看代码的同事感到困惑。

调试这段代码的另一个问题是，我们无法设置断点并检查随机数本身。我们只能在`return`语句上设置断点，然后逆向推导随机数是什么。

将随机数提取到其自身变量中的重构操作称为“*Extract Variable...*”（（在Mac上为`command-alt-v`，在Windows上为`ctrl-alt-v`））。如果突出显示调用`randint`函数的表达式，运行“提取变量...”命令，并将其命名为`roll`，您的代码将如下所示：

```
def get_defensive_roll(self):
    roll = random.randint(1, 12)
    return 3 * roll * self.level
```

在那次重构之后，你可以在该方法中设置一个断点，并检查`roll`的值。

注意

还有一种“`command-alt-n`”或“`ctrl-alt-n`”重构方式，它会提取一个变量并内联其值。这与“提取变量”相反。

3数值在此处是防御检定的增强修正值。如果我们想让这个值成为可配置的值，可以使用命令“*Extract Parameter...*”（`command-alt-p`，或者`ctrl-alt-p`）。你可以选中它并使用参数名称为`modifier`的命令来得到：

```
def get_defensive_roll(self, modifier=3):
    roll = random.randint(1, 12)
    return modifier * roll * self.level
```

我们重构了代码，使其意图更加清晰。我们还增加了更改`modifier`的功能，但默认值保持不变，因此该方法的行为与现有代码兼容。本节中我们研究的方法相对简单。不过，你也可以将这些相同的思路应用于更大、更复杂的代码。

6.5 创建常量

PyCharm也可以“*Extract Constant...*”（使用`Command + Option + C`或`Ctrl + Alt + C`）。Python语言没有常量的概念。但是我们可以采用一种约定，使用全大写变量来创建全局变量。

假设你要从我的应用程序向终端打印一个ASCII艺术横幅。你可以使用这样的代码，即在`print`函数中直接嵌入一个字符串：

```
print('''
  _ _ _ _ _
 | '_ \ | | | | _ | | | | _ _ _ _ _
 | |_) | | | | | | | | | | | | | |
 | . _/ \_, | \_ | | | | \_ / | | | | is cool!
 | | | | | | | | | | | | | | | | |
''')
```


6. Refactoring

如果你想在其他地方重复使用该横幅，可以复制并粘贴代码。或者，你可以选择该字符串并运行“提取常量”。如果你将其重命名为PYTHON_TEXT，你将得到：

```
PYTHON_TEXT = ''' _ _ _ _ | | | | _ _ _ _ _ _ _ _  
| '_ \ | | | | _ _ | '_ \ / _ \ | '_ \  
| |_) | | | | | | | ( ) | | | |  
| . _ / \ _ , | \ _ | | | \ _ _ / | | | is cool!  
|_| | _ _ /'''  
  
print(PYTHON_TEXT)
```

要在其他区域使用该文本，请引用 PYTHON_TEXT 变量。

6.6 移动代码

你还可以使用“Move” (F6)将代码移动到另一个模块。要应用此重构，请选择要移动的代码并运行“移动”命令。

假设你有一个包含所有代码的单一模块。如果你有一个处理命令行参数并运行命令行脚本的主函数，你可以将该方法重构到一个main.py文件中。为简洁起见，我们假设代码结构如下：

```
# process.py  
import argparse  
  
class Processor:  
  
    def process_data(self):  
        # compute stats  
  
def main():  
    p = argparse.ArgumentParser()  
    p.add_argument('-f', '--file')  
    opts = p.parse_args()  
    if opts.file:  
        proc = Processor()  
        proc.process_data()
```

在选择main函数并使用名为main.py的文件运行“移动”重构后，原始文件将如下所示：

```
# process.py  
class Processor:  
  
    def process_data(self):  
        # compute stats  
  
而main.py看起来像：  
  
import argparse  
  
from process import Processor  
  
def main():  
    p = argparse.ArgumentParser()
```

```
p.add_argument('-f', '--file')
opts = p.parse_args()
if opts.file:
    proc = Processor()
    proc.process_data()
```

这使您能够创建特定于某些任务的专用模块。您始终可以利用运行代码或执行测试的功能（正如我们将在后面的章节中看到的），来验证代码是否仍然有效。

PyCharm还支持将模块移动到包中，反之亦然。模块是一个以.py结尾的文件。包是一个目录，其中包含一个名为__init__.py的文件。如果某个模块变得越来越大，而您又想保留命名空间，则可以将其重构为一个包。该模块的实现将放在__init__.py文件中，但您以后可以向其中添加其他模块。

6.7 总结

在本章中，我们讨论了许多重构方法。这并非全面的讨论（因为有专门论述重构的书籍），但它应能激发你改进代码的兴趣。利用PyCharm中的功能查找代码中的重复部分，并通过重构加以清理。下一步可以研究常见的代码坏味道及其解决方法。PyCharm应能让清理代码变得容易得多，而且是以稳健的方式进行。

6.8 命令

- *"Locate Duplicates"* - （代码 -> “查找重复项...”）
- *"Refactor This"* - （Command + Option + Shift + T 或 Shift + Ctrl + Alt + T）
- *"Pull Members Up..."*
- *"Rename..."* - （按Shift+F6）
- *"Extract Variable..."* - （Command + Option + V 或 Ctrl + Alt + V）
- *"Inline"* - （Command + Option + N 或 Ctrl + Alt + N）
- *"Extract Constant..."* - （Command + Option + C 或 Ctrl + Alt + C）
- *"Move"* - （F6）

6.9 练习

1. 研究维基百科上的代码异味。看看你能否在自己的代码中发现其中任何一种。
2. 研究重构。有许多书籍和文章深入探讨了主题。看看你能否在自己的代码中找到应用它们的地方。PyCharm应该会让这件事变得更容易。

6. Refactoring

3. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/4-refactoring` 中关于新项目的说明，尝试找出一些代码异味并使用 PyCharm 修复它们。

第7章

数据库

7.1 引言

许多创建或读取数据的应用程序都使用数据库来简化工作。PyCharm 拥有强大的数据库支持，并且可以与几乎任何 SQL 数据库进行交互。

许多公司甚至设有专门的岗位来支持和维护数据库。这可能会让开发者觉得数据库是别人的领域。此外，使用数据库的工具可能很难掌握、不熟悉，或者会打断你的工作流程。而且，大多数数据库都有不同的工具，因此学习每种工具都是一件苦差事。支持和创建迁移以及处理对象关系不匹配问题也很麻烦。

PyCharm 缓解了其中一些问题。它具备集成的数据库支持。PyCharm 在不同数据库平台上也使用相同的用户界面。你只需花时间学习这一个工具即可。

本章大部分内容将聚焦于DataGrip，它随PyCharm专业版一同提供，但也是一款独立产品。

7.2 简单数据库

让我们使用SQLite创建一个简单的数据库。大多数数据库需要运行一个单独的服务器并进行大量配置。SQLite非常适合初学者，因为它仅使用一个文件作为数据库，并在我们的Python进程中运行。此外，SQLite已包含在Python中，因此无需安装任何内容。它将作为一个简单的存储来保存运行日志。本章展示的原理和工具适用于DataGrip支持的任何数据源。

创建一个文件，dbexample.py，其内容如下：

```
import sqlite3

conn = sqlite3.connect('runs.db')
with conn:
    c = conn.cursor()
    c.execute('CREATE TABLE Person(id INTEGER PRIMARY KEY, name TEXT)')
    c.execute('CREATE TABLE Location(id INTEGER PRIMARY KEY, place TEXT)')
    c.execute('''CREATE TABLE Run(id INTEGER PRIMARY KEY,
                                pid INTEGER,
                                lid INTEGER,
                                distance NUMBER,
                                FOREIGN KEY(pid) REFERENCES Person(id),
                                FOREIGN KEY(lid) REFERENCES Location(id))''')
```

7. Databases

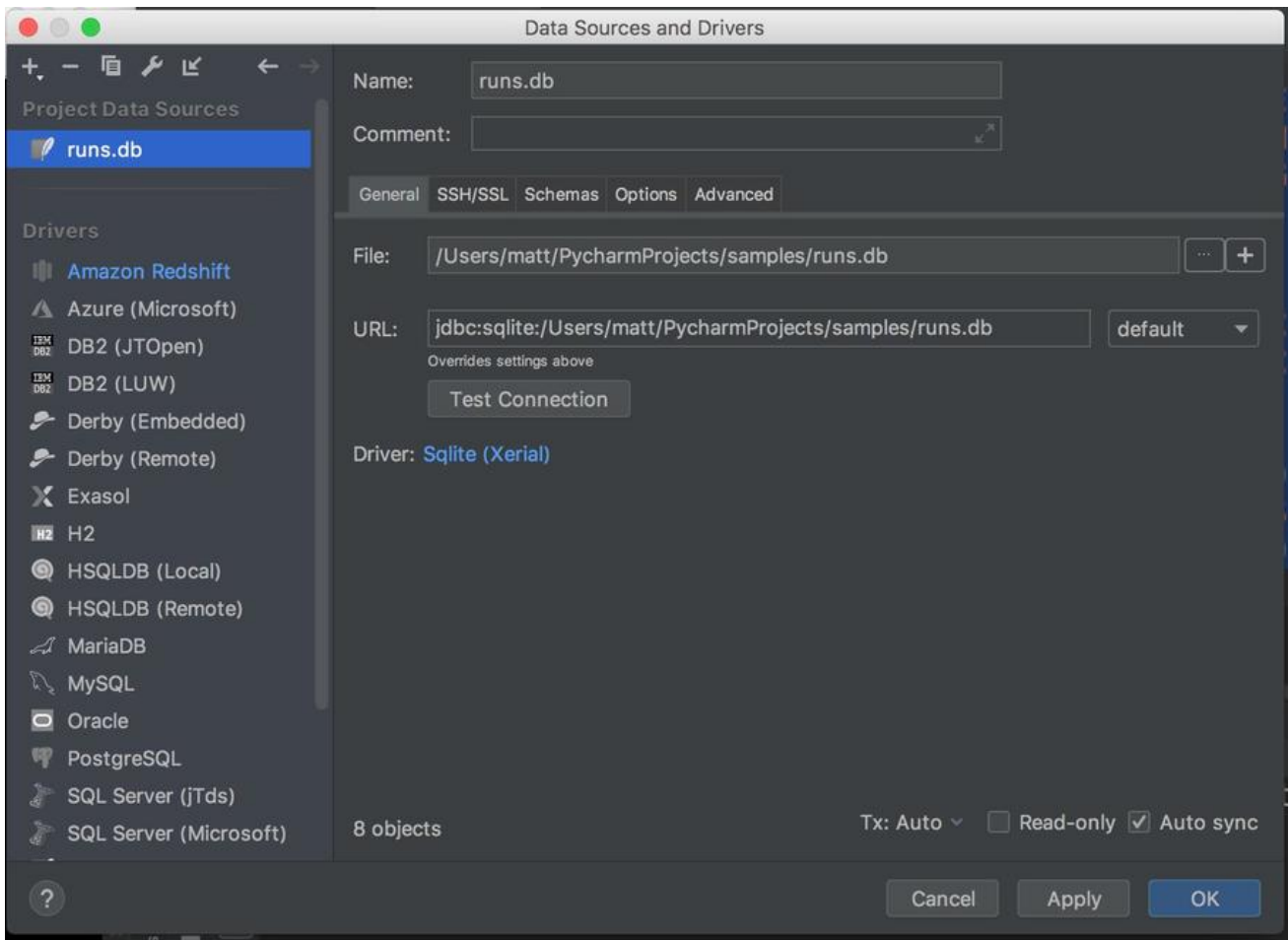


图71：说明为SQLite配置新数据源的图。

```
)  
'')  
c.execute("INSERT INTO Person VALUES(0, 'Matt')")  
c.execute("INSERT INTO Location VALUES(0, 'Squaw Peak')")  
c.execute("INSERT INTO Run VALUES(0, 0, 0, 52)")  
c.execute("INSERT INTO Run VALUES(1, 0, 0, 50)")
```

如果你从PyCharm运行此代码，它应该可以顺利执行。此代码会在你的工作目录（通常在dbexample.py文件旁边）的文件系统中创建一个文件runs.db。要让你的项目将其识别为数据源，在“项目”视图中右键单击项目，然后选择“新建”->“数据源”。它会要求输入数据源的“路径”，你可以将其指向runs.db的位置。此时，“驱动程序”选项应自动填充为“Sqlite (Xerial)”。如果你点击“确定”，会弹出一个新窗口来配置数据源。

配置窗口上的设置应该没问题。但是，如果您在驱动程序文件旁边看到红色字体，这表明您需要下载驱动程序。点击蓝色的“下载”链接，PyCharm 将处理其余事宜。此时点击“确定”，您会注意到编辑器最右侧有一个“数据库”选项卡。

数据库视图有一个树形视图，其中包含数据源的名称，在我们的例子中是runs.db。如果展开“模式”和“主项”，您将看到我们创建的所有表：Location、Person 和 Run。如果展开其中一个表，您将看到各列及其相应的数据类型。

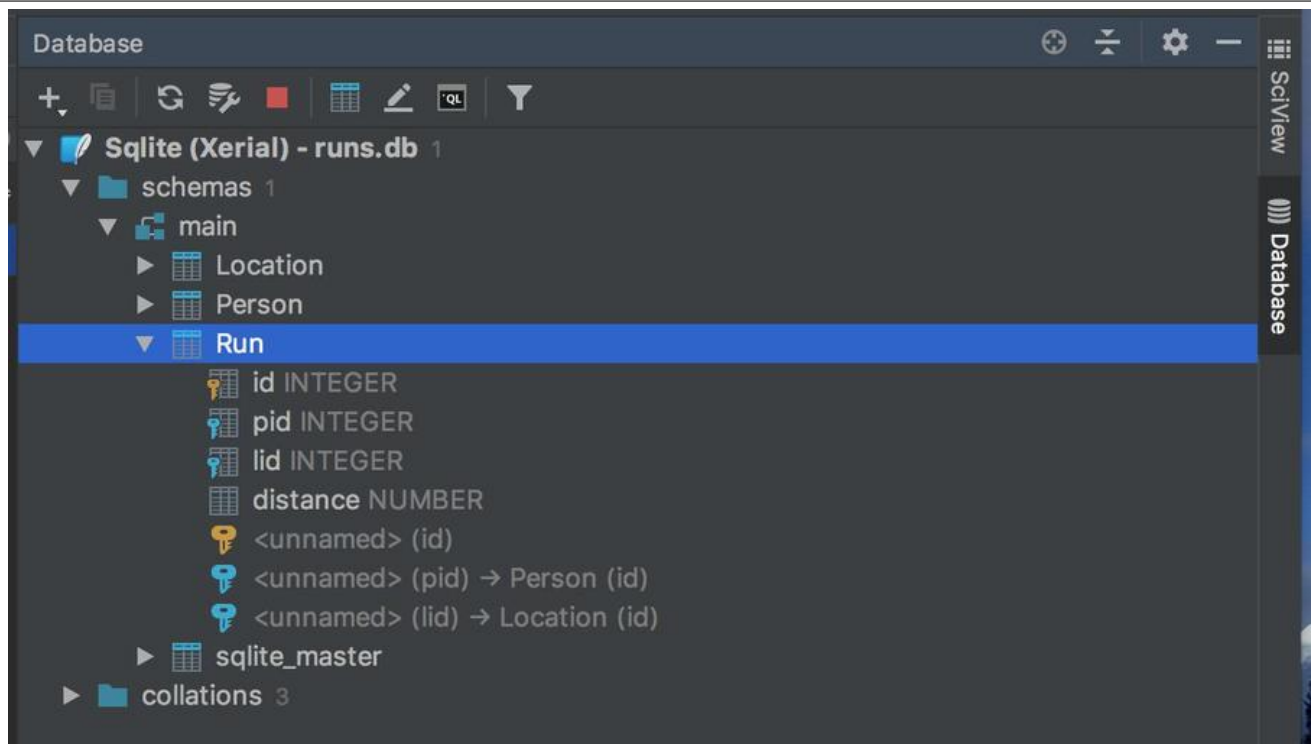


图72: 说明数据源视图的图。请注意, 您可以展开它以查看列类型。

[illegible]

提示

你也可以将SQLite文件拖放到“数据库”选项卡上，PyCharm会为你进行配置！

7.3 数据库关系图

我们的模式由三个表和几个列组成。你可能会处理大得多的表。阅读表的DDL（数据定义语言）是理解这些表之间关系的一种方法。通常，将这些关系可视化会更容易。DataGrip可以帮上忙。

“显示可视化效果”命令（右键单击数据源“图表”->“显示可视化效果”，Command + Shift + Alt + U 或 Ctrl + Alt + Shift + U）将显示表、列及其关系。在此视图顶部有隐藏列或键、放大或缩小以及保存可视化效果的选项。

如果你想要一个浮动窗口，“显示可视化效果”有一个同级选项“Show V “可视化弹出窗口”，它将创建一个包含可视化内容的弹出窗口。

最后一种可能一开始不太直观的方法是仅查看特定的表格。可视化是交互式的，你可以相对移动各个表格的位置。你还可以通过选中表格并按“Delete”键将其从视图中删除。或者，你也可以只选择特定的表格。

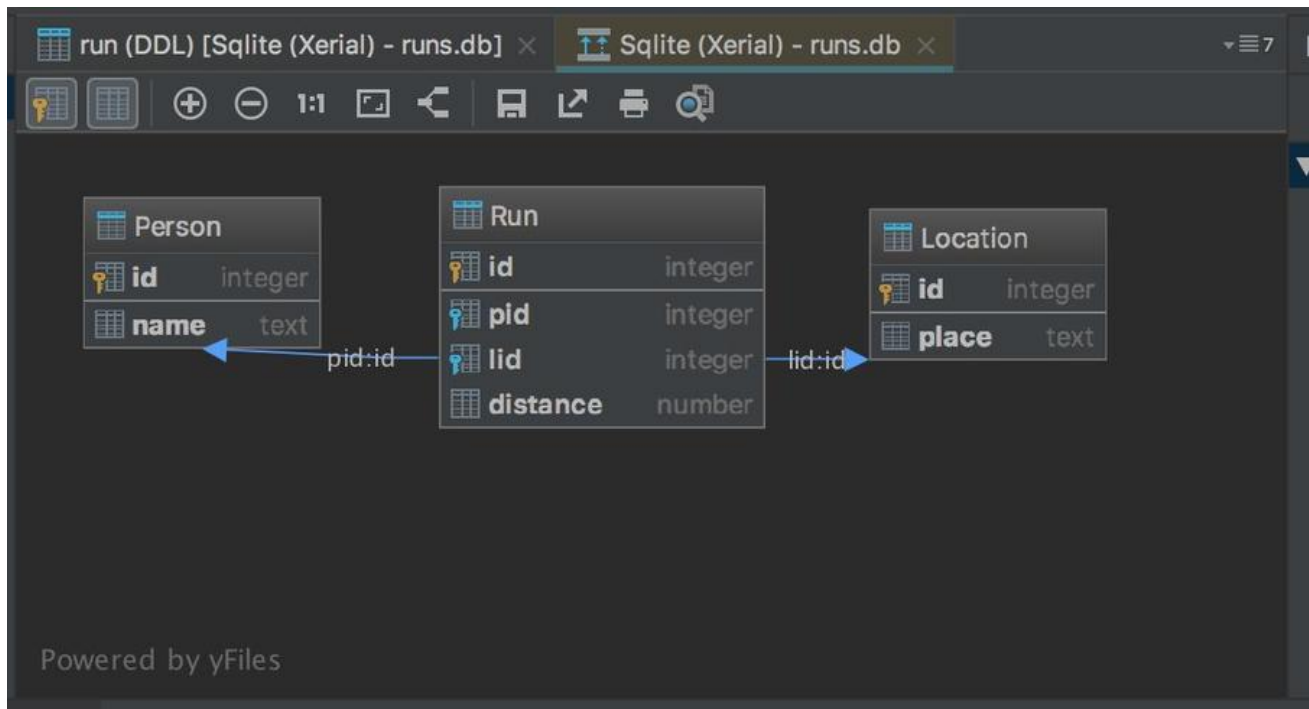


图73: 展示“显示可视化效果”命令结果的示意图。

你想要包含在数据源视图中，然后展示可视化效果，该可视化效果将限于所选的表。

7.4 查询数据

我们已经配置好了一个数据源。我们还有一个可视化工具来查看表中的关系。让我们来看看如何查询数据。查看数据最简单的方法是双击一个表。如果您这样做，DataGrip 将显示该表的前 501 行数据。

控制台是一个编辑器，您可以在其中输入SQL并执行它。由于我们已经配置了数据源，它知道表和列的信息，并且我们有代码补全功能。

如果你精通SQL，并且想使用结构化查询语言提取数据，可以运行“*Open Console*”命令（右键单击数据源或按F4）。我们输入以下查询：

```
SELECT r.id, l.place, p.name, r.distance
FROM Run AS r
      LEFT JOIN Person AS p on p.id = r.pid
      LEFT JOIN Location AS l on l.id = r.lid;
```

虽然DataGrip有代码补全功能，但如果你开始编写SELECT l.并等待帮助，那你得等上很长时间。在这种情况下，DataGrip不知道别名l指的是什么。即使你完整输入了SELECT Run., DataGrip也不会弹出选择Run表列的选项。骗过DataGrip的一个小技巧是先写上SELECT * FROM Run as r。此时，DataGrip就会知道r的别名是什么，并会针对该名称提供列补全。

如果您“*Execute*”（按命令+回车键或Ctrl+回车键）执行查询，将会出现一个包含结果的数据网格。显示结果的数据网格是实时的。因此，您可以通过点击列标题对表格进行排序。如果您想按距离排序，可以点击该列。

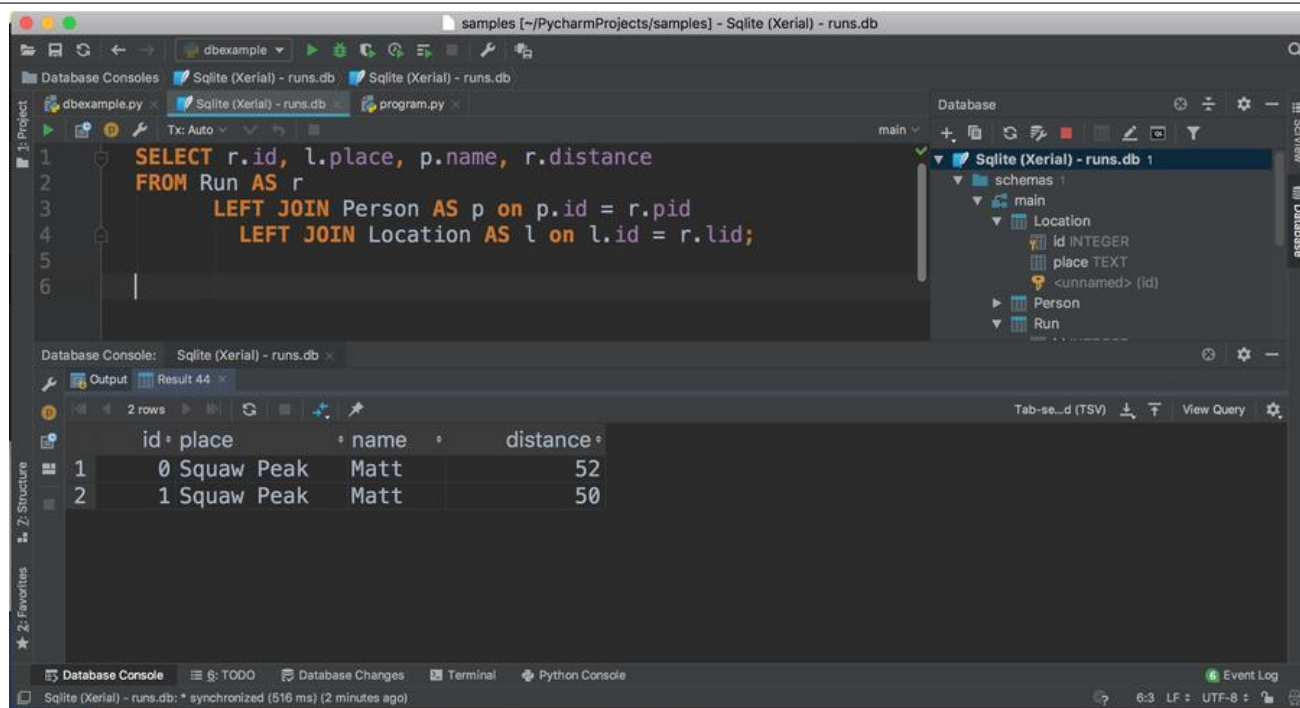


图74：说明从控制台执行查询的示意图

如果您直接查看表格（通过在数据源视图中双击它，或执行 `SELECT * FROM Runs` 操作），您还可以编辑数据。您可以双击某个字段来更改其值。更改值后，不会自动将其推送回数据库。您需要运行“Submit”命令（点击指向数据库的绿色箭头按钮，或按 `command + enter` 组合键，或按 `ctrl + enter` 组合键）。

您的结果视图还能够将结果导出为 TSV、CSV、JSON 格式，或导出到剪贴板。此外，您可以查看表的 DDL。如果您没有创建该表，或者没有创建该表的代码，这可能会很有用。您可以使用 DDL 和数据转储，根据自己的需要制作一份本地副本。

7.5 修改模式

从数据源视图中，我们可以更改数据库的结构。我们可以进行一些简单的重构，例如重命名数据库列。要进行更强大的更改，请运行“Modify Column...”（命令-F6 或 `ctrl-F6`）命令。这将调出“Modify Table”视图。在此处，您可以更改列的名称、类型、默认值和唯一性属性。您可以添加列、键和索引。DataGrip 可以为您运行这些命令，或者在控制台中为您提供相应的 SQL 供您运行。

更改数据库后，记得运行“Synchronize”命令（刷新按钮或 `command-alt-y` 组合键，或 `ctrl-alt-y` 组合键）。这将更新数据源视图，使其反映出新的更改。

7.6 编码的乐趣

语言注入是指PyCharm能够将字符串字面量视为包含另一种语言。当您点击一个字符串时，灯泡图标会出现，您可以选择“注入语言或引用”来告诉PyCharm嵌入的是什么语言。PyCharm还足够智能，当我们输入了足够多的SQL语句，使得一个字符串明显是SQL语句时，它会自动为我们进行转换。

PyCharm 会将其视为普通字符串。你可以点击灯泡图标并注入一种语言，或者多输入一些 SQL。然后 PyCharm 会自动将该字符串转换为 SQL。一旦我输入了：

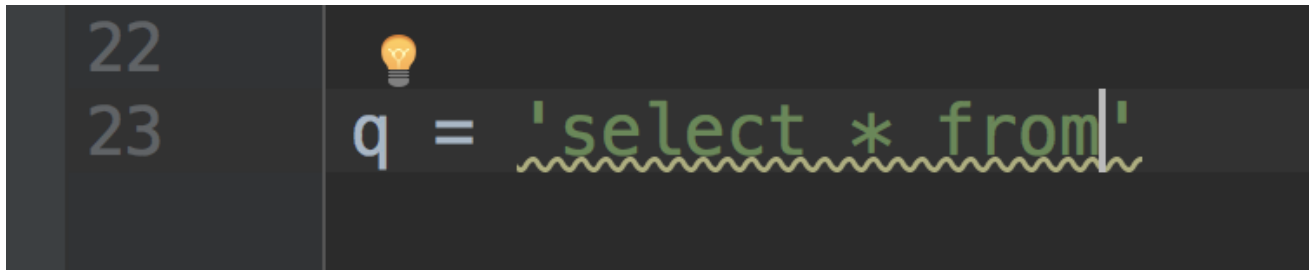


图76: 说明在字符串中输入SQL的示意图。当您开始输入时, 它会被视为普通字符串。如果您想向PyCharm提示该字符串包含SQL, 可以点击灯泡图标并进行语言注入。

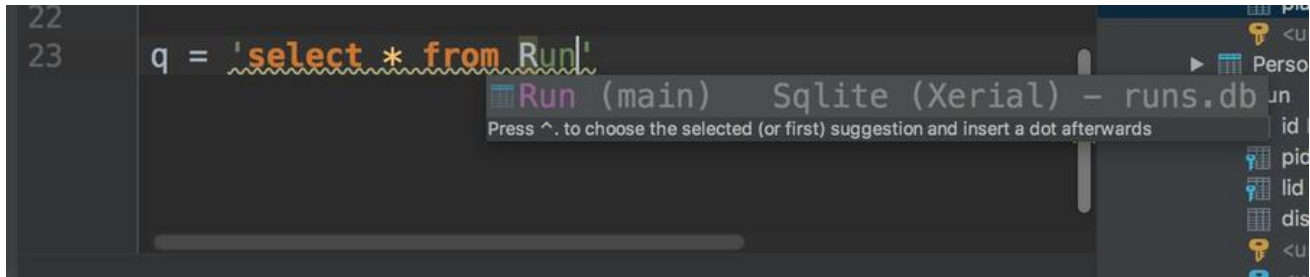


图77: 该图展示了如果我们配置了数据源并输入足够多的SQL, PyCharm将自动注入SQL语言并为我们提供代码补全功能。

```
q = 'select * from Run.'
```

PyCharm 会为您调出代码补全选项。它会补全您模式中独有的表名和列名。当您修改模式时, PyCharm 会修改 Python 中注入语言的 SQL 字符串中的符号。如我们所见, PyCharm 对符号的理解赋予了它强大的功能。您甚至还可以导航到符号。您还可以注入许多其他语言, 但 SQL 补全功能确实是 PyCharm 的一项出色功能。

7.7 总结

在本章中, 我们探讨了集成到PyCharm中的DataGrip工具。这是一套功能强大的端到端工具集, 可用于与大多数SQL数据库进行交互。DataGrip的优势之一在于, 一旦掌握了该工具, 你就可以将其用于任何其他受支持的数据库。我们还讨论了语言注入。此功能使我们能够在原生Python字符串中实现代码补全。PyCharm专业版中的数据库工具是选择该平台的一个极具吸引力的理由。

7.8 命令

- “显示可视化” - (右键单击数据源“图表” -> “显示可视化”, 按下 `command + shift + alt + u` 组合键, 或按下 `ctrl + alt + shift + u` 组合键)
- “Open Console” - (右键单击数据源或按 F4)
- “Execute” - (Command + Enter 或 Ctrl + Enter)

7. 数据库

- *"Submit"* - （点击指向数据库的绿色箭头按钮，或按 `command + enter` 组合键，或按 `ctrl + enter` 组合键）。
- *"Modify Column..."* - （按 `F6` 键或 `Ctrl + F6` 组合键）
- *"Synchronize"* - （刷新按钮或 `command - alt - y` 或 `ctrl - alt - y`）

7.9 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/5-databases` 中新项目的说明，尝试使用 DataGrip 打开现有数据库。
2. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/5-databases` 中新项目的说明，尝试从 DataGrip 运行查询。
3. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/5-databases` 中新项目的说明，尝试查看数据库表的图表。
4. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/5-databases` 中新项目的说明，尝试语言注入。

第8章

服务器端Python Web应用程序

8.1 服务器端 Web 应用程序简介

Python是目前非常流行的一种语言。在其他语言似乎停滞不前或受欢迎程度正在下降的情况下，Python的受欢迎程度却持续上升。其中一个原因是，Python是创建服务器端Web应用程序的绝佳选择。像Instagram、Pinterest、Dropbox、YouTube和Yelp这样的网站都是用Python实现的。

PyCharm 让服务器端项目的开发变得容易得多，因为它能理解整个项目（Python 代码、JavaScript、CSS 等等）。此外，它还支持流行的 Python 网络框架，如 Flask、Django、Pyramid 和 Google App Engine。PyCharm 可以启动服务器、运行外部工具等等。

Web开发的另一个方面是前端开发。本章将重点介绍后端支持，后续章节将探讨PyCharm在前端开发方面的一些特性。

PyCharm具备许多后端开发功能。你可以创建新项目或加载现有项目。PyCharm了解Python中常见的模板语言，因此你可以在动态HTML模板（如Jinja2和Chameleon模板）中实现代码补全。PyCharm与框架管理工具集成，因此它对Django上的`manage.py`和Pyramid上的`setup.py`都有集成。PyCharm支持资源目录，CSS、JavaScript和图像等静态文件可存放于此。你将获得资源目录中路径的代码补全以及CSS和JavaScript补全支持。你可以导航到模板中的符号。你甚至可以使用某些模板语言的调试器。最后，你还可以配置外部工具。如果你需要运行Gulp或Webpack，PyCharm也有针对它们的集成。

许多Web开发人员选择Python作为实现语言。PyCharm对创建和维护Web项目的开发人员提供了出色的支持。简而言之，PyCharm是开发基于Python的Web应用程序的利器。

8.2 创建服务器端项目

让我们从创建一个新项目开始。我们将为本章中的示例创建一个 Pyramid 项目。某些功能可能会略有不同，比如使用哪种模板语言或路由选项，但总体思路适用于 PyCharm 支持的所有框架。

要继续操作，请运行 “*New Project...*” 命令。在 “新建项目” 窗口中，选择 “Pyramid” 项目。你可以利用 PyCharm 的虚拟环境支持来创建一个

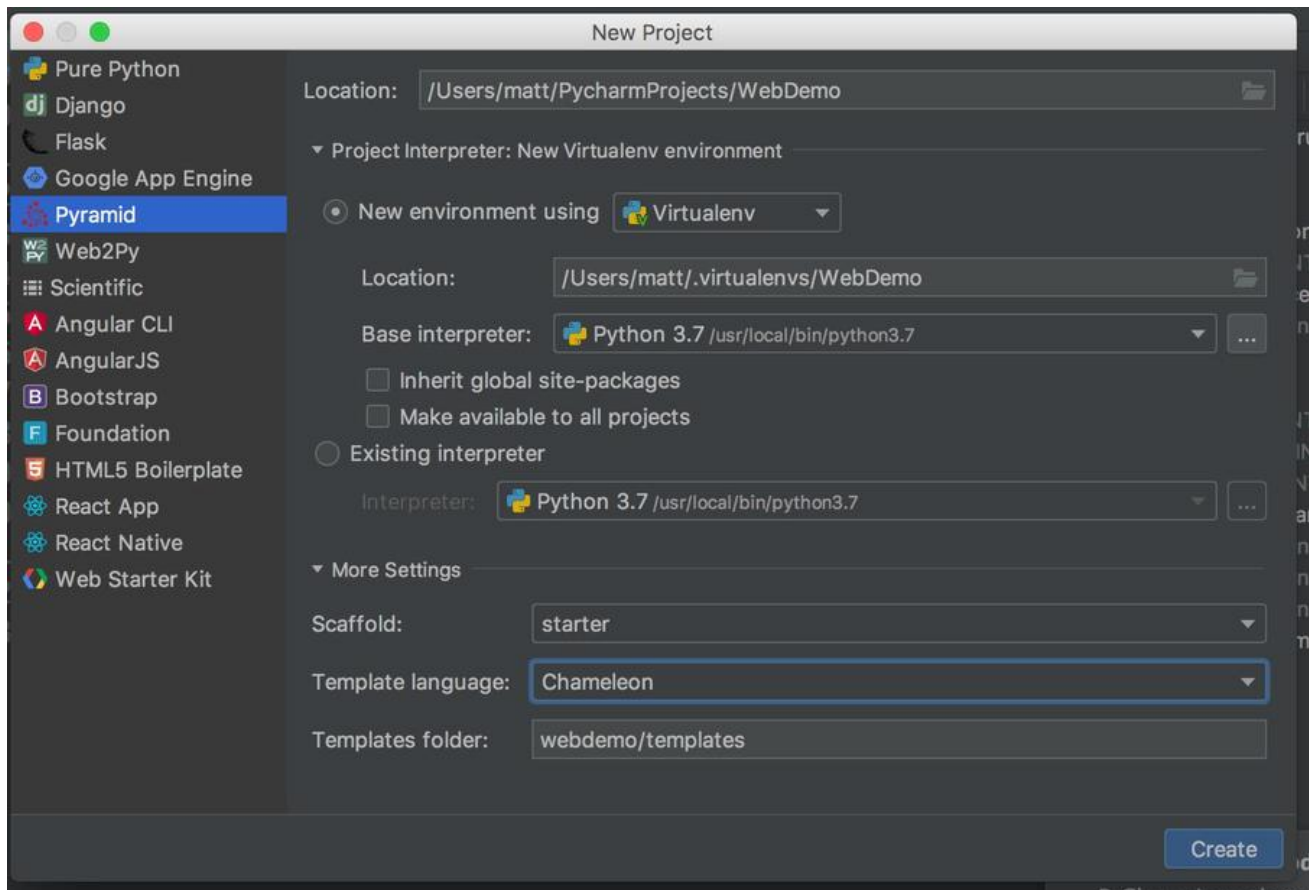


图81：说明创建新的Pyramid项目的图。

新的虚拟环境。在窗口底部的“更多设置”区域，您会看到“脚手架：”、“模板语言：”和“模板文件夹：”的选项。脚手架是 Pyramid 项目的内部项目布局。“starter”的默认值就可以。在本章中，我们将使用“Chameleon”模板语言，而不是默认的“Jinja2”。点击“创建”按钮，PyCharm 将为您生成一个项目。

创建项目后，PyCharm 可能会生成一些关于缺少软件包以及项目未安装用于开发的警告。你可以通过点击底部的“终端”并运行 `pip list` 来查看 PyCharm 为你安装了什么。我们需要安装这些依赖项，因此点击“Install requirements”（安装需求）。

你还需要运行 `python setup.py develop`。此命令会将我们的项目安装为可编辑包。从我们的虚拟环境导入时会找到项目源代码，但如果我们更改任何代码，无需重新安装。我们可以在终端中操作，点击“Run setup.py develop”，或者选择“Tools” -> “Run setup.py Task...” 并选择“develop”。

此时，您点击“Run (WebDemo)” (shift-F10)。PyCharm底部会出现一个“运行”控制台，其中包含指向开发服务器的链接。点击该链接以查看您的Web项目。如果看到错误页面，则需要打开 `webdemo/templates/layout.pt` 并将 `WebDemo:` 替换为 `webdemo:`。

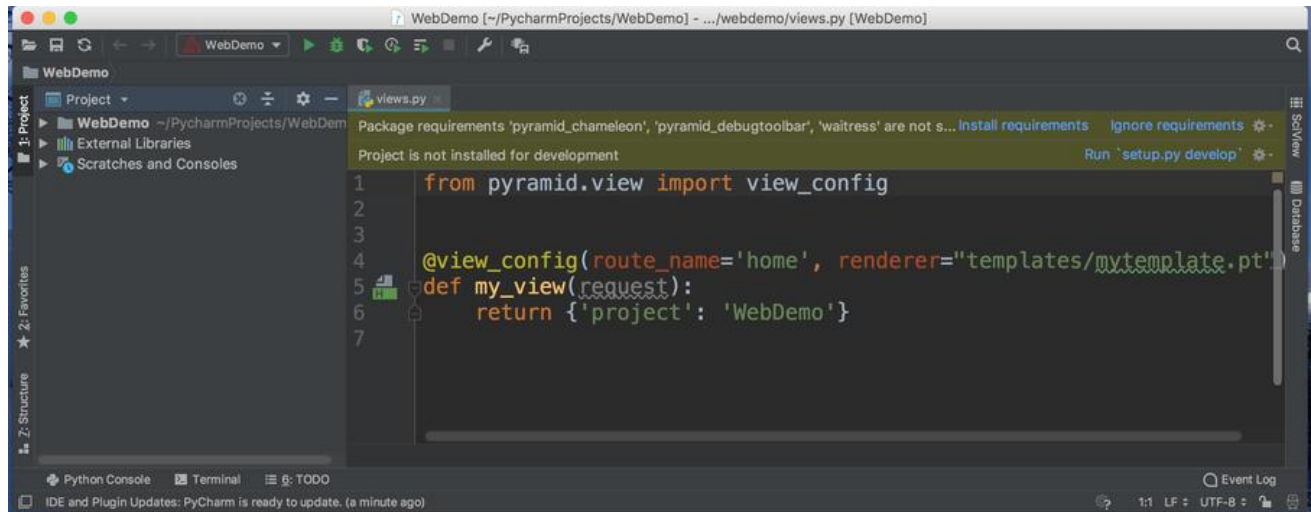


图82：生成项目的示意图。请注意，PyCharm生成了一些警告。

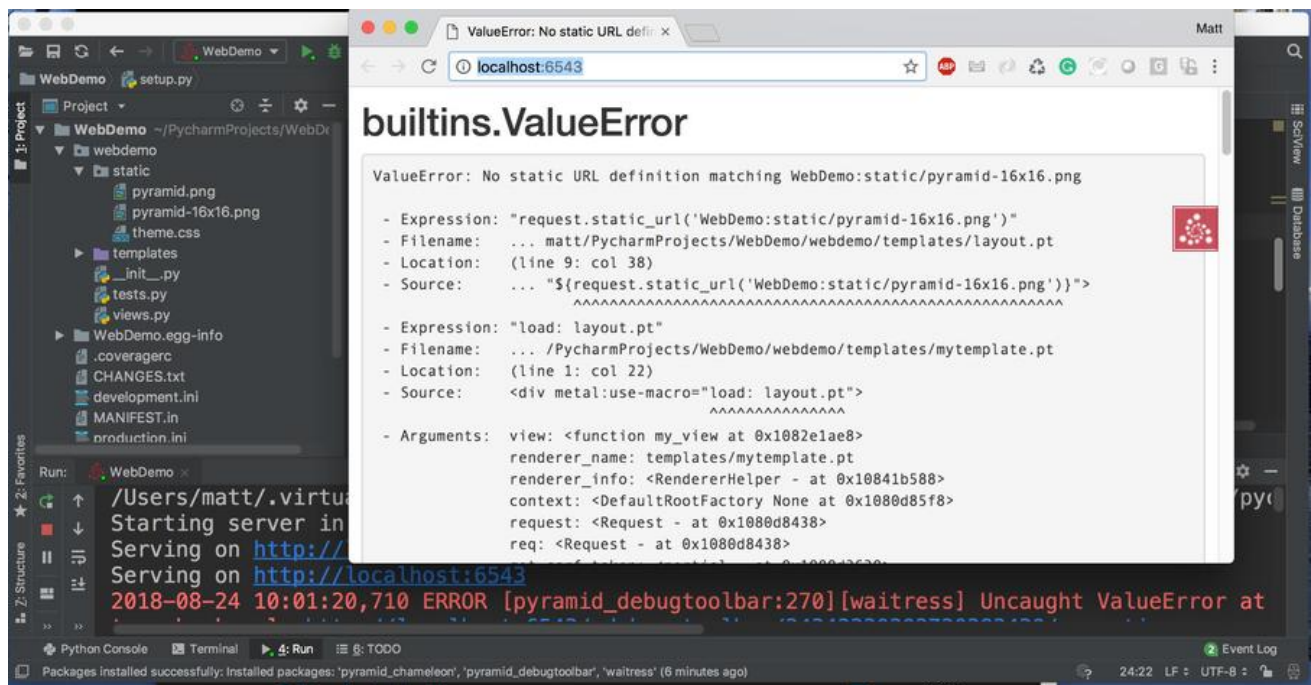


图83：说明如果静态调用指向WebDemo:而非webdemo:时的错误的图。

8. Server-side Python Web Applications

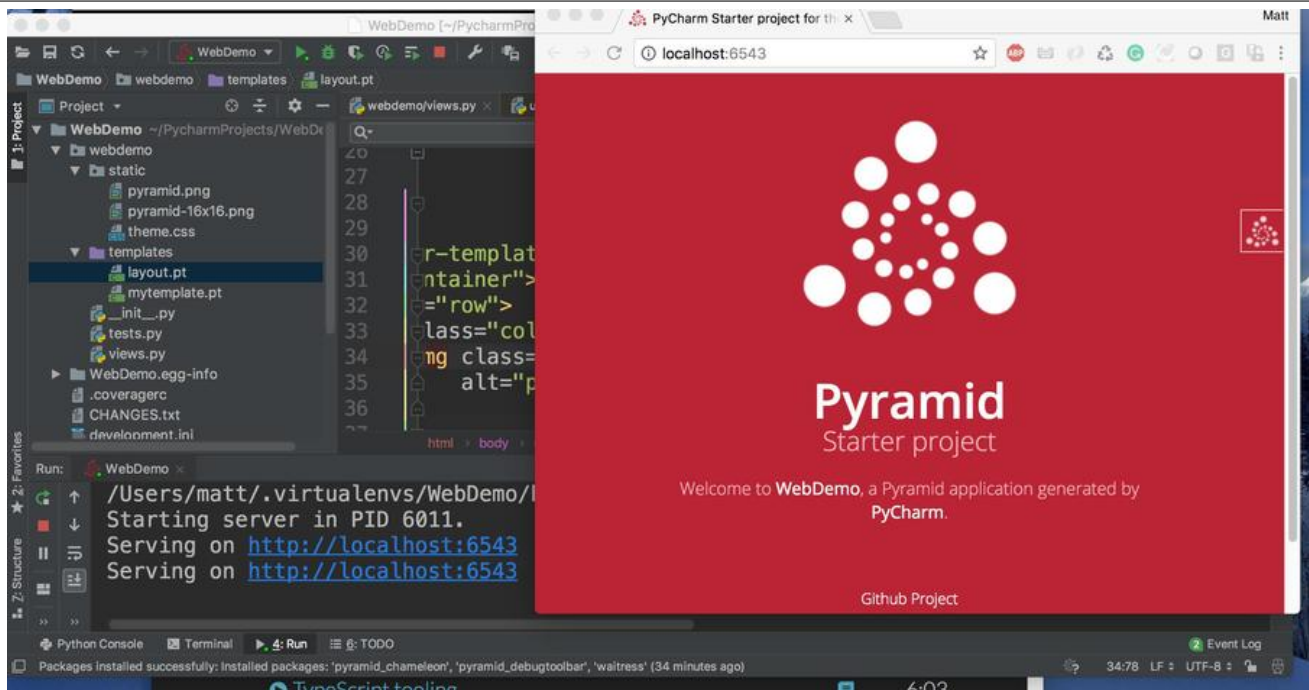


图84：展示成功启动网页的示意图。

提示

一旦你让运行程序为你的网页正常工作，你应该再做一处修改。如果你尝试再次运行它，你会得到一个错误，抱怨“地址已在使用中”。这是因为Web应用程序正在使用的端口不可用。一个小改动就能解决这个问题。

通过运行 “Edit Configurations...” 调整运行配置。然后选中“仅单实例”复选框。这样可以确保如果再次点击运行，PyCharm 会终止先前的版本，你就不会看到此错误。

金字塔将路由映射到渲染器。在webdemo/views.py中，你会看到以下代码：

```
@view_config(route_name='home',
              renderer="templates/mytemplate.pt")
def my_view(request):
    return {'project': 'WebDemo'}
```

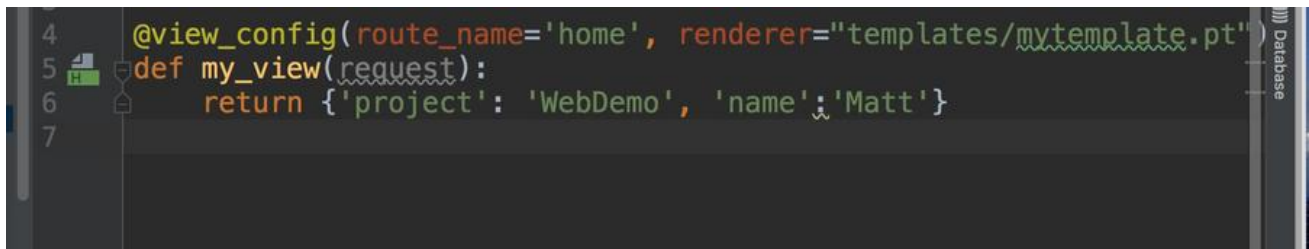
这段代码表示此视图名为home，使用templates/mytemplate.pt模板，并返回一个数据字典。

在webdemo/__init__.py中有一些代码将home名称映射到/的路由。代码行如下：

```
config.add_route('home', '/')
```

这种路由抽象可能看起来令人困惑，但它可能很强大。要验证Web应用程序是动态的，更新my_view的返回值以添加一个名称：

```
@view_config(route_name='home',
              renderer="templates/mytemplate.pt")
def my_view(request):
    return {'project': 'WebDemo',
            'name': 'Matt'}
```



财像影响说明该视图的模板。请注意左侧空白处带有H的图标。如果您单击它，您将b
ta肯符合模板。

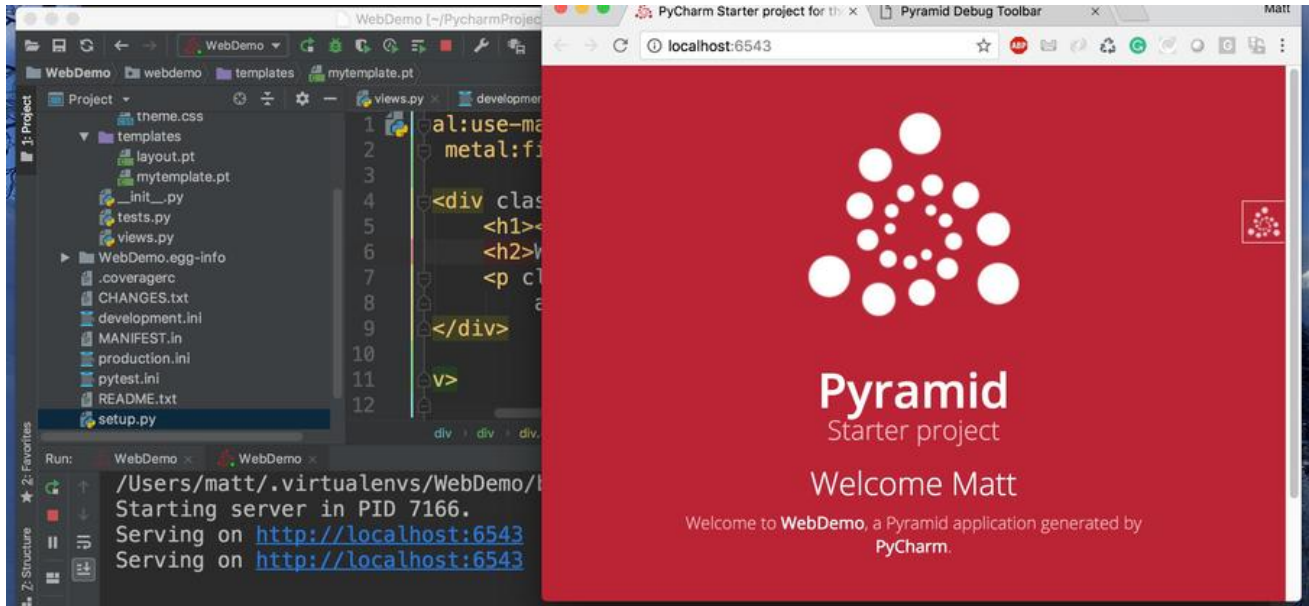


图86：展示从视图进行模板渲染数据的示意图。

你会在视图左侧的空白处看到一个图标，其下方有一个“H”。如果你点击它，它会带你进入该视图的模板。现在更改模板（templates/mytemplate.pt）。有一行内容如下：

```
<p class="lead">Welcome to <span class="font-normal">WebDemo</span>
```

在它前面添加一行，内容为：

```
<h2>Welcome ${name}</h2>
```

重新加载网页会让你看到模板正在使用从th提供的数据和呢。

注意

如果你不想一直重启Pyramid，可以在“运行配置”中将`--reload`作为“附加选项：”标志添加。这是Pyramid特有的，但其他框架也有类似功能。你可能需要在终端中启动Watchman（`watchman -f`）或安装Watchdog（`pip install watchdog`）。

这种调整使开发变得容易。你更改一些代码，服务器就会注意到并重新启动。

8.3 模板工具

在本节中，我们将探讨模板中的一些代码补全支持。回到th和我们对其进行添加更新，以提供一些数据和一个布尔标志：

```
@view_config(route_name='home',
              renderer="templates/mytemplate.pt")
def my_view(request):
    return {'project': 'WebDemo',
            'name': 'Matt',
            'data': [1, 1, 2, 3, 5, 8, 13],
            'show_odd': True}
```

回到mytemplate.pt，删除`<div class="content">`标签内的内容。添加以下内容以显示表示我们是否显示奇数的标志。由于我们使用的是Chameleon，因此将使用TAL（模板属性语言）来实现。其他模板语言有不同的语法，但思路是一样的：

```
<h1>Template Demo</h1>
<h2 tal:condition="show_odd">Showing odd numbers as well</h2>
<ul>
  <li tal:repeat="n data">Num ${n}</li>
</ul>
```

需要注意几点。PyCharm会自动为您关闭HTML标签。PyCharm还会处理TAL属性的代码补全。遗憾的是，PyCharm不会补全上下文变量（data），但它确实识别n。

如果将my_view中'show_odd'键的值更改为False，网页将不会过滤掉奇数。要隐藏这些奇数，你需要对模板代码稍作修改。TAL有一个tal:condition属性，可以打开或关闭标签，但不能将其应用于标签，因为我们正在使用它进行循环。为了解决这个问题，你可以将标签更改为<div>标签（选中li并输入div），然后添加一个新的内部标签：

```
<h1>Template Demo</h1>
<h2 tal:condition="show_odd">Showing odd</h2>
<ul>
  <div tal:repeat="n data" tal:omit-tag="True">
    <li tal:condition="n%2 == 0 or show_odd">
      Num ${n}
    </li>
  </div>
</ul>
```

如果你利用PyCharm在编辑HTML方面的智能功能，这个更改相当轻松。

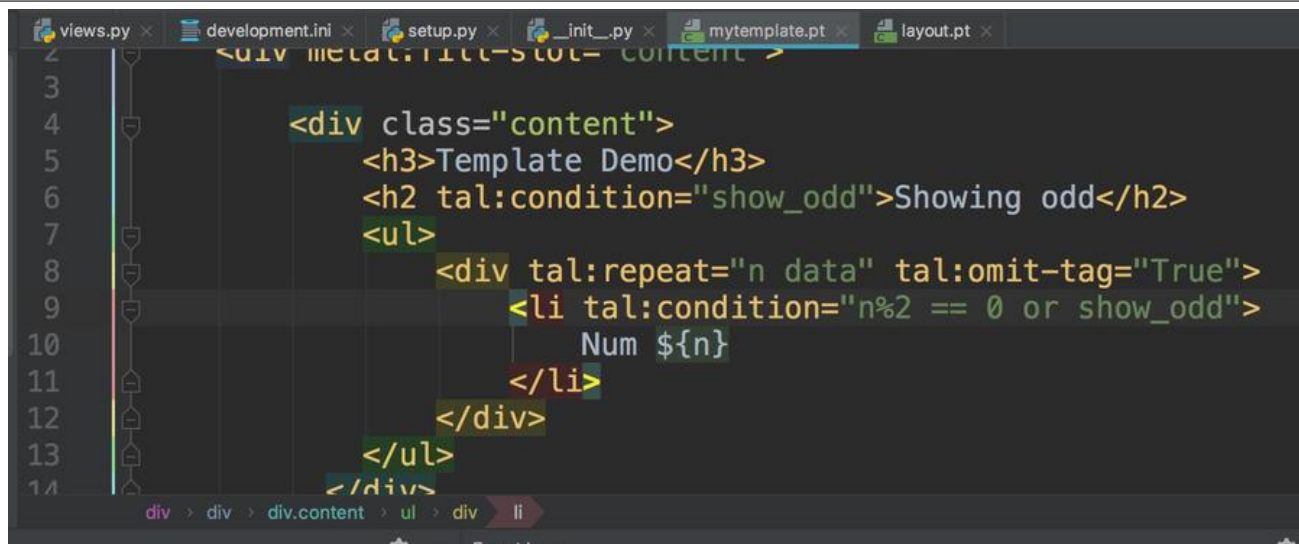


图87：编辑器底部说明HTML标签层级结构的图。点击底部栏中带颜色的元素。

另一件需要注意的事情是，PyCharm 会在编辑器底部跟踪光标所在的标签层次结构。如果你想使用这些标签进行导航，它们是可点击的。在下图中，你会看到 `div > div > div.content > ul > div > li`。这些标签中的每一个都可以点击，以便在 DOM 层次结构中快速导航，而不会迷失位置。

8.4 Emmet

PyCharm 集成了 Emmet。Emmet 以前称为 *Zen Coding*，它允许您从 CSS 创建 HTML。在最基本的层面上，您可以创建单个元素。要创建 `<div></div>` 标签，在模板编辑器中输入 `div` 并按 TAB 键。

如果你想制作一个 `<div class="blue" id="header"></div>` 标签，输入：

```
div.blue#header
```

然后按TAB键。

Emmet 能做的远不止这些。它支持子元素 (`>`)、同级元素 (`+`)、向上攀升 (`^`)、乘法 (`*`)、分组 (`()`)、ID (`#`)、类 (`.`)、自定义属性 (`[key="value"]`)、文本 (`{}`) 等等。

以下代码：

```
div.content>h1{Welcome}>^(div{Lorem}+div{Ipsum}+ul>li*3)
```

在其后按 TAB 键会展开为此内容：

```
<div class="content">
  <h1>Welcome</h1>
  <div>Lorem</div>
  <div>Ipsum</div>
  <ul>
    <li></li>
    <li></li>
    <li></li>
  </ul>
</div>
```

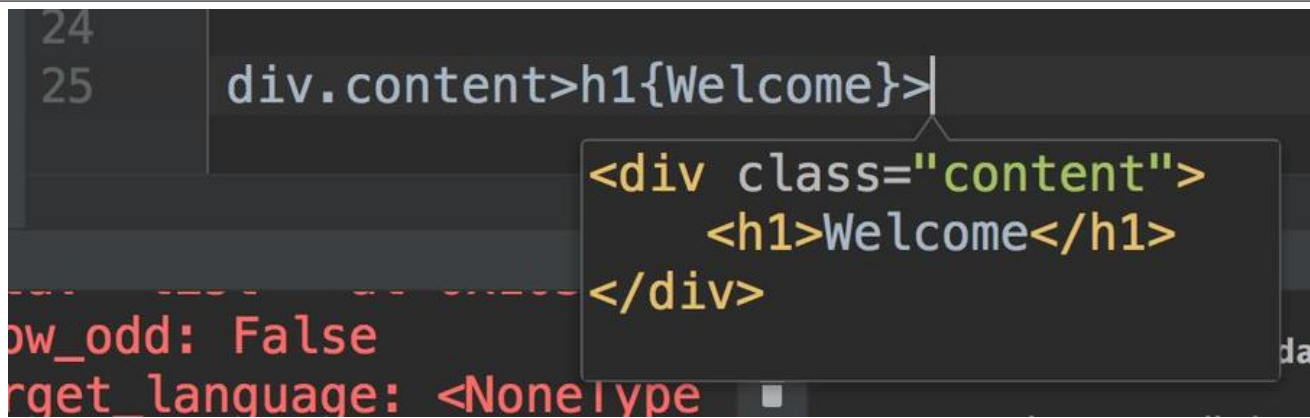


图88：说明Emmet预览弹出框的图。

有一个选项可以在展开缩写之前弹出一个带有缩写预览的窗口。转到偏好设置，搜索“emmet”，然后点击“启用缩写预览”。

8.5 静态文件

我们已经提到，PyCharm 允许我们将目录标记为特殊类型（源根目录、资源根目录、忽略目录等）。我们可以将模板目录标记为“模板文件夹”（如果创建新项目，PyCharm 会处理此操作，但如果加载现有项目则不会）。我们还可以将目录标记为“资源根目录”。资源根目录用于存放图像、CSS 和 JavaScript 等静态文件。

如果我们希望static成为URL的第一部分，就需要将根目录（包含静态文件夹的目录，在我们的例子中是webdemo）标记为资源根目录。在layout.pt模板中，我们使用了由CDN（内容分发网络）托管的jQuery和Bootstrap。这在生产环境中很好，但可能会增加开发和调试的难度。

让我们安装一个本地版的Bootstrap，以展示资源根的工作原理。创建一个目录webdemo/static/css。使用“终端”窗口cd进入该目录并运行：

```
$ npm install bootstrap
```

此命令要求在系统路径中安装node和npm。现在在layout.pt中，转到包含以下内容的行：

```
<link href="//oss.maxcdn.com/libs/twitter-bootstrap/3.0.3/css/
bootstrap.min.css" rel="stylesheet">
```

将href的值更改为/static/，你会发现有目录补全功能，因为根目录已被标记为资源根目录。用引导程序的新路径完成填写：

```
/static/css/node_modules/bootstrap/dist/css/bootstrap.css
```

此时，PyCharm 将识别并解析 CSS，并且它可用于代码补全。请注意，如果我们的项目引用了托管在 CDN 上的内容（不在资源根目录中），PyCharm 将不会使用它们进行代码补全。此外，PyCharm 在静态文件中找到的任何符号（JavaScript 函数、CSS 类名）都是 PyCharm 意义上的“符号”。因此，我们可以利用这一点进行导航、代码补全、重构等！

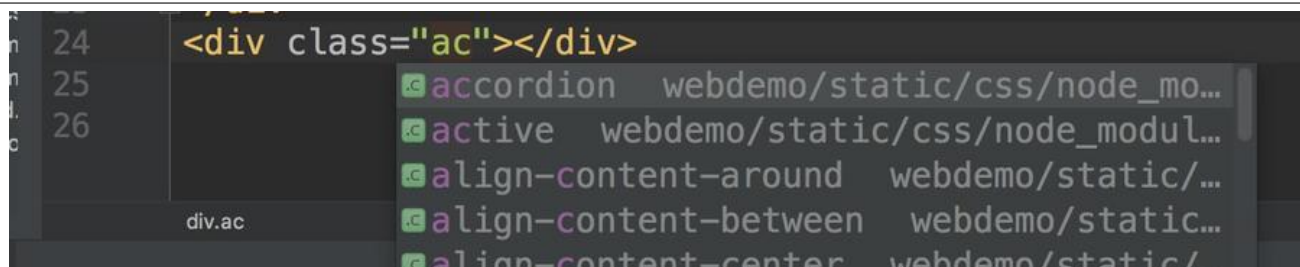


图89：该图展示了在资源根目录中添加了bootstrap的本地副本后代码补全的情况。

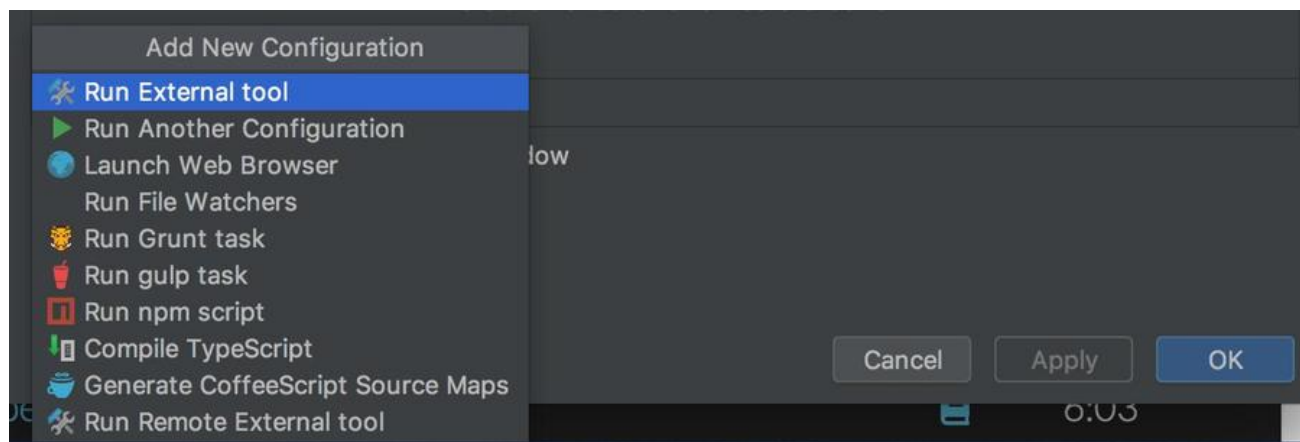


图810：说明在启动Web应用程序之前运行工具的各种选项的图。

8.6 其他工具

另一个可能有用的最终配置设置是与外部实用程序集成。许多 Web 应用程序在启动应用程序之前需要运行某些工具或脚本。如果这适用于您的应用程序，PyCharm 对此提供支持。在 Web 应用程序的“运行配置”窗口中，有一个“启动前：激活工具窗口”选项。

在这个区域，你可以添加各种工具。它内置了对运行Grunt、Gulp、NPM等工具的支持。选择适合你需求的选项。如有需要，你也可以运行多个工具。

8.7 总结

在本章中，我们探讨了PyCharm对服务器端开发的支持。PyCharm开箱即支持许多框架。有时，我们需要向PyCharm提供一些额外提示，以便它知道如何处理某些文件。一旦我们这样做了，就可以享受代码补全的好处。PyCharm还使我们能够通过集成Emmet快速创建HTML代码。

8.8 命令

- *"Run (WebDemo)"* - (按 Shift+F10)
- *"Edit Configurations..."*

8.9 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/6-web-applications` 中关于新项目的说明, 尝试创建一个基本的Pyramid应用程序。
2. 访问<https://github.com/talkpython/mastering-pycharm-course> , 按照 `tree/master/your-turn/6-web-applications` 中新项目的说明, 尝试实现一个视图方法。
3. 访问<https://github.com/talkpython/mastering-pycharm-course> , 按照 `tree/master/your-turn/6-web-applications` 中新项目的说明, 尝试从视图渲染数据。
4. 访问<https://github.com/talkpython/mastering-pycharm-course> , 按照 `tree/master/your-turn/6-web-applications` 中新项目的说明, 尝试更改Web应用程序的模板。

第9章

客户端 Web 应用程序

9.1 引言

PyCharm的主要优势之一在于它能够理解整个项目。除了Python之外，它还对JavaScript和Web开发提供强大支持。如果你正在开发一个前端使用JavaScript的Web应用程序，PyCharm可以满足你的需求。它能理解诸如Angular、React和Vue等框架。它还与TypeScript、LESS和SASS集成。最后，PyCharm可以帮助你使用ElectronJS创建桌面应用程序。在本章中，我们将探索PyCharm的这些功能。

9.2 基础HTML和JavaScript

让我们从使用PyCharm针对标准JavaScript的一些基本功能开始。首先，创建一个新项目。为简单起见，创建一个新的Bootstrap项目。我们的项目名为*JS Demo*。在根目录内，添加一个HTML文件，即*index.html*。向该文件添加以下内容。确保它有script标签：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
  <script src="js/site.js"></script>
</head>
<body>
  <h1>Welcome to JS Demo</h1>
</body>
</html>
```

在 js 文件夹中创建一个 *site.js* JavaScript 文件。向其中添加以下内容：

```
function runAtLoad() {
  person = "Paul"

  console.log("Nice to meet you" + person)

runAtLoad()
```

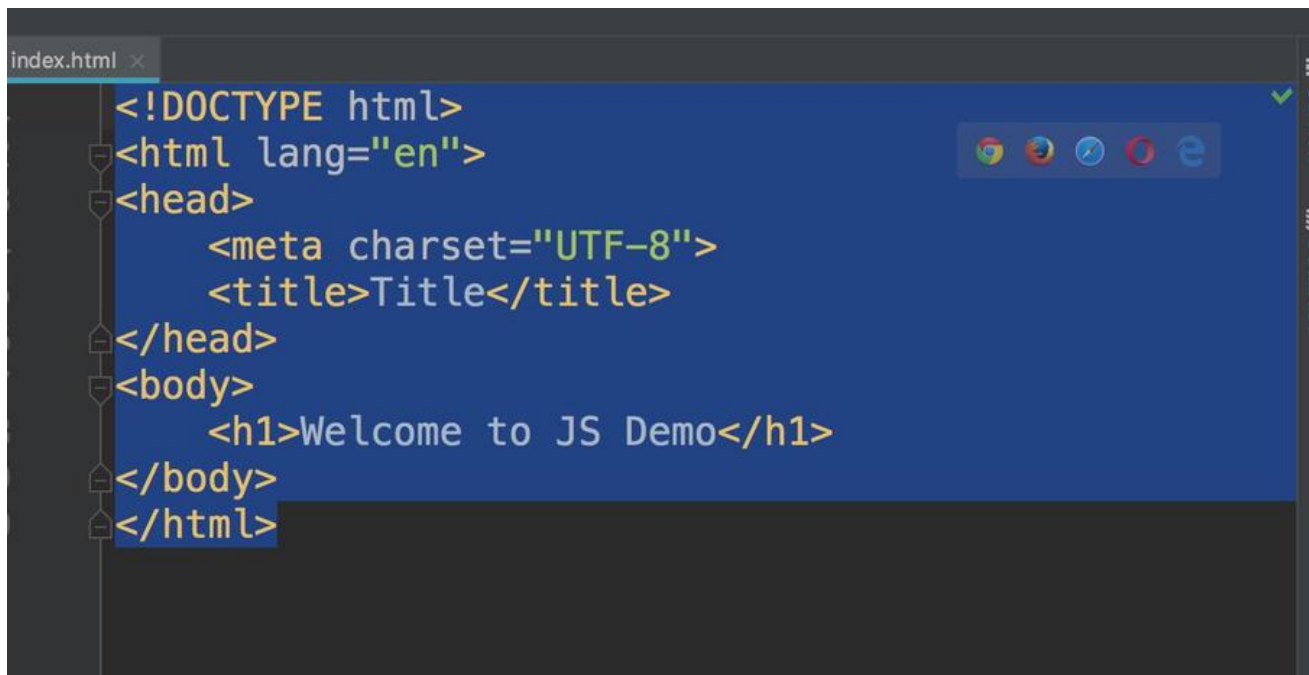


图91：展示HTML页面右上角浏览器图标示意图。

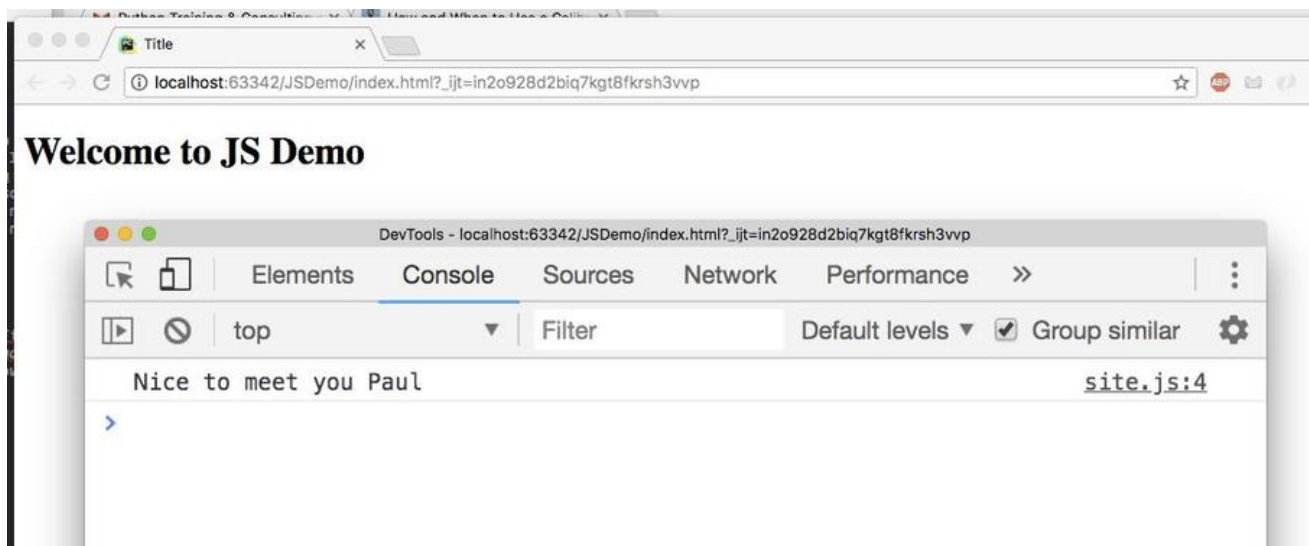


图92：展示打开Chrome JavaScript控制台的示意图。

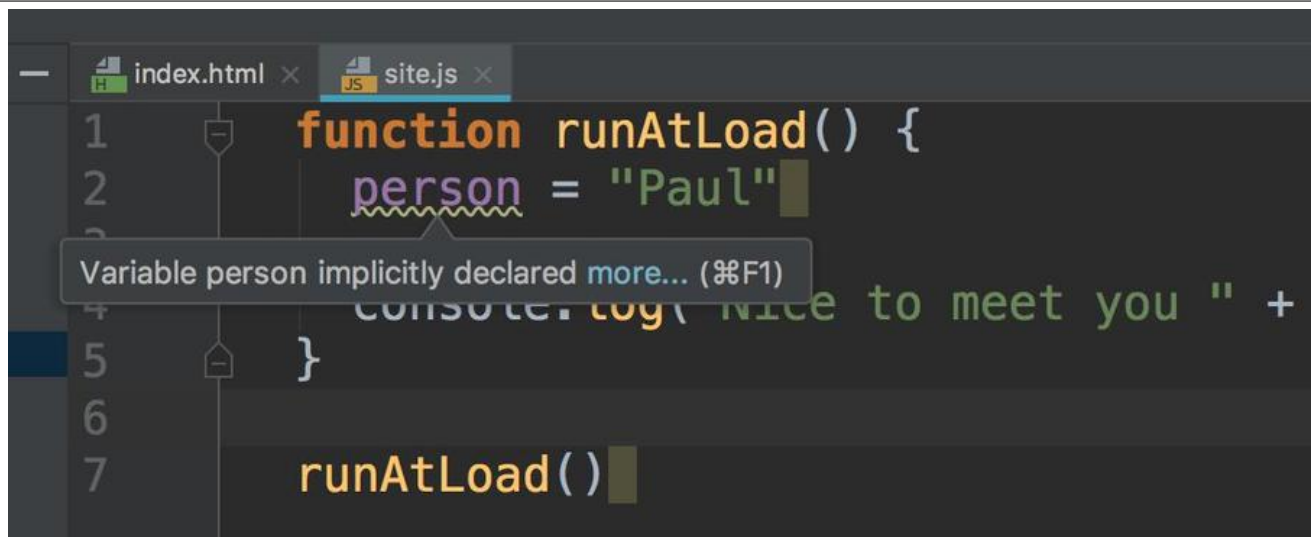


图93: 该图展示了PyCharm会提供修复缺少作用域声明的变量的意图。

如果您点击HTML页面上的Chrome图标，您应该会看到一个显示“欢迎来到JS演示”的网页。如果您打开JavaScript控制台（视图->开发者->“JavaScript控制台”），您应该会看到文本Nice to meet you Paul。

代码似乎已经运行，但存在一个问题。person变量没有作用域声明。与Python不同，在Python中变量的作用域仅限于其定义所在的范围，而JavaScript更倾向于在定义作用域时你能更加明确。

在JavaScript中，变量的默认作用域是全局作用域。这就是我们的person字符串变量所发生的情况。全局变量往往会引发问题，因为它们破坏了封装性，并且可以在代码的任何地方被修改。除非明确需要，否则最好避免使用全局变量。你会看到PyCharm会给person变量加上下划线，并提供一个代码意图来解决这个问题。

如果您点击该意图，它会将该行更改为：

```
let person = "Paul"
```

包含let作用域声明后，person变量不再是全局变量。从那时起，在JavaScript控制台中，你可以尝试访问person变量，但这已不再可能。

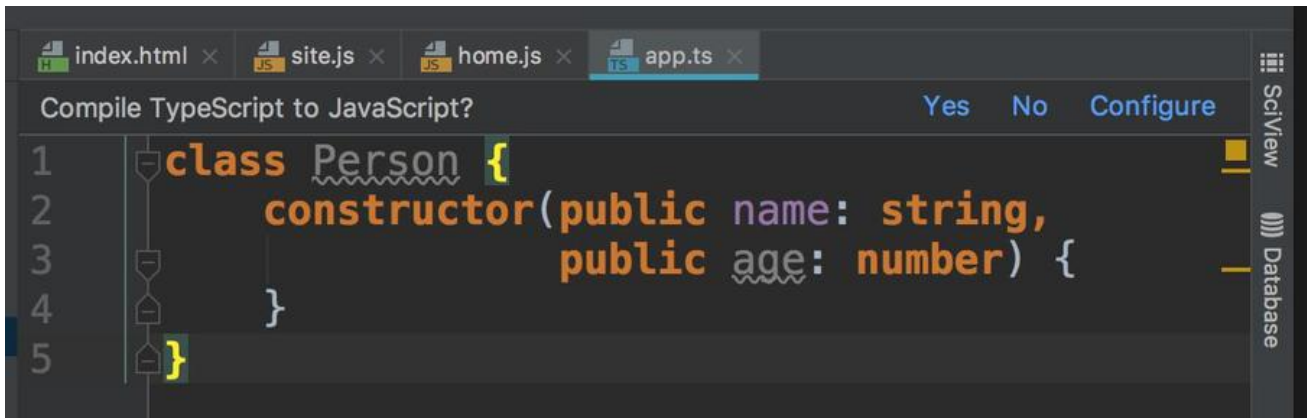
PyCharm 支持对 JavaScript 进行重构。突出显示代码"Nice to meet you " + person 并运行“Extract Method”命令（command-alt-m 或 ctrl-alt-m）。系统将提示您选择重构到全局作用域还是runAtLoad 内部。选择重构到全局作用域。您的代码将如下所示：

```
function getMessage(person) {
    return "Nice to meet you " + person;
}
```

```
function runAtLoad() {
    let person = "Paul"

    console.log(getMessage(person))
}
```

```
runAtLoad()
```



94 TypeScript JavaScript

我们也可以重构到另一个文件。选择`getMessage`函数并运行“*Move*”命令（F6）。告诉PyCharm将其移动到一个文件`js/home.js`中。PyCharm会为您创建一个包含重构内容的新文件。

注意

一些重构操作，比如将代码移动到另一个文件，需要用到ECMAScript 6的特性。浏览器对这些特性的支持也在不断提升。

9.3 TypeScript 工具

TypeScript是另一种在前端框架中很受欢迎的工具。TypeScript是JavaScript的超集，它添加了类型注释，以便更早地捕获某些错误。Angular 2及更高版本以及像Ionic这样的框架都是用TypeScript构建的，以利用这些优势。PyCharm也支持TypeScript。

TypeScript 并非一门全新的语言。它是 JavaScript 的类型化超集，可编译为普通的 JavaScript。这类似于 C++ 与 C 之间的关系。你的浏览器无需理解 TypeScript，因为它永远不会看到 TypeScript，浏览器只会看到其输出的 JavaScript。

让我们在项目中创建一个新的TypeScript文件。将其命名为`app.ts` (注意`ts`扩展名)。向其中添加以下内容:

```
class Person {
    constructor(public name: string,
                public age: number) {
    }
}
```

文件编辑器应在顶部弹出一个对话框，询问是否要将此文件编译为JavaScript。如果在项目资源管理器中点击“是”，该文件将变为可展开状态。当您点击该文件时，会看到已编译的JavaScript文件出现。在这种情况下，它将被称为`app.js`。

你的JavaScript文件可以利用TypeScript代码。在`site.js`文件中，你可以更新它以使用新的人员代码：

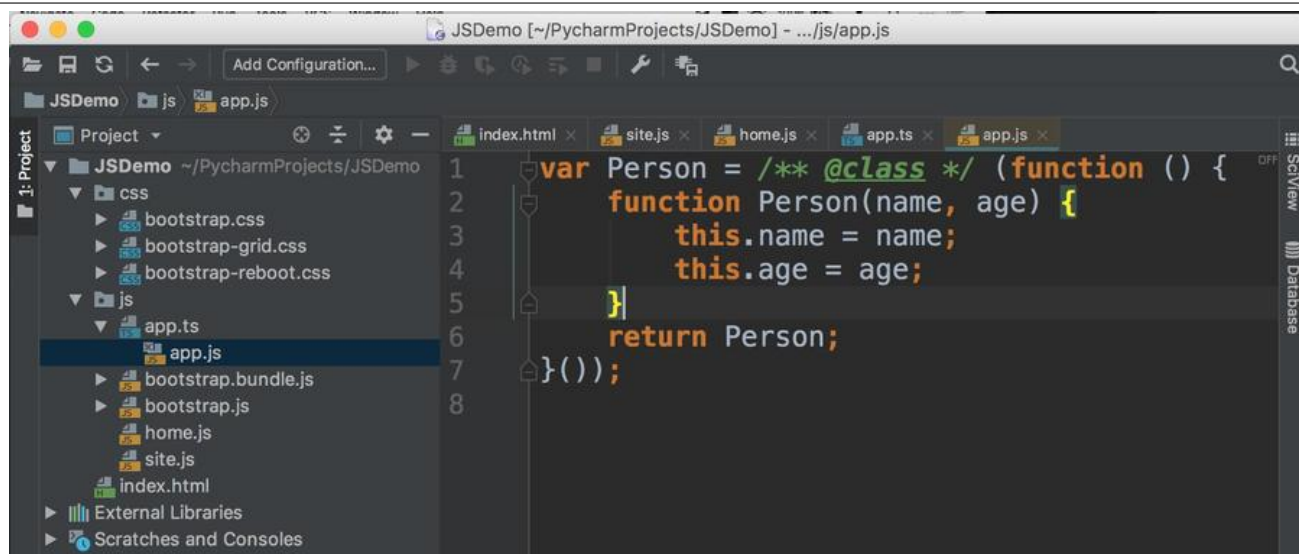


图95：说明从TypeScript派生而来的已编译JavaScript的图。

```
function runAtLoad() {
  let person = new Person('Paul', 64)
  console.log(getMessage(person.name))
}
```

9.4 使用角向

有很多关于Angular的专著。让我们快速了解一下如何开始运行和使用Angular应用程序。

创建一个新的Angular项目。PyCharm将创建一个目录结构和辅助文件。如果打开app/index.html文件，你会看到这是Angular应用程序的入口点。在右上角是在浏览器中打开页面的图标。因为这是一个Angular应用程序，我们想做更多一点的事情。我们想添加一个NPM运行配置。

有一个添加NPM运行配置的模板。我们可以用它来填写适当的信息。但这里有个快捷方式。打开package.json文件。在脚本名称旁边的空白处，你会看到绿色三角形。如果你点击其中一个按钮，它将运行NPM脚本（并创建运行配置）。

要启动一个Web服务器以便与Angular应用程序进行交互，请点击"start"旁边的按钮。它将下载所有必需的依赖项，并在编辑器底部打开一个终端。所有先决条件完成后，将出现一个指向http://localhost:8000的链接。如果点击"Run 'start'"，您将被带到一个运行该应用程序的网页。

在Angular项目的HTML文件中，PyCharm也能为你提供代码补全功能。如果你输入ng作为属性名，就会出现可用选项列表。

本节仅简要介绍了PyCharm对Angular的一些支持。此外，PyCharm还支持代码检查、调试、重构和导航。这与对React和Vue的支持类似。如果您使用的是单页JavaScript技术，请务必利用PyCharm的支持。

9. Client-side Web Applications

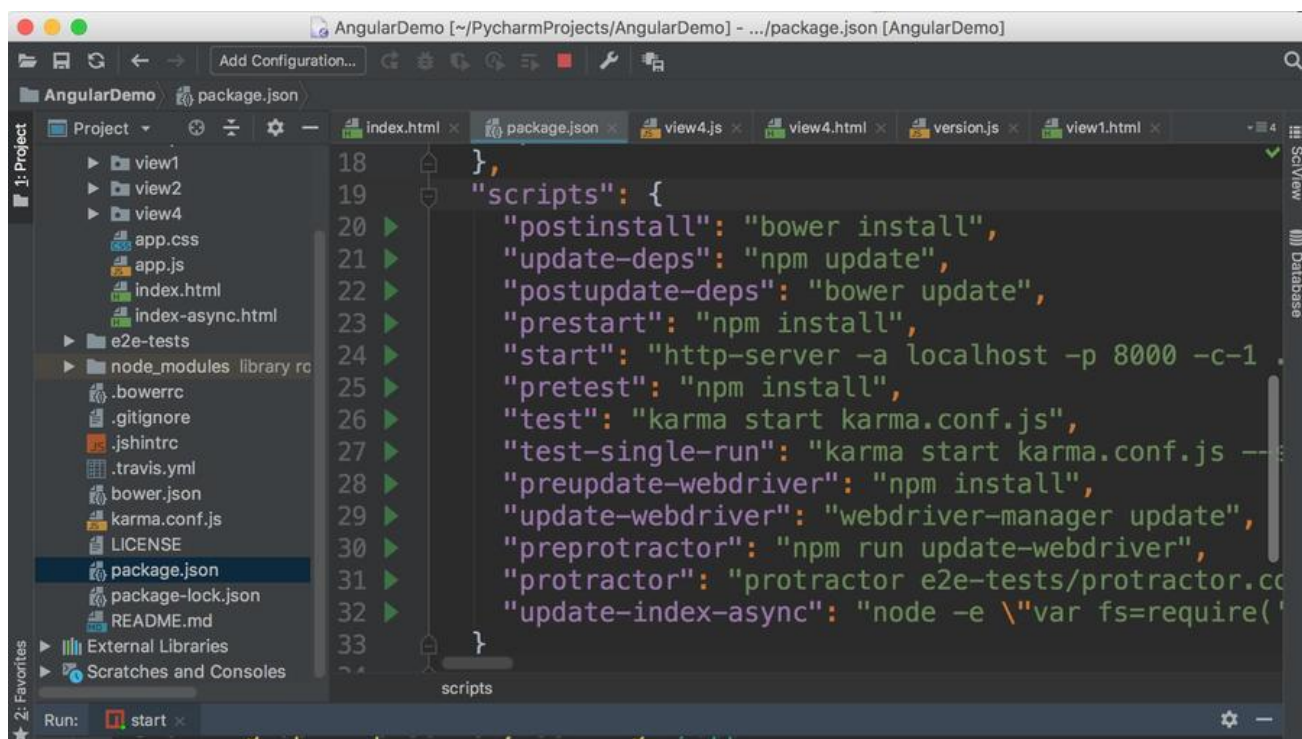


图96：说明package.json文件装订线中运行按钮的图。

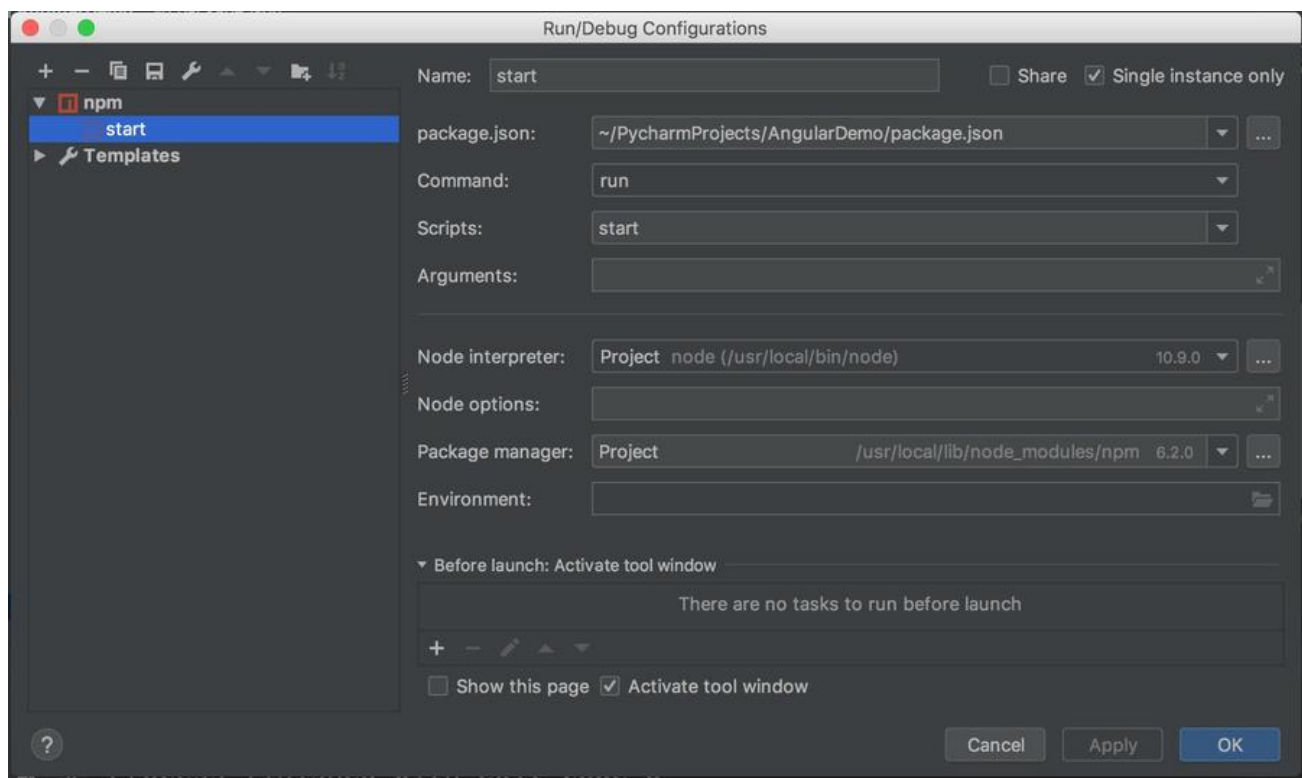


图97：该图展示了通过点击package.json文件中的“开始”生成的“运行配置”。

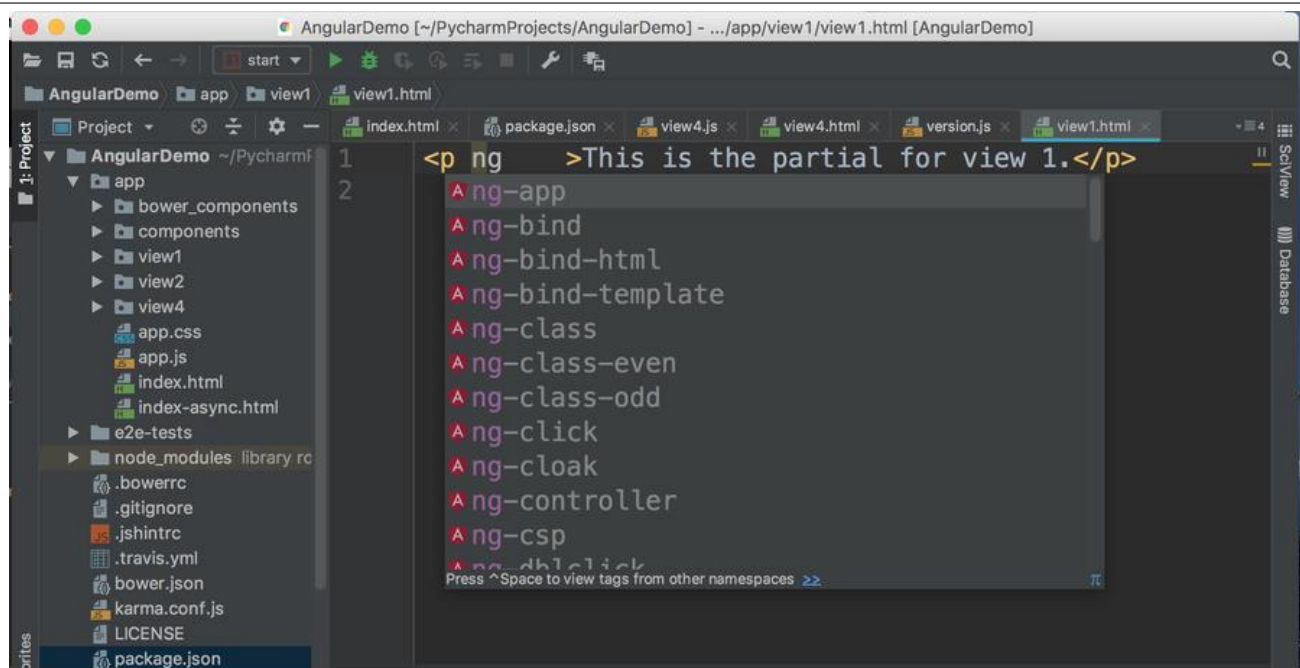


图98: 说明Angular代码补全的图。

9.5 Less

`CSS` `mixin` `Less` `CSS`

假设你有一个网页项目，并且想使用Less。在PyCharm中，在css目录下添加一个以.less结尾的新文件。在New菜单中有各种类型文件的选项，但没有Less文件的选项。没关系，创建一个新文件app.less，然后将扩展名改为.less。

打开文件时，PyCharm 会在顶部显示一个对话框，询问您是否要启用文件观察器。文件观察器将监视 Less 文件，每当该文件发生更改时，它将重新生成其创建的 CSS 文件。单击“是”以启用文件观察器。会有各种选项，但一般来说，使用默认设置就可以正常工作。

让我们添加一个变量@color1，用于存储绿色。将以下RGB十六进制值插入到Less文件中：

```
@green: #00ff00
```

在变量旁边的空白处会出现一个绿色按钮。如果您点击它，将会打开一个颜色选择器窗口。在那里，如果您愿意，可以自定义颜色。

注意

如果文件监视器未生成CSS，可能是因为在您的机器上未找到可执行文件。如果您的机器上安装了npm，则可以运行：

²¹<https://lesscss.org/>

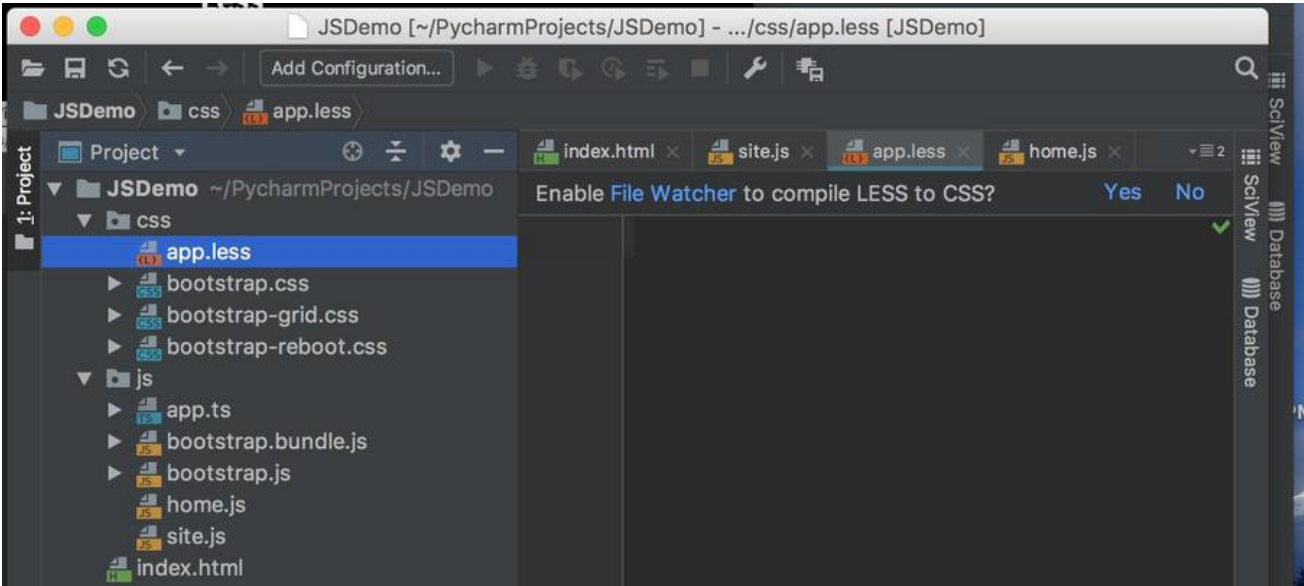
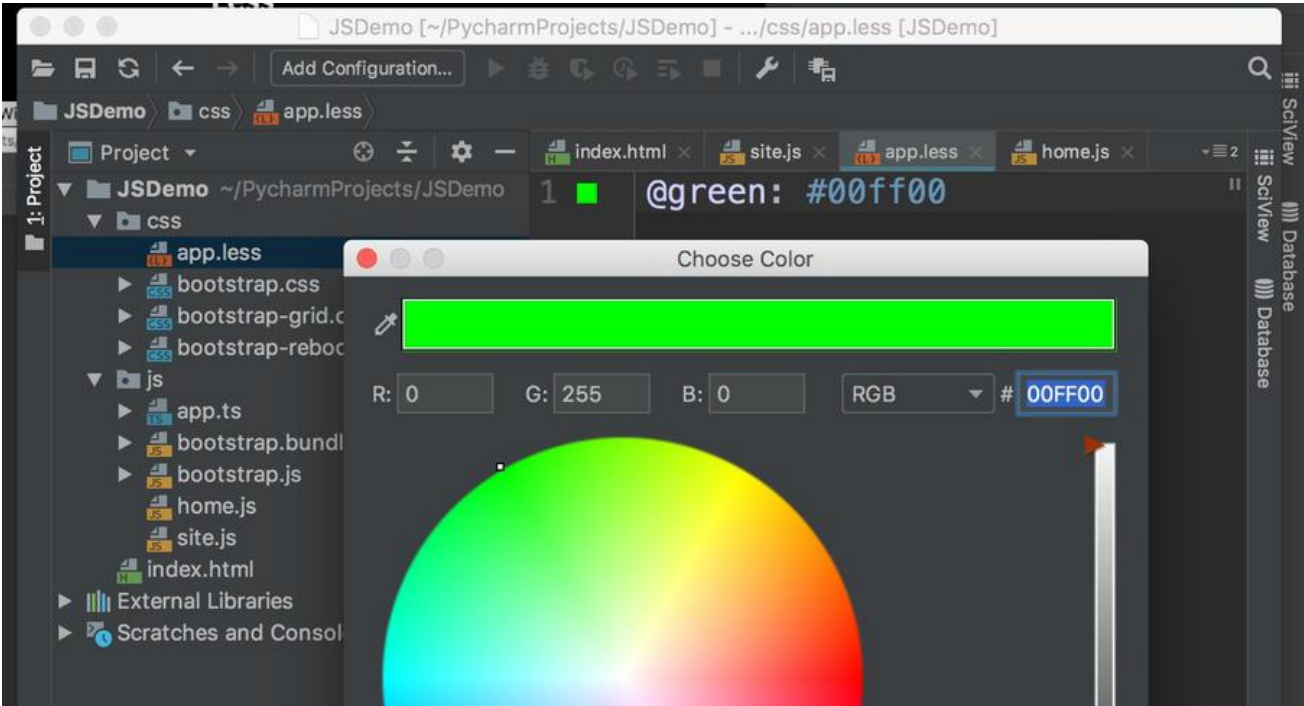


图99：说明告知PyCharm将Less编译为CSS这一选项的图。



910 PyCharm Less

```
$ npm install -g less
```

在 Mac 机器上，可执行文件将位于 `/usr/local/bin/lessc` 中。如果您使用的是 Windows 系统，则可在 `C:\Users\[username]\AppData\Roaming\npm\lessc` 中找到它。使用该路径在观察程序配置页面上更新“程序：”选项。另外，请注意，许多系统的路径中都有一个名为 `less` 的程序。这是一个分页实用程序，并非您所需的程序。如前所述，您需要的是 `lessc`，即 `less` 编译器。

将你的 Less 文件更新为如下内容：

```
@green: #68ff4c;
@lite-green: #d0ffcb;
@page-color: #888888;

h1 {
  color: @green;
  background-color: @page-color;

  &:hover {
    color: @lite-green;
  }
}

h2 {
  color: @green;
}
```

这将生成以下CSS文件：

```
h1 {
  color: #68ff4c;
  background-color: #888888;
}
h1:hover {
  color: #d0ffcb;
}
h2 {
  color: #68ff4c;
}
```

在这种情况下，CSS的代码行数确实更少，因此Less的优势并不明显。但是，当我们添加其他样式组件时，我们可能会像复用`@green`一样，想要复用颜色。通过使用Less变量，我们可以在一个地方更改颜色，而不必担心进行搜索和替换。

转到 `index.html` 页面并更新它，使其引用 CSS 文件：

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Title</title>
    <link rel="stylesheet" href="css/app.css">
  </head>
  <body>
    <h1>Welcome to JS (& CSS) Demo</h1>
  </body>
</html>
```

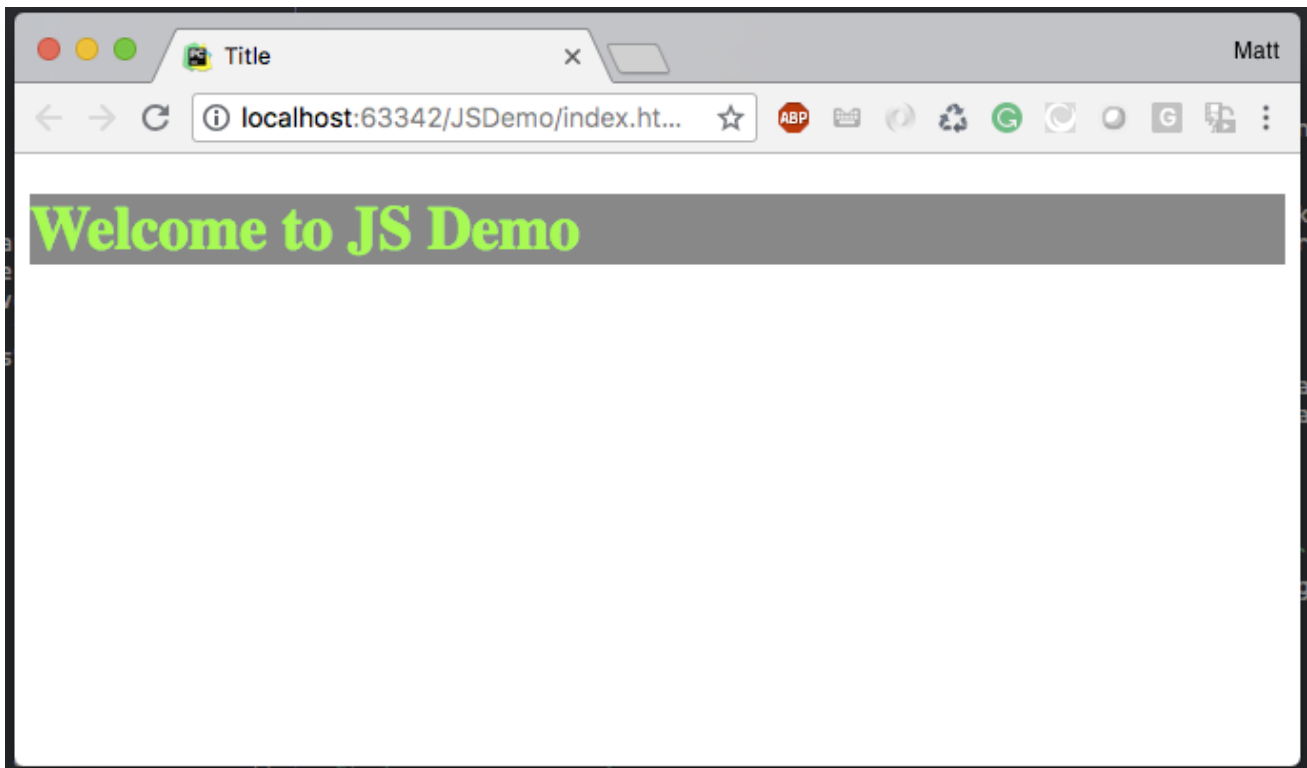


图911：展示从Less生成的CSS在HTML代码中渲染效果的示意图。

如果你点击浏览器图标，将看到一个基础演示。如果你将鼠标悬停在 `h1` 标签上，会注意到颜色发生变化。

Less是一款用于管理和分解CSS的强大工具。如果你正在进行一个项目，它是控制样式的绝佳方式。PyCharm允许你在编辑器中创建和管理CSS和Less，包括代码补全功能。充分利用这些工具。

9.6 Electron JS

在结束关于JavaScript的这一章之前，让我们介绍一下Electron JS。这个框架允许你使用JavaScript创建图形用户界面（GUI）应用程序。它将Chrome浏览器和Node打包成一个可执行文件。基于Electron JS构建的著名应用程序示例包括Slack、Atom编辑器、VS Code和Postman。Electron可在多个平台上运行，并允许你使用用于Web的技术来创建桌面应用程序。本节将探讨PyCharm对使用Electron JS的支持。

如果你已经安装了Git和NPM，安装Electron JS就很简单。让我们尝试一个简单的应用程序。运行以下命令：

```
$ git clone https://github.com/electron/electron-quick-start
$ cd electron-quick-start
$ npm install & npm start
```

这些命令将启动一个简单的应用程序。你应该注意，这不是一个网络浏览器，而是一个独立的应用程序。让我们来探索一下PyCharm对管理Electron JS代码的支持。

使用PyCharm从electron-quick-start目录创建一个新项目。遗憾的是，Electron JS使用JavaScript（Node）而非Python。你需要确保自己已经

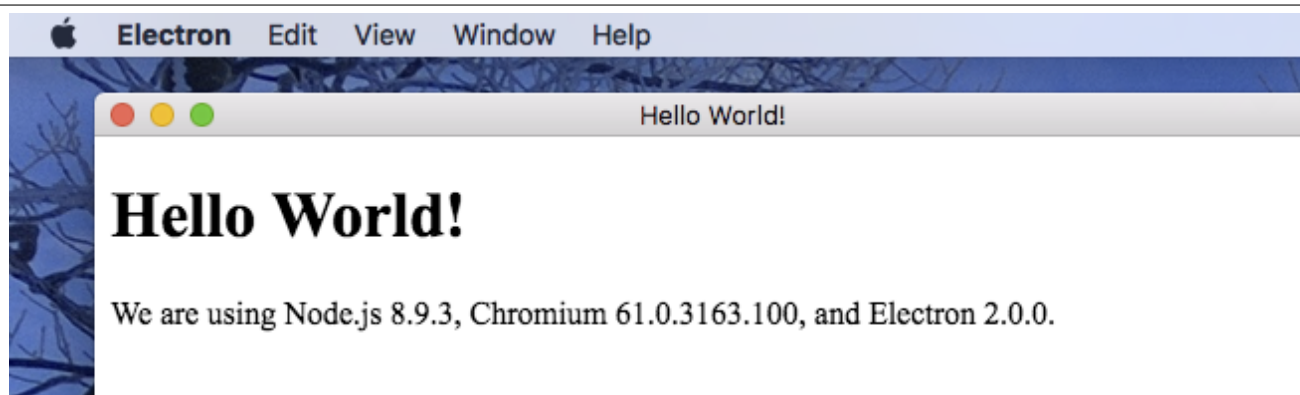


图912：展示一个简单的Electron JS应用程序的图。请注意，这不是一个网络浏览器，如您所见，它有自己的菜单。

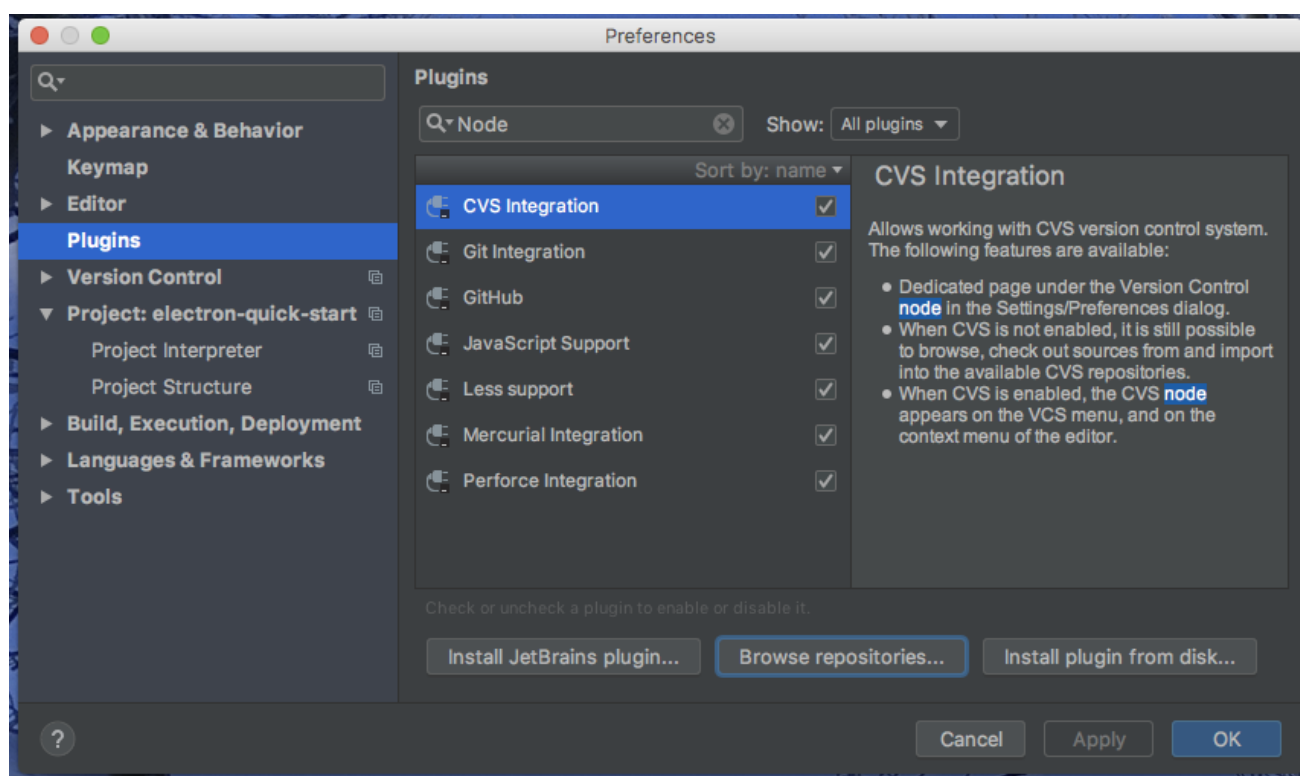


图913：说明缺少Node JS插件的图。

要使用本项目中的代码，需安装Node插件。打开“Preferences”，点击左侧的“插件”。在插件界面中有一个搜索区域。在其中输入“NodeJS”，查看该插件是否已安装。如果未找到该插件，点击“浏览仓库...”按钮，搜索名为“NodeJS”的插件。选中该插件，然后点击“安装”按钮。安装插件后，你需要重启PyCharm。

我们可以向……Electron应用程序添加一个配置。点击“Add Configuration...”按钮（或点击“运行” -> “编辑配置...”菜单）。在“运行/调试配置”窗口中，点击加号按钮添加配置。应该有一个“Node.js”选项。你可以将“名称：”字段更改为“Electron”，并将“Node解释器：”的值更改为electron解释器。你可以在node_modules/.bin/目录中找到它。

9. Client-side Web Applications

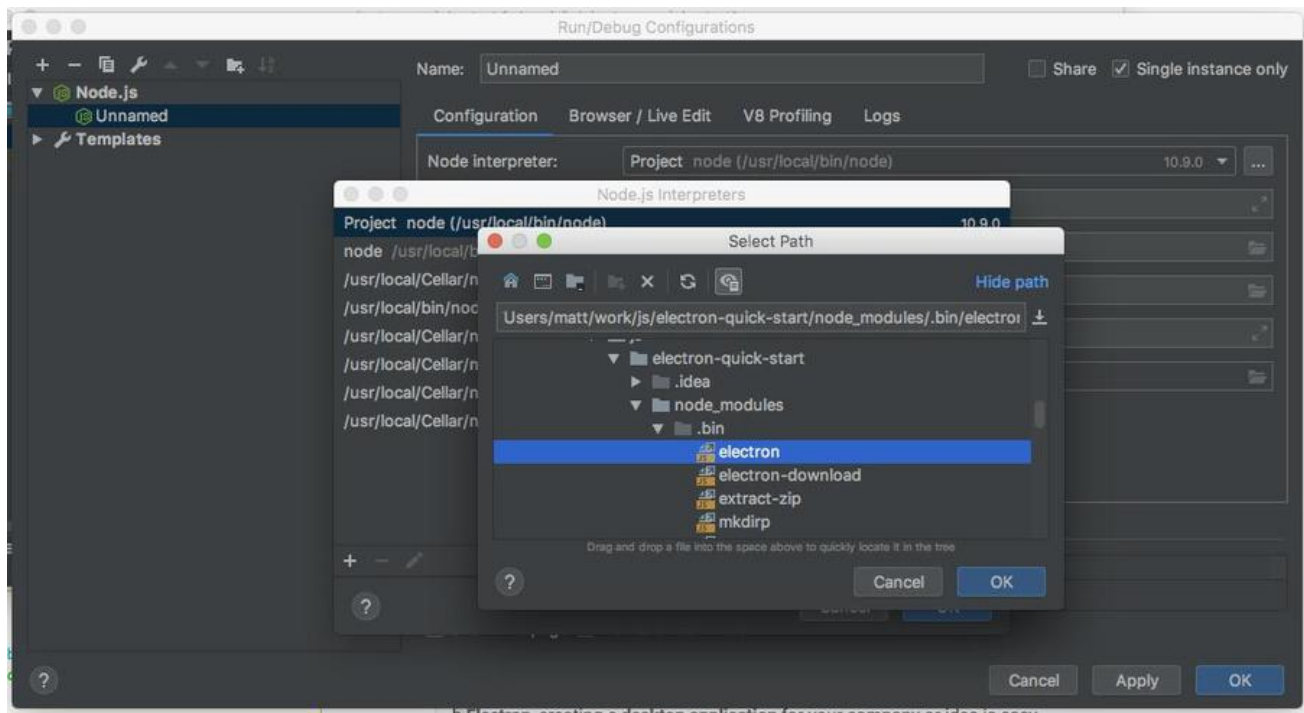


图914：说明为Electron JS配置解释器的图。

“工作目录：” 应设置为项目目录（electron-quick-start）。将“JavaScript 文件：” 留空，并将“应用程序参数：” 设置为 . (一个句点)。这会告知 Electron 在工作目录中运行应用程序。

如果运行该配置，Electron JS 应用程序应该会出现。此时，你应该能够编辑 index.html。你可以将 <h1> 标签更改为：<h1>Hello World from PyCharm!</h1>

如果重新加载Electron JS应用程序（视图-> 重新加载），它将显示更新后的内容。你甚至无需停止并重新启动应用程序。

9.7 总结

PyCharm是一款功能强大的编辑工具。不过，它可能需要一些配置。在本章中，我们详细介绍了如何配置PyCharm，以便它能够运行Electron JS应用程序，并展示了它与JavaScript语言和常见JavaScript框架的深度集成。

9.8 命令

- “Extract Method” - （Command + Option + M， 或 Ctrl + Alt + M）
- “Move” - （F6）

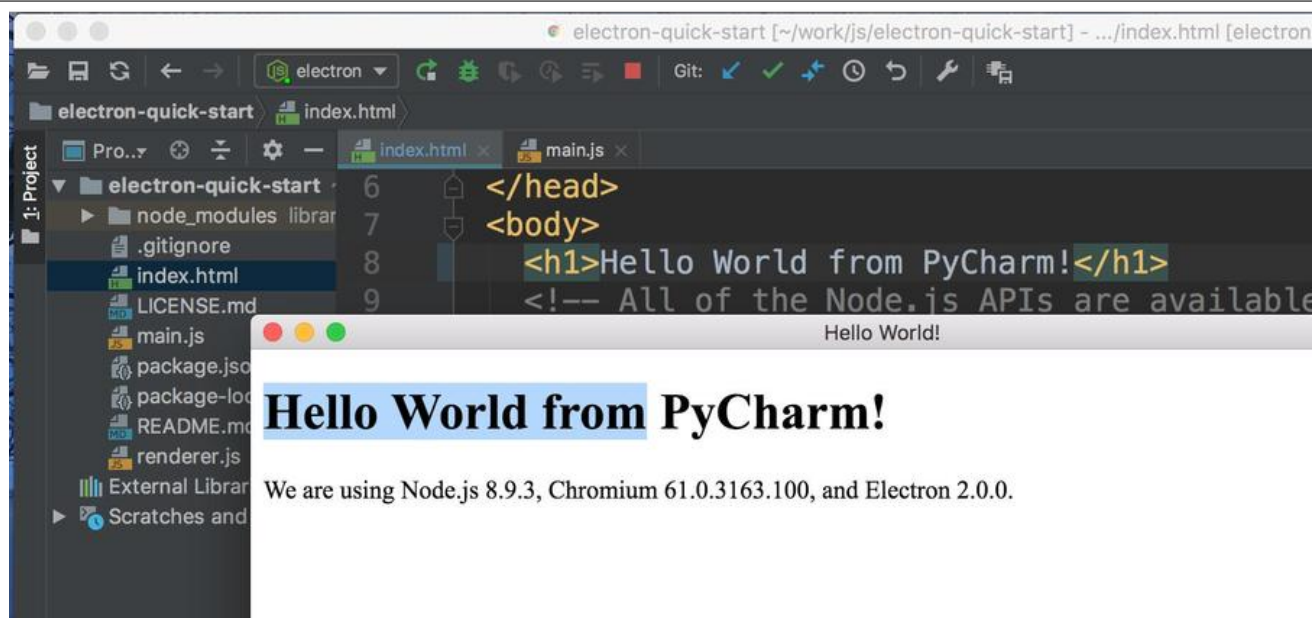


图915：说明从PyCharm运行Electron JS应用程序的图。

9.9 练习

1. 创建一个使用Angular的项目。利用PyCharm对Angular的支持。
2. 创建一个使用TypeScript的项目。利用PyCharm对TypeScript的支持。

第10章

调试Python应用程序

调试是开发人员的常见活动。很多时候，开发人员会使用`print`函数来调试他们的代码。有些人会使用像`pdb`模块这样的命令行调试器，但很少有编辑器能够在编辑环境中调试和检查代码。PyCharm 包含一个非常强大的调试环境，一定能让追踪错误变得更快、更容易。

10.1 调试工具

在前面的章节中，我们讨论了配置环境和运行配置。一旦拥有Python配置，你还可以使用该配置进行调试。不要执行“Run”，而是使用“Debug”命令（`command-alt-F9`）。

让我们用以下代码创建一个简单的Python模块`pets.py`，以说明调试过程：

```
def remove_pet(kennel, name):
    for other in kennel:
        if name == other:
            kennel.remove(other)
    return kennel

kennel = ['Snoopy', 'Fido', 'Fido', 'Pluto']
result = remove_pet(kennel, 'Fido')
print(result)
```

要快速创建配置，可以右键单击文件，然后单击“Run”或“Debug”命令。这将创建一个基本的Python配置并运行或调试模块。我们通过在代码行左侧的空白处单击来添加断点。在将`result`赋值给列表的位置添加断点，然后调试程序。

当你使用断点调试程序时，执行应在断点处停止。此外，会出现一个“调试”窗口。在“调试”窗口的左侧是用于“Rerun”（`control-F5`）、“Resume”（`F9`）、“Pause”和“Stop”（`command-F2`）的按钮。在这些按钮下方是用于“View Breakpoints”（`command-shift-F8`或...）和“Mute Breakpoints”的按钮。在顶部是用于“Show Execution Point”（`alt-F10`）、“Step Over”（`F8`）、“Step In”（`F7`）、“Step In (My Code)”（`alt-shift-F7`）、“Force Step Into”（`alt-shift-F7`）、“Step Out”（`shift-F8`）、“Run Cursor To”（`alt-F9`）以及“Evaluate Expression”（`command-F8`或...）的按钮。

“重新运行”命令将重新启动调试会话。“继续”用于在……之后继续。
哟你已暂停或遇到断点。“暂停”命令将停止执行。通常情况下，w

r
e

10. Debugging Python applications

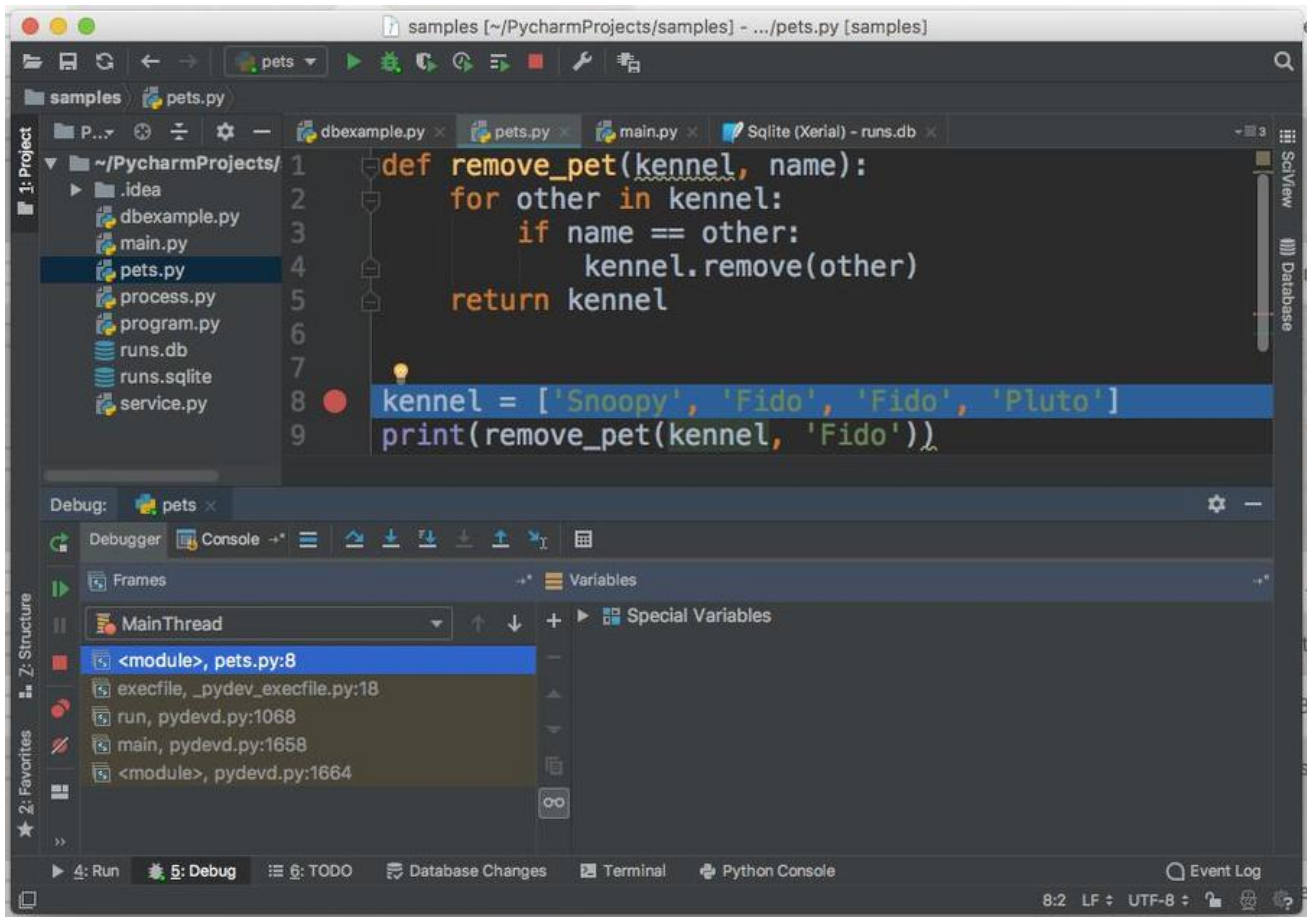


图101：说明装订线处断点和调试器窗口的图。

设置断点以停止执行并检查变量。但有时你会遇到一个长时间运行的程序，却不知道该在哪里设置断点。`pause` 命令可以帮助你找出执行在哪里卡住了，或者你的程序在做什么。“停止”命令会暂停当前程序。

“查看断点”命令看似简单。它会显示设置断点的所有位置。但你还可以通过此命令切换断点、记录断点命中情况以及设置启用断点的条件。例如，你可以在`print(result)`行添加一个断点，并指定仅在`'Fido' in results`时才停止。从代码来看，似乎应该从`result`中删除`'Fido'`。但条件断点仍会命中。你还可以记录断点命中情况，并在断点首次命中后将其禁用。这些都是非常实用的选项，在调试代码问题时确实能提供帮助。

“显示执行点”命令会将光标移回到最后执行的一行。如果您在四处跳转查看其他文件后，决定要快速导航回执行点，此命令会很有用。“单步跳过”命令会执行到下一行代码。“单步跳入”将进入可调用对象（函数或方法）。而“单步跳入（我的代码）”将跳入您的代码，但不会跳入第三方或标准库代码。PyCharm 允许您配置跳过某些代码段，“单步跳入”将忽略这些代码段。如果您确实想跳入这些代码段，请使用“强制单步跳入”命令。还有另一个选项，“Smart Step Into”（shift-F7），它没有对应的按钮。如果某一行上有多个方法，运行此命令，您就可以选择要单步跳入的方法。“单步跳出”会跳出

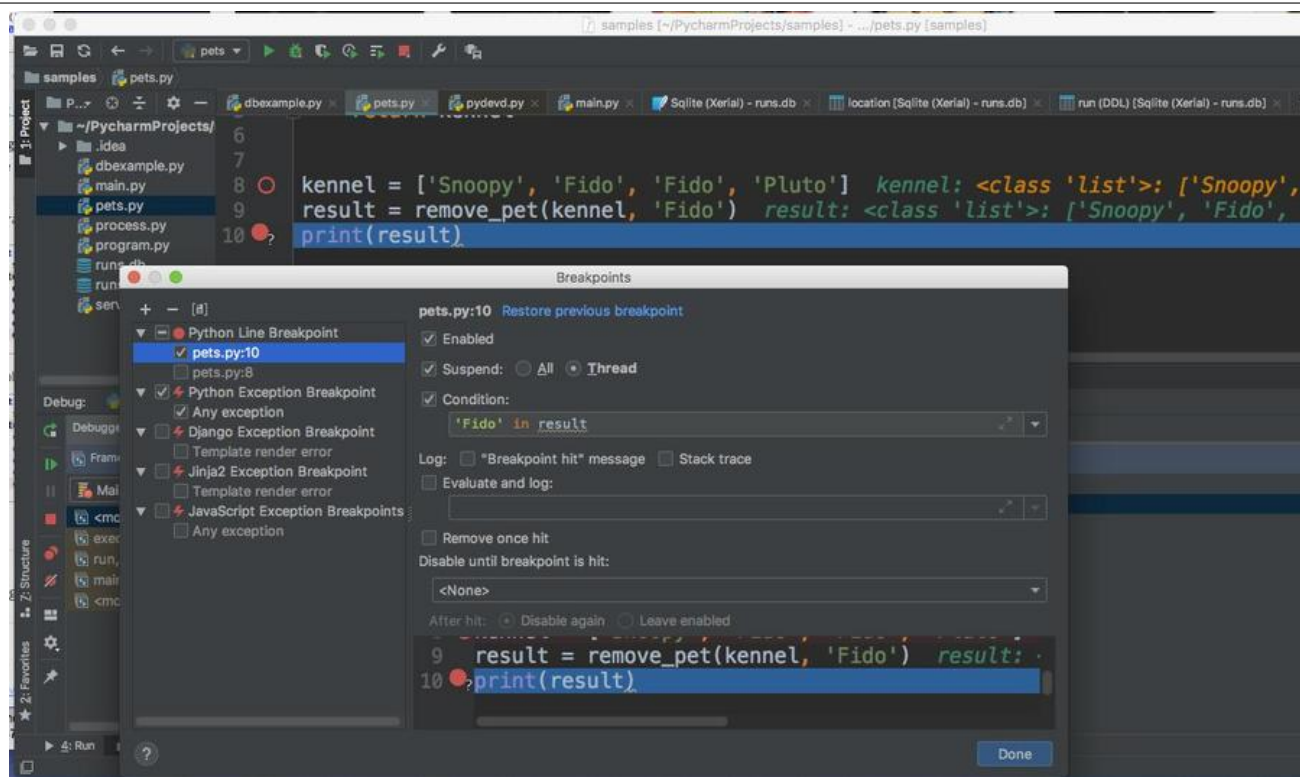


图102：说明条件断点的图。

当前的方法或函数。“运行到光标处”将允许您恢复执行到光标所在的行。“计算表达式”允许您像在交互式解释器（REPL）中一样输入Python表达式。

如果您正在逐步调试此程序，请进入 `remove_pet` 函数。您会看到 PyCharm 已 *inline debugging*，并在设置或修改变量的代码行旁边显示变量的值。内联调试值采用颜色编码。如果变量未发生变化，则显示为绿色；如果变量已更新或发生变化，则显示为橙色。

在“调试”窗口中有两个选项卡，“调试器”和“控制台”。在“调试器”选项卡中，您可以在“帧”部分看到堆栈，在“变量”部分看到对象。“变量”部分会显示变量的值。不过，它还允许我们与变量进行交互。有一个“Set Value...”命令（F2）可让你覆盖某个变量。你可以使用“Jump to Source”（F4）导航到变量的创建位置。

PyCharm 有 *watches* 的概念。监视点是存在于函数或方法内部的特殊变量。通常，你会为一个表达式设置监视点。就我们的代码而言，我们可以使用“New Watch”并赋予其 `'Fido' in result` 的值来创建一个监视点。PyCharm 会将其显示为布尔值，如果 `result` 变量中包含 `'Fido'`，其值会动态变化。

调试器窗口中还有“控制台”选项卡，它会显示标准输出的任何内容。你还可以向标准输入中输入内容。

如果你逐步执行 `remove_pet` 中的循环，你会发现代码只移除了第一个 `'Fido'`。这是由于 Python 的迭代行为。在遍历列表时，Python 会跟踪迭代的索引。如果你移除当前索引处的元素，所有其他元素都会向左移动。因此，当我们移除第一个 `'Fido'` 时，第二个 `'Fido'` 会移动到它的位置，但随后迭代会移动到下一个元素，而这个元素现在是 `'Pluto'`。解决这种情况的一种方法是使用切片构造遍历列表的浅拷贝，然后更新原始列表。

10. Debugging Python applications

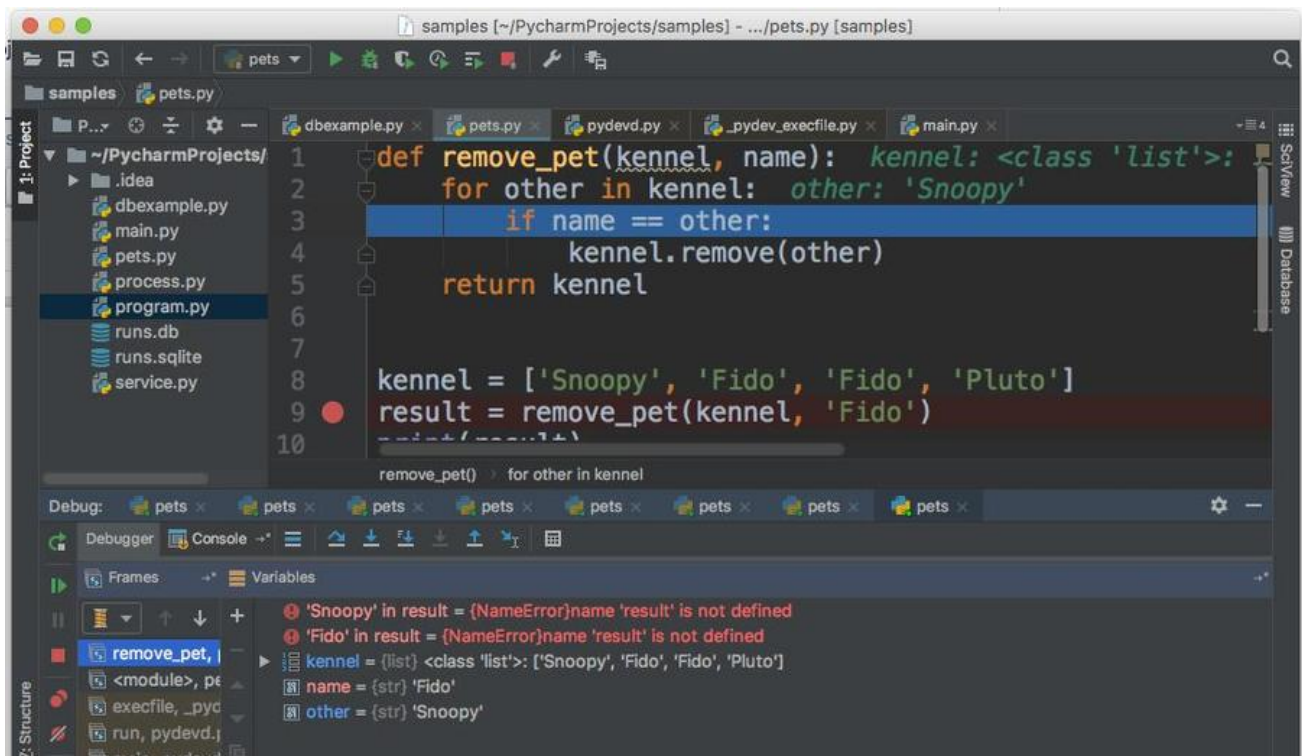


图103: 说明调试的图。请注意，我们已单步进入remove_pet，并且kennel和other的值已内联显示。

代码可以写成：

```
def remove_pet(kennel, name):  
    for other in kennel[:]:  
        if name == other:  
            kennel.remove(other)  
    return kennel
```

```
kennel = ['Snoopy', 'Fido', 'Fido', 'Pluto']  
result = remove_pet(kennel, 'Fido')  
print(result)
```

另一种选择是使用纯函数，不改变传入的列表，而是返回一个新列表：

```
def remove_pet(kennel, name):  
    result = []  
    for other in kennel:  
        if name != other:  
            result.append(other)  
    return result
```

```
kennel = ['Snoopy', 'Fido', 'Fido', 'Pluto']  
result = remove_pet(kennel, 'Fido')  
print(result)
```


10.2 总结

本节概述了逐行调试代码和设置断点的工具。这些强大的工具是调试和理解代码的绝佳方式。Python 是一种动态语言，熟悉诸如交互式解释器（REPL）、监视点和条件断点等工具，可以简化并加快代码的阅读、编写和调试过程。

10.3 命令

- *"Debug"* - (Command + Option + F9)。
- *"Rerun"* - (按Ctrl+F5)
- *"Resume"* - (F9)
- *"Pause"*
- *"Stop"* (命令-F2)
- *"View Breakpoints"* (Command + Shift + F8 或...)
- *"Mute Breakpoints"*
- *"Show Execution Point"* - (按 Alt+F10)
- *"Step Over"* - (F8)
- *"Step In"* - (F7)
- *"Step In (My Code)"* - (按Alt+Shift+F7组合键)
- *"Smart Step Into"* - (按 Shift+F7 组合键)
- *"Force Step Into"* - (按 Alt + Shift + F7 组合键)
- *"Step Out"* - (按 Shift+F8)
- *"Run Cursor To"* - (按Alt+F9)
- *"Evaluate Expression"* - (按 F8 键或...)
- *"Set Value..."* - (F2)
- *"Jump to Source"* - (F4)
- *"New Watch"*

10.4 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/7-debugging` 中关于新项目的说明，尝试创建断点。
2. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/7-debugging` 中新项目的说明，尝试使用调试器来停止和启动控制流。
3. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/7-debugging` 中新项目的说明，尝试使用调试器添加条件断点。

第11章

包

Python的主要吸引力之一是有大量可用的第三方库。Python软件包索引[[#]]拥有超过16万个软件包，并且每天都在不断增加。当你需要用Python执行一项新任务时，应该查看是否有可用的软件包提供该功能。如果不存在，你可能需要构建自己的软件包。PyCharm对这两种情况都提供支持。

11.1 使用现有软件包

在本节中，我们将了解如何获取现有软件包并创建自定义版本。开发人员的一项常见活动是更新软件包。您可能想要添加一项功能或修复一个错误。让我们逐步了解如何围绕现有软件包创建一个PyCharm项目、初始化一个虚拟环境，以及以开发人员模式安装该软件包以便在本地进行处理。

让我们展示如何创建流行的Requests HTTP客户端库的自定义版本。我们需要做的第一件事是检出项目。运行以下命令来检出项目：

```
git clone https://github.com/requests/requests.git
```

使用PyCharm在“位置：”字段中使用该目录创建一个新项目。展开“项目解释器”下拉菜单，然后选择“使用Virtualenv创建新环境”。

如果你打开setup.py文件，可能会注意到顶部显示“未为项目配置Python解释器”。如果是这种情况，将项目解释器指向虚拟环境。之后，你应该会看到PyCharm抱怨缺少包依赖项。点击“安装依赖项”。

让我们对requests/__init__.py文件做一个简单的修改。我们将添加一个print函数，以表明该文件已被修改。在该文件的文档字符串下方输入以下行：

```
print("THIS IS OUR CUSTOM VERSION")
```

此时，如果你尝试在虚拟环境中使用Requests库，除非已将其安装在系统库中，否则Python会报出ModuleNotFoundError (错误。尽管你创建了一个虚拟环境并安装了Requests的依赖项，但Requests库尚未安装。

你要做的是以*development*模式安装Requests。这会将其安装到虚拟环境中，并且还允许你编辑requests包的本地副本。任何编辑操作都会

²²<https://pypi.org>

11. Packages

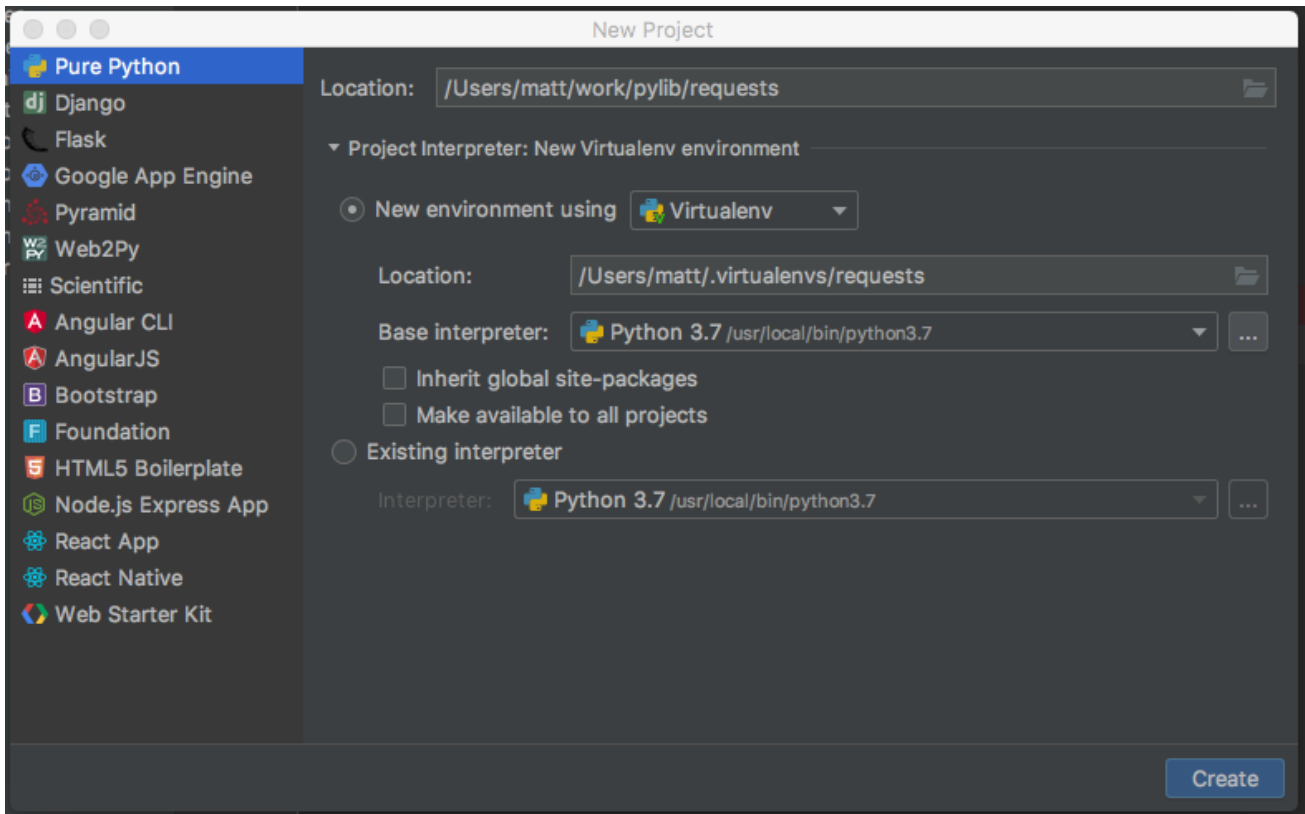


图111：说明使用新的虚拟环境创建新项目的示意图。

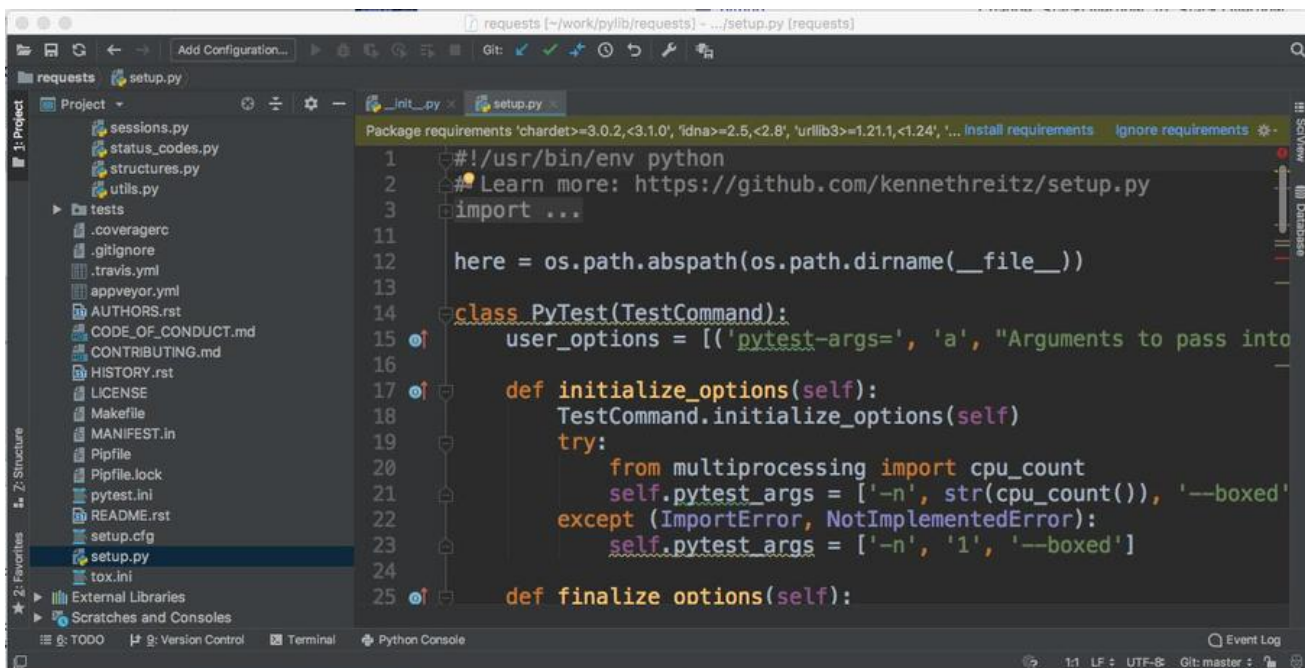


图112：说明缺失需求的图。

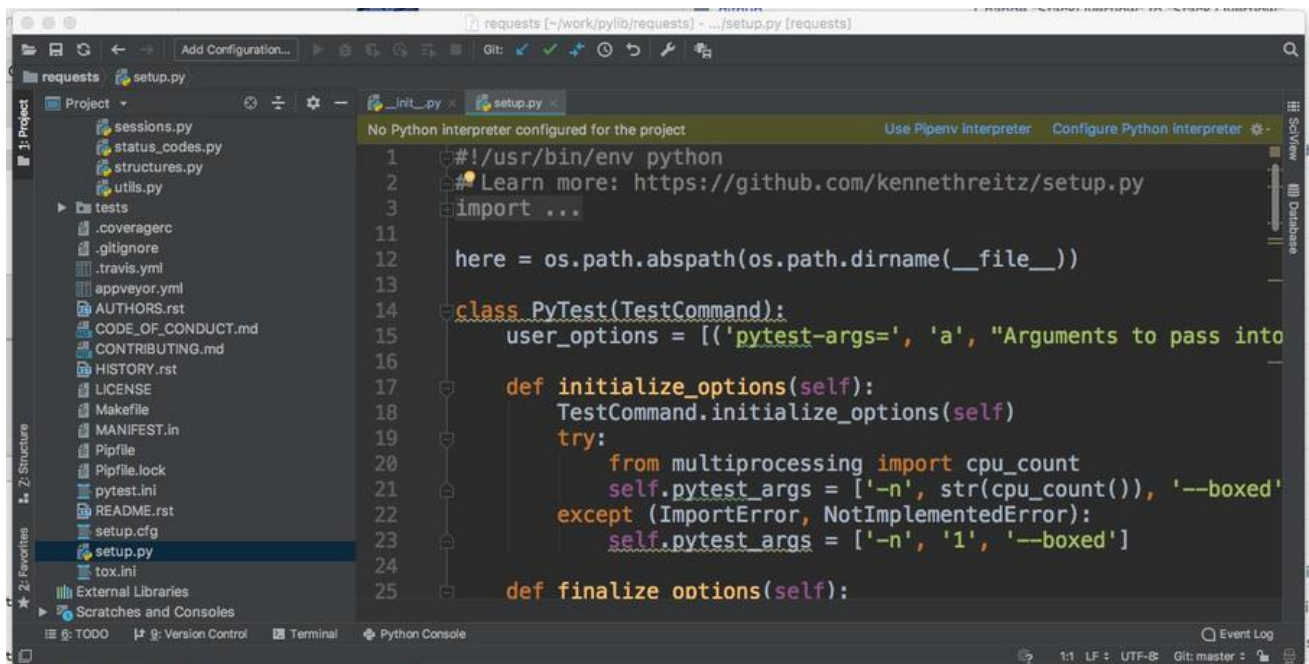


图113: 说明setup.py文件没有关联解释器的示意图。

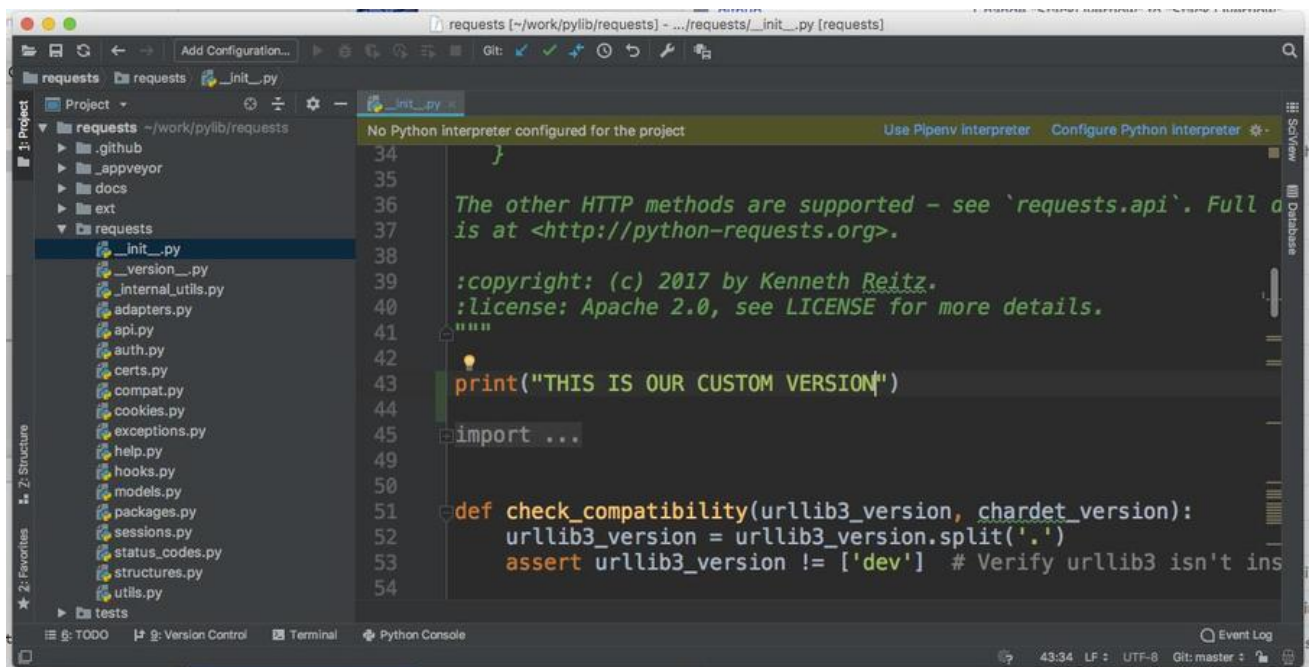


图114: 说明向Requests项目添加自定义代码行的示意图。

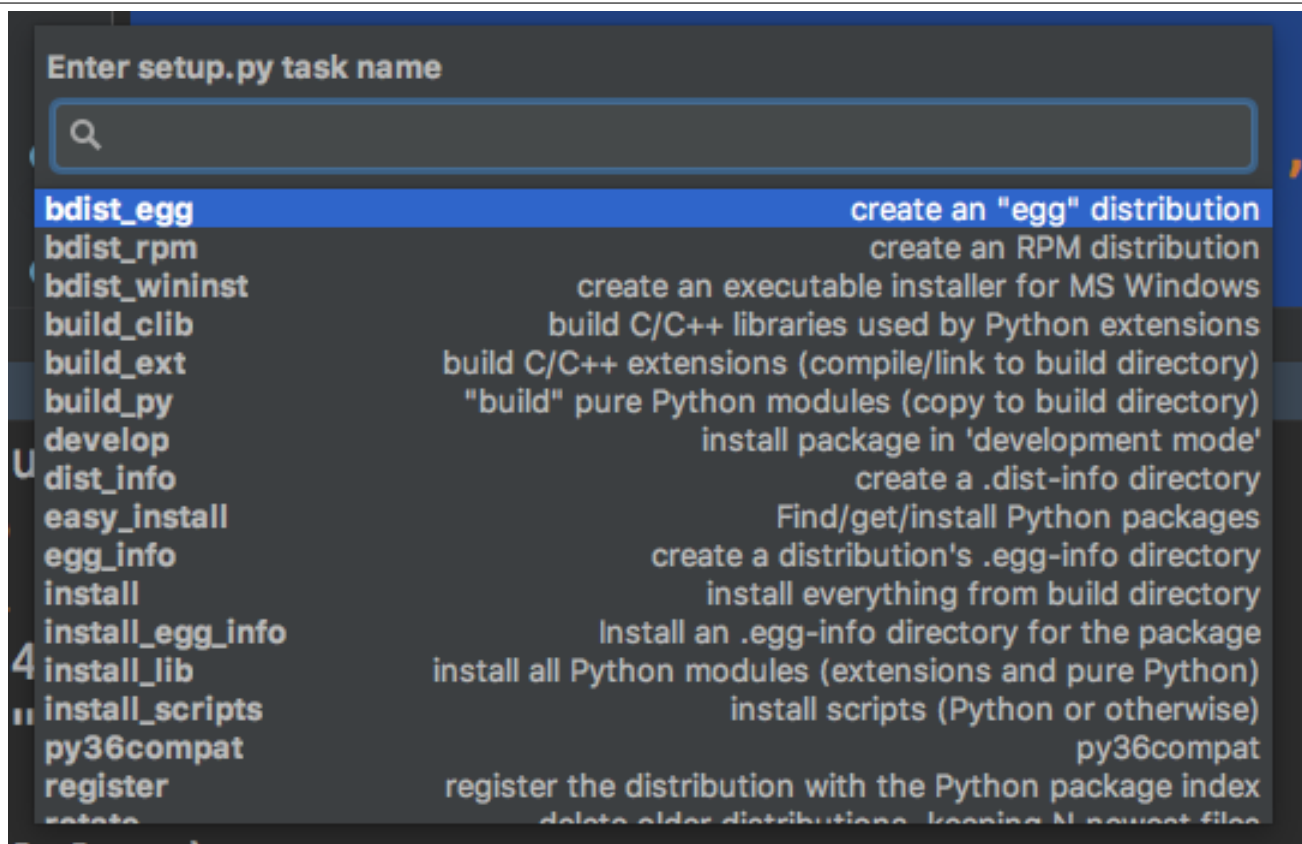


图115: 说明从PyCharm运行setup.py任务的图。

会立即反映在虚拟环境中。你可以通过进入虚拟环境并在命令行中输入以下内容来实现：

```
$ python setup.py develop
```

因为这是一本关于PyCharm的书，所以我们从PyCharm中进行操作。点击“工具” -> “运行setup.py任务...”。点击“开发”。

此时，当你从虚拟环境中加载Requests库时，你应该能够访问正在使用的版本。当你对该库进行更改时，你也能够获取这些更改。

使用新库的输出应该如下所示：

```
>>> import requests
THIS IS OUR CUSTOM VERSION
```

11.2 创建新库

在本节中，我们将用Python创建一个库。要拥有一个库，你需要几样东西。我们需要实现库功能的代码以及一个setup.py文件，该文件声明有关代码的元信息（如何安装和打包）。库的Python代码可以位于模块（只是一个以.py结尾的文件）或包（其中包含一个__init__.py文件的目录）中。

让我们创建一个计算器库。创建一个名为“Calculator”的新“纯Python”项目。在项目选项卡内会有一个名为“Calculator”的文件夹。运行“New...” (ctrl + n)命令。

从新弹出的窗口中选择“Python包”，并将包名输入为“calculator”（PEP 8 建议我们将包名命名为小写）。如果展开新的包文件夹，你会看到 PyCharm 为你创建了一个 `__init__.py` 文件。打开该文件并添加以下代码：

```
class Calculator:

    def add(self, x, y):
        return x + y

    def subtract(self, x, y):
        return x - y
```

如果你打开“Python控制台”，你将可以访问calculator包。你可以运行以下代码：

```
>>> import calculator
>>> calc = calculator.Calculator()
>>> calc.add(2, 5)
7
```

如果我们想共享calculator包，以便其他代码可以利用它，我们有几种选择。我们可以将该包复制到需要它的代码中。我们还可以更新PYTHONPATH环境变量，使其指向calculator所在的目录。不建议采用前者，因为一般来说，复制代码需要谨慎对待。如果要修复错误或添加功能，就必须在多个地方进行操作。也不建议采用后者，因为为每个包编辑环境变量很繁琐。更好的选择是创建一个合适的Python库，并使用标准的Python工具（虚拟环境和pip）来安装该库。

让我们逐步了解将此代码转换为可重用库所需的步骤。我们缺少一个`setup.py`文件，所以让我们创建一个。这是一个很少创建的文件，其格式有点复杂。因为这个文件通常只创建一次，所以你不会经常处理它们。此外，`setup.py`文件可能会让人困惑。它是注册并上传到PyPI的入口点。它指明了包的元数据是什么：作者是谁、版本、许可证、主页等等。它控制源压缩包、二进制文件和wheel文件的创建方式。它指定哪些文件（代码和数据文件）属于一个发行版。它还指导Python如何安装一个包。它可以指定依赖项以及应安装的控制台可执行文件。此外，它还可以指定测试套件和测试运行器。

PyCharm 有一个命令可帮助生成`setup.py`文件的框架。运行“工具” -> “创建`setup.py`”命令。这将弹出一个窗口，您可以在其中提供有关项目的数据。输入版本号（0.1.0是一个不错的起点）、网址、许可证、作者、电子邮件和描述。运行该命令后，您将得到一个如下所示的`setup.py`文件：

```
from setuptools import setup

setup(
    name='Calculator',
    version='0.1.0',
    packages=['calculator'],
    url='https://calculat.or',
    license='MIT',
    author='Matt Harrison',
    author_email='matt@calculat.or',
    description='A Pythonic calculator'
)
```

11. Packages

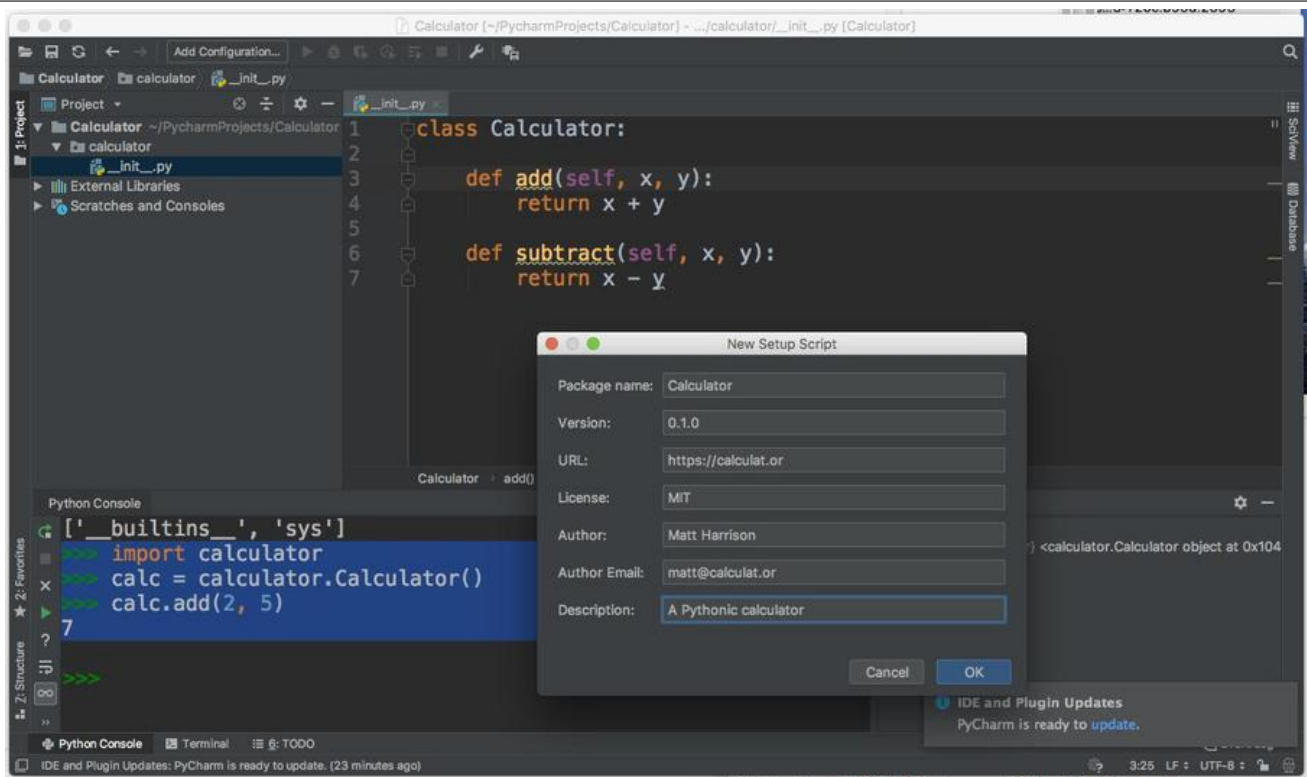


图116: 说明在“创建setup.py”命令窗口选项中进行填写的示意图。

在我们的例子中，此文件的重要部分在于，将packages参数设置为一个包含我们包名称的列表。此参数指明哪些文件夹将被打包到这个库中。

此时我们可以在PyPI上注册这个项目，创建一个源发行版，并上传到PyPI。由于这只是一个简单示例，我不会这么做，但命令应该是`python setup.py register sdist upload`。（此外，“Calculator”这个名称在PyPI上已被使用，所以上传会失败）。如果这个名称未被占用，我们可以使用`pip install Calculator`安装这个包。

如果你想在正在进行的另一个项目中使用这个库，可以通过上传到PyPI的流程，并使用pip进行安装。有时你可能想跳过使用PyPI。以下是两种无需上传到PyPI即可安装软件包的方法。如果你在进行新项目时想更新Calculator代码，更好的方法是在Calculator目录内使用项目虚拟环境运行`pip install -e .`。如果你不需要编辑Calculator，可以运行`pip install .`。

11.3 总结

在本章中，我们讨论了如何从现有代码编辑现有包，以及如何从头开始创建一个新的Python包。我们探究了如何设置环境以便在开发模式下使用包。我们还研究了如何从头开始创建一个新库。我们可以使用PyCharm为我们生成setup.py文件的内容框架。一旦有了该文件，我们就可以上传到PyPI，或者绕过PyPI在本地安装我们的库。

11.4 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/8-python-packages` 中关于新项目的说明创建一个项目。
2. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/8-python-packages` 中关于新项目的说明，从项目创建一个包。
3. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 `tree/master/your-turn/8-python-packages` 中新项目的说明，为项目添加一项功能。
4. 访问<https://github.com/talkpython/mastering-pycharm-course> ， 按照 `tree/master/your-turn/8-python-packages` 中关于新项目的说明创建一个 `setup.py` 文件。
5. 访问 <https://github.com/talkpython/mastering-pycharm-course> ， 按照 `tree/master/your-turn/8-python-packages` 中关于新项目的说明，使用你刚刚创建的库。

第12章

性能与分析

12.1 性能分析简介

Python针对开发者速度进行了优化。你可以利用丰富的库和易读性快速创建应用程序。你可能会发现，尽管你的代码能够运行，但它可能无法提供你期望的运行时性能。你可能有一个希望响应非常迅速的网站，但它返回响应需要超过一秒的时间。我们无需逐行检查代码并尝试优化每一行，而是可以使用PyCharm的分析工具来找出速度减慢发生在哪里。通常，我们对哪些部分速度慢的直觉是错误的，因此利用这些工具可能是一项巨大的优势。

12.2 分析应用程序

在本章中，我们将研究一个基本Python应用程序的性能。该应用程序是一个情感分类器应用程序。它将在Talk Python网站上查询与给定搜索词匹配的播客。然后，利用返回的描述来确定该描述是正面还是负面的。它还会报告运行所需的时间。这是一个简单的示例，但它说明了在进行性能分析时，各种程序中可能会出现输入/输出以及一些计算。

计算情感倾向的过程需要下载三个列表。其中两个列表分别包含积极词汇和消极词汇。最后一个列表是一组被称为*stop words*的词汇。这些词汇是我们想要忽略的，因为它们很常见（例如“a”“and”“the”），只会占用空间。为了计算描述中的词汇（自然语言处理工程师称之为*corpus*）是否为积极词汇，我们会去除所有停用词，然后计算差值，并除以词汇数量。最大值1.0表示所有词汇都是积极的，而-1.0则表示所有词汇都是消极的。

这是用于计算Talk Python描述情感倾向的程序。该文件名为*sentiment.py*。在GitHub上的Talk Python精通Pycharm课程仓库中也可获取²²。

```
"""
* get stopwords
* get positive words
* get negative words
```

²³https://github.com/talkpython/mastering-pycharm-course/tree/master/extra_samples/profiling/

12. 性能与分析

```
* get corpus
* score = (count positive - count negative) / (num words)
bounds[-1, 1]
"""

import os
import re
import time
import urllib

import requests

BASE = 'https://raw.githubusercontent.com/jeffreybreen/'\
'twitter-sentiment-analysis-tutorial-201107/master/'\
'data/opinion-lexicon-English/'
NEG_URL = f'{BASE}negative-words.txt'
POS_URL = f'{BASE}positive-words.txt'

STOPWORDS_URL = 'https://raw.githubusercontent.com/'\
'stanfordnlp/CoreNLP/master/data/edu/stanford/nlp/'\
'patterns/surface/stopwords.txt'

def get_url(url, fname):
    if os.path.exists(fname):
        return
    fin = requests.get(url)
    data = fin.text
    with open(fname, 'w') as fout:
        fout.write(data)

def get_line_words(fin):
    r"""
    >>> get_line_words([': .hello world, my name!\n', 'is matt.\n'])
    ['world', 'my', 'name', 'is', 'matt']
    """
    book_words = []
    word_re = re.compile(r'(\w*)')

    for line in fin:
        for word in line.split():
            # don't use match it checks start of line
            valid = word_re.search(word)
            if valid and valid.group(1):
                book_words.append(valid.group(1))
    return book_words

def get_posts(query):
    enc = urllib.parse.urlencode({'q': query})
    url = f'http://search.talkpython.fm/api/search?{enc}'
    res = requests.get(url)
    return [d for d in res.json().get('results')]

def get_score(words):
    if not len(words):
```



```

        return 0.0

    neg = [line.lower().strip() for line in open('neg.txt')]
    pos = [line.lower().strip() for line in open('pos.txt')]
    stop = [line.lower().strip() for line in open('stop.txt')]

    pcount = 0
    ncount = 0
    remove = 0
    for word in words:
        if word in stop:
            remove += 1
            continue
        if word in pos:
            pcount += 1
        if word in neg:
            ncount += 1

    score = (pcount - ncount) / (len(words) - remove)
    return score

def main():
    get_url(NEG_URL, 'neg.txt')
    get_url(POS_URL, 'pos.txt')
    get_url(STOPWORDS_URL, 'stop.txt')
    posts = get_posts('numpy')

    for post in posts:
        words = get_line_words([post['description']])
        print(f"{post['title'][:20]:20} Score: {get_score(words):6.3f}")

if __name__ == '__main__':
    start = time.time()
    main()
    print(f"\ntook {time.time() - start:.3} seconds")

```

请注意，此应用程序使用第三方Requests库。你需要将其安装到活动的虚拟环境中才能运行代码。

创建一个运行配置并执行该文件。输出将打印出描述的标题和分数。在我的机器（2015 年款 MacBook）上，它打印的内容为：

```

Python in Finance      Score: -0.045
Serverless software    Score:  0.000
...
PyPy - The JIT Compi   Score: -0.034
Effective Python       Score:  0.000
Python in Brain Rese   Score:  0.093
took 2.04 seconds

```

报告指出，运行该程序需要近两秒的时间。让我们使用PyCharm的分析功能，找出时间花在了哪里。

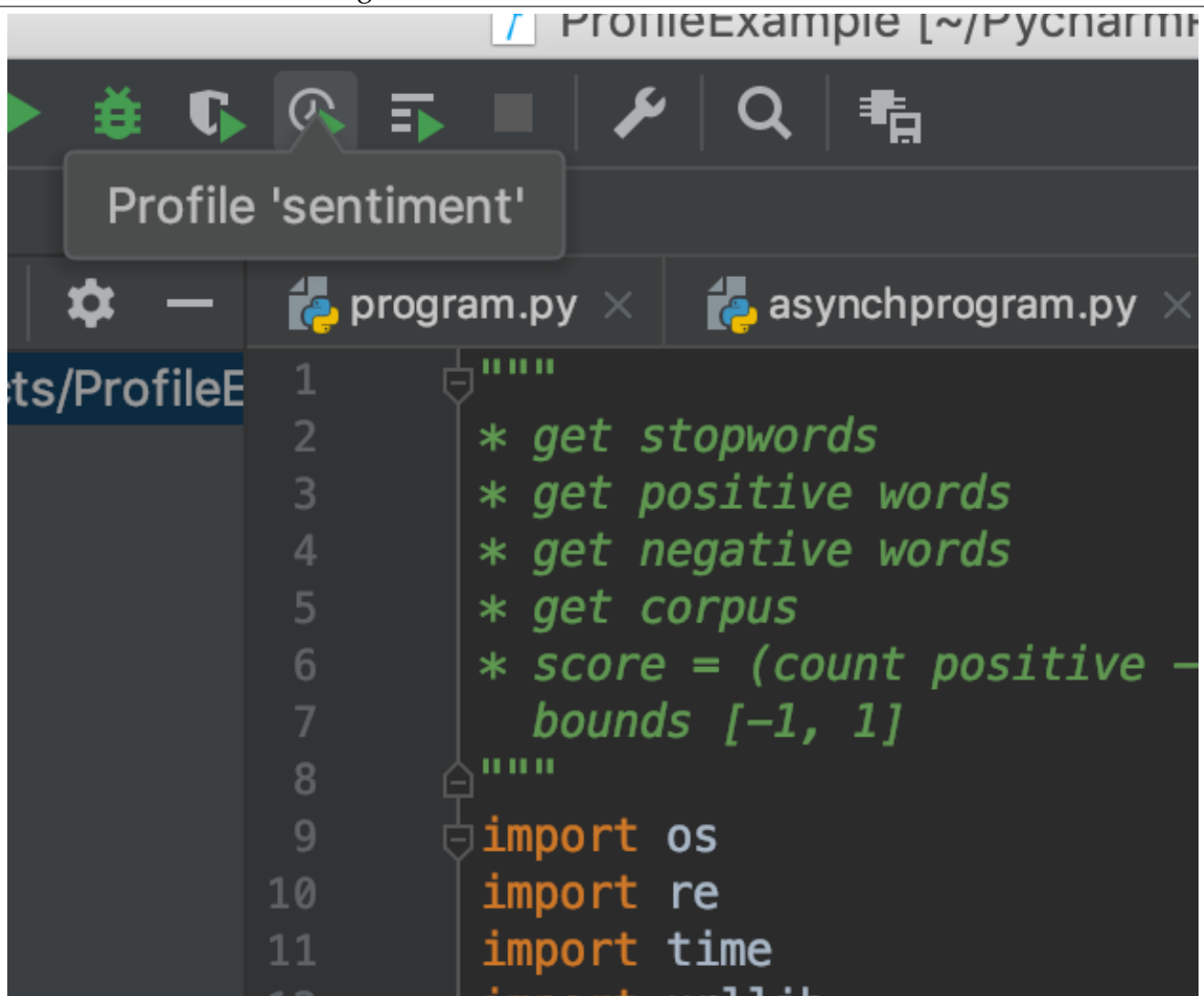


图121: 说明点击个人资料按钮的示意图

12.3 捕获配置文件信息

要捕获配置文件信息，请运行“Profile 'sentiment'”命令（其中sentiment是PyCharm项目中运行配置的名称，该配置将执行我们命名为sentiment.py的上述文件）。你需要点击带有时钟的按钮来执行此操作。

运行分析器后，您应该会看到打开了一个选项卡，其中包含两个子选项卡。默认选项卡“统计信息”包含一个分析信息表。表格顶部是列名。“名称”列列出正在分析的函数或方法。默认视图应按函数（或方法）运行时间排序。运行时间最长的函数排在第一位。“调用次数”列指示该函数被调用的次数。“时间（毫秒）”列显示该函数及其调用的任何函数中的代码所花费的时间。最后一列“自身时间（毫秒）”指示该函数中的代码花费了多少时间。在不调用任何其他函数的函数中，最后两列将相同。

该表告诉我们，get_score 函数占用了超过72%的时间。其中，27%的时间花在了 get_score 函数本身。该表没有告诉我们是什么代码调用了 get_score 或者 get_score 调用了什么。这就是“调用图”选项卡发挥作用的地方。此选项卡有一个图，

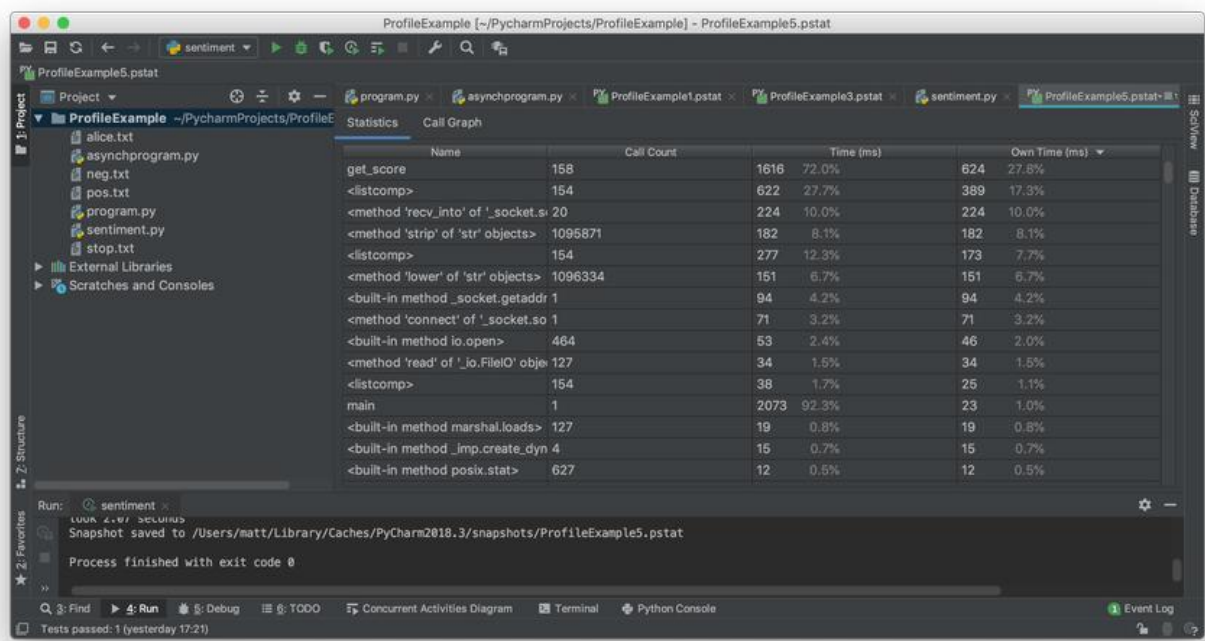


图122：说明配置文件信息表的图。

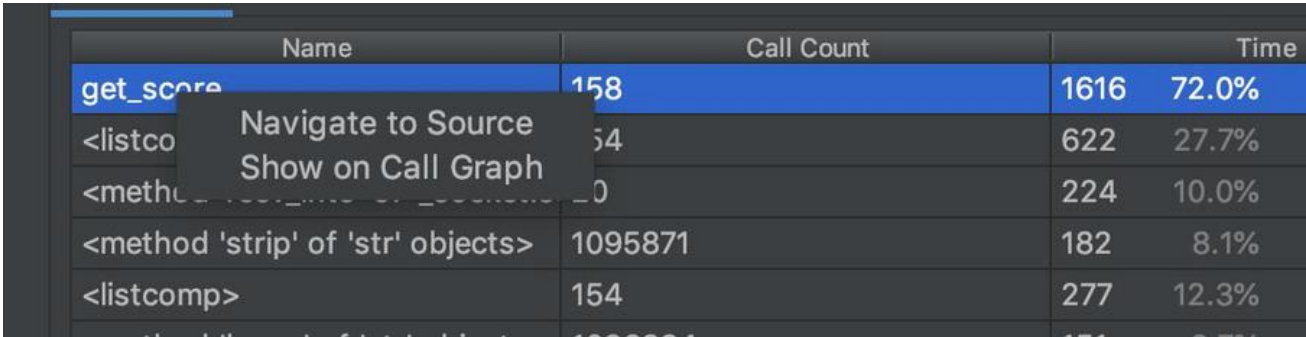
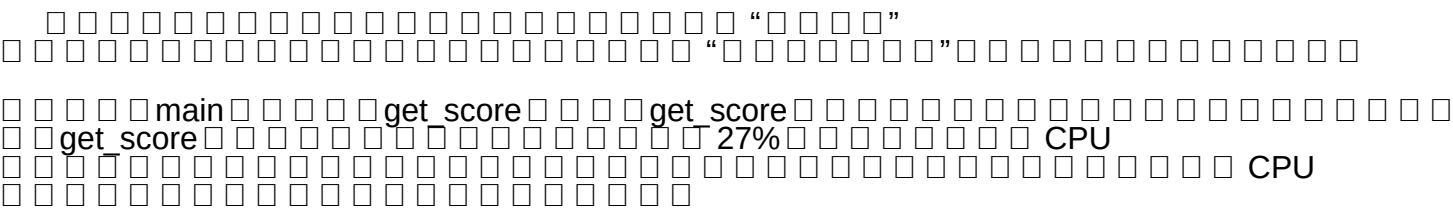


图123：说明右键单击该函数以进入图形的示意图。



如果我们在代码中四处查看，就会发现也存在一些I/O 绑定的代码。对get_url和get_posts的调用会通过网络获取数据。这段代码是I/O 绑定的，我们在等待网络返回页面结果。为了加快速度，我们需要迁移到异步代码，使用线程将等待操作批量处理，或者使用缓存。

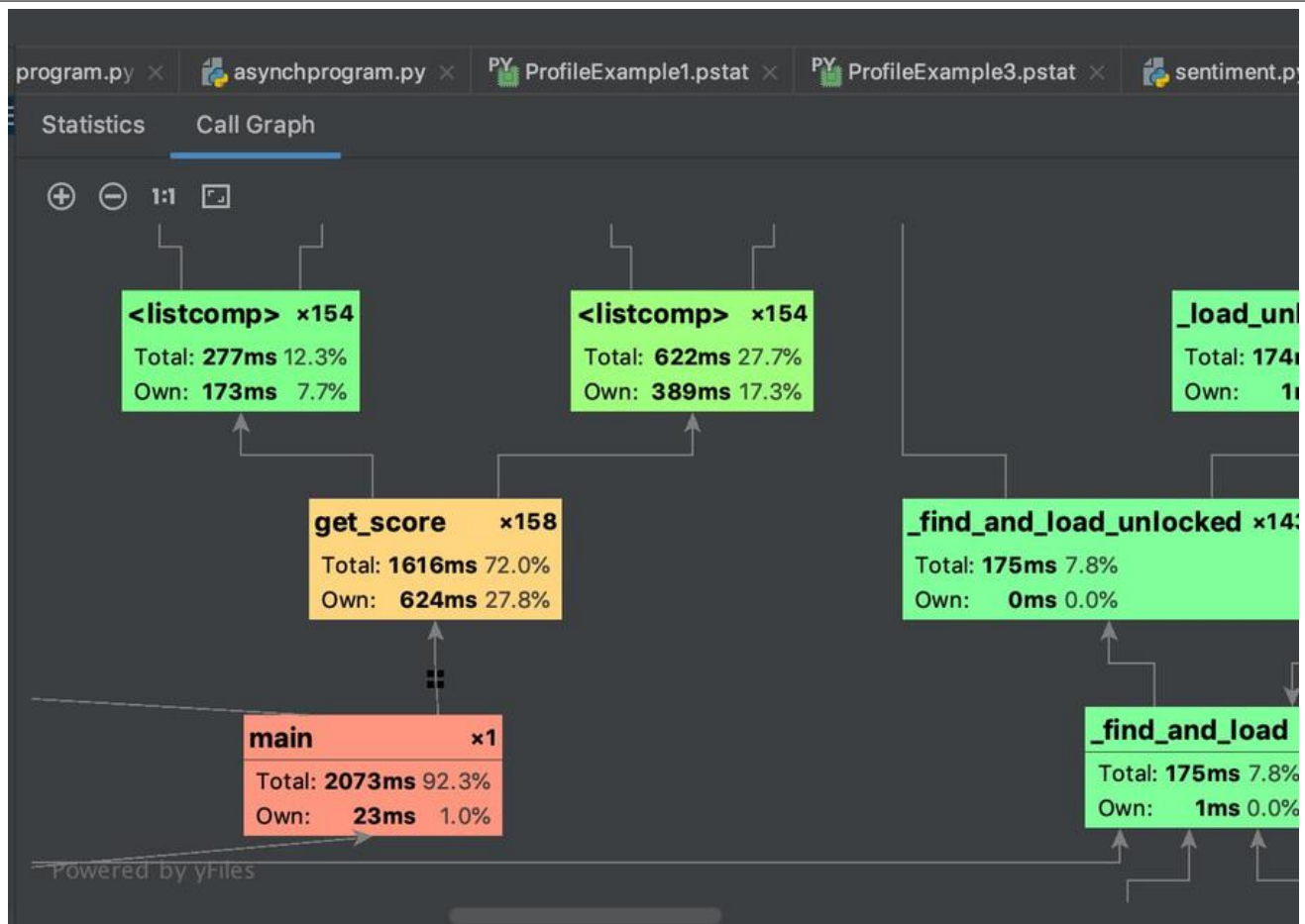


图124：说明调用图信息的图。

注意

如果已安装yappi，PyCharm将使用它进行分析。默认情况下，PyCharm使用标准库中的cProfile库。Yappi具有每个线程的调试信息、分析装饰器和其他功能。

12.4 处理 CPU 密集型代码

大部分时间都花在 `get_score` 函数上。这是最容易解决的问题，也是我们首先应该考虑优化的地方。在该函数中，我们遍历 `words` 变量，同时也遍历 `stop`、`pos` 和 `neg` 列表。Python 中的 `in` 运算符通过遍历列表来对其进行操作。因此我们有嵌套循环，这是寻找更好算法或数据结构的主要对象。我们需要遍历语料库中的单词，但无需反复遍历停用词列表或正负词列表。通过简单的重构，将列表推导式改为集合推导式，我们应该能够对集合进行快速哈希查找，而不是遍历列表。

将 `get_score` 改为以下内容并重新运行代码：

```
def get_score(words):
    if not len(words):
        return 0.0
```

```

neg = {line.lower().strip() for line in open('neg.txt')}
pos = {line.lower().strip() for line in open('pos.txt')}
stop = {line.lower().strip() for line in open('stop.txt')}

pcount = 0
ncount = 0
remove = 0
for word in words:
    if word in stop:
        remove += 1
        continue
    if word in pos:
        pcount += 1
    if word in neg:
        ncount += 1

score = (pcount - ncount) / (len(words) - remove)
return score

```

在我的机器上，运行时间从2秒缩短到了1.4秒。通过修改6个字符，性能提升了30%。这是从工具中获得的深刻见解。但我们仍可以做得更好。

再次运行性能分析，我们发现现在大部分时间都花在了集合推导上。实际上，neg、pos 和 stop 集合被反复创建。当我们发现自己多次做同一件事时，这表明我们应该缓存结果（并且只计算一次）。

让我们对main和get_score再进行一次重构。我们将在main函数中计算这些集合，然后将它们传入get_score函数：

```

def get_score(words, neg, pos, stop):
    if not len(words):
        return 0.0

    pcount = 0
    ncount = 0
    remove = 0
    for word in words:
        if word in stop:
            remove += 1
            continue
        if word in pos:
            pcount += 1
        if word in neg:
            ncount += 1

    score = (pcount - ncount) / (len(words) - remove)
    return score

def main():
    get_url(NEG_URL, 'neg.txt')
    get_url(POS_URL, 'pos.txt')
    get_url(STOPWORDS_URL, 'stop.txt')
    posts = get_posts('numpy')

    neg = {line.lower().strip() for line in open('neg.txt')}

```

12. Performance and Profiling

```
pos = {line.lower().strip() for line in open('pos.txt')}
stop = {line.lower().strip() for line in open('stop.txt')}

for post in posts:
    words = get_line_words([post['description']])
    score = get_score(words, neg, pos, stop)
    print(f"{post['title'][:20]:20} Score: {score:6.3f}")
```

现在代码运行时间为0.44秒，提升了78%！

此时，配置文件输出中的统计信息表明，耗时最长的调用是`_socket.so`的`recv_into`方法。仅表格视图本身并不能告诉我们是誰在调用这段代码。但是你可以右键单击并选择“在调用图中显示”，从而发现是`requests`库在调用这段代码。结果发现，现在耗时最长的是等待数据从Web服务器返回。这段代码现在受限于输入/输出（IO）。我们在等待IO操作，而不是等待CPU完成任务。

处理I/O受限代码的典型方法包括使用多线程或异步代码，以消除对多个调用的等待。缓存常见调用也能解决这个问题，这是以存储更多数据为代价来实现更快的访问。（`get_url`代码正在缓存对获取积极、消极和停用词列表的调用。）在这种情况下，我们每个URL只进行一次HTTP调用，因此缓存对此并无帮助。

12.5 总结

本章以一个简单应用为例，对其运行时性能进行了优化。我们使用了PyCharm的分析功能，快速查看哪些代码耗时最多。通过调用图，我们能够了解代码的调用位置。通过使用合适的数据结构和一些缓存，我们将运行时间缩短了78%。分析功能并非PyCharm所独有，但它与开发环境的紧密集成，让你可以在同一应用中快速进行分析，无需分散注意力，也无需在不同工具间切换。这对提高开发者的工作效率大有裨益。

12.6 命令

- *"Profile 'program'"* - 点击带时钟的按钮
- *"Show on Call Graph"* - 右键单击个人资料统计信息

12.7 练习

1. 如果你想尝试进行性能分析，可以在以下网址获取本章所涉及的情感分析应用程序的原始版本：https://github.com/talkpython/mastering-pycharm-course/tree/master/extra_samples/profiling/

尝试在这个项目上运行分析器，看看你是否能发现与我们相同的问题。看看你还能做出哪些其他性能改进。

2. 打开一个最近的项目，并在分析器下执行它。什么占用的时间最多？

第13章

单元测试

13.1 测试简介

一般来说，单元测试及测试并不总是软件开发的重点。但它们常常能区分业余开发者和专业开发者。PyCharm 与单元测试及相关功能（如代码覆盖率）深度集成。在本章中，我们将探讨如何利用 PyCharm 和 pytest 库来测试代码并确保质量。

13.2 测试逻辑

在本章中，我们将创建一个函数，该函数接受一个字符串并将其转换为数字列表。你可能在打印机对话框中见过类似的情况，在那里你可以指定要打印第1页到第3页以及第6页。你可以输入1-3,6。我们需要一个函数，它接受这个字符串并将其转换为列表[1,2,3,6]。

让我们使用测试驱动开发（TDD）来创建这个函数，`parse_int`。TDD背后的理念是，先为你想要的功能编写一个测试。这个测试最初应该是失败的。然后你可以实现相应逻辑，使测试通过。TDD的支持者声称，它将开发重点放在你实际需要的内容上，而且你还能获得测试。创建代码不一定要使用TDD，但我们将在本章展示一个相关示例。

13.3 编写测试

在PyCharm中创建一个具有如下目录结构的新项目：

```
TestExample/  
  numparser.py  
  test_parser.py
```

由于我们将使用`pytest`库，因此将测试文件命名为`test_parser.py`。`pytest`库将以`test_`开头或以`_test.py`结尾的文件识别为测试文件。我们将遵循这一约定。

打开`test_parser.py`并创建一个测试。创建pytest测试最简单的方法是创建一个以`test`开头的函数，并在其中使用`assert`语句。以下是一个基本测试：

```
from numparser import parse
```

```
def test_basic():
```

13. 单元测试

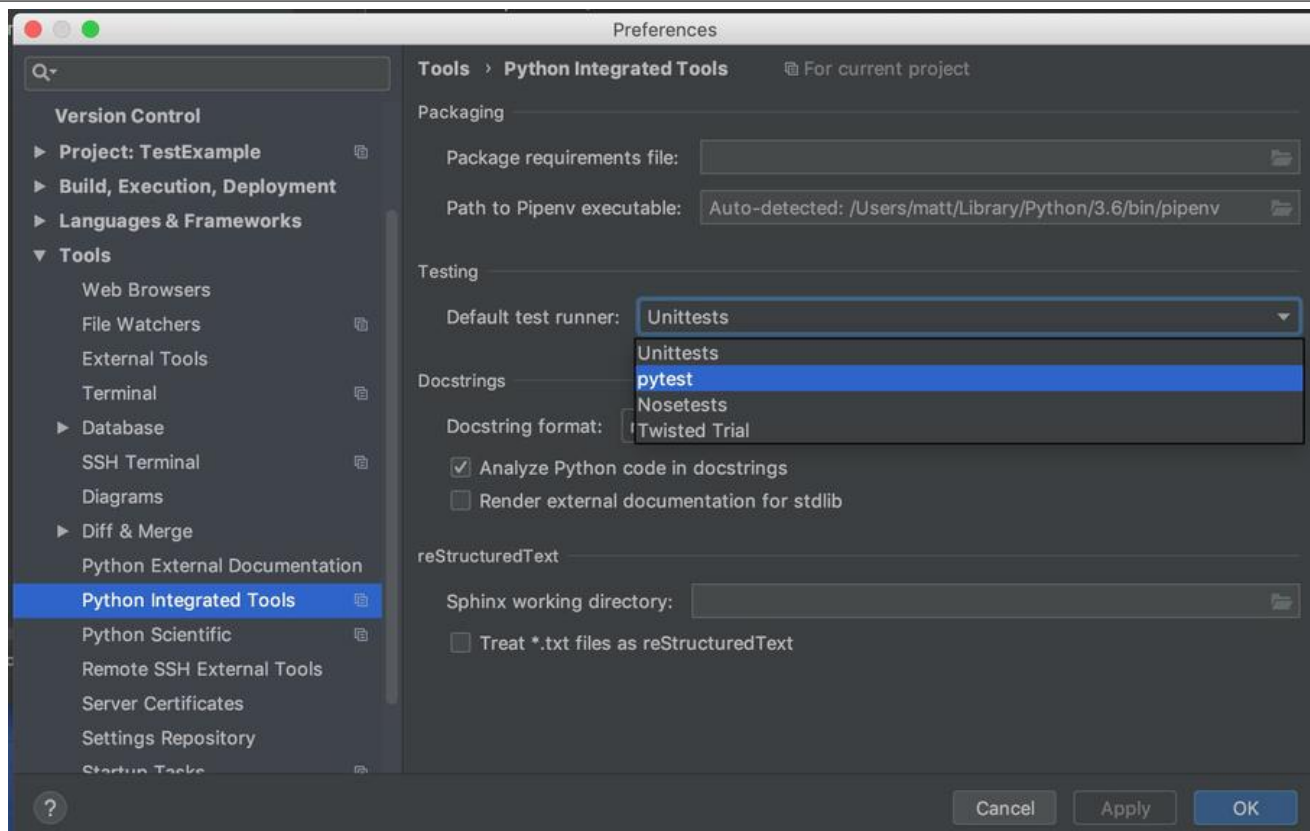


图131：将“默认测试运行器：”更改为pytest的示意图。

```
result = parse('1,3')
assert result == [1, 3]
```

PyCharm 中的默认测试运行器是 unittest 库。让我们配置 PyCharm 使用 pytest 来运行测试。打开“Preferences”并搜索“Python 集成工具”。在该窗口中，你可以将“测试” -> “默认测试运行器：”选项更改为 pytest。

将默认测试运行器设置为 pytest 后，你应该会看到 test_basic 旁边的代码空白处出现一个绿色三角形。如果你点击它，就可以运行测试（或对其进行性能分析，或收集覆盖率信息）。点击“Run 'pytest for test_pars...'” (ctrl-shift-F10)，你会看到下方出现一个“运行”选项卡，并显示错误。

`pytest` 库不在标准库中。我的项目没有安装 `pytest`，所以我需要安装它。你有几种安装它的方法。“Python 集成工具”窗口底部有一条警告，内容为“在所选解释器中未找到 `pytest` 运行器”。有一个“修复”按钮，你可以点击它来安装。或者，你可以添加这一行：

```
import pytest
```

进入 test_parser.py 并使用代码意图修复导入。

此时，我们会得到一个 ImportError，因为我们还没有实现 parse。咱们继续。
在这段英文似乎不完整或有误，不太明确确切意思，你可以检查并补充更准确内容以便更好翻译
)

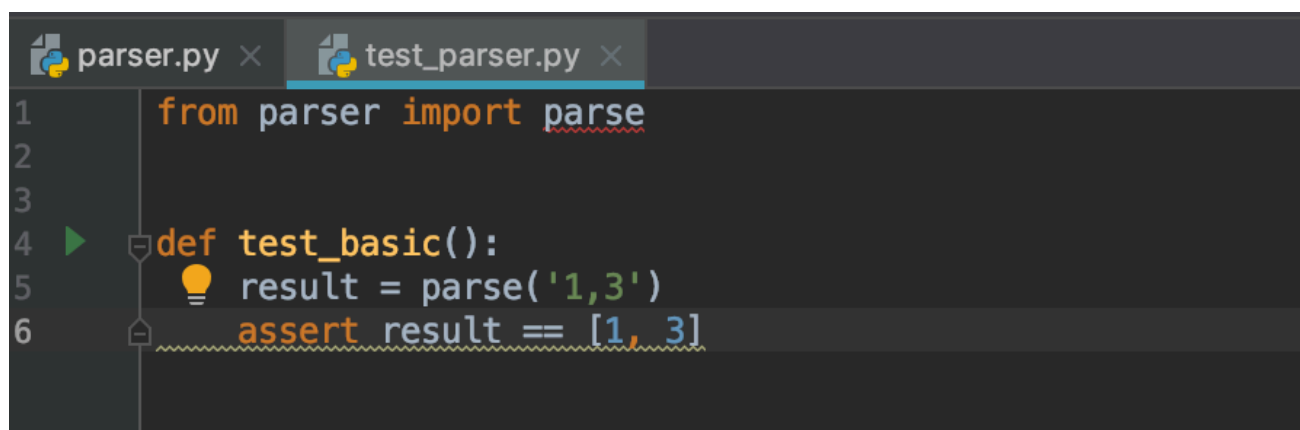


图132：当将pytest设置为默认测试运行器时，在测试函数旁边出现的三角形的示意图。

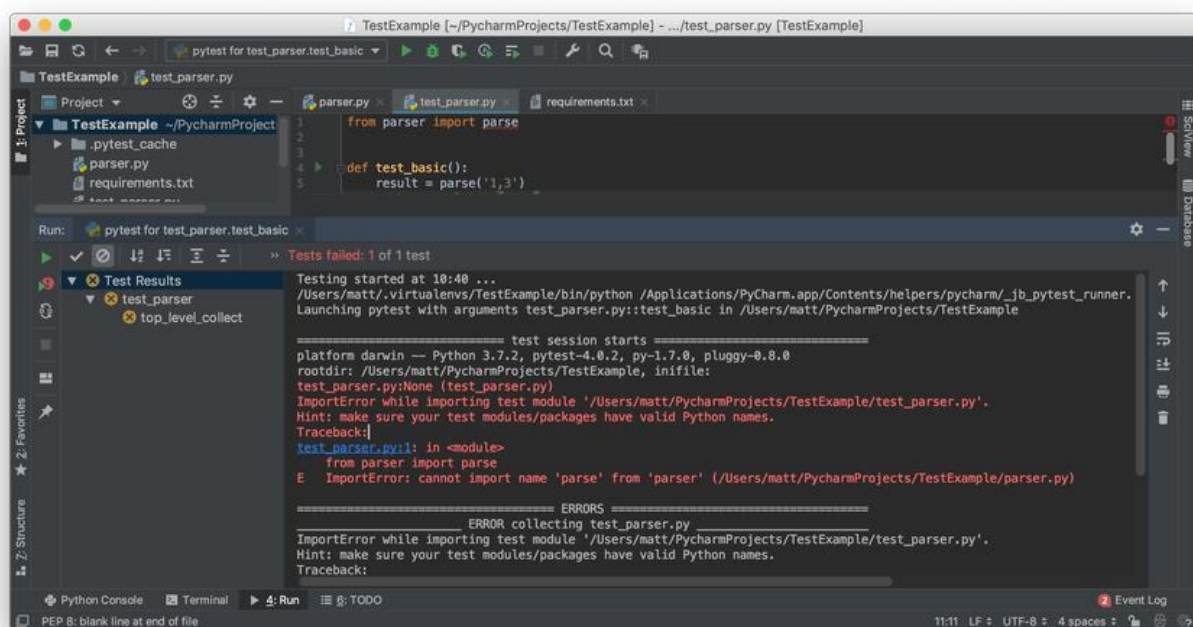


图133：说明TDD错误的图。我们正在测试的代码尚未实现。

13. Unit testing

13.4 实现逻辑

因为我们正在使用测试驱动开发（TDD），所以parse函数的功能由我们的测试来确定。我们需要实现指定以逗号分隔的整数列表的功能。以下是一种可能的实现方式：

```
def parse(txt):
    result = []
    for num in txt.split(','):
        result.append(int(num))
    return result
```

如果重新运行测试，你应该会看到类似这样的输出：

```
===== test session starts =====
platform darwin -- Python 3.7.2, pytest-4.0.2, py-1.7.0, pluggy-0.8.0
rootdir: /Users/matt/PycharmProjects/TestExample, inifile:
collected 1 item

test_parser.py . [100%]

===== 1 passed in 0.02 seconds =====
Process finished with exit code 0
```

此输出会告知您所使用的操作系统（darwin 代表 MacOS），以及您正在使用的 Python 和 pytest 库的版本。（py 和 pluggy 库是 pytest 的依赖项。）“rootdir”是 pytest 用于搜索测试文件的目录。“inifile”是一个 pytest.ini（或 setup.cfg 或 tox.ini 文件），用于自定义 pytest 的行为。在这种情况下，不存在 inifile。下一行报告 pytest 找到了 1 个测试。在 test_parser.py 中有一个测试，且它占有所有测试的 100%。如果您仔细查看该行，会看到一个句点。这表明在 test_parser.py 中的测试通过了。pytest 库对测试还有其他状态，其中包括 s（跳过）、F（失败）和 E（异常）。

13.5 审视失败

pytest 库试图让调试失败变得非常容易。让我们使测试失败并查看输出。将测试代码更改为：

```
def test_basic():
    result = parse('1,3')
    assert result == []
```

现在 pytest 的输出如下：

```
===== test session starts =====
platform darwin -- Python 3.7.2, pytest-4.0.2, py-1.7.0, pluggy-0.8.0
rootdir: /Users/matt/PycharmProjects/TestExample, inifile:
collected 1 item

test_parser.py F
test_parser.py:3 (test_basic)
[1, 3] != []

Expected :[]
Actual   :[1, 3]
<Click to see difference>
```

```

def test_basic():
    result = parse('1,3')
>    assert result == []
E    assert [1, 3] == []
E        Left contains more items, first extra item: 1
E        Use -v to get the full diff

test_parser.py:6: AssertionError

[100%]

===== FAILURES =====
----- test_basic -----

    def test_basic():
        result = parse('1,3')
>        assert result == []
E        assert [1, 3] == []
E            Left contains more items, first extra item: 1
E            Use -v to get the full diff

test_parser.py:6: AssertionError
===== 1 failed in 0.10 seconds =====
Process finished with exit code 0

```

pytest 不仅会通知我们断言失败，还会使用一些巧妙的方法向你展示断言失败的原因。在 PyCharm 中，如果点击 <Click to see the difference>，还会集成弹出“比较失败”窗口。

在 PyCharm 底部“运行”选项卡的左侧，有用于“Rerun pytest” (Ctrl+F5)、*“Rerun Failed Tests”* 和 *“Toggle auto-test”* 的按钮。重新运行失败的测试是快速修复失败测试的好方法，不会受到通过测试的干扰。自动测试是一种在代码每次更改时运行测试的模式。代码更改后会有轻微延迟，然后测试会自动运行。

注意

pytest 有许多 PyCharm 未显示的功能，但你可以从“终端”窗口访问这些功能。例如，运行：

```
pytest -x --lf
```

将首先运行上次失败的测试。并且在首次失败后停止。这是一种很好的调用方式，有助于专注于一次修复一个失败的测试用例。当你将代码迁移到不同平台或重构代码时，这非常实用。

在 PyCharm 中拥有 pytest 运行配置后，你可以打开右上角通常用于选择要运行的配置的“Edit configurations”，并在“Additional Arguments”中输入 `-x -lf`，以便 PyCharm 使用此选项运行。

13.6 添加更多测试

让我们为 `parse` 函数添加另一个特性，即能够接受用短横线表示范围。首先，我们将在 `test_parser.py` 中创建一个测试：

13. Unit testing

```
def test_dash():
    result = parse('1-3')
    assert result == [1, 2, 3]
```

请注意，如果您之前点击了代码行号栏中的三角形来运行`test_basic`，那么点击“运行”选项卡上的绿色三角形或顶部的三角形（“Run”命令）时，PyCharm 将只运行`test_basic`测试，而不会运行新的测试。在运行配置的名称中，您会看到“`pytest for test_parser.test_basic`”，这表明该运行配置仅适用于单个测试。

您可以使用“*Edit Configurations...*”命令将“Target:”从`test_parser.test_basic`更改为`test_parser`，然后“Run”（shift - F10）命令将运行文件中的所有测试。

注意

在实际项目中，我们可能会有多个测试文件。如果PyCharm能一键运行所有测试，那就太好了。

如果将“Target:”改为“Script path”并将其设置为“.”（不带引号的一个句点），PyCharm 将运行在工作目录中找到的所有测试文件。

运行所有测试的输出应该如下所示：

```
Testing started at 14:12 ...
~/venvs/bin/python /Applications/PyCharm.app/_jb_pytest_runner.py
--path test_parser.py
Launching pytest with arguments test_parser.py in
/Users/matt/PycharmProjects/TestExample

===== test session starts =====
platform darwin -- Python 3.7.2, pytest-4.0.2, py-1.7.0, pluggy-0.8.0
rootdir: /Users/matt/PycharmProjects/TestExample, inifile:
collected 2 items

test_parser.py .F
test_parser.py:7 (test_dash)
def test_dash( ):
>         result = parse('1-3')

test_parser.py:9:
- - - - -

txt = '1-3'

    def parse(txt):
        result = []
        for num in txt.split(','):

>         result.append(int(num))
E         ValueError: invalid literal for int() with base 10: '1-3'

numparser.py:15: ValueError

[100%]

===== FAILURES =====
----- test_dash -----
```

```

def test_dash( ):
>     result = parse('1-3')

test_parser.py:9:
-----

txt = '1-3'

def parse(txt):
    result = []
    for num in txt.split(','):

>         result.append(int(num))
E         ValueError: invalid literal for int() with base 10: '1-3'

numparser.py:15: ValueError
===== 1 failed, 1 passed in 0.15 seconds =====
Process finished with exit code 0

```

其中隐藏着这样一句话：

```
test_parser.py .F
```

其中包含一个句点和一个“F”。句点表示通过测试，“F”表示失败。在调试时，它正在引发一个ValueError。

让我们更新numparser.py中的实现以支持短横线：

```

def parse(txt):
    result = []
    for num in txt.split(','):
        if '-' in num:
            start, end = num.split('-')
            result.extend(range(int(start), int(end)+1))
        else:
            result.append(int(num))
    return result

```

现在测试应该可以正常运行。

13.7 测试异常

我们还希望能够测试异常情况。parse 函数不会显式引发异常，但如果我们向其传入错误数据，该函数应该引发异常。我们可以使用 `pytest` 来断言是否引发了异常。让我们弄清楚会引发什么异常。

查看引发何种异常最简单的方法是使用错误输入运行代码。我更喜欢在解释器中这样做。如果你右键单击源代码（在编辑器中，而不是在“项目”选项卡中的文件名上），会有一个命令，“Run File in Console”。如果你点击该命令，“Python控制台”将打开并加载文件内容。

我在控制台中输入了以下内容以查看错误：

```

>>> parse('bad')
Traceback (most recent call last):
  File "<input>", line 1, in <module>
  File "TestExample/numparser.py", line 8, in parse
    result.append(int(num))
ValueError: invalid literal for int() with base 10: 'bad'

```


13. Unit testing

看起来如果传入一个字符串，它会抛出一个`ValueError`。我们来编写一个测试来处理这个问题。到目前为止，我们的`test_parser.py`文件中没有任何地方引用`pytest`。当我们使用`pytest`调用测试时，`pytest`能够确定什么是测试以及如何处理`assert`语句。然而，`assert`语句无法处理异常。为此，我们需要使用一个特殊的上下文管理器`raises`，它可以在`pytest`中找到。将`test_parser.py`文件更新为：

```
from numparser import parse

import pytest

def test_basic():
    result = parse('1,3')
    assert result == [1, 3]

def test_dash():
    result = parse('1-3')
    assert result == [1, 2, 3]

def test_bad():
    with pytest.raises(ValueError):
        parse('bad')
```

我们可以使用`with`语句以及`pytest.raises`上下文管理器来测试上下文管理器块中是否发生了异常。

`pytest`库还有许多其他特性，这些特性足以单独写成一本书。我们鼓励你进一步探索这个库，并在Python测试中加以利用。除了其自带的众多特性外，它还有一个丰富的插件生态系统来进一步增强其功能。

13.8 代码覆盖率

代码覆盖率是一份关于有多少行代码被执行的报告。它通常与测试紧密相关。由于Python不像Java或C那样经过编译，我们无法获得编译代码可能带来的那种安全感。通过了解我们已经对代码进行了测试，我们可以获得那种感觉。毕竟，在许多情况下，Python代码的语法直到执行时才会被验证。经过测试的代码行必然具有有效的语法。而正是代码覆盖率会告诉我们在运行测试时执行了哪些代码行。

PyCharm让运行代码覆盖率分析变得很容易。我们需要点击“*Run 'pytest for test_parser' with Coverage*”命令。该图标在绿色运行按钮后面，形似盾牌。此外，一旦你针对一组特定测试设置了运行配置，“运行代码覆盖率分析”（图标相同）也会出现在运行和调试按钮附近。

注意

你需要安装一个用于覆盖率的库。当你执行“*Run 'pytest for test_parser' with Coverage*”命令时，你将有选项安装`coverage.py`库。

当为运行覆盖率正确配置PyCharm后，运行覆盖率命令后将出现一个“覆盖率”选项卡。此选项卡将显示代码覆盖率的汇总统计信息。通常，我们最关心的是非测试代码的覆盖率数字。与PyCharm的另一个出色集成是，覆盖率结果会显示在代码空白处。代码行旁边的空白处会有绿色或红色，以指示是否存在代码覆盖。

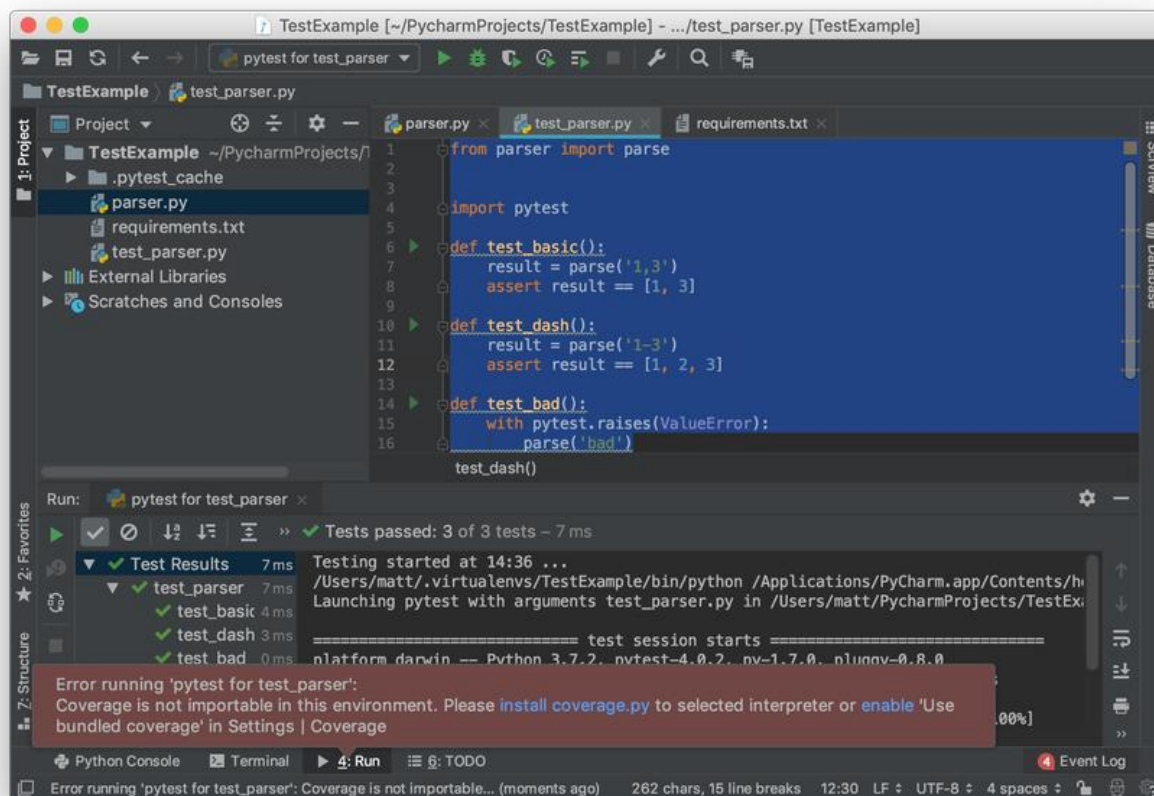


图134：说明缺失覆盖依赖关系的图。

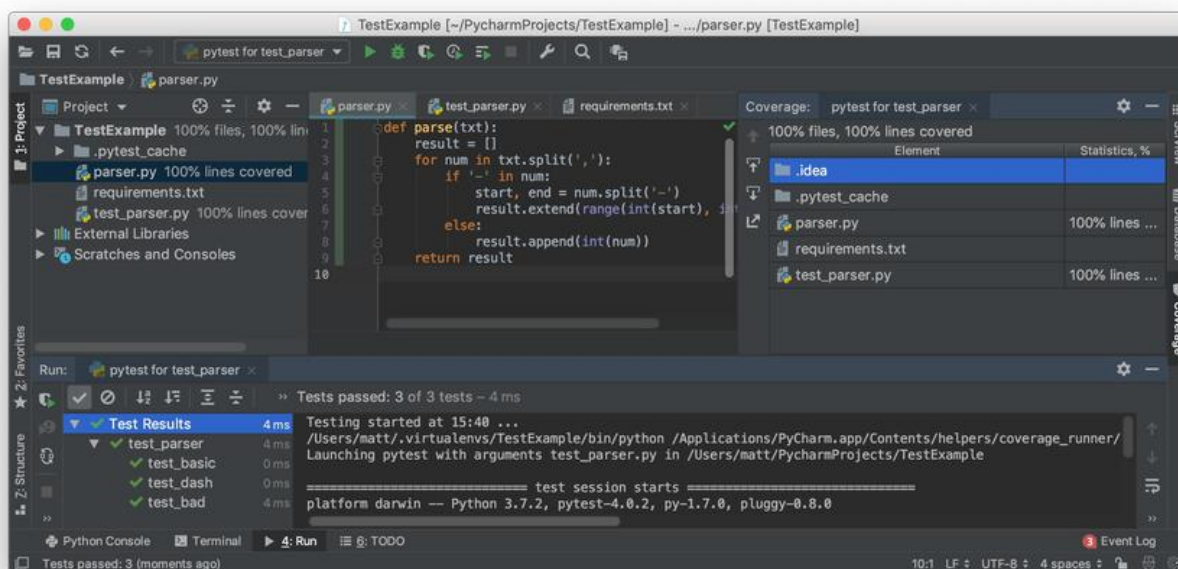


图135：说明覆盖范围摘要的图。

13. Unit testing

对于我们的`parse`函数，我们实现了100%的覆盖率。关于覆盖率数值，有几点需要说明。数值越高越好。但是，100%的覆盖率仅意味着代码行被执行了，并不意味着代码没有逻辑错误。此外，大多数代码覆盖率工具只处理行覆盖率。在程序执行过程中，代码中存在“分支”和“路径”，它们可能会根据代码的输入而变化。

一种更详尽的测试和覆盖类型是全路径覆盖，即测试代码中不同的可能路径。在实践中，这种覆盖类型很难实现，因为大多数工具并不支持它，但它也需要投入大量精力来理解代码以及执行路径的组合。大多数有覆盖相关指标的公司会选定一个他们希望达到的数字，并确保新的代码提交不会降低这个数字。

13.9 总结

在本章中，我们使用测试驱动开发来实现一个将字符串解析为数字列表的函数。我们使用了`pytest`库，因为它使测试创建变得非常容易。我们仅仅触及了这个库功能的皮毛。最后，我们利用PyCharm中的集成功能查看代码覆盖率统计信息。在集成环境中能够使用这些库对Python开发者来说是一大福音。

13.10 命令

- *"Run 'pytest for test_pars...'"* - (按Ctrl+Shift+F10)
- *"Rerun pytest"* - (按Ctrl+F5)
- *"Rerun Failed Tests"*
- *"Toggle auto-test"*
- *"Run File in Console"*
- *"Run 'pytest for test_parser' with Coverage"*

13.11 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照tree/master/your-turn/10-testing中新项目的说明，在现有项目中使用`pytest`。
2. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照tree/master/your-turn/10-testing中新项目的说明，使用覆盖率工具。

第14章

数据科学工具

14.1 科学观点简介

Python常用于数值处理。像numpy、pandas和matplotlib这样的库提供了基础，使得如今Python成为数据科学和机器学习领域最受欢迎的技能之一。使用Python进行分析需要一种与应用开发略有不同的工作流程。与构建工具并进行部署不同，它通常更具交互性和迭代性。你对某些数据进行过滤、排序和调整，然后查看结果。PyCharm有一个“科学模式”来辅助这种使用方式。它还集成了Jupyter。

14.2 科学模式

在创建PyCharm项目时，除了“纯Python”项目外，还有一个选项是“科学”项目。这将开启“科学模式”，该模式提供了用于查看图表和检查数据的视图。另一种启用科学模式的方法是在代码中添加以下行：

```
import numpy as np
```

当你导入numpy库时，PyCharm会检测到这一点，并询问你是否要开启科学模式。在下一个示例中，我们将提取燃油经济性数据，并绘制每年汽车厂商的品牌数量。有了合适的数据，pandas库可以轻松完成这项工作。使用以下布局创建一个新项目：

```
Data/
```

```
    explore.py
```

将以下代码放在explore.py中：

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
import numpy as np
```

```
###
```

```
auto = pd.read_csv(
```

```
    'https://www.fueleconomy.gov/feg/epadata/vehicles.csv.zip')
```

```
###
```

```
# Just look at Ford, Lexus, & Toyota
```

```
(auto
```

```
    .groupby(['year', 'make']))
```

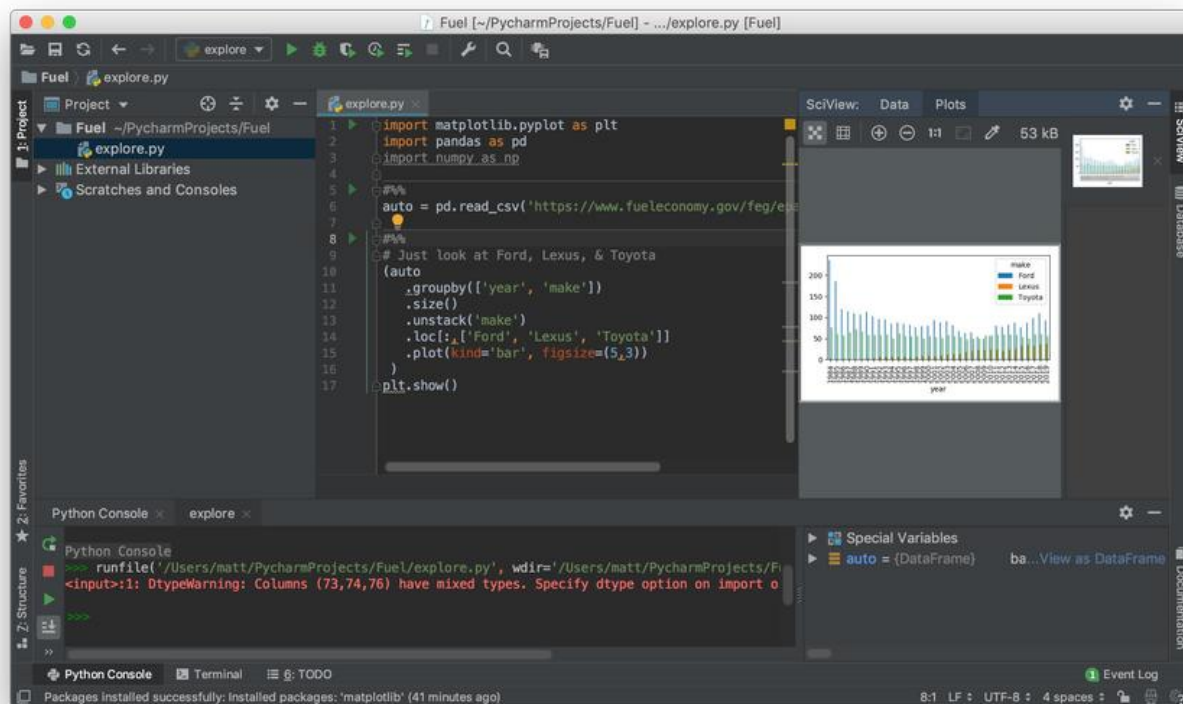


图141：展示在科学模式下运行带绘图代码的示意图。

```

.size()
.unstack('make')
.loc[:, ['Ford', 'Lexus', 'Toyota']]
.plot(kind='bar', figsize=(14,10))
)
plt.show()

```

注意

对于大多数Python开发者来说，pandas代码周围的括号看起来有点奇怪。这在pandas中很常见，因为我们倾向于链式操作。有了括号，我们就不需要把所有代码都写在一行上。我们将链式操作中的每个操作都单独写在一行，这样更容易理解发生了什么。

这段代码将按年份和品牌对数据进行分组，然后计算每个组的大小，接着将品牌列（实际上是一个索引）拉成一行（这样我们就能得到每年的每个品牌），然后只提取福特、雷克萨斯和丰田，最后绘制一个柱状图。

如果你输入这段代码，PyCharm 在看到numpy这一行时会询问你是否要进入“科学模式”。我的代码导入了matplotlib，但没有调用它。我这样做是为了能通过代码意图轻松安装它。（或者，你可以打开终端并对其执行pip install操作）。你还需要安装pandas。

如果你运行这段代码，它可能会思考一会儿，但在“SciView”中，会有一个“Plots”选项卡，其中包含生成的图表。

你还会注意到，在“Python控制台”窗口中，有一个带有解释器的“探索”选项卡。该解释器具有`explore.py`的命名空间。从那里我们可以访问`auto`数据框，并与数据进行交互。如果你已经安装了iPython（从“终端”安装`pip install ipython`），你将得到一个iPython提示符，而不是普通的Python提示符。

右侧是命名空间中变量的列表。在这种情况下，只有`auto`。有一个名为“以DataFrame形式查看”的可点击区域。当您点击它时，“数据”的SciView选项卡将填充一个数据表。

还有一点需要注意。由于我们的代码中包含以`###`开头的注释，PyCharm 将其识别为代码“单元格”。该单元格由紧跟该注释之后的代码组成，直到下一个`###`注释。它会在每个单元格旁边放置一个绿色的执行三角形，我们可以独立运行或重新运行这些单元格。PyCharm 还会方便地为第一个`###`之前的初始代码创建一个单元格。

使用带有解释器的Python并能够访问先前的变量，对于交互式探索来说非常棒。

14.3 笔记本模式

PyCharm用于科学计算的另一个特性是与Jupyter集成。Jupyter是一个开源工具，用于创建笔记本，以运行代码“单元”并显示标记。该工具曾被用于公布引力波的发现，2018年诺贝尔奖得主也曾使用Jupyter开展研究并发表成果。让我们来探索PyCharm中的特性。

我们可以使用“New”命令（Ctrl+N）来创建一个新的“Jupyter Notebook”。让我们通过使用“新建”命令并将笔记本命名为“自动探索”，将之前的自动探索代码迁移到一个笔记本中。如果未安装Jupyter，系统将弹出一个安装对话框。你可以点击“安装jupyter包”。

在PyCharm 2019.1中的笔记本编辑器会显示一个编辑器以及其旁边类似笔记本的窗格。PyCharm会将代码`###`插入到编辑器中。当你在编辑器中输入内容时，该内容也会显示在笔记本窗格中。我们给编辑器添加一些代码。我喜欢让笔记本的顶部单元格仅包含代码的导入部分。在`###`行下方添加`matplotlib`和`pandas`的导入：

```
import matplotlib.pyplot as plt
import pandas as pd
```

如果您为该项目创建了新的虚拟环境，则需要安装这些库。点击带下划线的库名称，您将获得安装库的代码意图。如果点击绿色三角形，即按“Execute Code Cell”（Ctrl+Enter），PyCharm 应启动Jupyter 并运行代码。

执行该单元格后。笔记本视图将在单元格旁边显示一个数字，并显示生成的任何输出。由于此单元格没有输出，它将显示“无输出”。

要创建一个新单元格，请在希望单元格开始的位置上方添加注释。或者，你可以点击边栏中的加号来创建代码单元格或Markdown单元格。

将以下代码添加到笔记本中：

```
###
auto = pd.read_csv(
    'https://www.fueleconomy.gov/feg/epadata/vehicles.csv.zip')

###
```

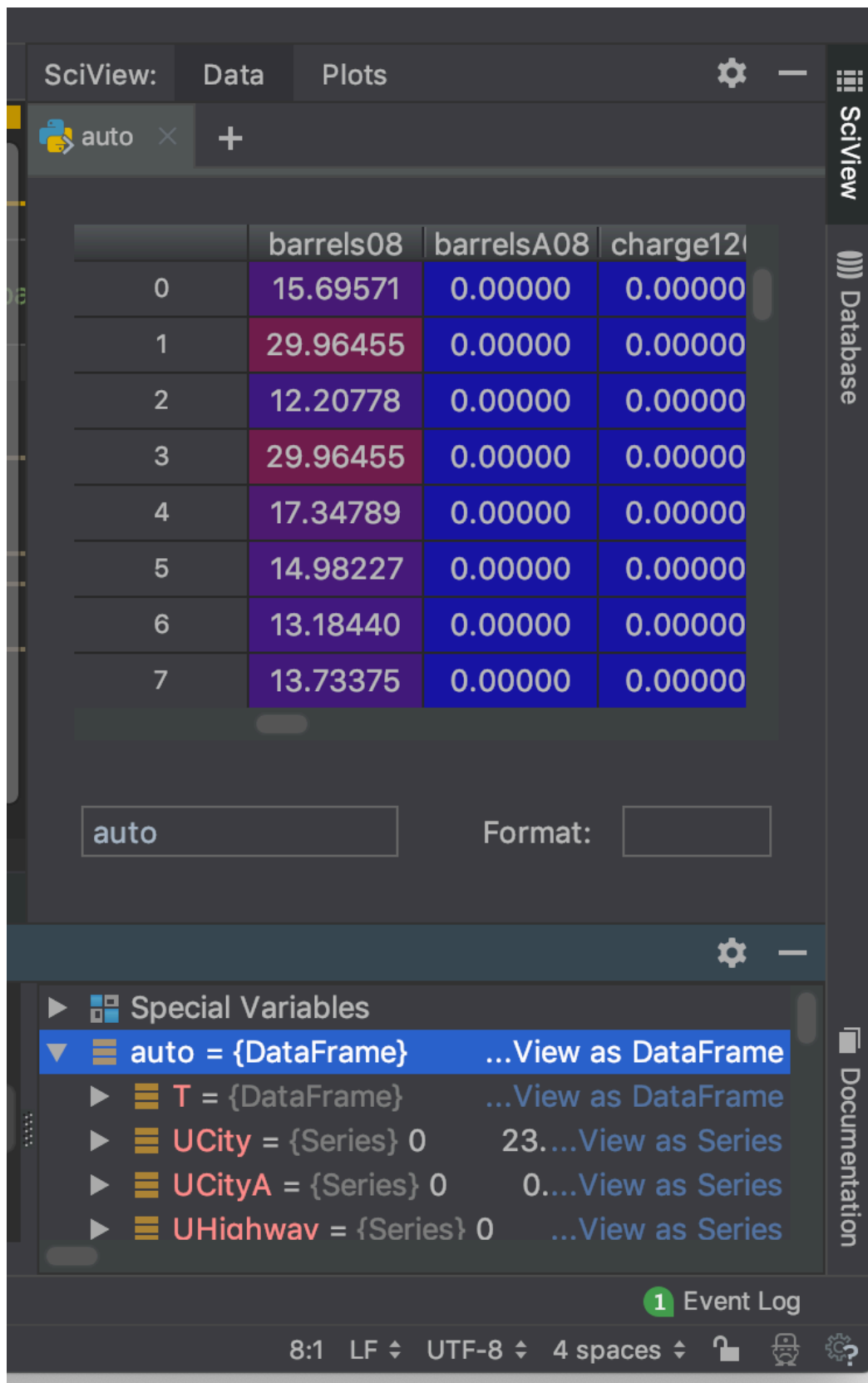


图142: 说明点击“以数据框形式查看”结果的图。

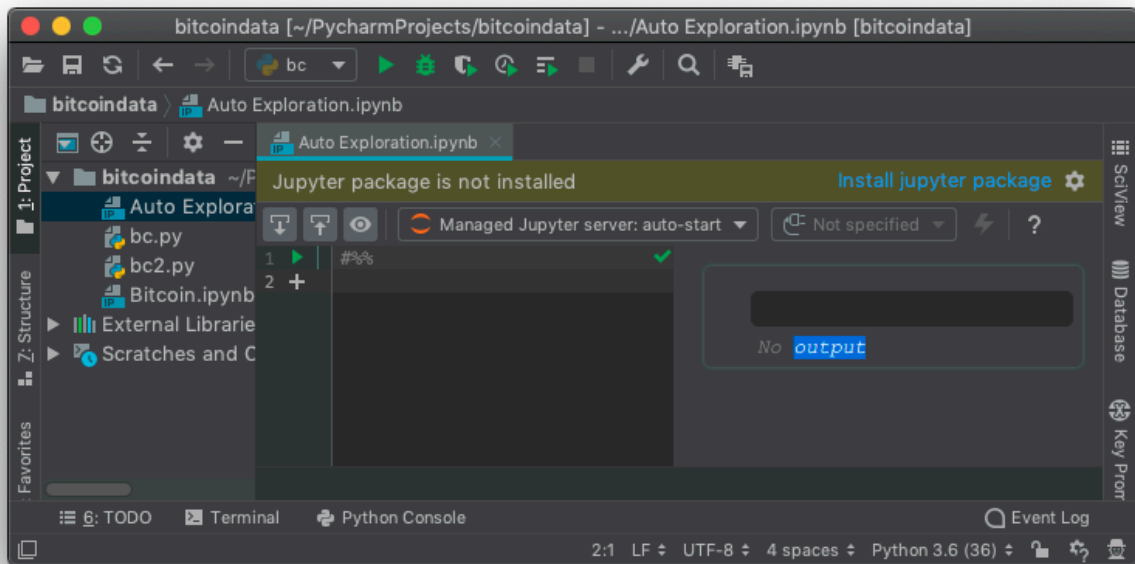


图143：展示新Jupyter笔记本以及安装“jupyter”包提示的示意图。

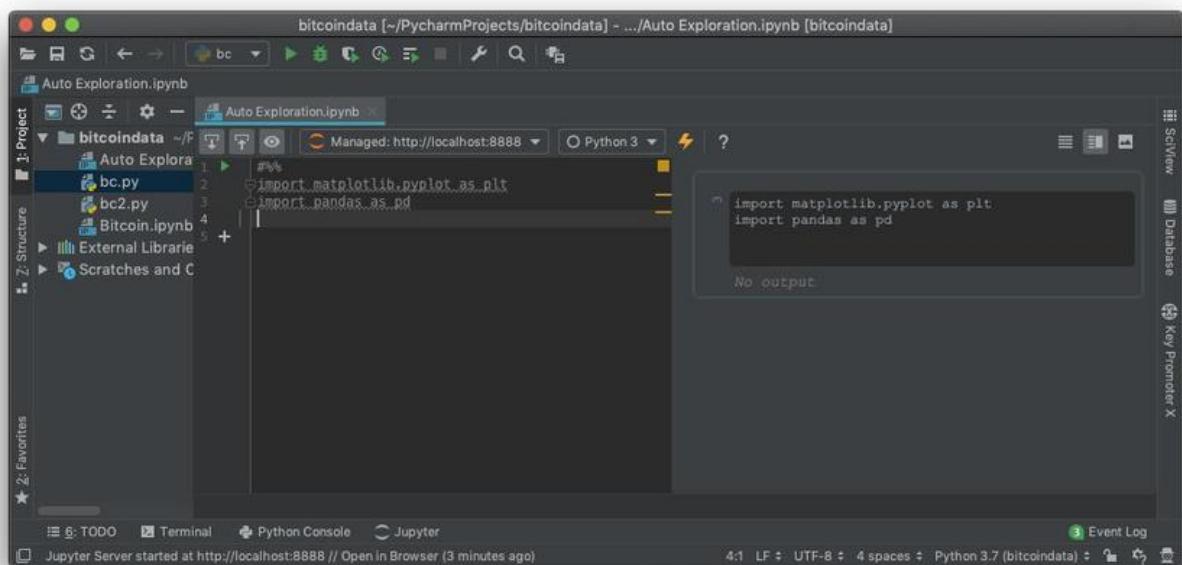


图144：说明启动新Jupyter笔记本时运行单元格的示意图。

14. Data science tools

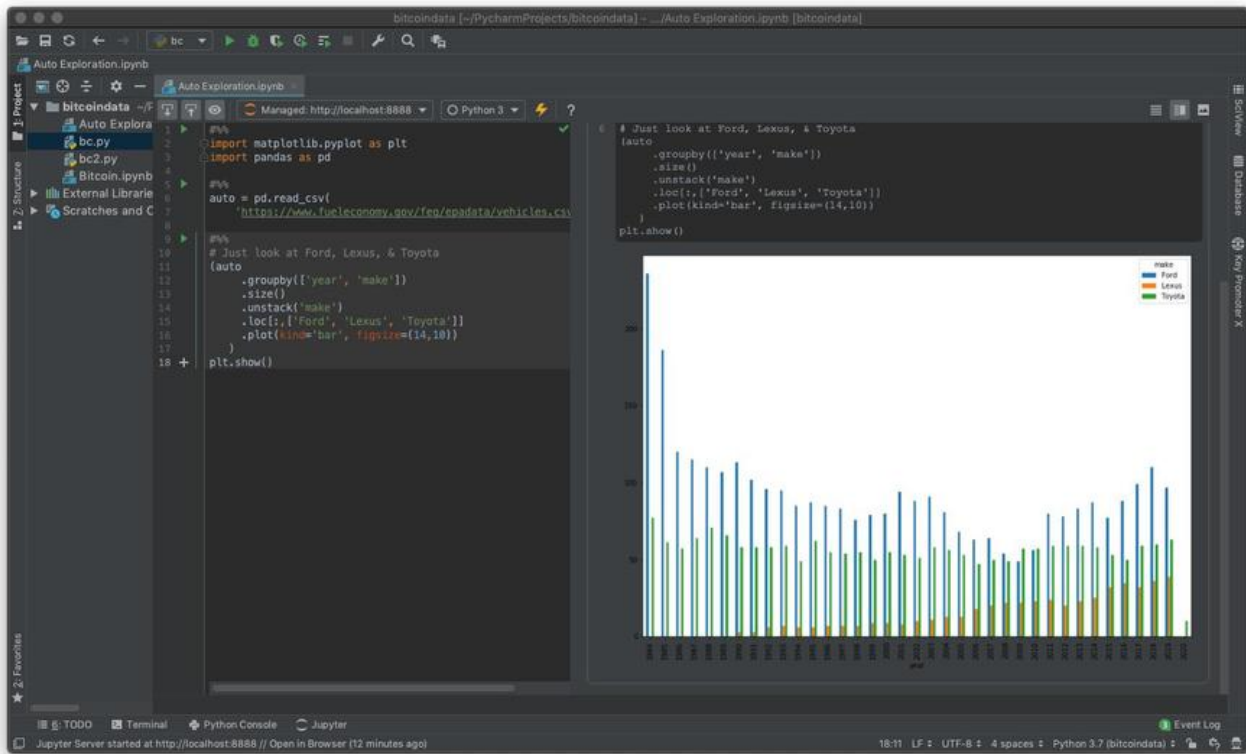


图145：说明笔记本单元格绘图的图。

```
# Just look at Ford, Lexus, & Toyota  
(auto  
    .groupby(['year', 'make'])  
    .size()  
    .unstack('make')  
    .loc[:, ['Ford', 'Lexus', 'Toyota']]  
    .plot(kind='bar', figsize=(14,10))  
)  
plt.show()
```

如果运行这些代码单元格，你会看到PyCharm将图表嵌入到笔记本区域中。

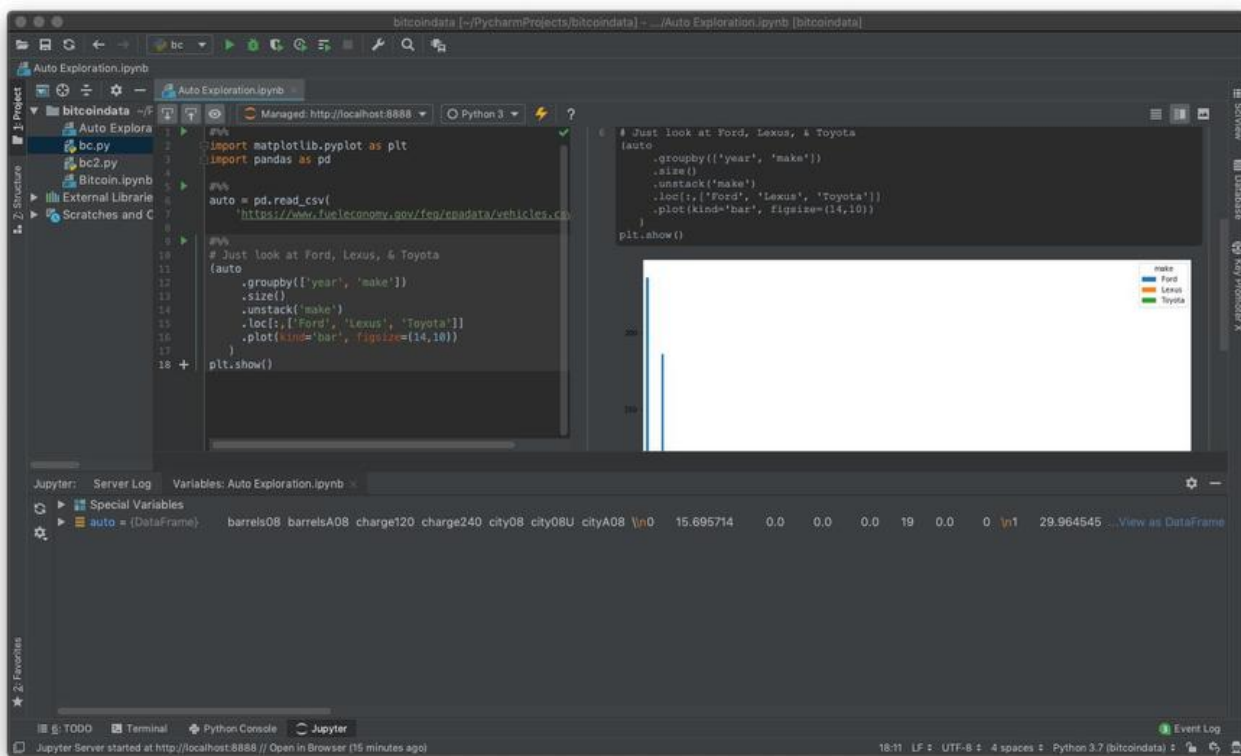


图146：展示PyCharm的Jupyter选项卡的示意图。

注意

如果你还打算在Jupyter浏览器界面中使用此笔记本，则应使用所有用于在浏览器中显示图表的 *cell magic*。单元格魔法命令是用于控制Jupyter行为的指令。我会在包含导入内容的单元格中添加以下行：

```
%matplotlib inline
```

此外，PyCharm 提供了一个“Jupyter”选项卡，用于显示笔记本中的变量。你可以点击该选项卡来浏览变量。在这种情况下，它只显示 `auto` 变量。

14.4 总结

本章介绍了科学模式以及对Jupyter笔记本的支持。在进行分析时，通常会利用REPL和交互特性来尝试各种操作并快速检查数据。这种编程风格与应用程序开发有很大不同，在应用程序开发中，我们尽量避免使用全局变量，并且倾向于编写函数和类。这并不是说普通的Python开发人员不使用REPL（他们绝对应该利用它），或者支持使用全局变量等不良编程习惯。当我为客户提供一些数据科学咨询服务时，我会使用Jupyter来探索和创建模型。在对结果满意后，我会重构代码，并应用工程实践，将函数、类、测试和打包引入到分析中。PyCharm涵盖了各种开发风格。

14.5 命令

- `import numpy` - 将此内容包含在科学模式的提示中
- `#%` - 包含此项以划分单元格
- *"Execute Code Cell"* - (按Ctrl+Enter或点击绿色按钮)

14.6 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/11-scientific-computing` 中新项目的说明，以科学模式创建一个图表。
2. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照 `tree/master/your-turn/11-scientific-computing` 中新项目的说明，使用Jupyter创建一个图表。

第15章

工具窗口

15.1 工具窗口介绍

在本章中，我们将介绍一系列工具。这些工具和实用程序对各种任务都很有帮助。我们将了解待办事项注释、运行工具窗口、Python 控制台、终端、收藏夹和结构窗口。这些是用于与代码和环境交互的工具，但并不明确属于我们已经介绍过的类别。

15.2 待办事项注释

有时你想给自己留下关于代码的注释。提醒未来的自己修复某个问题、添加某个功能或重构某些代码行。你可以通过提交一个错误报告、添加一条注释、修改README文件或在笔记本上草草记下的方式来做到这一点。PyCharm有一个“TODO窗口”来支持这种用例。如果你在文档字符串或注释中使用“FIXME”或“TODO”这个词，PyCharm会追踪这些内容，并以不同的颜色突出显示它们。

滚动条内部有一条可点击的线，用于从编辑器窗口快速导航到这些项目。此外，还有一个“TODO” (command-6或alt-6)命令，用于切换显示项目中所有待办事项的窗口。您可以展开文件夹和文件以查看这些项目，然后单击它们以跳转到编辑器中的相应待办事项。待办事项窗口的侧边有用于“Previous TODO” (alt-command-up或ctrl-alt-up)、 “Next TODO” (alt-command-down或ctrl-alt-down)、 “Filter TODO Items”、 “Autoscroll to Source”、 “View Options”以及“PreviewSource”的按钮。

默认情况下，PyCharm 会忽略大小写突出显示“todo”和“fixme”。如果你打开设置并搜索“TODO”，可以添加 PyCharm 识别的其他模式。如果你添加一个新模式，可以使用“Filter TODO Items”指定要突出显示哪些模式。

注意

这并不能替代使用合适的错误跟踪工具。

15.3 Python控制台窗口

“Python Console”窗口（“工具” -> “Python 控制台...” 或点击 “Python 控制台” 选项卡）是 Python 开发中非常有用的工具。如果运行该命令，它将在虚拟环境中启动一个 Python 交互式解释器（REPL）。它还会将项目目录添加到 sys.path 属性中。

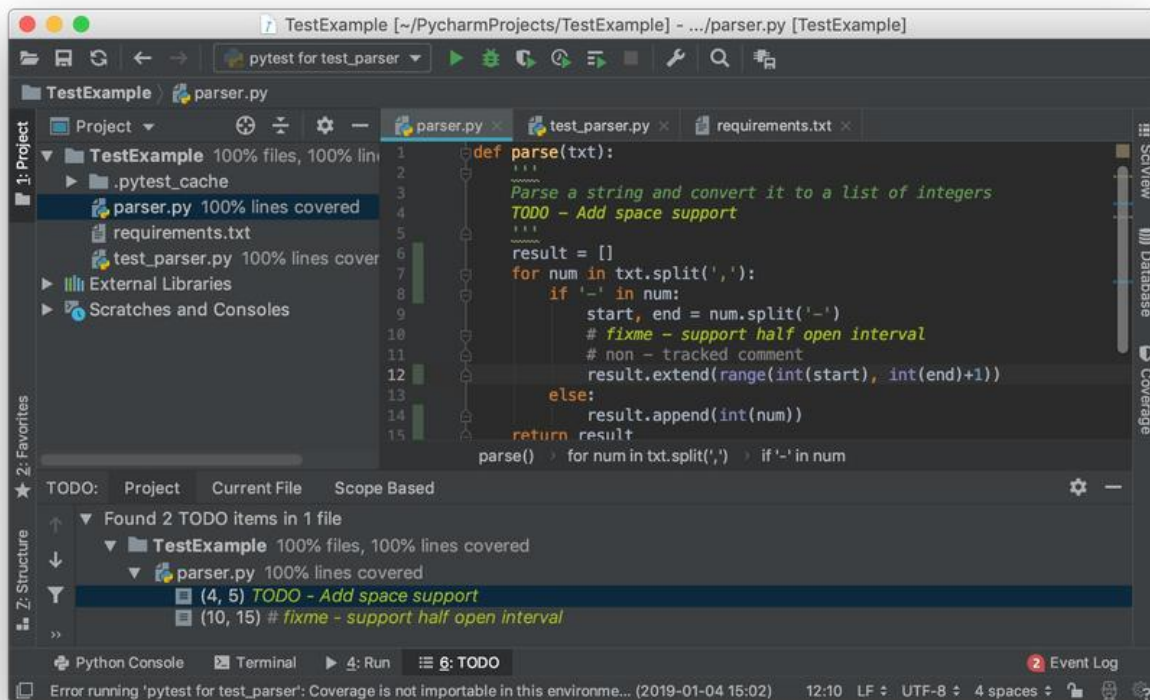


图151：用于说明待办事项文档字符串和待修复注释的图。

这使得导入项目中的源代码并在虚拟环境中使用依赖项变得很容易。

虽然这看起来与你在命令行调用python3时看到的Python交互式解释器类似，但它有一些独特之处。它包含了Python编辑器中的代码补全功能。只需在开始输入变量名或句点后按Tab键。你也可以按“Basic Code Completion” (Ctrl + Space)组合键。

你可以使用向上和向下箭头浏览命令历史记录。如果你在Python控制台中输入函数时出现拼写错误，可以按向上箭头，然后修正错误，而无需再次完整输入该函数。

标签窗口还有一个变量检查区域。局部变量会在那里显示。导入的模块可以在“特殊变量”部分下找到。可以在那里检查变量。诸如列表和字典之类的嵌套变量可以展开以查看变量的部分内容。此外，你可以运行“Set Value...” (f2) 命令来更改变量或嵌套值的值。

当你运行“运行配置”时，Python 控制台也会出现（Python 配置中有一个“使用 Python 控制台运行”选项）。通过使用运行配置，你无需导入文件中的所有全局变量即可访问它们。

如果你想运行代码的特定部分，可以直接在 Python 控制台中输入。你也可以在编辑器中突出显示代码，然后按“Execute Selection in Console...” (alt-shift-e)运行。如果你通过执行所选内容来执行函数定义，除非展开“特殊变量”层次结构，否则在变量检查区域中不会看到函数名。

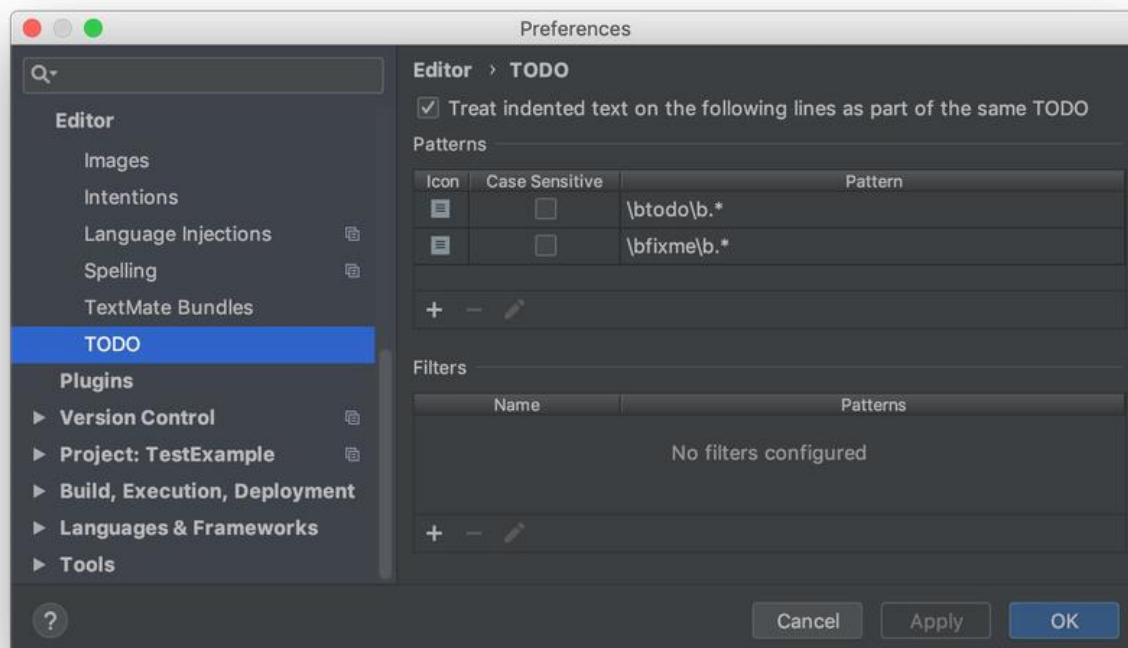


图152: 说明待办事项配置的图。

Python 控制台的一个不错的特性是，执行后它会保持打开状态。这使你即使在代码执行完毕后，也能够检查变量并与全局命名空间进行交互。

15.4 “运行” 工具窗口

“Run”窗口（Command+4 或 Alt+4）可切换上次运行的调用。当测试配置为使用测试运行器执行时，此窗口会出现。如果使用 PyCharm 启动“Jupyter Notebook”配置，也会使用此窗口。Web 运行配置（Pyramid、Flask 或 Django）同样会使用“运行”窗口。它也可用于 Python 执行（取消选中“在 Python 控制台中运行”），不过我不推荐这样做。

“运行”窗口的显著特点包括“Rerun”命令（Ctrl+F5）、“Stop”（命令-F2或Ctrl+F2）以及“Pause Output”。如果在服务器进程期间生成了大量输出，并且您想要停止并检查这些输出，那么后者会很有用。在左侧有用于这些命令的按钮。

15.5 终端窗口

PyCharm 包含一个集成的“Terminal”（Alt+F12）。在 Unix 平台上，它会在项目目录中启动一个 bash shell。在 Windows 上，它将启动一个命令提示符(cmd.exe)。它还会激活项目的虚拟环境。这使得安装软件包变得非常容易，并且您可以在终端中使用pip命令。

15. Tool Windows

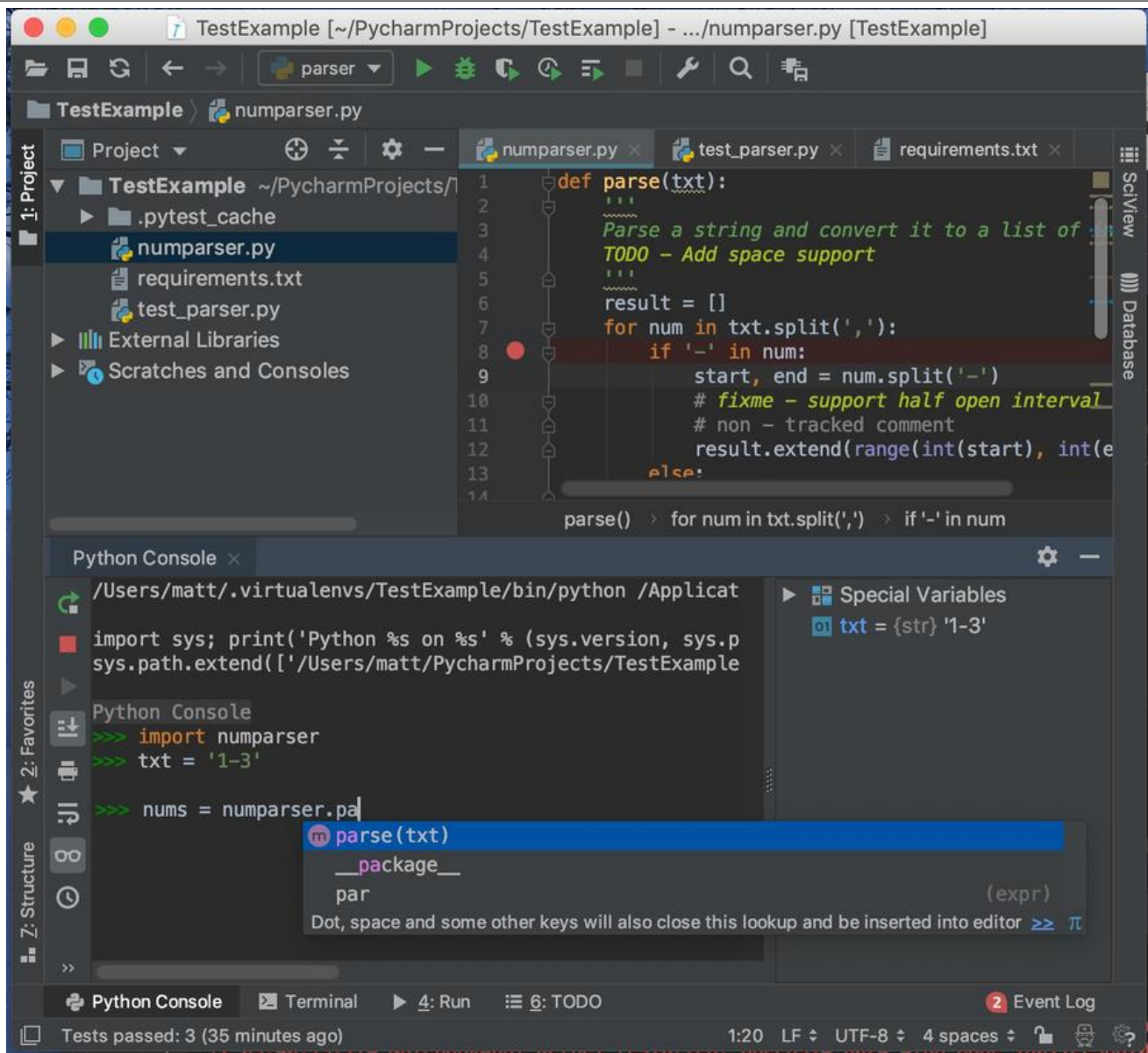


图153: 展示具有代码补全和变量检查功能的Python控制台的示意图。

要查看已安装的软件包，你可以运行以下命令：
pip list

在Windows或Unix上。

如果您更喜欢其他 shell，可以转到设置窗口的“终端”部分。在“应用程序设置”区域下有一个“Shell 路径”条目。在 Windows 上，您可以指定 powershell，对于 Zsh，可以指定 bin/zsh。

如果您需要另一个终端，请点击“New Session” (加号按钮)命令。如果您在某个终端中有一个长时间运行的进程，并且希望在其运行时执行其他操作，这会很有用。其他一些技巧包括使用鼠标中键粘贴（真希望这在编辑器窗口中也能生效），以及使用上下箭头循环浏览之前的命令。

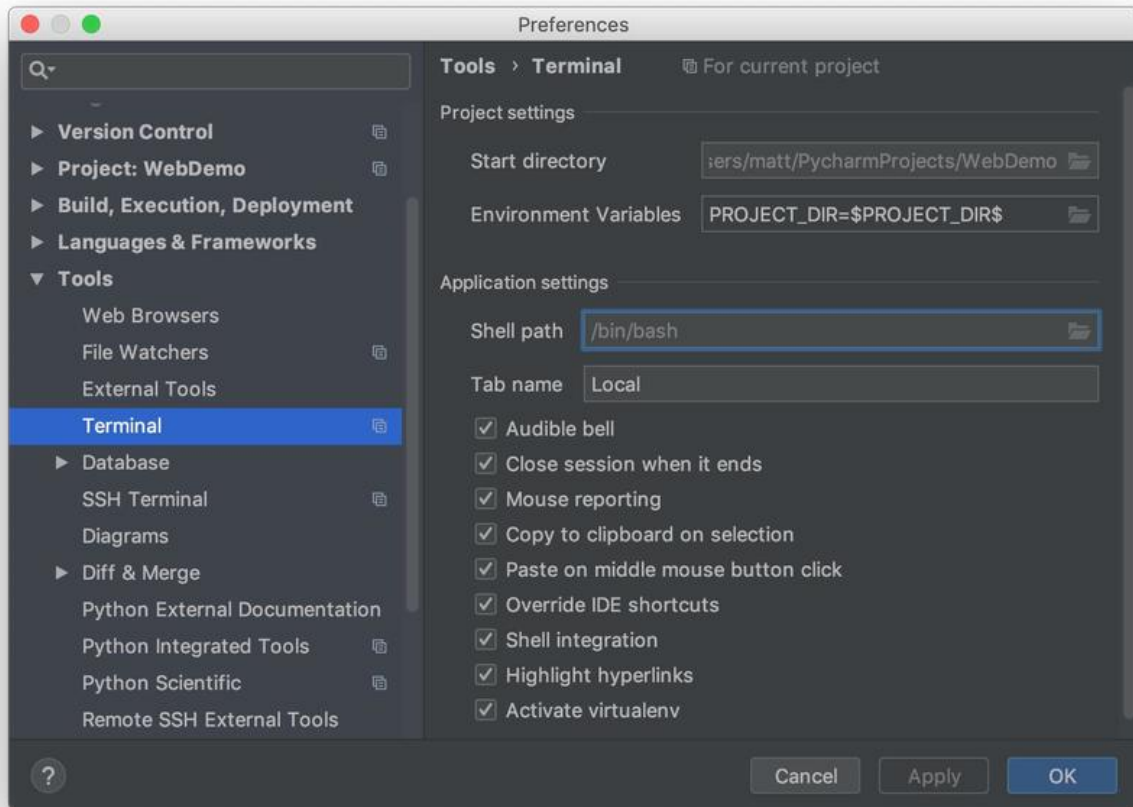


图156：说明“偏好设置”窗口中“终端”设置的图。

在Web应用程序中，添加功能或修复错误时，您可能需要编辑Python中的模型和视图代码，以及JavaScript和CSS。您可以创建一个新的收藏夹，并将所有文件添加到其中。

要将文件添加到收藏夹，请使用“Add to Favorites”命令（按alt-shift-f组合键，选择“文件”->“添加到收藏夹”，在“项目”视图中右键单击文件，或在编辑器中的选项卡上右键单击）。如果要将所有打开的文件添加到收藏夹，可以在文件编辑器的其中一个选项卡上右键单击，然后单击“Add All To Favorites”。

书签类似于收藏，但更细粒度。书签不是指向文件，而是指向文件中的某一行。当光标位于某行代码上时，你可以按“Toggle Bookmark”（F11 键，或者依次选择“导航”->“书签”->“切换书签”）。

此外，你可以创建一个“助记”书签。使用“Toggle Bookmark with Mnemonic”（command - F11或Ctrl - F11）命令，PyCharm将弹出一个对话框来选择一个单元格。单元格是单字符字母数字。如果你选择“1”作为单元格，将会有一个新的键盘“Go to Bookmark<1>”快捷键（Ctrl - 1）来导航到助记书签。你也可以使用“收藏夹”窗口来导航到书签。

书签对于跟踪文件中的特定行非常有用。利用助记书签可快速导航到常用的代码行。

最后，“收藏夹”窗口便于查看和导航到断点。当您运行“Toggle Breakpoint”（命令 - F8 或 Ctrl - F8），或者在代码旁边的空白处单击时，

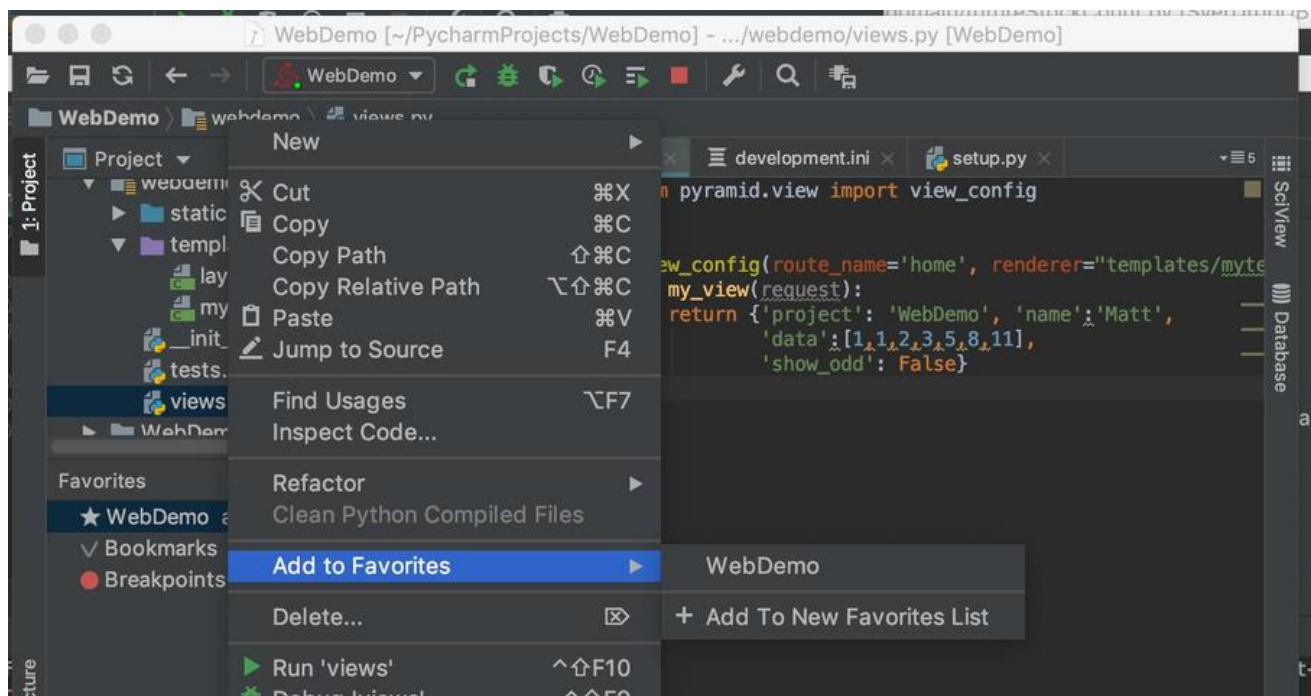


图157: 重新说明通过右键单击将文件添加到收藏夹仪式窗口。

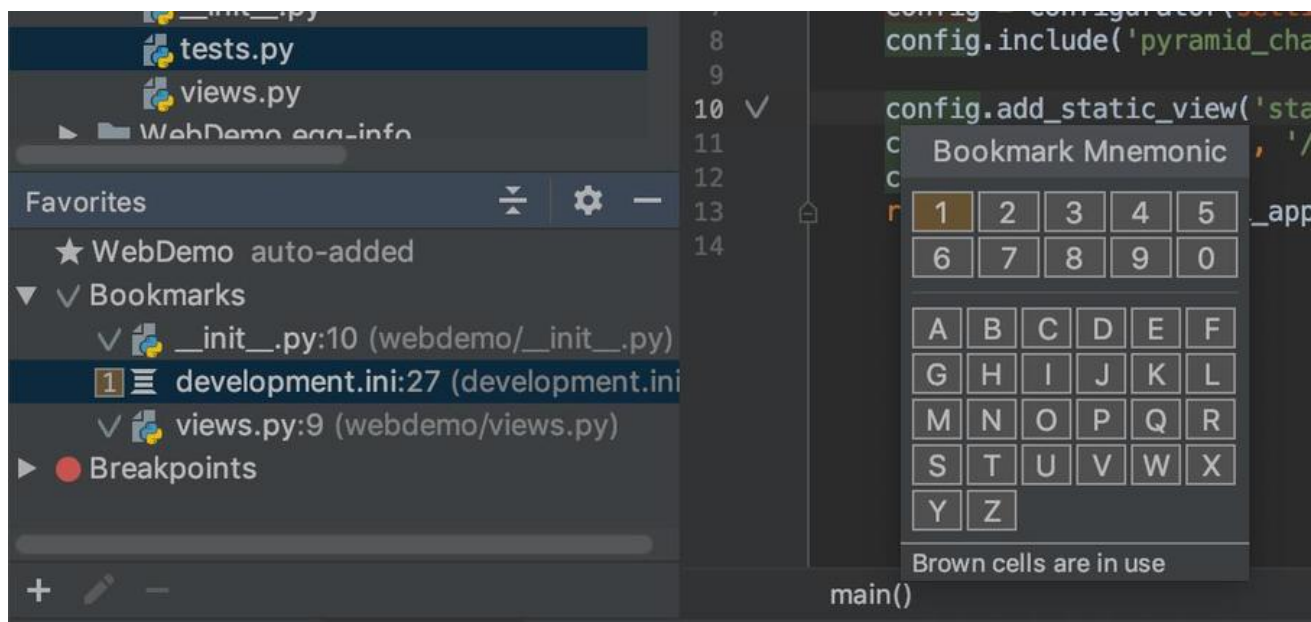


图158: 说明添加助记书签的图。

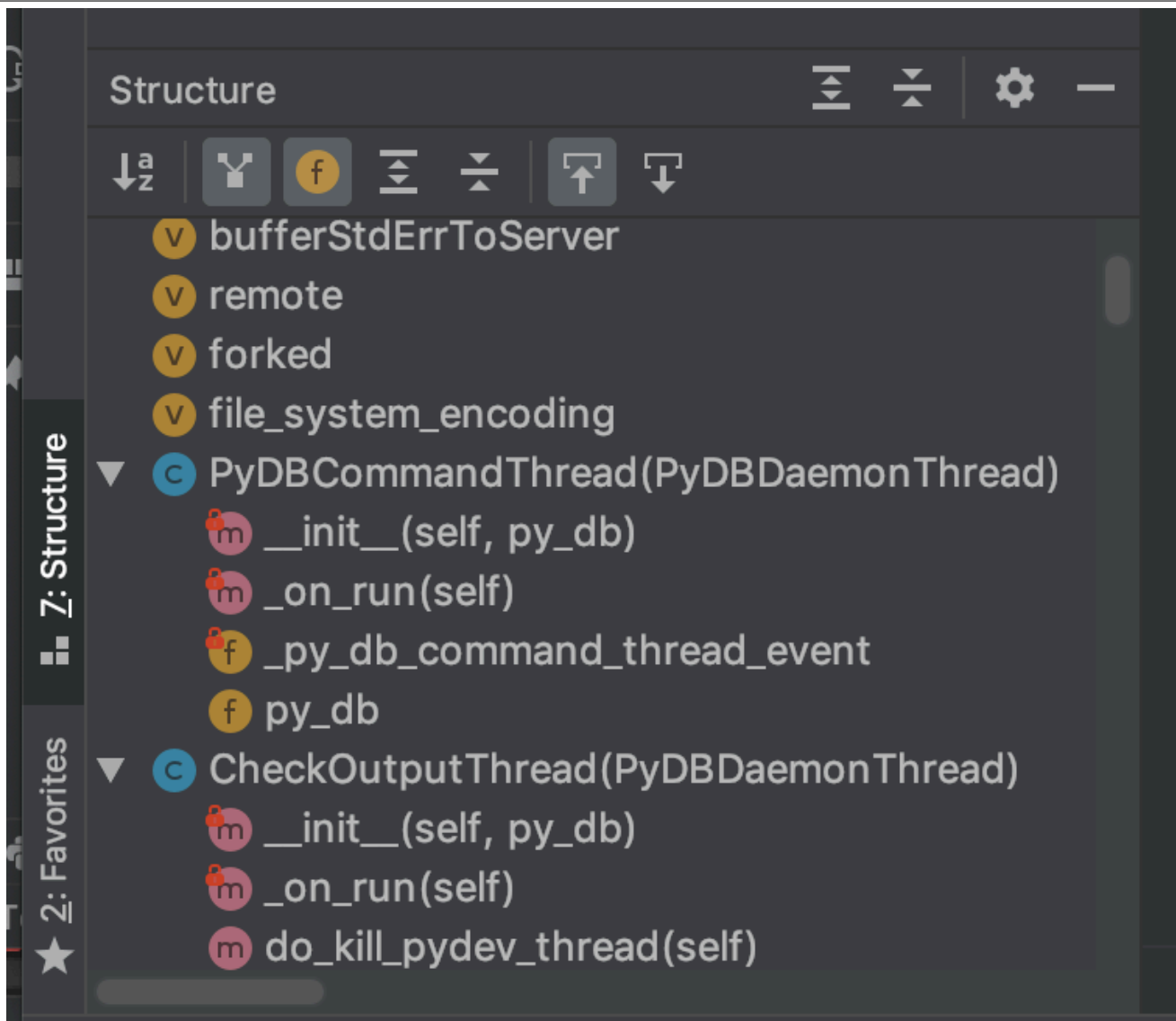


图159：说明“结构”窗口的图

断点将出现在收藏夹视图中。同样，通过在收藏夹窗口中单击，您可以导航到一个断点。

15.7 结构工具窗口

“Structure”窗口（Command-7 或 Alt-7）显示编辑器窗口的内容。它将显示类、方法、函数和全局变量。您可以通过在窗口中单击来导航到这些内容。

结构窗口对于浏览包含许多类和函数的大型文件很有用。它对于导航到类的父成员也很有用。

15.8 总结

有一系列工具，虽然并非绝对必要，但能极大提升你的编码体验。“待办事项”窗口可让你追踪需要更新或更改代码的区域。控制台窗口集成了Python交互式解释器（REPL），便于你探索和试验项目。运行代码时会出现“运行”窗口，它会显示代码执行的输出。终端窗口可让你快速访问虚拟环境和命令行脚本。“收藏夹”窗口可让你导航并记住常用的文件和代码。最后，“结构”窗口会总结文件中的代码。在处理项目时，不妨利用这些窗口。

15.9 命令

- *"TODO"* - (Command - 6 或 Alt - 6)
- *"Previous TODO"* - (Option - Command - 向上箭头 或 Ctrl - Alt - 向上箭头)
- *"Next TODO"* - (Option - Command - 向下箭头 或 Ctrl - Option - 向下箭头)
- *"Filter TODO Items"*
- *"Autoscroll to Source"*
- *"View Options"*
- *"Preview Source"*
- *"Python Console"* - (工具 -> "Python 控制台..." 或点击 "Python 控制台" 选项卡)
- *"Basic Code Completion"* - (按Ctrl + 空格)
- *"Set Value..."* - (f2)
- *"Execute Selection in Console..."* - (按alt + shift + e)
- *"Run"* - (Command - 4 或 Alt - 4)
- *"Rerun"* - (按Ctrl+F5)
- *"Stop"* - (Command+F2 或 Ctrl+F2)
- *"Pause Output"*
- *"Terminal"* - (按下alt + f12)
- *"New Session"*
- *"Favorites"* - (Command - 2 或 Alt - 2)
- *"Add to Favorites"* - (按 Alt + Shift + F, "文件" -> "添加到收藏夹", 在项目面板中右键单击某个查看, 或在编辑器中右键单击该选项卡)
- *"Add All To Favorites"*

15. 工具窗口

- *"Toggle Bookmark"* - (f11 或 导航 -> “书签” -> “切换书签”)
- *"Toggle Bookmark with Mnemonic"* - (Command+F11 或 Ctrl+F11)
- *"Go to Bookmark <1>"* - (按Ctrl+1)
- *"Toggle Breakpoint"* - (Command+F8 或 Ctrl+F8)
- *"Structure"* - (Command - 7 或 Alt - 7)

15.10 练习

1. 在您喜欢的项目中，尝试使用“待办事项”窗口。
2. 在您喜欢的项目中，尝试通过“控制台”窗口运行代码。
3. 使用你喜欢的项目，打开终端窗口并运行一些命令。尝试调用Python。列出pip已安装的软件包。
4. 使用您喜欢的项目，尝试使用“收藏夹”窗口在函数和文件之间导航。
5. 使用你喜欢的项目，尝试使用“结构”窗口来探索项目中的代码。 t.

第16章

使用插件扩展 PyCharm

16.1 插件介绍

PyCharm 实际上是基于 IntelliJ 平台上的一系列插件构建的。通过利用通用代码，我们可以在 PyCharm 中使用 DataGrip 和 JavaScript 等功能。在本章中，我们将介绍如何管理、安装和更新插件。

PyCharm还支持编写您自己的自定义插件。您可以出于个人使用目的进行此操作，也可以让其他人使用该插件。我们不会讨论插件开发，因为这需要Java知识。JetBrains提供了有关插件开发的详尽文档^[#]。

16.2 探索插件

如果您打开“*Preferences...*”窗口（command-, 或ctrl-alt-S）并点击“插件”，您将看到插件设置。插件设置有三个选项卡，“市场”、“已安装”和“更新”。

在“市场”选项卡中，你可以搜索插件、了解更多插件信息并安装插件。如果你点击插件名称，将看到有关该插件的更多详细信息。让我们以“Key Promoter X”插件为例进行探讨。我们非常热衷于学习PyCharm的键盘快捷键，而这个插件对此大有裨益。Key Promoter X插件使用IntelliJ消息系统，告知你通过鼠标执行的命令对应的键盘快捷键。

详情页面会显示版本、发布日期、下载次数、评分以及插件作者提供的任何文本。还有一个安装插件的按钮。点击“安装”按钮即可使用该插件。您会看到一份“第三方插件隐私声明”，您可以选择接受。

注意

这些插件中有许多是开源的，而有些则是专有的。与互联网上的许多事物一样，可能存在恶意代码。买家需谨慎。

PyCharm 中的插件页面会显示一个“重启 IDE”按钮。点击该按钮以使用新插件。现在，当您使用鼠标执行命令时，此插件将指示键盘快捷键。

²⁴<http://www.jetbrains.org/intellij/sdk/docs/welcome.html>

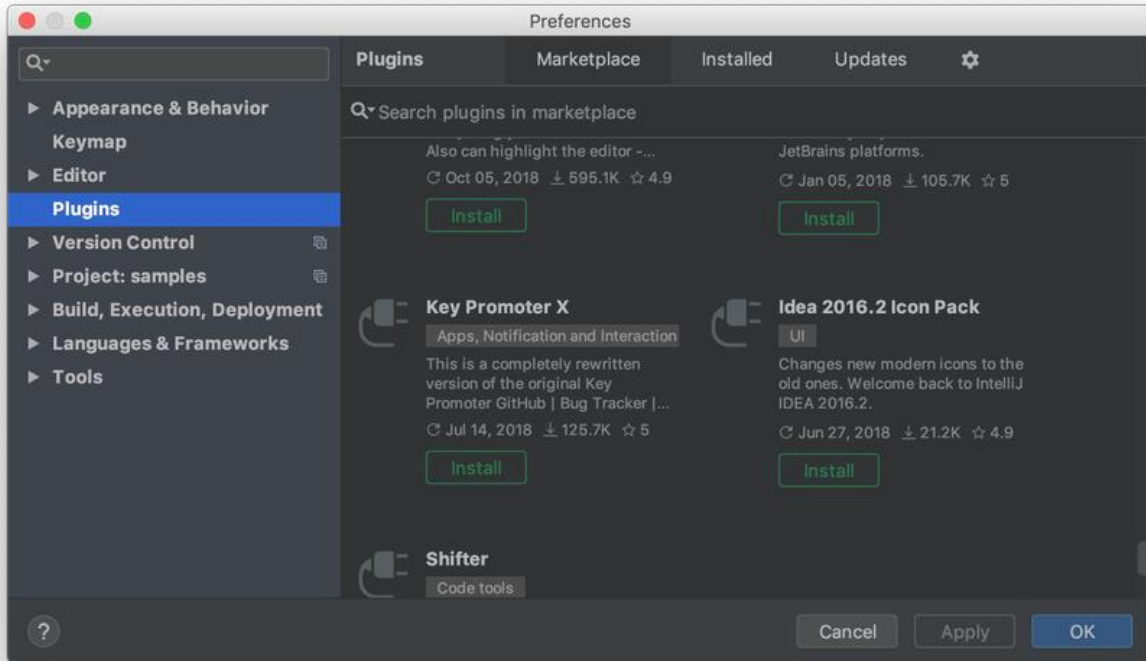


图161：说明“偏好设置”中“插件”窗口的图。

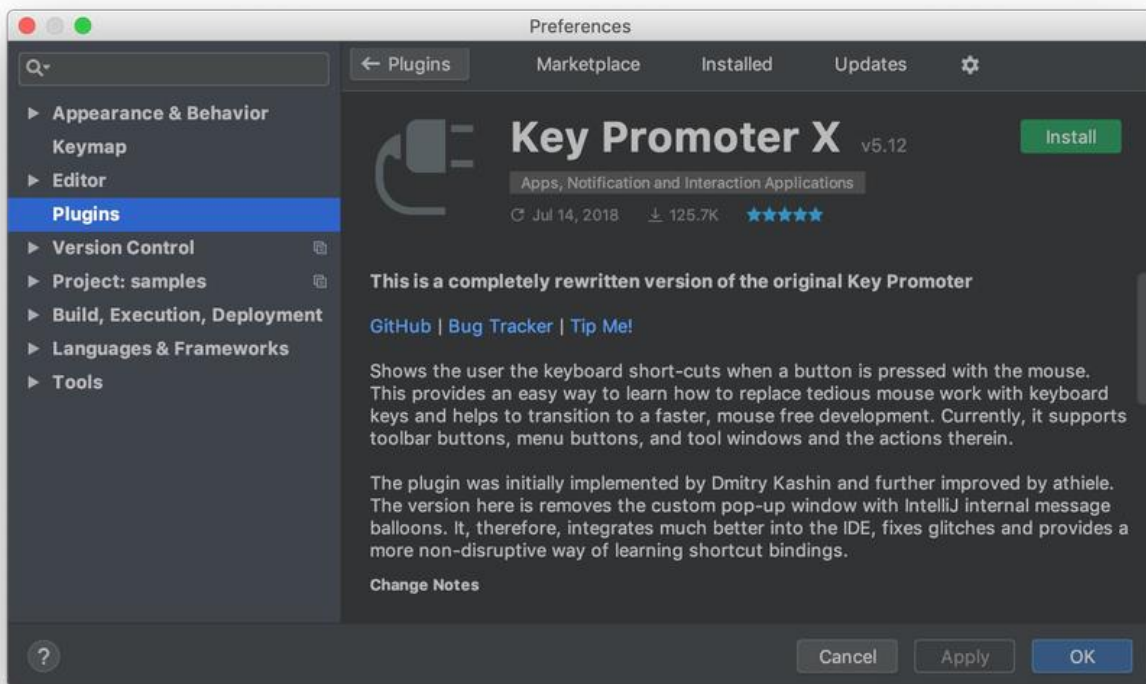


图162：展示插件细节的示意图。

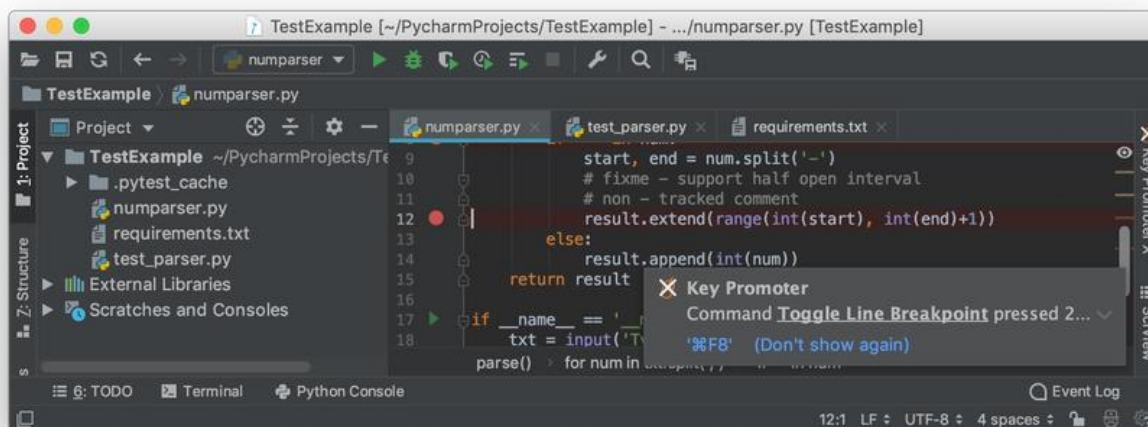


图163：展示关键启动器X的图，该图显示了断点命令的快捷方式。

16.3 已安装的插件

“插件”设置的“已安装”选项卡会显示PyCharm中已安装的插件。在此页面上，你也可以禁用插件。有“已下载”插件和“内置”插件部分。内置插件是预先配置的插件，为PyCharm提供基本功能。通常，你不会想要禁用内置插件。“CVS集成”可能是一个你可以安全禁用的内置插件。使用PyCharm的原因是利用其插件和集成功能。

“更新”选项卡会显示已安装的任何插件是否有更新。这使您能够使用最新、最出色的版本。

16.4 总结

PyCharm 通过使用插件来工作。内置插件定义了 PyCharm 的行为。此外，还有一千多个插件可供你安装。如果你需要 PyCharm 具备特定行为，请查看是否有支持该行为的插件。

16.5 命令

- “Preferences...” 窗口 - (Command + S 或 Ctrl + Alt + S)

16.6 练习

1. 访问<https://github.com/talkpython/mastering-pycharm-course> 按照 tree/master/your-turn/13-plugins 中新项目的说明来探索现有插件。

16. 使用插件扩展PyCharm

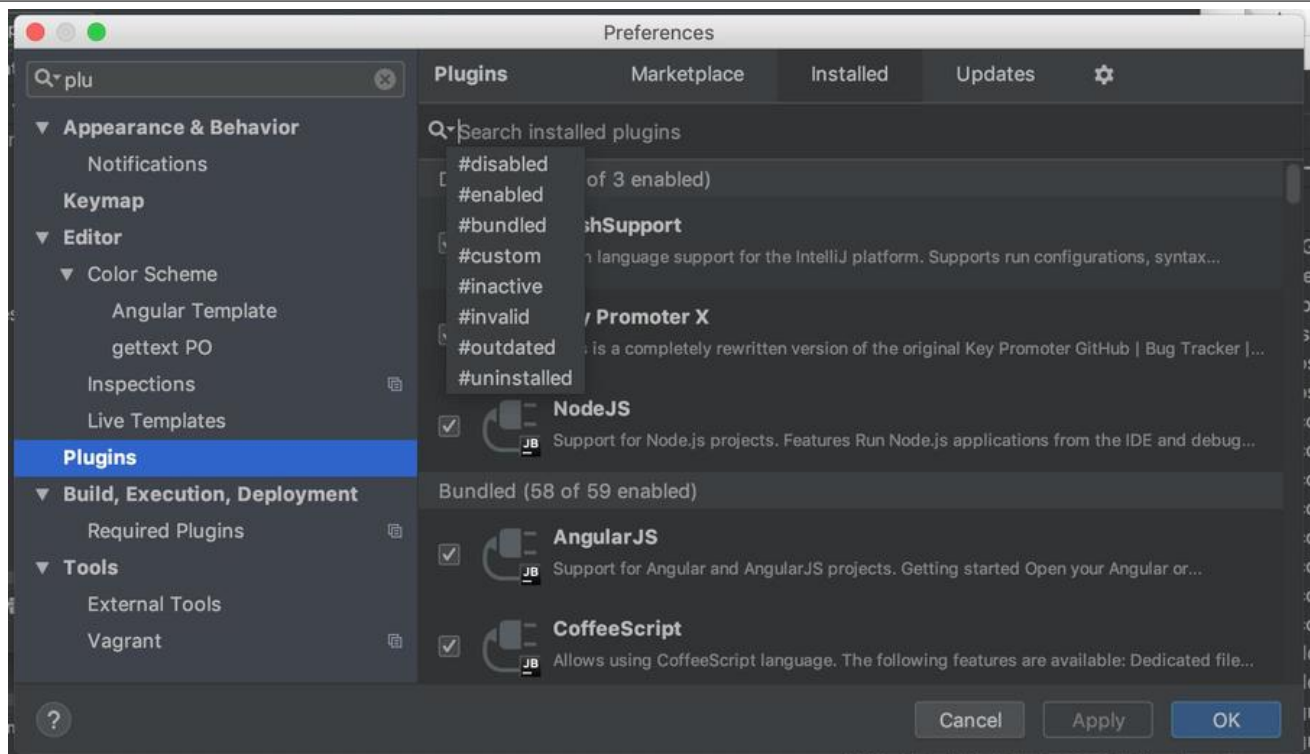


图164：说明已安装插件设置的图。

2. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照tree/master/your-turn/13-plugins中新项目的说明来发现新插件。3. 访问<https://github.com/talkpython/mastering-pycharm-course>，按照tree/master/your-turn/13-plugins中新项目的说明来安装新插件。

第17章

结论

17.1 你做到了！

恭喜！你已经探索了PyCharm的诸多强大功能。PyCharm是一款功能强大的工具，可助力Python开发的各个方面。它能够处理简单脚本、复杂应用程序、网站开发以及数据分析。

在本书中，我们研究了PyCharm的许多功能。我们探讨了项目。项目包含一组代码的配置。它们指明了PyCharm如何处理目录和文件，以及如何配置Python环境。确保正确配置您的项目，以便充分利用PyCharm的功能。

我们探索了PyCharm编辑器的功能，并了解了代码意图。这个强大的功能使您能够清理代码并自动修复错误。PyCharm拥有强大的代码补全和文档辅助工具，让创建代码变得轻松无痛。“查找操作”功能使我们能够快速发现或重新发现命令。此外，我们可以更改键盘映射或将其自定义为我们喜欢的样子。

PyCharm 具有出色的 Git 集成功能。你可以使用它来创建分支、提交更改以及查看差异。它与GitHub 也能很好地配合使用。PyCharm 还会维护一份独立于版本控制的本地历史记录。此外，它还有针对其他版本控制系统的插件和支持，但 Git 非常受欢迎。

清理代码和修复代码异味是程序员经常要做的事。PyCharm 凭借其对代码的深入理解，能够为您执行安全的重构。PyCharm 有一个“查找重复项”功能，可让您看到潜在的重构点。它还支持重命名、移动代码以及提取常量和变量。

PyCharm专业版包含DataGrip插件。这个强大的工具为许多数据库提供了一个通用界面。我们在SQLite中展示了一个示例，但它也适用于Oracle、Postgres、MySQL、SQLServer等数据库。PyCharm会绘制表和关系的结构示意图。它允许您查询和查看数据。此外，您可以从DataGrip更新模式。最后，PyCharm具有语言注入功能，因此如果您在字符串或代码文件中开始输入SQL，PyCharm将识别它并为SQL提供代码补全建议。

许多Python开发者从事Web应用程序的开发工作。PyCharm具备完善的功能，可支持多种常见的Python框架，包括Flask、Django和Pyramid。它提供了用于创建项目的模板，并且与模板语言深度集成。借助运行器实用工具，您无需离开PyCharm即可快速试用应用程序。

17. Conclusion

PyCharm 专业版还附带强大的 JavaScript 工具。你可以在 PyCharm 中重构 JavaScript。你也可以从 PyCharm 创建 TypeScript。PyCharm 拥有支持 Angular、Vue、React、LESS 等的插件。

开发者会花费大量时间阅读和调试代码。PyCharm 拥有图形化调试器，让您能够设置断点、逐行浏览代码并检查值。PyCharm 还具备“内联调试”功能，会在代码旁边显示变量的值。使用调试功能应能让您以最小的工作量理解代码并修复错误。

PyCharm 支持 Python 打包。你可以在现有的 `setup.py` 文件上运行任务。你可以使用“创建 `setup.py`”命令，轻松创建新的包。除了便于创建 Python 代码外，你还可以创建包并与他人共享代码。PyCharm 会在整个过程中为你提供支持。

除了调试功能，PyCharm 还具备性能分析功能。无需额外配置。只要能运行程序，就能对其进行性能分析。性能分析报告将以表格形式展示调用次数和耗时。还可以查看调用图，以确定哪些代码调用了运行缓慢的代码。

由于 Python 不是编译型语言，因此测试对于确保代码正确运行和工作非常重要。（并非编译就能保证代码正确运行）。Python 支持内置的 `unittest` 库，也能与 `pytest` 库配合使用。花些时间学习 `pytest` 库，在编写更好的 Python 代码时会有所回报。PyCharm 还提供代码覆盖率支持。你可以快速查看代码行号旁边的空白区域，判断是否有测试覆盖某一行代码。同样，点击一个按钮就能获得此功能。一旦你有了运行配置，就能免费获得性能分析和代码覆盖率分析功能。

Python 是数据科学家最常用的工具之一。PyCharm 支持这类工作所需的工作流程。科学模式使 PyCharm 能够快速反馈您运行的数据分析和模型结果。PyCharm 还支持 Jupyter 笔记本。

PyCharm 中有许多工具窗口，能让你的工作更轻松。“待办事项”窗口可跟踪你未来想要更改的事项。运行工具和 Python 控制台可让你运行代码并检查实时变量。终端可让你与虚拟环境进行交互。你可以使用“收藏夹”窗口跟踪文件和文件中的行。最后，“结构”窗口可让你从高层次查看类、函数和变量。

PyCharm 有许多你可以利用的插件。如果你需要额外的功能，请务必查看是否有相关插件。我们展示了如何使用 Key Promoter X 插件来帮助记忆键盘命令。

17.2 轮到你了

我们介绍了 PyCharm 的许多功能。你如何记住所有这些功能呢？本书的每一章都列出了所讨论的命令。我们建议将这些命令复制到便利贴上，并将它们放在你能看到的地方。学习这些命令对提高工作效率大有裨益。

此外，每章都有练习题。务必利用好这些练习题。通过做练习，你能形成使用 PyCharm 的肌肉记忆，从而加强学习效果。

17.3 感谢与道别

感谢阅读本书。我们很享受创作本书的过程，希望你在阅读和做练习的过程中学到了很多知识。欢迎在推特上关注我们，我们的账号是@__mharrison__和@mkennedy。如果你喜欢本书，烦请考虑在亚马逊上留下评论。这是让更多人了解本书的最佳方式。也欢迎在社交媒体上分享本书链接。

关于作者



马特·哈里森（Matt Harrison）经营着MetaSnake，这是一家专注于Python和数据科学的咨询与企业培训机构。过去，他曾涉足搜索、构建管理与测试、商业智能以及存储等领域。

他曾在Strata、SciPy、SCALE、PyCON和OSCON等会议以及本地用户会议上展示并讲授教程。本书的结构和内容基于向众多个人教授Python的第一手经验。

他在hairysun.com撰写博客，偶尔会在@__mharrison__上发布与Python相关的有用信息。



迈克尔·肯尼迪（Michael Kennedy）是《Python字节》（Python Bytes）和《与Python对话》（Talk Python to Me）的主持人。他还是Talk Python培训的创始人，以及Python软件基金会成员。迈克尔在开发者领域工作了20多年，并在众多会议上发表过演讲。

在推特上关注迈克尔，他在那里的账号是@mkennedy。